

CERTIFICATION PREP • NVIDIA AI NETWORKING

NCP-AIN Hands-On Lab Guide

Step-by-Step Labs on NVIDIA DSX Air for the NCP-AIN Certification Guide

Build • Configure • Break • Fix — Spectrum-X, InfiniBand, Kubernetes and Automation

Vakeesan Thevarajah

CCIE #43898

Lab Guide • First edition, 2026

September 2026

Cloudfoxy Ltd

NCP-AIN Hands-On Lab Guide: Step-by-Step Labs on NVIDIA DSX Air

Companion to *NCP-AIN Certification Guide: NVIDIA AI Networking with Spectrum-X and InfiniBand*

First edition, 2026

Copyright © 2026 Vakeesan Thevarajah. All rights reserved.

Published by **Cloudfoxy Ltd**, United Kingdom.

Independent publication. This lab guide is not produced, sponsored, endorsed or reviewed by NVIDIA Corporation. NVIDIA, DSX Air, Spectrum-X, Cumulus, NVUE, NetQ, UFM, ConnectX, BlueField, DOCA and other names are trademarks of NVIDIA Corporation, used here only to identify the products discussed.

Lab environment. The labs were designed for an NVIDIA DSX Air free-trial account (individual organisation) and the software versions available in September 2026 (Cumulus VX 5.16, Ubuntu 24.04). Air, Cumulus Linux and the open-source tools change; where a step behaves differently on your release, follow the version notes and the current NVIDIA documentation.

How this guide was produced. Portions of this guide were drafted with the assistance of generative AI tools. The InfiniBand simulator steps were executed and checked on Ubuntu 24.04; other outputs are labelled illustrative where they could not be captured from a live system.

Errata and lab files. Corrections and the lab files (topology JSON, InfiniBand fabric file, templates and playbooks) are published at <https://cloudfoxy-ltd.github.io/ncp-ain-guide/>.

Contents

Before You Start.....	6
What this guide is.....	6
What you can and cannot do in Air.....	6
Lab map.....	6
Conventions.....	8
Saving your credits.....	8
Lab 0 — DSX Air Account, Free Trial and the Helix Lab Topology.....	9
Lab at a glance.....	9
Objectives.....	9
Background.....	9
Step-by-step.....	11
Verify.....	15
Break and fix.....	15
Clean-up / save state.....	16
Exam tie-in.....	16
Review questions.....	16
Lab 1 — NVUE Essentials on Cumulus VX.....	17
Lab at a glance.....	17
Objectives.....	17
Background.....	17
Step-by-step.....	18
Verify.....	29
Break and fix.....	29
Clean-up / save state.....	31
Exam tie-in.....	31
Review questions.....	31
Lab 2 — Underlay with eBGP Unnumbered.....	33
Lab at a glance.....	33
Objectives.....	33
Background.....	33
Step-by-step.....	34
Verify.....	42
Break and fix.....	43
Clean-up / save state.....	45
Exam tie-in.....	45
Review questions.....	45
Lab 3 — Lossless RoCE QoS and Soft-RoCE Hosts.....	47
Lab at a glance.....	47
Objectives.....	47
Background.....	47
Step-by-step.....	48
Verify.....	58
Break and fix.....	58
Clean-up / save state.....	60
Exam tie-in.....	61
Review questions.....	61
Lab 4 — Adaptive Routing, ECMP and Telemetry Configuration.....	62
Lab at a glance.....	62
Objectives.....	62
Background.....	62
Step-by-step.....	63
Verify.....	69
Break and fix.....	70
Clean-up / save state.....	71

Exam tie-in.....	71
Review questions.....	71
Lab 5 — BGP-EVPN Multi-Tenancy.....	73
Lab at a glance.....	73
Objectives.....	73
Background.....	73
Step-by-step.....	75
Verify.....	86
Break and fix.....	86
Clean-up / save state.....	88
Exam tie-in.....	88
Review questions.....	89
Lab 6 — Air Power Skills: Checkpoints, Cloning, Services and the SDK.....	90
Lab at a glance.....	90
Objectives.....	90
Background.....	90
Step-by-step.....	92
Verify.....	104
Break and fix.....	104
Clean-up / save state.....	104
Exam tie-in.....	105
Review questions.....	105
Lab 7 — Monitoring and Troubleshooting a Spectrum-X-Style Fabric.....	106
Lab at a glance.....	106
Objectives.....	106
Background.....	106
Step-by-step.....	107
Break and fix.....	110
Verify.....	111
Clean-up / save state.....	111
Exam tie-in.....	111
Review questions.....	111
Lab 8 — An InfiniBand Fabric with ibsim: SM, HA, Partitions and Diagnostics.....	113
Lab at a glance.....	113
Objectives.....	113
Background.....	113
Step-by-step.....	115
Verify.....	122
Break and fix.....	122
Clean-up / save state.....	123
Exam tie-in.....	123
Review questions.....	123
Lab 9 — Verifying End-to-End Performance with perftest over Soft-RoCE.....	124
Lab at a glance.....	124
Objectives.....	124
Background.....	124
Step-by-step.....	125
Verify.....	129
Break and fix.....	130
Clean-up / save state.....	130
Exam tie-in.....	130
Review questions.....	130
Lab 10 — Kubernetes and the NVIDIA Network Operator.....	131
Lab at a glance.....	131
Objectives.....	131
Background.....	131

Step-by-step.....	133
Verify.....	139
Break and fix.....	139
Clean-up / save state.....	139
Exam tie-in.....	140
Review questions.....	140
Lab 11 — NVUE Templates and Configuration Management.....	141
Lab at a glance.....	141
Objectives.....	141
Background.....	141
Step-by-step.....	142
Verify.....	148
Break and fix.....	148
Clean-up / save state.....	149
Exam tie-in.....	149
Review questions.....	149
Lab 12 — Ansible Automation: VLANs and RoCE with nvidia.nvue.....	150
Lab at a glance.....	150
Objectives.....	150
Background.....	150
Step-by-step.....	151
Verify.....	155
Break and fix.....	155
Clean-up / save state.....	155
Exam tie-in.....	156
Review questions.....	156
Lab 13 — Capstone: Troubleshooting Challenge.....	157
Lab at a glance.....	157
Objectives.....	157
Background.....	157
Step-by-step.....	159
Break and fix.....	165
Solutions.....	169
Verify.....	175
Clean-up / save state.....	175
Exam tie-in.....	175
Review questions.....	175
Appendix A — helix-b-air.json.....	178
Resource budget.....	179
Appendix B — helix-a.net (InfiniBand simulator fabric).....	180
Appendix C — Quick Reference.....	182
Credentials used in this guide.....	182
Everyday commands.....	182
Appendix D — Troubleshooting the Lab Environment.....	183
Appendix E — Where Each Exam Objective Is Practised.....	184

Before You Start

What this guide is

This guide turns the *NCP-AIN Certification Guide* into hands-on practice. You build a small version of the book's running example, the **Helix AI factory**, inside **NVIDIA DSX Air**, NVIDIA's cloud-hosted digital twin, and then configure, verify, break and repair it lab by lab. Every lab maps to chapters of the book and to NCP-AIN exam objectives.

The labs run on the **free trial**. That shapes the design: the whole topology fits inside the trial's limits of 60 concurrent vCPUs and 60 GiB of memory, with room to spare.

What you can and cannot do in Air

Be clear about this from the start, because the exam tests real-hardware behaviour.

Topic	In this lab guide	What you still need to learn from the book
Cumulus Linux and NVUE	Fully real (Cumulus VX runs the same NVUE and FRR as a Spectrum switch)	—
BGP unnumbered, EVPN, VRFs, VXLAN	Fully real control plane and data plane	—
RoCE QoS (trust, maps, PFC, ECN)	Configuration and verification of intent; DSCP marking visible in captures	Lossless behaviour under load, pause frames, ECN marking (needs a Spectrum ASIC)
Adaptive routing, telemetry, WJH	Configuration only	Forwarding behaviour and drop reasons (ASIC features)
RDMA and perfest	Real RDMA verbs over Soft-RoCE (software RoCE in the Linux kernel)	Hardware numbers (ConnectX line-rate, sub-2 μ s latency)
InfiniBand	Real OpenSM and InfiniBand diagnostic tools against a simulated fabric (ibsim)	ibdiagnet, UFM, SHARP and hardware counters
Kubernetes Network Operator	Real operator, CRDs and secondary networks (Multus, NV-IPAM, macvlan)	RDMA device plugins and DOCA drivers (need NVIDIA NICs)
NVUE templates and Ansible	Fully real	—
NCCL tests	Not possible (no GPUs)	Chapter 19 methodology

Exam focus: When a lab can only verify configuration (for example PFC on Cumulus VX), the lab says so and points you to the chapter that explains the hardware behaviour. The exam asks about both.

Lab map

Lab	Title	Book chapters	Exam objectives	Time
0	Air account, trial and the Helix-B lab topology	9	2.4	60 min
1	NVUE essentials on Cumulus VX	4	2.1, 6.1	60 min
2	Underlay: eBGP unnumbered	4, 7	2.1, 2.3	60 min
3	Lossless RoCE QoS and Soft-RoCE hosts	3, 5	2.1, 2.2	75 min

Lab	Title	Book chapters	Exam objectives	Time
4	Adaptive routing, ECMP and telemetry configuration	6	2.2, 2.5	45 min
5	BGP-EVPN multi-tenancy	7	2.3	90 min
6	DSX Air power skills	9	2.4	45 min
7	Monitoring and troubleshooting	10, 18	2.5, 2.6, 5.1, 5.2	60 min
8	InfiniBand fabric with ibsim	11–15, 17, 20	3.1–3.4, 5.4, 5.5	90 min
9	perftest over Soft-RoCE	19	5.3, 5.5	60 min
10	Kubernetes and the NVIDIA Network Operator	16	4.1, 4.2	90 min
11	NVUE templates and configuration management	21	6.1	60 min
12	Ansible automation: VLANs and RoCE	22	6.2	60 min
13	Capstone troubleshooting challenge	All	All	120 min

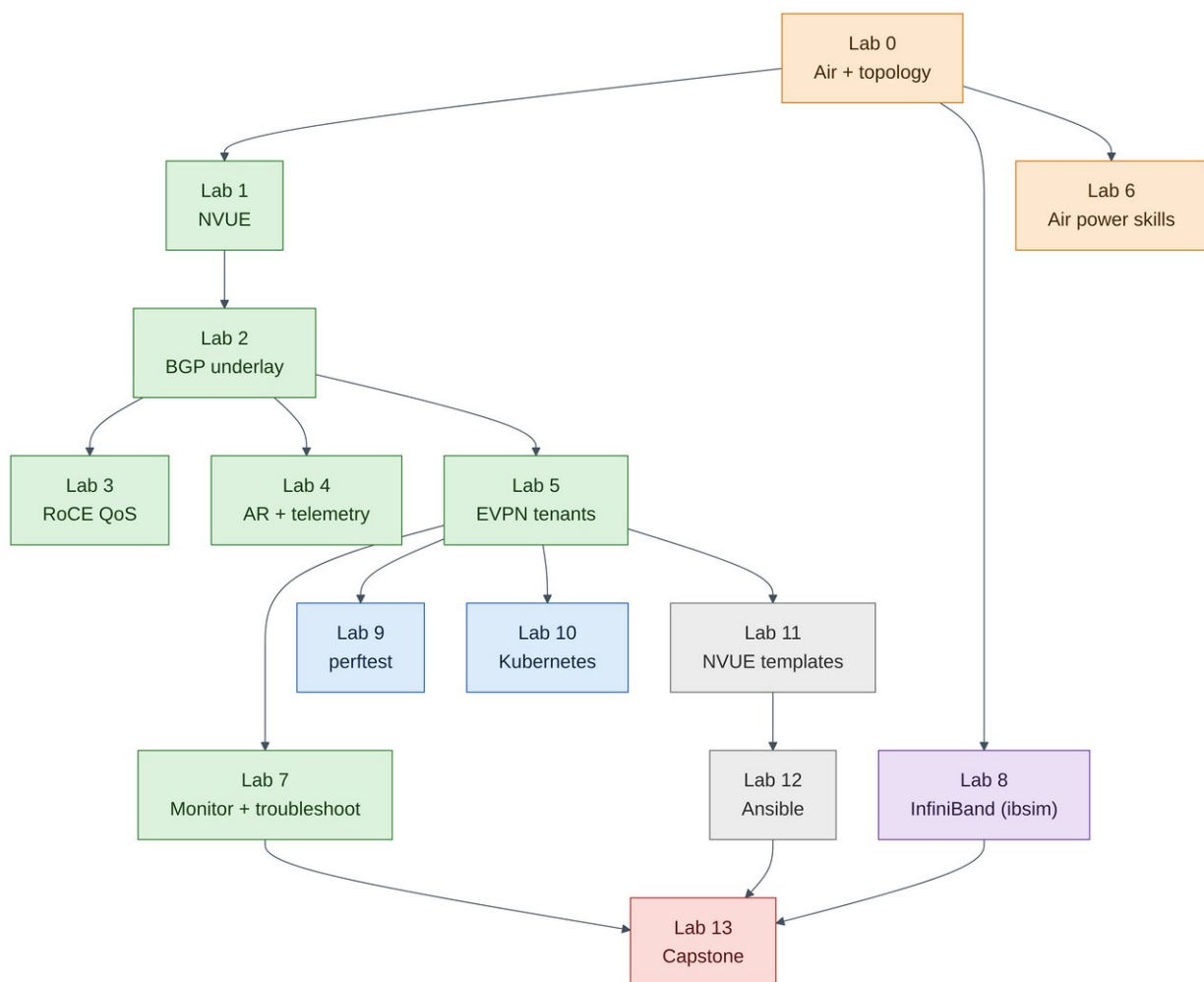


Figure 0.1: Lab dependency map

Labs 0–2 build the platform. Lab 5 depends on Labs 1–2; Labs 9 and 10 run across the EVPN fabric from Lab 5. Lab 8 (InfiniBand) only needs Lab 0. Lab 13 draws on everything.

Conventions

- **Prompts** show where to type: `cumulus@hxb-leaf-r1:mgmt:~$` is a switch, `ubuntu@hxb-gpu01:~$` a server, `ubuntu@oob-mgmt-server:~$` the jump host, and `sim>` the InfiniBand simulator console.
- **Expected output** blocks show what you should see. Outputs marked *illustrative* show the shape of the output; values such as MAC addresses, uptimes and counters will differ.
- **Checkpoint** lists tell you what must be true before you move on. Don't skip them: most later problems come from an earlier checkpoint that was never met.
- Callouts follow the book: **Exam focus**, **Warning**, **Field note**, **Version note** and **Check your understanding**.

Saving your credits

The free trial gives you compute-hour credits for a year. The lab topology uses about 26 vCPUs while it runs.

Always sleep the simulation (store a checkpoint) when you stop for the day, and save switch configuration with `nv config save` first. Lab 6 shows how to estimate and manage credits in detail.

Warning: A simulation left running overnight spends credits for nothing. Set a sleep date in the simulation settings as a safety net (Lab 0, Task 5).

Lab 0 — DSX Air Account, Free Trial and the Helix Lab Topology

Lab at a glance

Book chapters	9 (Lab Simulation with NVIDIA Air), 1 (running example)
Exam objectives	2.4 Simulate a Spectrum-X network with NVIDIA Air
Time	60 minutes (plus 10–15 minutes of boot time)
Air resources	About 26 vCPU and 46 GiB while running
Prerequisites	A web browser, an SSH client (Windows Terminal, macOS Terminal or PuTTY)

Objectives

By the end of this lab you will have:

- activated the DSX Air free trial and understood its limits;
- imported the **helix-b-air** topology from JSON and started it;
- logged in to every switch and server through the out-of-band (OOB) management network;
- verified the cabling with LLDP;
- protected your credits with a sleep date and your first checkpoint.

Background

DSX Air runs real network operating systems as virtual machines. The switches run **Cumulus VX**, the virtual edition of Cumulus Linux, with the same NVUE CLI, FRR routing suite and Linux kernel data path that a Spectrum switch uses. The servers run Ubuntu. Air wires them together exactly as the topology file describes and adds an **OOB management network**: an **oob-mgmt-switch** connected to **eth0** on every server and the management port on every switch, and an **oob-mgmt-server** that acts as jump host, DHCP and DNS server, and NAT gateway to the internet.

The lab topology is a compressed Helix-B (Chapter 1): two spines, four rail leaves and four "GPU servers" (Ubuntu VMs without GPUs), plus one Ubuntu node, **hxa-ibsim**, that hosts the InfiniBand simulator used in Lab 8.

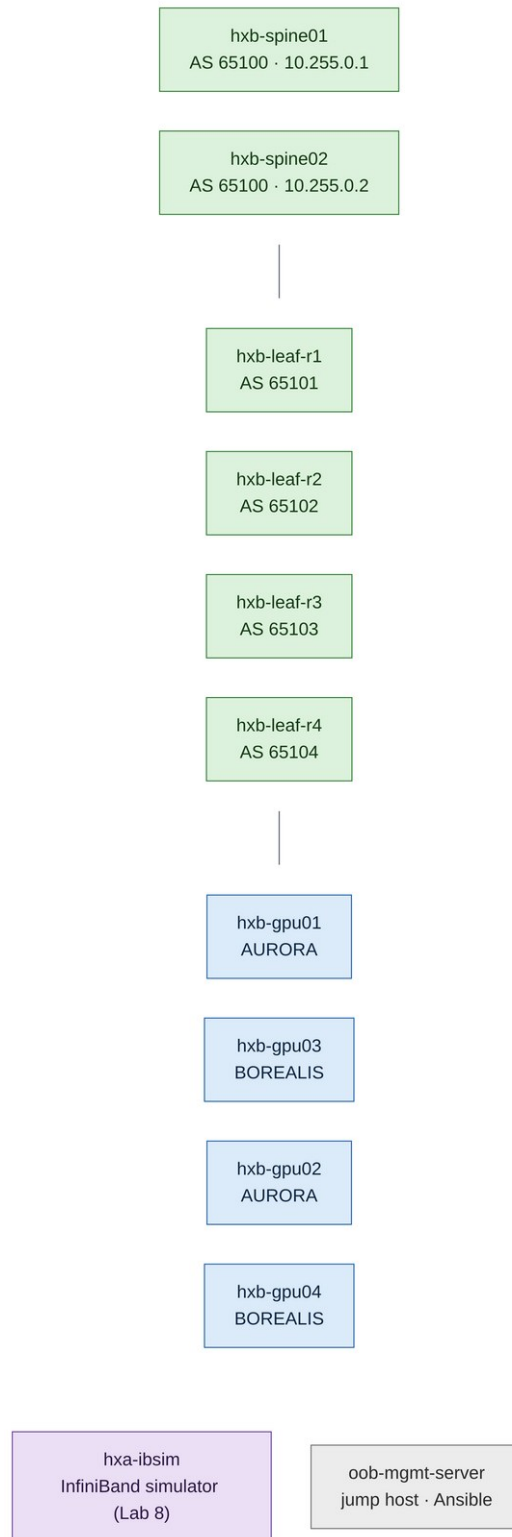


Figure 0.2: The helix-b-air lab topology

Every leaf connects to both spines (swp31 → spine01, swp32 → spine02). Each server has two data NICs: eth1 to an odd leaf and eth2 to an even leaf. AURORA servers are gpu01 and gpu02, BOREALIS servers gpu03 and gpu04. The OOB network (not drawn) reaches every node's eth0 or management port.

Addressing and cabling plan

Node	Loopback / role	Uplinks
hxb-spine01	10.255.0.1/32, AS 65100	swp1–swp4 → leaf-r1...r4 swp31

Node	Loopback / role	Uplinks
hxb-spine02	10.255.0.2/32, AS 65100	swp1–swp4 → leaf-r1...r4 swp32
hxb-leaf-r1 ... r4	10.255.0.11–.14/32, AS 65101–65104	swp31 → spine01, swp32 → spine02

Server NIC	Leaf port	Tenant / VLAN (Lab 5)	Host IP (Lab 5)
hxb-gpu01 eth1	hxb-leaf-r1 swp1	AURORA VLAN 110	172.16.10.101/24
hxb-gpu01 eth2	hxb-leaf-r2 swp1	AURORA VLAN 111	172.16.11.101/24
hxb-gpu02 eth1	hxb-leaf-r3 swp1	AURORA VLAN 110	172.16.10.102/24
hxb-gpu02 eth2	hxb-leaf-r4 swp1	AURORA VLAN 111	172.16.11.102/24
hxb-gpu03 eth1	hxb-leaf-r1 swp2	BOREALIS VLAN 210	172.17.10.103/24
hxb-gpu03 eth2	hxb-leaf-r2 swp2	BOREALIS VLAN 211	172.17.11.103/24
hxb-gpu04 eth1	hxb-leaf-r3 swp2	BOREALIS VLAN 210	172.17.10.104/24
hxb-gpu04 eth2	hxb-leaf-r4 swp2	BOREALIS VLAN 211	172.17.11.104/24

Step-by-step

Task 1 — Sign in and activate the free trial

1. Go to <https://dsx-air.nvidia.com> and sign in with your NVIDIA account. DSX Air uses NVIDIA NGC organisations: when you first sign in you are placed in (or asked to create) an NGC organisation.
2. If you are the **owner** of your NGC organisation, Air shows a prompt to start the free trial. Click **Start Trial**. If you are not the owner, ask the organisation owner to start it; only NGC organisation owners can activate the trial.
3. Note the trial limits. For an individual organisation they are:

Limit	Free trial
Concurrent vCPUs	60
Concurrent memory	60 GiB
Compute-hour credits	10,000
Duration	1 year

Field note: "Concurrent" means *while simulations are running*. A sleeping (checkpointed) simulation uses no vCPU or memory quota. You can keep several lab simulations and run one at a time.

Checkpoint

- [] You can open the **Simulations** page without an error or a trial prompt.

Task 2 — Add your SSH key (optional but recommended)

Console access in the browser works for every lab. SSH from your own terminal is faster for copying and pasting long command blocks.

1. On your laptop, create a key if you don't already have one:

```
you@laptop:~$ ssh-keygen -t ed25519 -C "air-lab"
you@laptop:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3Nza... air-lab
```

2. In Air, open your user menu → **Settings**, find the SSH keys section, give the key a **Name** and paste the **Public Key** line, then save.

Lab 6 (Task 4) shows how to publish SSH access to the running simulation.

Task 3 — Import the helix-b-air topology

1. Download **helix-b-air.json** from the book's companion site (<https://cloudfoxy-ltd.github.io/ncp-ain-guide/>), or copy it from **Appendix A** of this guide into a file on your laptop.
2. In Air, go to **Simulations** and click **Create Simulation**.
3. Name it **helix-b-air**, choose **JSON** as the file type, and upload **helix-b-air.json**.
4. Leave **Apply ZTP Template** unticked. You configure every switch by hand in Labs 1 and 2, which is the point of the exercise.
5. Click **Create**. Air validates the file and draws the topology on the canvas.

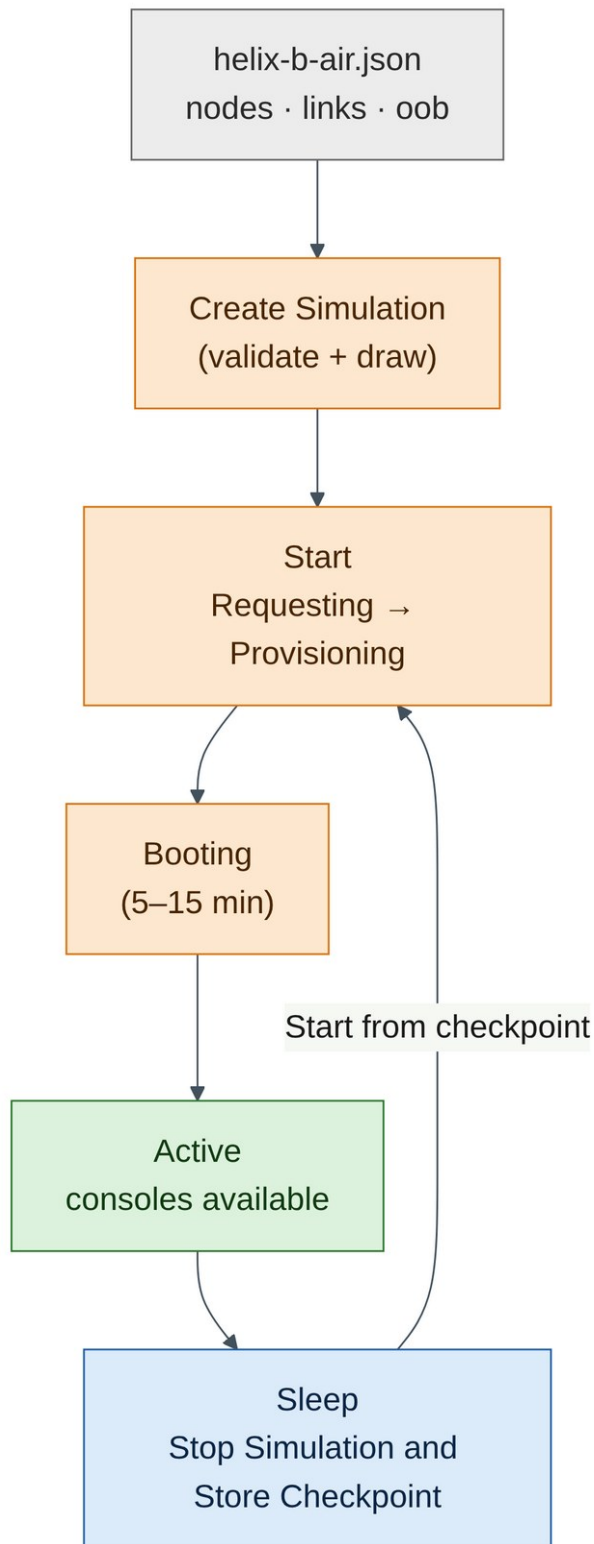


Figure 0.3: From JSON file to running simulation

Air validates the JSON, builds the nodes and links, adds the OOB network, then boots every VM. The simulation is usable when it reaches Active and every node shows as running.

Version note: Air validates node resources on import. Cumulus VX nodes need at least **20 GB of storage**; a smaller value fails with "Resource amount of 10 does not reach the minimum requirement of 20". The file in Appendix A already uses 20. If validation reports another minimum on your account, raise that value in the JSON and import again.

Version note: The JSON uses the operating-system image names `cumulus-vx-5.16.1` and `generic/ubuntu2404`, as shown in the DSX Air documentation. If Air rejects an image name, open the simulation in the canvas, select a node, and pick the nearest available Cumulus VX 5.x or Ubuntu 24.04 image from the **OS** list. Keep all six switches on the same Cumulus version.

Checkpoint

- [] The canvas shows 2 spines, 4 leaves, 4 gpu servers, `hxa-ibsim`, and the OOB switch and server.

Task 4 — Start the simulation and log in

1. Click **Start**. The state moves through *Requesting* → *Provisioning* → *Booting* → *Active*. Allow 5–15 minutes.
2. Double-click **oob-mgmt-server** to open its console. Log in as `ubuntu`. Air's Ubuntu images commonly use the password `nvidia`; if that fails, check the node's details panel or console banner for the credentials your image uses.
3. From the oob-mgmt-server, reach any node by name. Start with a switch:

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-leaf-r1
cumulus@hxb-leaf-r1's password:          # default: cumulus
You are required to change your password immediately (administrator enforced).
Current password:                        # cumulus
New password:                            # CumulusLinux!
Retype new password:                     # CumulusLinux!
```

4. Repeat for every switch (`hxb-spine01`, `hxb-spine02`, `hxb-leaf-r1` to `hxb-leaf-r4`). This guide uses `CumulusLinux!` as the new password; choose your own if you prefer, and use it consistently.

Warning: Cumulus Linux 5.x forces a password change for the `cumulus` user at first login. Automation in Labs 11 and 12 uses the new password, so write it down.

5. Log in to each server and check its interfaces:

```
ubuntu@oob-mgmt-server:~$ ssh ubuntu@hxb-gpu01
ubuntu@hxb-gpu01:~$ ip -br link
lo                UNKNOWN      00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0              UP           44:38:39:00:00:1a <BROADCAST,MULTICAST,UP,LOWER_UP>
eth1              DOWN        44:38:39:00:00:21 <BROADCAST,MULTICAST>
eth2              DOWN        44:38:39:00:00:23 <BROADCAST,MULTICAST>
```

(*Illustrative.*) `eth0` is the OOB interface. `eth1` and `eth2` are the data NICs you configure in Labs 3 and 5.

6. Make passwordless SSH easier for later labs. On the oob-mgmt-server:

```
ubuntu@oob-mgmt-server:~$ ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
ubuntu@oob-mgmt-server:~$ for h in hxb-spine01 hxb-spine02 hxb-leaf-r1 hxb-leaf-r2 hxb-leaf-r3
hxb-leaf-r4; do ssh-copy-id cumulus@$h; done
ubuntu@oob-mgmt-server:~$ for h in hxb-gpu01 hxb-gpu02 hxb-gpu03 hxb-gpu04 hxa-ibsim; do ssh-
copy-id ubuntu@$h; done
```

Checkpoint

- [] You can `ssh cumulus@hxb-leaf-r1` from the oob-mgmt-server without a password prompt.
- [] Every switch password has been changed.

Task 5 — Verify the cabling with LLDP

Cabling mistakes cause more lab problems than anything else, so prove the wiring before you configure anything. Bring the data ports up and read LLDP on every leaf.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp1-2,swp31-32 link state up
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r1:mgmt:~$ sudo lldpctl | grep -E "Interface|SysName|PortID"
Interface:    swp31, via: LLDP
  SysName:    hxb-spine01
  PortID:     ifname swp1
Interface:    swp32, via: LLDP
  SysName:    hxb-spine02
  PortID:     ifname swp1
```

(*Illustrative.*) Servers only appear in LLDP if they run an LLDP agent. Install one on each server so the switch can see it:

```
ubuntu@hxb-gpu01:~$ sudo apt-get update && sudo apt-get install -y lldpd
ubuntu@hxb-gpu01:~$ sudo ip link set eth1 up && sudo ip link set eth2 up
```

Do the same on the spines (`nv set interface swp1-4 link state up`) and compare what you see against the cabling table.

Checkpoint

- [] Each leaf sees spine01 on swp31 and spine02 on swp32.
- [] Each leaf sees the expected servers on swp1 and swp2.

Task 6 — Protect your credits and save your first checkpoint

1. On every switch, save the configuration so it survives a reboot: `nv config save`.
2. Open the simulation settings and set a **sleep date** a few hours ahead as a safety net. Free-trial simulations may already have one that you cannot remove.
3. Click **Stop Simulation and Store Checkpoint**. When the simulation is *Inactive*, open **Checkpoints**, find the new checkpoint and mark it as a favourite named something like `lab00-cabled`.

Verify

- [] Free trial active; quota visible.
- [] helix-b-air imported, started and reached Active.
- [] Passwords changed on all six switches; passwordless SSH from the oob-mgmt-server.
- [] LLDP matches the cabling plan.
- [] A favourite checkpoint saved.

Break and fix

Fault: "I can't reach hxb-leaf-r2 by name."

- *Symptom:* `ssh cumulus@hxb-leaf-r2` from the oob-mgmt-server fails with "Could not resolve hostname".
- *Diagnosis:* the node is still booting, or it has not taken its OOB DHCP lease yet. Check its state on the Air canvas and open its console.
- *Fix:* wait for the node to finish booting, then retry. If it stays unreachable, reboot the node from its menu in the canvas.

Fault: "ssh says No route to host" and the switch's eth0 has a 169.254.x.x address.

- *Symptom*: the name resolves on the oob-mgmt-server, but SSH fails with *No route to host*. On the switch console, `ip -br addr show eth0` shows a `169.254.x.x` (link-local) address.
- *Diagnosis*: the switch's DHCP request on the OOB network got no answer, so it fell back to a link-local address. Check the oob-mgmt-server: `ip -br addr` (it should own `192.168.200.1/24`) and `ps aux | grep -E '[d]hcpd|[d]nsmasq'` (a DHCP service should be running).
- *Fix*: restart the DHCP service on the oob-mgmt-server (`sudo systemctl restart isc-dhcp-server` or `dnsmasq`, whichever is installed), then renew on the switch with `sudo ifdown eth0 && sudo ifup eth0`. If the oob-mgmt-server has no `192.168.200.1` address, stop and start the simulation. As a temporary workaround, give the switch the address that `getent hosts <switch>` shows on the oob-mgmt-server: `nv set interface eth0 ip address <ip>/24, nv set interface eth0 ip gateway 192.168.200.1, nv config apply -y`.

Fault: "A leaf sees spine02 on swp31."

- *Symptom*: LLDP on a leaf shows the spines swapped.
- *Diagnosis*: the topology file was edited and the links swapped.
- *Fix*: stop the simulation, correct the link in the canvas or JSON (`swp31` → `spine01`, `swp32` → `spine02`), and start it again. Every later lab assumes this wiring.

Clean-up / save state

Always finish a session with `nv config save` on the switches and **Stop Simulation and Store Checkpoint** in Air.

Exam tie-in

- Objective 2.4 expects you to know how Air builds a digital twin: a topology file (JSON recommended, DOT supported), real NOS images, and an OOB network with a management server.
- Know that Cumulus VX runs the same NVUE and FRR as Cumulus Linux on Spectrum hardware, but has no ASIC. That's why lossless behaviour, WJH and adaptive routing forwarding can't be tested in Air.
- Know the purpose of the oob-mgmt-server: jump host, DHCP/DNS, ZTP server and NAT gateway.

Review questions

1. Which two limits of the free trial are "concurrent", and what does that mean for a sleeping simulation?
2. Why is JSON preferred over DOT for Air topologies?
3. What does the oob-mgmt-server provide to the other nodes?
4. Why do you verify cabling with LLDP before configuring BGP?
5. What must you do on each switch before sleeping a simulation, and why?

Answers

1. vCPUs (60) and memory (60 GiB) are concurrent limits: they only count while simulations run. A sleeping simulation uses none of them, only storage for its checkpoint.
2. NVIDIA recommends JSON because it has better validation, scales better and is more widely adopted; DOT is still supported.
3. A jump host reachable by SSH, DHCP and DNS for the OOB network (so nodes are reachable by name), a ZTP server, and a NAT gateway to the internet for package downloads.
4. Because BGP unnumbered, EVPN and every later lab assume the documented port mapping. A swapped cable produces confusing symptoms later; LLDP proves the wiring in seconds.
5. Run `nv config save` so the applied configuration is written to `/etc/nvue.d/startup.yaml`. Only the startup configuration is loaded at boot, so unsaved changes would be lost when the simulation restarts from its checkpoint.

Lab 1 — NVUE Essentials on Cumulus VX

Lab at a glance

Book chapters	Chapter 4 (Cumulus Linux and NVUE Essentials); Chapter 21 (NVUE Templates and Configuration Management) for the revision model
Exam objectives	2.1 Configure Spectrum-X switches (foundation: the switch OS and NVUE) · 6.1 Manage switch configuration with NVUE · supports 2.4 (NVIDIA Air)
Time	75–90 minutes
Air resources used	Simulation helix-b-air : hxb-spine01, hxb-spine02, hxb-leaf-r1 ... hxb-leaf-r4 (6 × Cumulus VX 5.16.1) and oob-mgmt-server. The GPU servers can stay powered but are not used.
Prerequisite labs	Lab 0 (simulation imported from helix-b-air.json , started, and oob-mgmt-server reachable)

Objectives

By the end of this lab you will be able to:

- Log in to a Cumulus VX switch from the out-of-band jump host and change the default password.
- Use **nv show**, **nv set** and **nv unset**, with Tab completion, to read and change the NVUE object model.
- Explain and observe the **pending**, **applied** and **startup** configurations, and use **nv config diff**, **nv config apply** and **nv config save**.
- Find past changes with **nv config history** and **nv config revision**, and roll back with **nv config apply <revision>**.
- Make a risky change safely with **nv config apply --confirm** and let it roll back by itself.
- Give every switch in Helix-B its hostname, port descriptions, jumbo MTU (9216) and loopback address, ready for the underlay in Lab 2.
- Read configuration through the NVUE REST API with **curl** on port 8765.

Background

Cumulus Linux 5.x is configured through **NVUE**, an object model of the whole switch. Every **nv** command addresses a path in that model, for example **interface swp31 link mtu** or **system hostname**. The same paths are exposed by the NVUE REST API on TCP port 8765, which is why the CLI, the API, templates (Chapter 21) and Ansible (Chapter 22) all describe configuration in the same way.

NVUE never changes the running switch while you type. **nv set** and **nv unset** only build a **pending** revision. **nv config apply** turns pending into the **applied** configuration, and NVUE then renders the Linux files (**/etc/network/interfaces**, **/etc/frr/frr.conf** and others) and reloads the services. With auto-save enabled (the default in Cumulus Linux 5.x), each apply is also written to the **startup** configuration, **/etc/nvue.d/startup.yaml**, which is what the switch loads at boot. With auto-save disabled you need an explicit **nv config save**.

Every apply is stored as a numbered revision. You can compare revisions with **nv config diff**, see who applied what with **nv config history**, and return to an earlier revision with **nv config apply <revision>**. For changes that could cut you off, **nv config apply --confirm <time>** applies the change on trial and rolls it back automatically unless you confirm it in time.

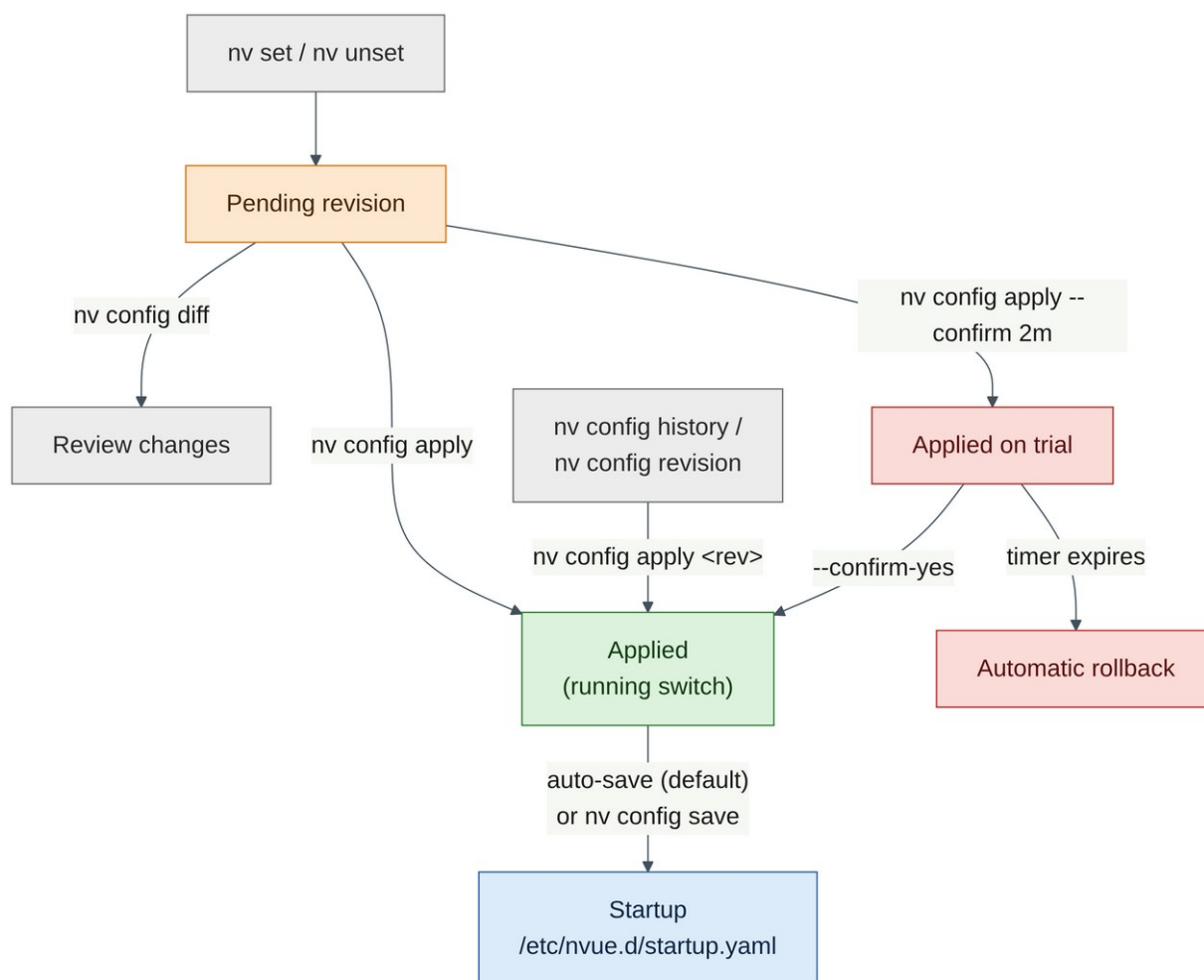


Figure L1.1: Pending, applied and startup in NVUE

nv set and nv unset only change the pending revision. nv config apply makes pending the applied (running) configuration and, with auto-save on, also writes startup.yaml. nv config apply --confirm applies on trial and rolls back unless confirmed. nv config apply with a revision number returns to an earlier state.

Version note: This guide uses Cumulus VX 5.16.1. NVUE paths occasionally move between 5.x releases (for example, SVI addressing moved under an `ipv4` object in 5.15; Lab 5 covers that). The commands in this lab were checked against the Cumulus Linux 5.16 documentation. If a command is rejected on your release, press Tab after the last accepted word to see what NVUE expects.

Step-by-step

The addressing and naming plan for this lab is below. Keep it open; every task refers to it.

Switch	Loopback (lo)	Uplink ports	Downlink ports
hxb-spine01	10.255.0.1/32	–	swp1–swp4 → leaf-r1...r4 swp31
hxb-spine02	10.255.0.2/32	–	swp1–swp4 → leaf-r1...r4 swp32
hxb-leaf-r1	10.255.0.11/32	swp31 → spine01 swp1, swp32 → spine02 swp1	swp1 → hxb-gpu01 eth1, swp2 → hxb-gpu03 eth1
hxb-leaf-r2	10.255.0.12/32	swp31 → spine01 swp2, swp32 → spine02 swp2	swp1 → hxb-gpu01 eth2, swp2 → hxb-gpu03 eth2

Switch	Loopback (lo)	Uplink ports	Downlink ports
hxb-leaf-r3	10.255.0.13/32	swp31 → spine01 swp3, swp32 → spine02 swp3	swp1 → hxb-gpu02 eth1, swp2 → hxb-gpu04 eth1
hxb-leaf-r4	10.255.0.14/32	swp31 → spine01 swp4, swp32 → spine02 swp4	swp1 → hxb-gpu02 eth2, swp2 → hxb-gpu04 eth2

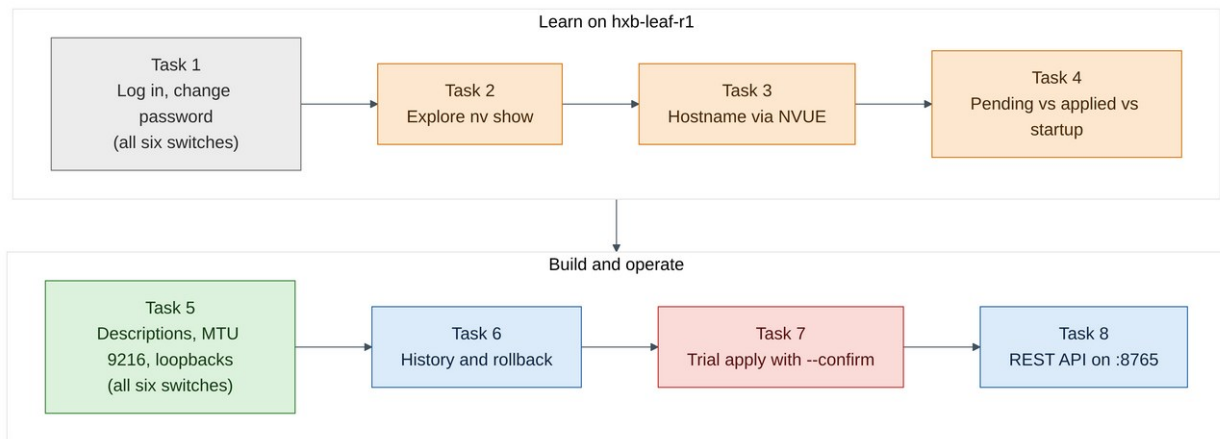


Figure L1.2: What Lab 1 builds on each switch

Tasks 1–4 are learned on *hxb-leaf-r1*. Task 5 then applies the same pattern to all six switches, and Tasks 6–8 exercise history, trial applies and the REST API on *hxb-leaf-r1*.

Task 1 — First login and password change

Do this on all switches

Switch	Log in from oob-mgmt-server with	New password
hxb-spine01	<code>ssh cumulus@hxb-spine01</code>	CumulusLinux!
hxb-spine02	<code>ssh cumulus@hxb-spine02</code>	CumulusLinux!
hxb-leaf-r1	<code>ssh cumulus@hxb-leaf-r1</code>	CumulusLinux!
hxb-leaf-r2	<code>ssh cumulus@hxb-leaf-r2</code>	CumulusLinux!
hxb-leaf-r3	<code>ssh cumulus@hxb-leaf-r3</code>	CumulusLinux!
hxb-leaf-r4	<code>ssh cumulus@hxb-leaf-r4</code>	CumulusLinux!

1. In the Air console, open the **oob-mgmt-server** terminal and log in as **ubuntu / nvidia** (check the login banner or your Air node panel if these differ). If Air asks you to change this password too, do so and note it.
2. From the jump host, connect to the first leaf:

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-leaf-r1
```

3. Accept the host key (**yes**), enter the default password **cumulus**, and then follow the forced password change: enter **cumulus** again as the current password, then **CumulusLinux!** twice.

Expected output (illustrative; wording varies by release):

```
cumulus@hxb-leaf-r1's password:
You are required to change your password immediately (administrator enforced).
Changing password for cumulus.
Current password:
New password:
Retype new password:
```

```
passwd: password updated successfully
Connection to hxb-leaf-r1 closed.
```

- The session closes after the change on many releases. Log in again with the new password:

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-leaf-r1
cumulus@hxb-leaf-r1:mgmt:~$
```

- Repeat steps 2–4 for the other five switches, using the table above. Use **exit** to return to the jump host each time.

Field note: **:mgmt:** in the prompt means your shell runs in the management VRF, because you arrived over eth0. **ping** and **traceroute** still use the default (data-plane) VRF unless you add **-I mgmt**. You will rely on that in Lab 2.

Warning: Use the same new password on every switch. The REST API step in Task 8 and the automation labs later assume **cumulus** / **CumulusLinux!**. If you choose something else, substitute it everywhere.

Checkpoint

- You can log in to all six switches from oob-mgmt-server with **CumulusLinux!**.
- The prompt on each switch ends in **:mgmt:~\$**.

Task 2 — Explore the object model with **nv show**

Work on **hxb-leaf-r1** for Tasks 2–4.

- Look at the system object and the interface summary:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show system
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface
```

Expected output (illustrative, trimmed):

	operational		applied					
-----	-----	-----	-----	-----				
hostname	hxb-leaf-r1		cumulus					
build	Cumulus Linux 5.16.1							
uptime	0:24:10							
timezone	Etc/UTC							
Interface	Admin Status	Oper Status	Speed	MTU	Type	Remote Host	Remote Port	Summary
-----	-----	-----	-----	-----	-----	-----	-----	
eth0	up	up	1G	1500	eth	oob-mgmt-sw...	swp2	IP
Address:	192.168.200.x/24							
lo	up	up		65536	loopback			IP
Address:	127.0.0.1/8							
swp1	down	down		9216	swp			
...								

- Note the two columns in **nv show system**: **operational** is what the switch is actually doing; **applied** is what NVUE has been told. If they differ (for example, Air set the hostname at boot outside NVUE), NVUE does not yet own that setting.
- Use Tab completion to discover the tree. Press Tab twice after each partial command:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show <TAB><TAB>
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp31 <TAB><TAB>
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp31 link <TAB><TAB>
```

4. Try machine-readable output, which automation uses in Labs 11 and 12:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface lo -o json
cumulus@hxb-leaf-r1:mgmt:~$ nv config show
```

`nv config show` prints the whole applied configuration as YAML. On a fresh switch it is short.

5. Ask NVUE where a setting lives with `nv config find` or `nv config lookup` if your release has them (Tab after `nv config` to check). This is useful when you remember a value but not its path.

Checkpoint

- You can explain the difference between the operational and applied columns.
- `nv show interface` lists eth0, lo and the swp ports.

Task 3 — Set the hostname through NVUE

1. Stage the hostname:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set system hostname hxb-leaf-r1
```

2. Look at what is pending before you apply it:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
```

Expected output (illustrative):

```
- set:
  system:
    hostname: hxb-leaf-r1
```

3. Apply it. If NVUE asks **Are you sure? [y/N]**, answer **y**. The output below is illustrative; your revision number will differ:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply
applied [rev_id: 2]
```

4. Confirm that the applied and operational values now agree:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show system
```

Field note: Air often sets the hostname from the node name, so your prompt may already have shown `hxb-leaf-r1`. Setting it through NVUE still matters: it puts the hostname in the applied and startup configuration, so NVUE owns it. The shell prompt only changes when you next log in.

Checkpoint

- `nv show system` shows `hxb-leaf-r1` in both the operational and applied columns.
- `nv config diff` now prints nothing (nothing is pending).

Task 4 — Pending vs applied vs startup

1. Stage a description on the first uplink but **do not apply it**:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp31 description to-hxb-spine01-swp1
```

2. Look at the same object in each revision:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp31 --pending
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp31 --applied
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp31 --startup
```

Expected result: only **--pending** shows the description. Applied and startup do not have it yet.

3. Check the auto-save setting:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show system config auto-save
```

Expected output (illustrative):

```

      operational  applied
-----
state  enabled      enabled
```

4. Now turn auto-save off, so you can see the three revisions separate. Apply both changes together:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set system config auto-save state disabled
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply
```

5. Compare applied with startup:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff applied startup
```

Expected output (illustrative): the description (and the auto-save change itself) appear, because they are applied but not yet in **startup.yaml**.

```
- set:
  interface:
    swp31:
      description: to-hxb-spine01-swp1
  system:
    config:
      auto-save:
        state: disabled
```

6. Save applied to startup and compare again:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config save
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff applied startup
```

The second diff should now be empty.

7. Turn auto-save back on, because the rest of this guide assumes it:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set system config auto-save state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply
```

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff applied startup
```

8. Look at the startup file itself (read only; never edit it by hand):

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo cat /etc/nvue.d/startup.yaml
```

Exam trap: `nv config save` does **not** apply anything. It copies the *applied* configuration to startup. Only `nv config apply` changes the running switch. A change that is only pending is lost at reboot.

Checkpoint

- You saw a change exist in pending only, then in applied but not startup, then in all three.
- `nv show system config auto-save` shows **enabled** again.

Task 5 — Descriptions, MTU 9216 and loopbacks on every switch

Now give every switch its identity and port settings. Ports that NVUE configures are brought up; we also set **link state up** explicitly so the intent is clear in the configuration.

Do this on all switches

Setting	Spines (hxb-spine01, hxb-spine02)	Leaves (hxb-leaf-r1 ... r4)
Hostname	<code>nv set system hostname <name></code>	<code>nv set system hostname <name></code>
Ports	swp1–swp4	swp1, swp2, swp31, swp32
Admin state	<code>link state up</code>	<code>link state up</code>
MTU	<code>link mtu 9216</code>	<code>link mtu 9216</code>
Description	to-hxb-leaf-rN-swp31/32	server or spine port name
Loopback	10.255.0.1/32, 10.255.0.2/32	10.255.0.11/32 ... 10.255.0.14/32
Finish with	<code>nv config diff</code> , then <code>nv config apply -y</code>	<code>nv config diff</code> , then <code>nv config apply -y</code>

The values differ per switch, so a full block is given for each. Paste each block into the matching switch.

hxb-spine01

```
cumulus@hxb-spine01:mgmt:~$ nv set system hostname hxb-spine01
cumulus@hxb-spine01:mgmt:~$ nv set interface swp1 description to-hxb-leaf-r1-swp31
cumulus@hxb-spine01:mgmt:~$ nv set interface swp2 description to-hxb-leaf-r2-swp31
cumulus@hxb-spine01:mgmt:~$ nv set interface swp3 description to-hxb-leaf-r3-swp31
cumulus@hxb-spine01:mgmt:~$ nv set interface swp4 description to-hxb-leaf-r4-swp31
cumulus@hxb-spine01:mgmt:~$ nv set interface swp1-4 link state up
cumulus@hxb-spine01:mgmt:~$ nv set interface swp1-4 link mtu 9216
cumulus@hxb-spine01:mgmt:~$ nv set interface lo ip address 10.255.0.1/32
cumulus@hxb-spine01:mgmt:~$ nv config diff
cumulus@hxb-spine01:mgmt:~$ nv config apply -y
```

hxb-spine02

```
cumulus@hxb-spine02:mgmt:~$ nv set system hostname hxb-spine02
cumulus@hxb-spine02:mgmt:~$ nv set interface swp1 description to-hxb-leaf-r1-swp32
cumulus@hxb-spine02:mgmt:~$ nv set interface swp2 description to-hxb-leaf-r2-swp32
cumulus@hxb-spine02:mgmt:~$ nv set interface swp3 description to-hxb-leaf-r3-swp32
cumulus@hxb-spine02:mgmt:~$ nv set interface swp4 description to-hxb-leaf-r4-swp32
cumulus@hxb-spine02:mgmt:~$ nv set interface swp1-4 link state up
```

```
cumulus@hxb-spine02:mgmt:~$ nv set interface swp1-4 link mtu 9216
cumulus@hxb-spine02:mgmt:~$ nv set interface lo ip address 10.255.0.2/32
cumulus@hxb-spine02:mgmt:~$ nv config diff
cumulus@hxb-spine02:mgmt:~$ nv config apply -y
```

hxb-leaf-r1 (the hostname and swp31 description are already applied from Tasks 3–4; setting them again is harmless)

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set system hostname hxb-leaf-r1
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp31 description to-hxb-spine01-swp1
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp32 description to-hxb-spine02-swp1
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp1 description hxb-gpu01-eth1
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp2 description hxb-gpu03-eth1
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp1-2,swp31-32 link state up
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp1-2,swp31-32 link mtu 9216
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface lo ip address 10.255.0.11/32
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

hxb-leaf-r2

```
cumulus@hxb-leaf-r2:mgmt:~$ nv set system hostname hxb-leaf-r2
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp31 description to-hxb-spine01-swp2
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp32 description to-hxb-spine02-swp2
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp1 description hxb-gpu01-eth2
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp2 description hxb-gpu03-eth2
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp1-2,swp31-32 link state up
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp1-2,swp31-32 link mtu 9216
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface lo ip address 10.255.0.12/32
cumulus@hxb-leaf-r2:mgmt:~$ nv config diff
cumulus@hxb-leaf-r2:mgmt:~$ nv config apply -y
```

hxb-leaf-r3

```
cumulus@hxb-leaf-r3:mgmt:~$ nv set system hostname hxb-leaf-r3
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface swp31 description to-hxb-spine01-swp3
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface swp32 description to-hxb-spine02-swp3
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface swp1 description hxb-gpu02-eth1
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface swp2 description hxb-gpu04-eth1
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface swp1-2,swp31-32 link state up
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface swp1-2,swp31-32 link mtu 9216
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface lo ip address 10.255.0.13/32
cumulus@hxb-leaf-r3:mgmt:~$ nv config diff
cumulus@hxb-leaf-r3:mgmt:~$ nv config apply -y
```

hxb-leaf-r4

```
cumulus@hxb-leaf-r4:mgmt:~$ nv set system hostname hxb-leaf-r4
cumulus@hxb-leaf-r4:mgmt:~$ nv set interface swp31 description to-hxb-spine01-swp4
cumulus@hxb-leaf-r4:mgmt:~$ nv set interface swp32 description to-hxb-spine02-swp4
cumulus@hxb-leaf-r4:mgmt:~$ nv set interface swp1 description hxb-gpu02-eth2
cumulus@hxb-leaf-r4:mgmt:~$ nv set interface swp2 description hxb-gpu04-eth2
cumulus@hxb-leaf-r4:mgmt:~$ nv set interface swp1-2,swp31-32 link state up
cumulus@hxb-leaf-r4:mgmt:~$ nv set interface swp1-2,swp31-32 link mtu 9216
cumulus@hxb-leaf-r4:mgmt:~$ nv set interface lo ip address 10.255.0.14/32
cumulus@hxb-leaf-r4:mgmt:~$ nv config diff
cumulus@hxb-leaf-r4:mgmt:~$ nv config apply -y
```

Version note: On Cumulus Linux 5.16 the loopback address is `nv set interface lo ip address <prefix>`, as in Chapter 4. If your release rejects `ip address` for `lo`, press Tab after `nv set interface lo` and use the address path it offers (newer releases have reorganised some addressing under `ipv4`). The command ranges `swp1-4` and `swp1-2, swp31-32` are standard NVUE syntax.

1. After applying on each switch, check the ports and the loopback:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp31 link
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface lo ip address
```

Expected output (illustrative, trimmed):

Interface	Admin Status	Oper Status	Speed	MTU	Type	Remote Host	Remote Port	Summary
-----	-----	-----	----	-----	-----	-----	-----	
lo	up	up		65536	loopback			IP
Address:	10.255.0.11/32							
swp1	up	up	1G	9216	swp	hxb-gpu01	eth1	
swp2	up	up	1G	9216	swp	hxb-gpu03	eth1	
swp31	up	up	1G	9216	swp	hxb-spine01	swp1	
swp32	up	up	1G	9216	swp	hxb-spine02	swp1	

2. The **Remote Host** and **Remote Port** columns come from LLDP, which is on by default. Use them to prove the cabling matches the plan table. A mismatch here is the cheapest cabling bug you will ever find.

Warning: Cumulus VX ports are virtual links and report a nominal speed (often 1G). Real Spectrum ports would show 400G or 800G. MTU 9216 is still applied and visible in `nv show`; that is what matters here.

Checkpoint

- All six switches show their correct hostname (log out and back in to see it in the prompt).
- On every switch, the configured ports are admin **up**, oper **up**, MTU **9216**, with the right description.
- LLDP neighbours match the cabling table on every leaf and spine.
- `nv show interface lo` shows the right 10.255.0.x/32 on each switch.

Task 6 — History, revisions and rollback

Return to **hxb-leaf-r1**.

1. List the stored revisions and the apply history:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config revision
cumulus@hxb-leaf-r1:mgmt:~$ nv config history
```

Expected output (illustrative; your revision numbers and dates will differ):

Rev ID	State	Apply ID	Apply Date	Type	User	Reason
2	applied_and_saved	rev_2_apply_1	2026-09-28 10:02:11	CLI	cumulus	Config update
3	applied	rev_3_apply_1	2026-09-28 10:06:40	CLI	cumulus	Config update
4	applied_and_saved	rev_4_apply_1	2026-09-28 10:07:15	CLI	cumulus	Config update
5	applied_and_saved	rev_5_apply_1	2026-09-28 10:15:02	CLI	cumulus	Config update

2. Write down the revision ID of your current **good** state (the latest one, here 5). Call it **GOOD**.

3. Make a deliberate "bad" change: wrong description and wrong MTU on swp32.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp32 description WRONG-PORT
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp32 link mtu 1500
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

4. Compare the bad revision with GOOD. Replace 5 and 6 with your numbers:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config revision
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff 5 6
```

Expected output (illustrative):

```
- set:
  interface:
    swp32:
      description: WRONG-PORT
      link:
        mtu: 1500
```

5. Roll back by applying the GOOD revision:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply 5 -y
```

6. Confirm the rollback and look at how history records it:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp32
cumulus@hxb-leaf-r1:mgmt:~$ nv config history
```

Field note: Rollback with `nv config apply <rev>` applies the **whole** configuration of that revision, not just the reverse of the last change. Anything applied after that revision is also undone. Always run `nv config diff <good> applied` first so you know exactly what will change.

Checkpoint

- swp32 shows the description `to-hxb-spine02-swp1` and MTU 9216 again.
- `nv config history` shows the bad apply and the rollback as separate entries.

Task 7 — Trial apply with `--confirm`

A trial apply protects you from changes that could lock you out. You will let one trial expire, and confirm a second.

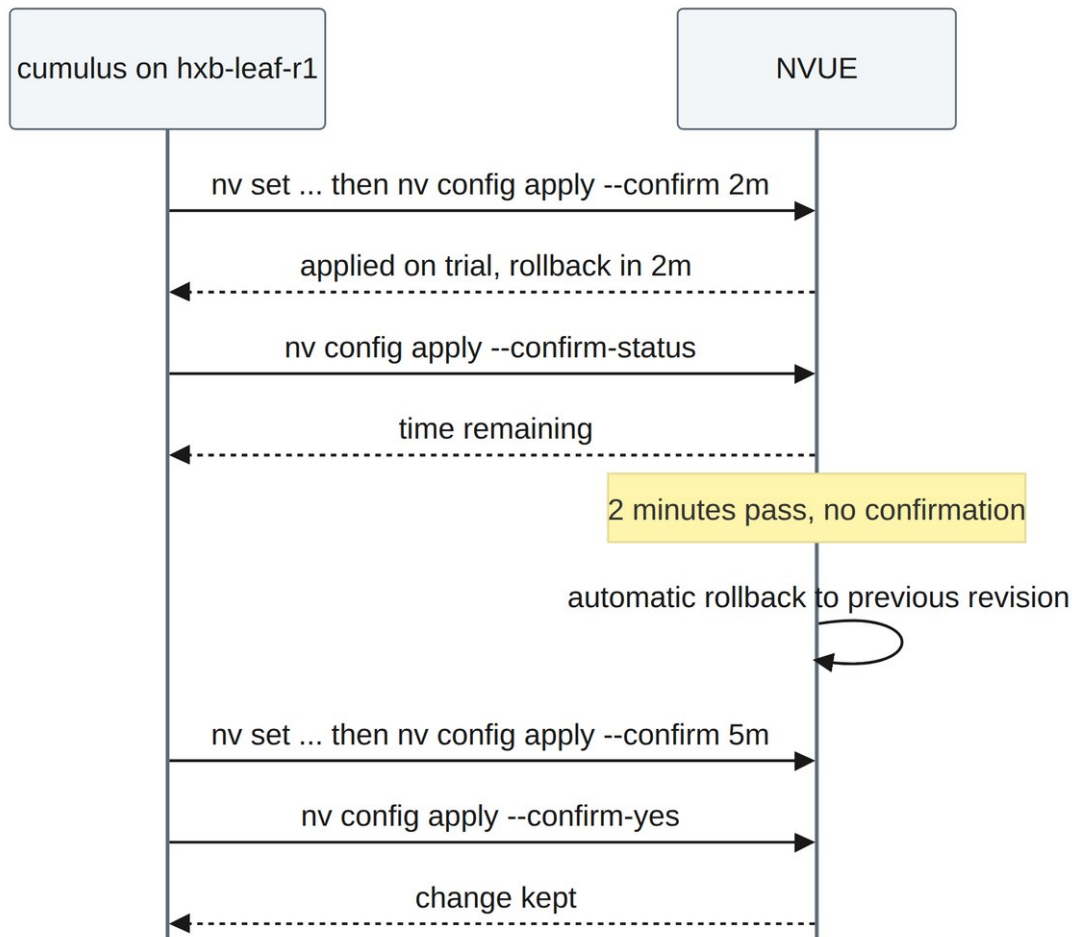


Figure L1.3: A trial apply with automatic rollback

The first trial is not confirmed, so NVUE restores the previous configuration when the timer expires. The second trial is confirmed within the window and is kept.

1. Stage a harmless test change and apply it on a two-minute trial:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp2 description TRIAL-CHANGE
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply --confirm 2m
```

2. Check the change is live and how long is left:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp2 description
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply --confirm-status
```

3. Do nothing for just over two minutes, then check again:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp2
cumulus@hxb-leaf-r1:mgmt:~$ nv config history
```

Expected result: the description is back to **hxb-gpu03-eth1**, and the history shows the rollback.

4. Run a second trial and confirm it this time. For the test we set a useful value: a longer description that includes the tenant:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp2 description hxb-gpu03-eth1-BOREALIS
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply --confirm 5m
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply --confirm-yes
```

5. Verify the change stayed and is in startup:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp2 description
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff applied startup
```

Version note: The NVUE command reference documents `nv config apply --confirm [<time>], --confirm-status`, and `nv config apply [<revision>] --confirm-yes` to keep a trial change. The default window is ten minutes, and the timer accepts `s`, `m` or `h`. On some releases the confirmation is also offered as an interactive prompt; if `--confirm-yes` is rejected on yours, press Tab after `nv config apply --` to see the confirm options (check on your release).

Field note: In production, use `--confirm` for anything that touches eth0, the management VRF, ACLs, BGP or the NVUE API itself. If the change cuts you off, the switch rolls back without you.

Checkpoint

- The first trial rolled back by itself.
- The second trial's description `hxb-gpu03-eth1-BOREALIS` is applied and, with auto-save on, also in startup.

Task 8 — Read configuration through the NVUE REST API

The REST API is enabled by default on TCP port 8765 and, on current releases, listens on `localhost` unless you add a listening address. You will call it from the switch itself.

1. Check the API service:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show system api
```

Expected output (illustrative):

	operational	applied
-----	-----	-----
port	8765	8765
state	enabled	enabled
certificate	self-signed	self-signed
[listening-address]	localhost	localhost

2. Read the applied loopback configuration. `-k` accepts the self-signed certificate. The single quotes stop Bash from interpreting the `!` in the password:

```
cumulus@hxb-leaf-r1:mgmt:~$ curl -s -k -u 'cumulus:Cumuluslinux!' \
'https://127.0.0.1:8765/nvue_v1/interface/lo?rev=applied' | python3 -m json.tool
```

Expected output (illustrative, trimmed):

```
{
  "ip": {
    "address": {
      "10.255.0.11/32": {}
    }
  },
  "type": "loopback"
}
```

3. Read operational state for an uplink, the API equivalent of `nv show interface swp31 link`:

```
cumulus@hxb-leaf-r1:mgmt:~$ curl -s -k -u 'cumulus:CumulusLinux!' \
https://127.0.0.1:8765/nvue_v1/interface/swp31/link | python3 -m json.tool
```

4. Read the revision list, which is what automation uses to track changes:

```
cumulus@hxb-leaf-r1:mgmt:~$ curl -s -k -u 'cumulus:CumulusLinux!' \
https://127.0.0.1:8765/nvue_v1/revision | python3 -m json.tool | head -30
```

5. Compare the JSON paths with the CLI paths. `interface/lo?rev=applied` is `nv show interface lo --applied`. That one-to-one mapping is the point of the NVUE object model.

Warning: `-k` and a password on the command line are fine in a lab. In production use a dedicated automation account, keep credentials in a vault, and validate the switch certificate. To reach the API from oob-mgmt-server you would add a listening address (`nv set system api listening-address <ip>`) in the management VRF; later labs use SSH and Ansible instead, so leave it on localhost.

Checkpoint

- `curl` returns JSON showing `10.255.0.11/32` on `lo`.
- You can name the CLI command equivalent to each API URL you used.

Verify

Tick each item before you move on to Lab 2.

- [] You can log in to all six switches as `cumulus` with `CumulusLinux!`.
- [] `nv show system` shows the correct hostname in both columns on every switch.
- [] On every leaf, `swp1`, `swp2`, `swp31` and `swp32` are up, MTU 9216, with correct descriptions; on every spine, `swp1`–`swp4` are the same.
- [] LLDP (`nv show interface`) matches the cabling plan.
- [] Loopbacks: `spine01 10.255.0.1/32`, `spine02 10.255.0.2/32`, `leaf-r1 ... r4 10.255.0.11–.14/32`.
- [] `nv config diff` is empty and `nv config diff applied startup` is empty on every switch.
- [] `nv show system config auto-save` is `enabled` on `hxb-leaf-r1`.
- [] You rolled back with `nv config apply <rev>` and watched a `--confirm` trial expire.
- [] `curl` to `https://127.0.0.1:8765/nvue_v1/...` returns JSON.

Break and fix

Fault 1 — "My change was there, now it has gone"

Inject (on `hxb-leaf-r2`):

```
cumulus@hxb-leaf-r2:mgmt:~$ nv set system config auto-save state disabled
cumulus@hxb-leaf-r2:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp2 description hxb-gpu03-eth2-BOREALIS
cumulus@hxb-leaf-r2:mgmt:~$ nv config apply -y
```

Symptom: a colleague reports that the new `swp2` description "disappears after every reboot". (Optionally reboot the switch to see it: `sudo reboot`, wait for it to return, and log in again.)

Diagnosis:

1. Compare applied with startup:

```
cumulus@hxb-leaf-r2:mgmt:~$ nv config diff applied startup
```

The description appears in the diff: it is applied but was never written to startup. (After a reboot, it is missing from applied too, because the switch loaded startup.)

2. Check why:

```
cumulus@hxb-leaf-r2:mgmt:~$ nv show system config auto-save
```

Auto-save is **disabled**.

Fix: re-enable auto-save and make sure the change is applied and saved.

```
cumulus@hxb-leaf-r2:mgmt:~$ nv set system config auto-save state enabled
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp2 description hxb-gpu03-eth2-BOREALIS
cumulus@hxb-leaf-r2:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r2:mgmt:~$ nv config save
cumulus@hxb-leaf-r2:mgmt:~$ nv config diff applied startup
```

The last diff must be empty.

Fault 2 — A hand edit that NVUE overwrites

Inject (on hxb-leaf-r3): edit a file NVUE manages directly.

```
cumulus@hxb-leaf-r3:mgmt:~$ sudo sed -i 's/mtu 9216/mtu 1500/' /etc/network/interfaces
cumulus@hxb-leaf-r3:mgmt:~$ sudo ifreload -a
cumulus@hxb-leaf-r3:mgmt:~$ ip link show swp31 | grep mtu
```

The kernel now shows MTU 1500 on the ports.

Symptom: `nv show interface` shows MTU 1500 in the operational view, but `nv show interface swp31 link --applied` still says 9216. Later, after an unrelated `nv config apply`, the MTU silently returns to 9216.

Diagnosis:

1. Compare the views:

```
cumulus@hxb-leaf-r3:mgmt:~$ nv show interface swp31 link
cumulus@hxb-leaf-r3:mgmt:~$ nv show interface swp31 link --applied
```

Operational and applied disagree: someone changed the switch outside NVUE.

2. `nv config history` shows no apply that set MTU 1500. The change was never an NVUE change.

Fix: let NVUE put back its own configuration, and agree the team rule: NVUE only.

```
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface swp31 description to-hxb-spine01-swp3
cumulus@hxb-leaf-r3:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r3:mgmt:~$ ip link show swp31 | grep mtu
```

If a plain apply reports nothing to apply and the MTU stays at 1500, force NVUE to re-render the files by applying the current revision again with `nv config apply <current-rev> -y`, or reboot the switch (startup still has MTU 9216). Check on your release which of these re-renders the files.

Warning: This is exactly the Chapter 4 scenario. Hand edits to `/etc/network/interfaces` or `/etc/frr/frr.conf` on an NVUE-managed switch are overwritten by the next apply. Choose NVUE for a switch and never mix the two.

Clean-up / save state

1. On every switch, confirm nothing is pending and startup matches applied:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff applied startup
```

2. If either shows output, run `nv config apply -y` and `nv config save`.
3. Check that auto-save is enabled on hxb-leaf-r2 again (Fault 1).
4. Keep the configuration. Lab 2 builds directly on these loopbacks, descriptions and MTUs.
5. If you are stopping here, put the simulation to sleep from the Air console to save compute-hour credits. Sleeping keeps the switch disks, so your configuration survives.

Exam tie-in

- **6.1** — The order of operations is **set → diff → apply (→ save)**. Know the three revisions and how to show each (`--pending`, `--applied`, `--startup`), and that `nv config save` writes applied to `/etc/nvue.d/startup.yaml`.
- **6.1** — Safe change: `nv config apply --confirm <time>` rolls back automatically; `nv config history / nv config revision` plus `nv config apply <rev>` is the manual rollback.
- **2.1** — All Spectrum-X switch configuration in later objectives (RoCE with `nv set qos roce`, adaptive routing, EVPN) uses this same NVUE workflow, and RoCE can only be configured with NVUE.
- **2.1 / 6.1** — The REST API runs on **port 8765** with base path `/nvue_v1/`, and its URL paths mirror the `nv` CLI paths.
- **2.4** — NVIDIA Air lets you rehearse all of this on Cumulus VX, with the caveat that VX has no Spectrum ASIC.

Review questions

1. You run `nv set interface swp1 link mtu 9216` and then `nv config save`, and reboot. What MTU does swp1 have after the reboot, and why?
2. Which command shows only the changes you have staged but not yet applied?
3. A change applied with `nv config apply --confirm 5m` is not confirmed. What happens, and how could you have kept it?
4. You find a bad change was applied as revision 12, and revision 11 was good. Which command returns the switch to the good state, and what should you check first?
5. What is the REST API equivalent of `nv show interface lo --applied` on a switch, and which port does it use by default?

Answers

1. The old MTU. `nv config save` copies the **applied** configuration to startup; the MTU change was only pending and was never applied, so it was neither running nor saved.
2. `nv config diff` (with no arguments it compares pending with applied). `nv show <path> --pending` shows the pending view of one object.
3. After five minutes NVUE rolls back to the previous configuration automatically. To keep it, confirm within the window with `nv config apply --confirm=yes`.
4. `nv config apply 11` (add `-y` to skip the prompt). First run `nv config diff 11 applied` to see everything the rollback will change, because it restores the whole revision.

5. GET `https://<switch>:8765/nvue_v1/interface/lo?rev=applied`, on port **8765**, using HTTP basic authentication.

Lab 2 — Underlay with eBGP Unnumbered

Lab at a glance

Book chapters	Chapter 4 (4.4 BGP unnumbered with NVUE); Chapter 2 (rail-optimised leaf-spine); Chapter 7 (the underlay that EVPN rides on)
Exam objectives	2.1 Configure Spectrum-X switches (underlay foundation) · 2.3 Multi-tenant BGP-EVPN (prerequisite underlay) · 1.2 Rail-optimised topologies · supports 2.4 (NVIDIA Air)
Time	75–90 minutes
Air resources used	Simulation helix-b-air : hxb-spine01, hxb-spine02, hxb-leaf-r1 ... hxb-leaf-r4 and oob-mgmt-server
Prerequisite labs	Lab 1 (hostnames, MTU 9216, port descriptions and loopbacks applied and saved on all six switches)

Objectives

By the end of this lab you will be able to:

- Explain why AI fabrics use eBGP with a private ASN per leaf and a shared spine ASN, and what "unnumbered" removes from the design.
- Configure BGP on all six Helix-B switches with NVUE: ASN, router ID, interface neighbours with **remote-as external**, and loopback advertisement.
- Confirm ECMP multipath is active so each leaf has two equal-cost paths to every other leaf.
- Verify the underlay with **nv show vrf default router bgp neighbor**, **vttysh (show ip bgp summary, show ip route)**, a loopback-sourced **ping**, and **traceroute**.
- Diagnose and fix a wrong **remote-as** and a downed uplink.

Background

Helix-B uses the design most Spectrum-X AI fabrics use: a **Layer 3 leaf-spine underlay running eBGP**. Each leaf has its own private ASN (hxb-leaf-r1 ... r4 are AS 65101–65104) and both spines share AS 65100. Sharing the spine ASN is deliberate. A path from one spine through a leaf to the other spine would contain AS 65100 twice, so BGP's own loop prevention rejects it. That keeps spine-to-leaf-to-spine "valley" paths out of the routing table without extra policy.

Unnumbered means the point-to-point links have no IPv4 addresses. Each switch port has an IPv6 link-local address and sends IPv6 router advertisements, so the neighbour learns that address automatically. BGP then peers over the link-local address, and carries IPv4 routes with an IPv6 next hop (RFC 5549 / RFC 8950). The result: you configure **neighbor swp31 remote-as external** instead of an IP address and a numeric ASN per link, there is no /31 addressing plan to maintain, and every leaf's BGP configuration is identical except for its ASN, router ID and loopback. That is exactly what makes templates (Lab 11) and Ansible (Lab 12) easy.

Each switch advertises only its **loopback (/32)**. In Lab 5 the leaf loopbacks become the VXLAN tunnel endpoints (VTEPs), so the underlay's only job is to make every loopback reachable from every other one over **two equal-cost paths** — one through each spine. Cumulus Linux enables BGP multipath by default, so both paths are installed and traffic is spread by ECMP hashing.

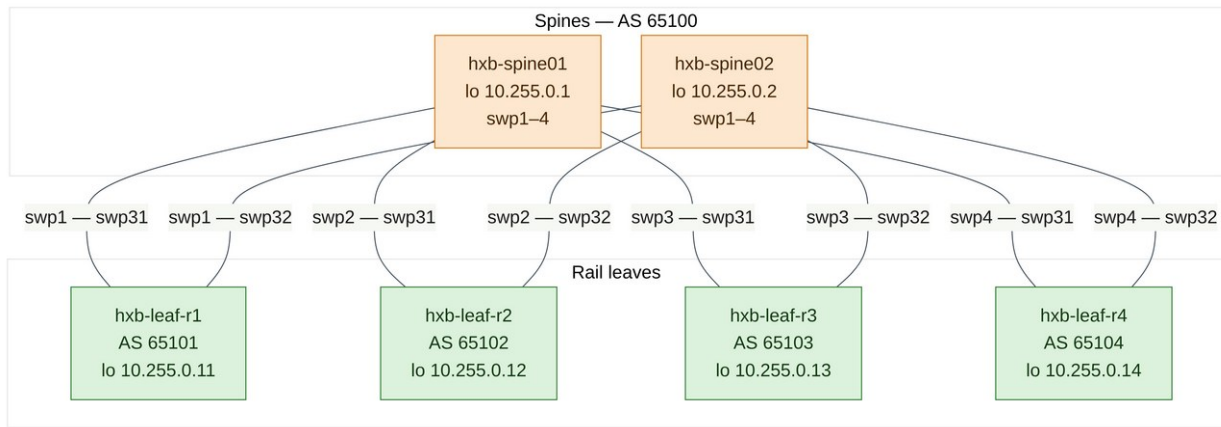


Figure L2.1: Helix-B underlay in Air

Each leaf connects swp31 to hxb-spine01 and swp32 to hxb-spine02. Leaves have their own ASN; both spines share AS 65100. Every link is an eBGP unnumbered session, and each switch advertises only its loopback.

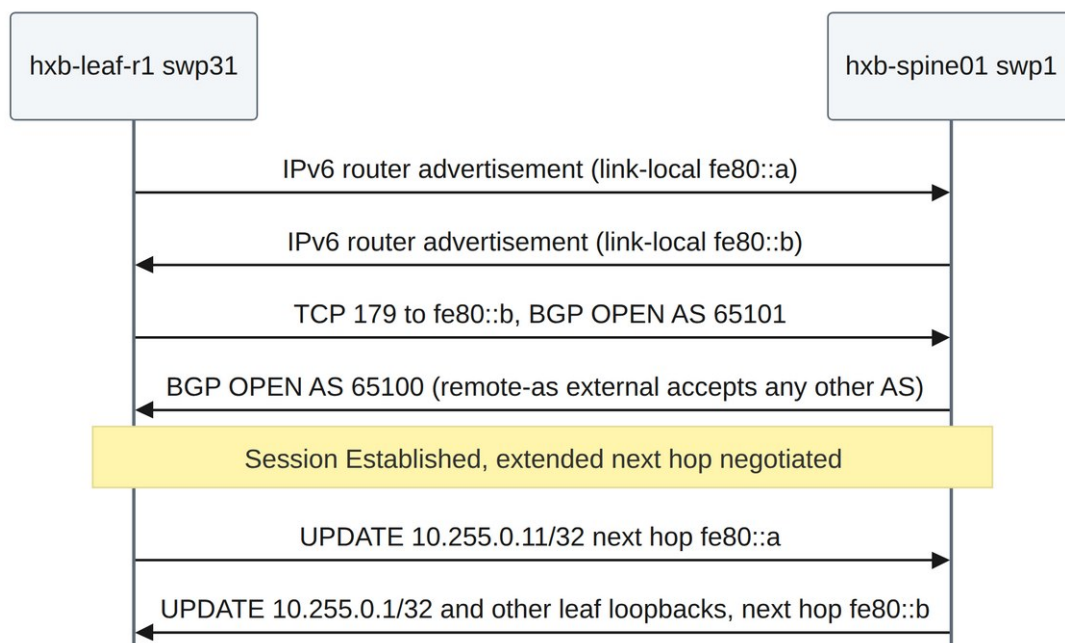


Figure L2.2: How an unnumbered session comes up

No IPv4 address is configured on the link. Router advertisements reveal the neighbour's IPv6 link-local address, BGP peers over it, and IPv4 loopback routes are carried with an IPv6 next hop.

Version note: Cumulus VX forwards in the Linux kernel, not on a Spectrum ASIC. BGP, the routing table and ECMP behave as on a real switch, but the exact hash the kernel uses to choose between the two paths may differ from Spectrum hardware. The routing table is the authoritative proof of ECMP in this lab.

Step-by-step

Task 1 — Check the starting point

Do this on all switches

Check	Command	Expected
Nothing pending	<code>nv config diff</code>	No output
Loopback present	<code>nv show interface</code> <code>lo</code>	Correct 10.255.0.x/32 from Lab 1

Check	Command	Expected
Fabric ports up with LLDP neighbours	<code>nv show interface</code>	Leaves: swp31/32 up to spine01/02; spines: swp1-4 up to leaf-r1...r4
No BGP yet	<code>nv show router bgp</code>	No ASN configured

1. From oob-mgmt-server, log in to each switch in turn (`ssh cumulus@hxb-spine01` and so on) and run the checks in the table. For example:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface
cumulus@hxb-leaf-r1:mgmt:~$ nv show router bgp
```

2. If a port is down or an LLDP neighbour is wrong, fix it before going further (see Lab 1, Task 5). BGP cannot fix cabling.

Checkpoint

- Every fabric port is up with the neighbour the plan expects, and every loopback is correct.

Task 2 — Plan the BGP values

This is the only information that differs between switches. Everything else is identical.

Switch	ASN	Router ID	Unnumbered neighbours	Advertise
hxb-spine01	65100	10.255.0.1	swp1, swp2, swp3, swp4	10.255.0.1/32
hxb-spine02	65100	10.255.0.2	swp1, swp2, swp3, swp4	10.255.0.2/32
hxb-leaf-r1	65101	10.255.0.11	swp31, swp32	10.255.0.11/32
hxb-leaf-r2	65102	10.255.0.12	swp31, swp32	10.255.0.12/32
hxb-leaf-r3	65103	10.255.0.13	swp31, swp32	10.255.0.13/32
hxb-leaf-r4	65104	10.255.0.14	swp31, swp32	10.255.0.14/32

Do this on all switches (the pattern)

Step	NVUE command
1. Local ASN	<code>nv set router bgp autonomous-system <ASN></code>
2. Router ID = loopback	<code>nv set router bgp router-id <loopback IP></code>
3. One unnumbered neighbour per fabric port	<code>nv set vrf default router bgp neighbor <swpX> remote-as external</code>
4. Advertise the loopback	<code>nv set vrf default router bgp address-family ipv4-unicast network <loopback>/32</code>
5. ECMP (default 64; set explicitly for clarity)	<code>nv set vrf default router bgp address-family ipv4-unicast multipaths ebgp 64</code>
6. Review and apply	<code>nv config diff</code> then <code>nv config apply -y</code>

Field note: `remote-as external` means "any ASN other than mine". That is why one line works on every leaf uplink without typing the spine's ASN, and why a spine can use the same line for four different leaves. Use `remote-as internal` for iBGP, or a number when you want BGP to enforce a specific peer ASN.

Field note: The guide uses a `network` statement to advertise exactly one prefix, as in Chapter 4. The alternative, `nv set vrf default router bgp address-family ipv4-unicast redistribute connected`, advertises every connected route in the default VRF. That is common in labs, but on a real leaf it can leak link or server subnets into the underlay unless you add a route map, so prefer `network` for loopbacks.

Task 3 — Configure the spines

Paste each block into the matching spine.

hxb-spine01

```
cumulus@hxb-spine01:mgmt:~$ nv set router bgp autonomous-system 65100
cumulus@hxb-spine01:mgmt:~$ nv set router bgp router-id 10.255.0.1
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp neighbor swp1 remote-as external
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp neighbor swp2 remote-as external
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp neighbor swp3 remote-as external
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp neighbor swp4 remote-as external
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network
10.255.0.1/32
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast multipaths
ebgp 64
cumulus@hxb-spine01:mgmt:~$ nv config diff
cumulus@hxb-spine01:mgmt:~$ nv config apply -y
```

hxb-spine02

```
cumulus@hxb-spine02:mgmt:~$ nv set router bgp autonomous-system 65100
cumulus@hxb-spine02:mgmt:~$ nv set router bgp router-id 10.255.0.2
cumulus@hxb-spine02:mgmt:~$ nv set vrf default router bgp neighbor swp1 remote-as external
cumulus@hxb-spine02:mgmt:~$ nv set vrf default router bgp neighbor swp2 remote-as external
cumulus@hxb-spine02:mgmt:~$ nv set vrf default router bgp neighbor swp3 remote-as external
cumulus@hxb-spine02:mgmt:~$ nv set vrf default router bgp neighbor swp4 remote-as external
cumulus@hxb-spine02:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network
10.255.0.2/32
cumulus@hxb-spine02:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast multipaths
ebgp 64
cumulus@hxb-spine02:mgmt:~$ nv config diff
cumulus@hxb-spine02:mgmt:~$ nv config apply -y
```

Expected `nv config diff` output on hxb-spine01 (illustrative, trimmed):

```
- set:
  router:
    bgp:
      autonomous-system: 65100
      router-id: 10.255.0.1
  vrf:
    default:
      router:
        bgp:
          address-family:
            ipv4-unicast:
              multipaths:
                ebgp: 64
              network:
                10.255.0.1/32: {}
          neighbor:
```

```

swp1:
  remote-as: external
  type: unnumbered
...

```

Version note: Recent releases may also need `nv set router bgp state enabled` or `nv set vrf default router bgp state enabled` before BGP starts, and on some releases setting the ASN enables BGP implicitly. After applying, if `nv show router bgp` shows the state as disabled, set it to `enabled` and apply again (check on your release). The multipaths value 64 is already the default on Cumulus Linux; if your release rejects `multipaths ebgp`, press Tab after `multipaths` and skip the line — ECMP still works.

Checkpoint

- Both spines applied without errors.
- `nv show router bgp` on each spine shows AS 65100 and the correct router ID.
- Sessions are not up yet; the leaves have no BGP.

Task 4 — Configure the leaves

hxb-leaf-r1

```

cumulus@hxb-leaf-r1:mgmt:~$ nv set router bgp autonomous-system 65101
cumulus@hxb-leaf-r1:mgmt:~$ nv set router bgp router-id 10.255.0.11
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf default router bgp neighbor swp31 remote-as external
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf default router bgp neighbor swp32 remote-as external
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network
10.255.0.11/32
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast multipaths
ebgp 64
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y

```

hxb-leaf-r2

```

cumulus@hxb-leaf-r2:mgmt:~$ nv set router bgp autonomous-system 65102
cumulus@hxb-leaf-r2:mgmt:~$ nv set router bgp router-id 10.255.0.12
cumulus@hxb-leaf-r2:mgmt:~$ nv set vrf default router bgp neighbor swp31 remote-as external
cumulus@hxb-leaf-r2:mgmt:~$ nv set vrf default router bgp neighbor swp32 remote-as external
cumulus@hxb-leaf-r2:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network
10.255.0.12/32
cumulus@hxb-leaf-r2:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast multipaths
ebgp 64
cumulus@hxb-leaf-r2:mgmt:~$ nv config diff
cumulus@hxb-leaf-r2:mgmt:~$ nv config apply -y

```

hxb-leaf-r3

```

cumulus@hxb-leaf-r3:mgmt:~$ nv set router bgp autonomous-system 65103
cumulus@hxb-leaf-r3:mgmt:~$ nv set router bgp router-id 10.255.0.13
cumulus@hxb-leaf-r3:mgmt:~$ nv set vrf default router bgp neighbor swp31 remote-as external
cumulus@hxb-leaf-r3:mgmt:~$ nv set vrf default router bgp neighbor swp32 remote-as external
cumulus@hxb-leaf-r3:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network
10.255.0.13/32
cumulus@hxb-leaf-r3:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast multipaths
ebgp 64
cumulus@hxb-leaf-r3:mgmt:~$ nv config diff

```

```
cumulus@hxb-leaf-r3:mgmt:~$ nv config apply -y
```

hxb-leaf-r4

```
cumulus@hxb-leaf-r4:mgmt:~$ nv set router bgp autonomous-system 65104
cumulus@hxb-leaf-r4:mgmt:~$ nv set router bgp router-id 10.255.0.14
cumulus@hxb-leaf-r4:mgmt:~$ nv set vrf default router bgp neighbor swp31 remote-as external
cumulus@hxb-leaf-r4:mgmt:~$ nv set vrf default router bgp neighbor swp32 remote-as external
cumulus@hxb-leaf-r4:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network 10.255.0.14/32
cumulus@hxb-leaf-r4:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast multipaths ebgp 64
cumulus@hxb-leaf-r4:mgmt:~$ nv config diff
cumulus@hxb-leaf-r4:mgmt:~$ nv config apply -y
```

1. After each leaf applies, wait 10–30 seconds for the sessions to establish.

Field note: Notice how little differs between the four leaf blocks: three values. In Lab 11 you will generate these blocks from one Jinja template and a small variables file.

Checkpoint

- All six switches applied without errors.
- `nv config diff applied startup` is empty on each switch (auto-save).

Task 5 — Verify the BGP sessions

1. On **hxb-leaf-r1**, check the neighbours with NVUE:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show vrf default router bgp neighbor
```

Expected output (illustrative; column names vary by release):

Neighbor	AS	State	Uptime	PfxSent	PfxRcvd
swp31	65100	established	00:01:12	...	4
swp32	65100	established	00:01:10	...	4

2. Look at one neighbour in detail:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show vrf default router bgp neighbor swp31
```

Note the remote ASN (65100), the remote router ID (10.255.0.1), the state and the IPv6 link-local address the session uses.

3. Check the same thing from FRR:

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show ip bgp summary"
```

Expected output (illustrative):

```
IPv4 Unicast Summary (VRF default):
BGP router identifier 10.255.0.11, local AS number 65101 vrf-id 0
BGP table version 7
RIB entries 11, using 2112 bytes of memory
Peers 2, using 47 KiB of memory
```

Neighbor Desc	V	AS	MsgRcvd	MsgSent	TblVer	InQ	OutQ	Up/Down	State/PfxRcd	PfxSnt
hxb-spine01(swp31) N/A	4	65100	45	44	0	0	0	00:01:12	4	5
hxb-spine02(swp32) N/A	4	65100	45	44	0	0	0	00:01:10	4	5
Total number of neighbors 2										

A number under **State/PfxRcd** means the session is Established and shows how many prefixes were accepted. A word such as **Active**, **Connect** or **Idle** means it is not up.

- On **hxb-spine01**, confirm all four leaves are neighbours:

```
cumulus@hxb-spine01:mgmt:~$ sudo vtysh -c "show ip bgp summary"
```

Expect four Established neighbours, **hxb-leaf-r1(swp1) ... hxb-leaf-r4(swp4)**, with remote AS 65101–65104 and one prefix received from each.

Do this on all switches (quick check)

Switch	Command	Expected
hxb-spine01, hxb-spine02	<code>sudo vtysh -c "show ip bgp summary"</code>	4 neighbours Established, AS 65101–65104
hxb-leaf-r1 ... r4	<code>nv show vrf default router bgp neighbor</code>	swp31 and swp32 Established, AS 65100

Checkpoint

- 8 sessions in total (each leaf has 2, each spine has 4), all Established.
- Each leaf receives 4 prefixes per spine.

Task 6 — Verify the routing table and ECMP

- On **hxb-leaf-r1**, list the BGP routes:

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show ip route bgp"
```

Expected output (illustrative; link-local addresses will differ):

```
B>* 10.255.0.1/32 [20/0] via fe80::4638:39ff:fe00:1, swp31, weight 1, 00:01:30
B>* 10.255.0.2/32 [20/0] via fe80::4638:39ff:fe00:2, swp32, weight 1, 00:01:28
B>* 10.255.0.12/32 [20/0] via fe80::4638:39ff:fe00:1, swp31, weight 1, 00:01:05
*                          via fe80::4638:39ff:fe00:2, swp32, weight 1, 00:01:05
B>* 10.255.0.13/32 [20/0] via fe80::4638:39ff:fe00:1, swp31, weight 1, 00:00:58
*                          via fe80::4638:39ff:fe00:2, swp32, weight 1, 00:00:58
B>* 10.255.0.14/32 [20/0] via fe80::4638:39ff:fe00:1, swp31, weight 1, 00:00:50
*                          via fe80::4638:39ff:fe00:2, swp32, weight 1, 00:00:50
```

- Read it carefully:
 - The spine loopbacks each have **one** path (you reach spine01 only through swp31).
 - Every other leaf loopback has **two** next hops, one per spine. The ***** on both lines means both are installed in the forwarding table. That is ECMP.
 - The next hops are IPv6 link-local addresses: that is BGP unnumbered.
- Look at one route in detail, and at the kernel's view:

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show ip route 10.255.0.14"
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show ip bgp 10.255.0.14/32"
cumulus@hxb-leaf-r1:mgmt:~$ ip route show 10.255.0.14
```

`show ip bgp 10.255.0.14/32` should list two paths, both with AS path **65100 65104**, and mark them **multipath**. The kernel output shows two **nexthop via inet6 fe80::...** lines.

4. On **hxb-spine01**, check what a spine knows:

```
cumulus@hxb-spine01:mgmt:~$ sudo vtysh -c "show ip route bgp"
```

Expect the four leaf loopbacks, each via one port. There is **no** route to 10.255.0.2 (spine02). A path spine02 → leaf → spine01 carries AS path **65101 65100**, which contains spine01's own ASN, so BGP rejects it. This is the valley-free behaviour described in the Background, not a fault.

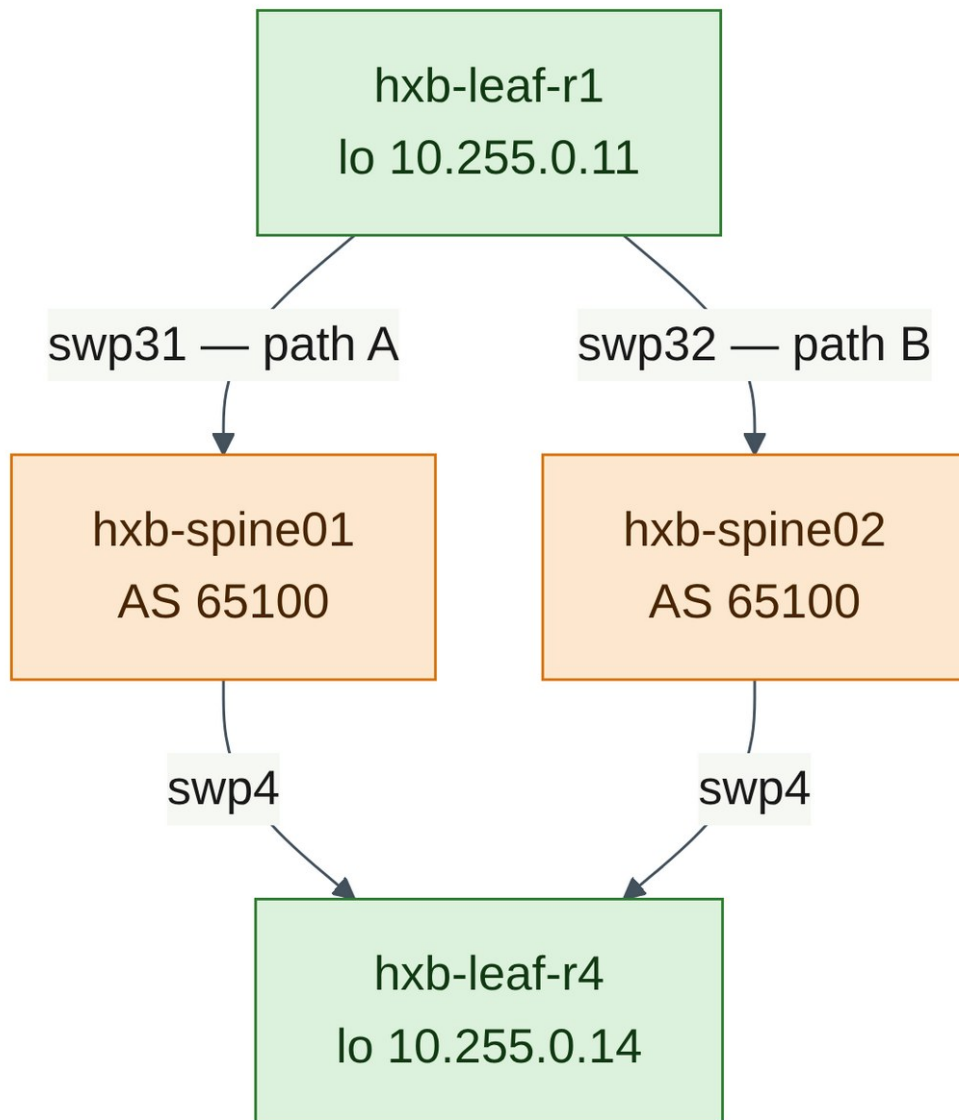


Figure L2.3: Two equal-cost paths from hxb-leaf-r1 to hxb-leaf-r4

Both paths have the same AS path length (65100 65104), so BGP installs both. Traffic sourced from 10.255.0.11 to 10.255.0.14 is hashed per flow onto swp31 or swp32.

Checkpoint

- Every leaf has one path to each spine loopback and two paths to each other leaf loopback.

- Each spine has four leaf loopbacks and no route to the other spine's loopback.

Task 7 — Loopback-to-loopback ping and traceroute

`ping` and `traceroute` use the default VRF even though your shell is in the management VRF (Lab 1, Task 1). Always source from the loopback: it is the address the other leaf has a route back to, and it is the address VXLAN will use in Lab 5.

1. Ping hxb-leaf-r4's loopback from hxb-leaf-r1's loopback:

```
cumulus@hxb-leaf-r1:mgmt:~$ ping -c 3 -I 10.255.0.11 10.255.0.14
```

Expected output (illustrative):

```
PING 10.255.0.14 (10.255.0.14) from 10.255.0.11 : 56(84) bytes of data.
64 bytes from 10.255.0.14: icmp_seq=1 ttl=63 time=0.912 ms
64 bytes from 10.255.0.14: icmp_seq=2 ttl=63 time=0.845 ms
64 bytes from 10.255.0.14: icmp_seq=3 ttl=63 time=0.801 ms

--- 10.255.0.14 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
```

TTL 63 means one routed hop (a spine) between the leaves.

2. Test jumbo frames end to end. 8972 bytes of ICMP data plus 28 bytes of headers is 9000 bytes, below the 9216 MTU; `-M do` forbids fragmentation:

```
cumulus@hxb-leaf-r1:mgmt:~$ ping -c 3 -M do -s 8972 -I 10.255.0.11 10.255.0.14
```

This should succeed. If it fails with `message too long`, a port on the path is not at MTU 9216 (Lab 1, Task 5).

3. Trace the path:

```
cumulus@hxb-leaf-r1:mgmt:~$ traceroute -n -s 10.255.0.11 10.255.0.14
```

Expected output (illustrative):

```
traceroute to 10.255.0.14 (10.255.0.14), 30 hops max, 60 byte packets
 1  10.255.0.1  0.702 ms  10.255.0.2  0.811 ms  10.255.0.1  0.765 ms
 2  10.255.0.14  1.120 ms  1.082 ms  1.051 ms
```

Traceroute sends three UDP probes per hop, each to a different destination port, so each probe is a different flow. When the ECMP hash includes the ports, the probes land on different spines and hop 1 lists both 10.255.0.1 and 10.255.0.2. The spines answer from their loopback because their fabric ports have no IPv4 address.

4. If hop 1 always shows the same spine, repeat with a few different starting ports to create different flows:

```
cumulus@hxb-leaf-r1:mgmt:~$ traceroute -n -s 10.255.0.11 -p 33434 10.255.0.14
cumulus@hxb-leaf-r1:mgmt:~$ traceroute -n -s 10.255.0.11 -p 40000 10.255.0.14
cumulus@hxb-leaf-r1:mgmt:~$ traceroute -n -s 10.255.0.11 -p 50000 10.255.0.14
```

Version note: Which path a flow takes depends on the kernel's multipath hash policy on Cumulus VX, which may hash on addresses only (check on your release with `sysctl net.ipv4.fib_multipath_hash_policy`: 0 is L3, 1 is L3+L4). If every traceroute shows the same spine, ECMP is still working: the two installed next hops in Task 6 prove it. A Spectrum switch hashes in the ASIC and, with adaptive routing (Lab 4), can go further than per-flow ECMP.

Do this on all switches (full-mesh reachability)

From	Command	Expected
hxb-leaf-r1	<code>ping -c 2 -I 10.255.0.11 10.255.0.12 / .13 / .14</code>	0% loss to each
hxb-leaf-r2	<code>ping -c 2 -I 10.255.0.12 10.255.0.11 / .13 / .14</code>	0% loss to each
hxb-leaf-r3	<code>ping -c 2 -I 10.255.0.13 10.255.0.11 / .12 / .14</code>	0% loss to each
hxb-leaf-r4	<code>ping -c 2 -I 10.255.0.14 10.255.0.11 / .12 / .13</code>	0% loss to each
hxb-spine01	<code>ping -c 2 -I 10.255.0.1 10.255.0.1114</code>	0% loss to each leaf
hxb-spine02	<code>ping -c 2 -I 10.255.0.2 10.255.0.1114</code>	0% loss to each leaf

Checkpoint

- hxb-leaf-r1 pings 10.255.0.14 from 10.255.0.11 with 0% loss, including 9000-byte packets.
- Traceroute shows one spine hop, and you have seen (or can explain why you have not seen) both spines at hop 1.
- Every leaf reaches every other leaf loopback.

Verify

- [] Every switch shows the planned ASN and router ID in `nv show router bgp`.
- [] 8 unnumbered sessions are Established (2 per leaf, 4 per spine).
- [] Each leaf has two ECMP next hops (swp31 and swp32) to every other leaf loopback.
- [] Spines have the four leaf loopbacks and no route to each other's loopback.
- [] `ping -I 10.255.0.11 10.255.0.14` succeeds, including with `-M do -s 8972`.
- [] `nv config diff` and `nv config diff applied startup` are empty on every switch.

Break and fix

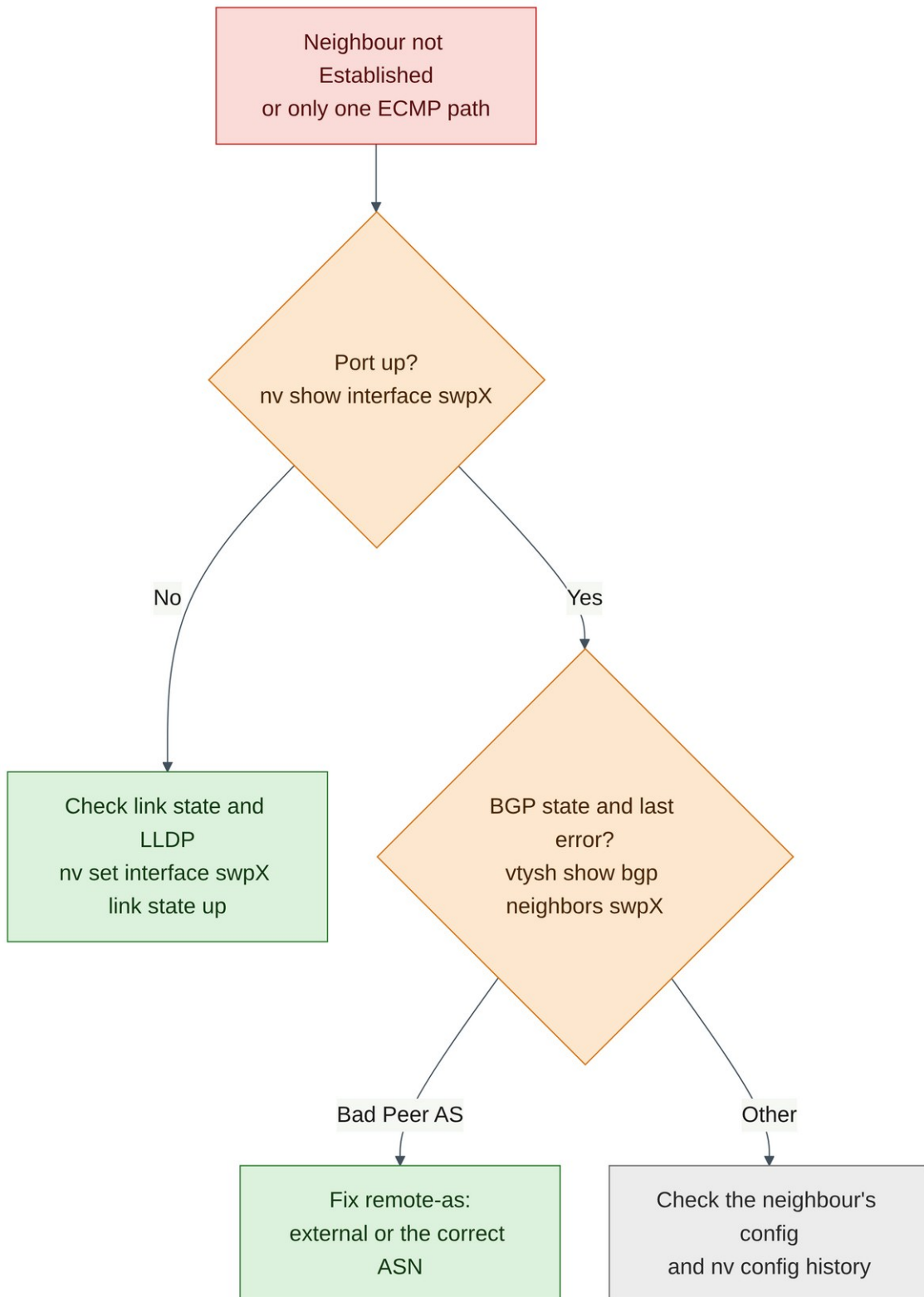


Figure L2.4: Diagnosing a missing underlay session

Start from the symptom on the leaf, check the link layer first, then the BGP state and the last notification, then compare the configured remote-as with what the neighbour really uses.

Fault 1 — Wrong remote-as

Inject (on hxb-leaf-r4): someone "tightens" the configuration by typing the spine ASN, but gets it wrong.

```
cumulus@hxb-leaf-r4:mgmt:~$ nv set vrf default router bgp neighbor swp31 remote-as 65200
cumulus@hxb-leaf-r4:mgmt:~$ nv config apply -y
```

Symptoms:

- On hxb-leaf-r4, `nv show vrf default router bgp neighbor` shows swp31 not established.
- On hxb-leaf-r1, `sudo vtysh -c "show ip route 10.255.0.14"` shows only **one** next hop (via swp32). Traffic still flows, so nothing alarms — you have silently lost half the bandwidth to leaf-r4.
- On hxb-spine01, `show ip bgp summary` shows the neighbour on swp4 as **Idle** or **Active**.

Diagnosis:

```
cumulus@hxb-leaf-r4:mgmt:~$ sudo vtysh -c "show ip bgp summary"
cumulus@hxb-leaf-r4:mgmt:~$ sudo vtysh -c "show bgp neighbors swp31" | grep -iE "remote AS|state|
notification|last reset"
cumulus@hxb-leaf-r4:mgmt:~$ nv config history
```

Expected output (illustrative):

```
BGP neighbor on swp31: fe80::4638:39ff:fe00:1, remote AS 65200, local AS 65104, external link
BGP state = Active
Last reset 00:00:41, Notification sent (OPEN Message Error/Bad Peer AS)
```

The leaf expects AS 65200, the spine opens with AS 65100, so the leaf rejects the OPEN with **Bad Peer AS**. `nv config history` shows who applied the change and when.

Fix:

```
cumulus@hxb-leaf-r4:mgmt:~$ nv set vrf default router bgp neighbor swp31 remote-as external
cumulus@hxb-leaf-r4:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r4:mgmt:~$ nv show vrf default router bgp neighbor
```

Then confirm on hxb-leaf-r1 that 10.255.0.14 has two next hops again. (Setting `remote-as 65100` would also work; `external` keeps every leaf identical.)

Fault 2 — Uplink administratively down**Inject** (on hxb-leaf-r1):

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp32 link state down
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

Symptoms:

- `nv show vrf default router bgp neighbor` shows swp32 down; `nv show interface` shows swp32 admin down.
- `sudo vtysh -c "show ip route 10.255.0.14"` shows a single next hop via swp31.
- Route to 10.255.0.2 (spine02) is gone from hxb-leaf-r1.
- Traceroute to 10.255.0.14 shows only 10.255.0.1 at hop 1, whatever the port.
- On hxb-spine02, only three leaves are Established.

Diagnosis: start at the link layer, as the figure shows.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp32
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp32 link --applied
cumulus@hxb-leaf-r1:mgmt:~$ nv config history
```

Admin state is **down** in the applied configuration: this was a configuration change, not a cable fault. (A cable fault would show admin up, oper down, and no LLDP neighbour.)

Fix:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp32 link state up
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show ip route 10.255.0.14"
```

Two next hops should return within a few seconds of the session re-establishing.

Field note: Both faults leave the fabric "working", because the other spine carries the traffic. In an AI fabric that means half the bandwidth for every flow to or from that leaf, and slower collective operations. Monitor the number of ECMP next hops and Established sessions, not just reachability (Lab 7).

Clean-up / save state

1. On every switch, confirm that nothing is pending and startup matches applied:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff applied startup
```

2. Make sure both faults are fixed: all 8 sessions Established, two next hops on every leaf-to-leaf route.
3. Optional but recommended: in the Air console, take a checkpoint of the simulation now (Lab 6 covers checkpoints in depth). "Underlay complete" is a useful point to return to before the RoCE and EVPN labs.
4. Keep the configuration. Labs 3–5 build on this underlay; Lab 5 uses the leaf loopbacks as VTEPs.
5. If you are stopping, put the simulation to sleep from the Air console.

Exam tie-in

- **2.1 / 2.3** — The Spectrum-X underlay is eBGP unnumbered: **nv set router bgp autonomous-system, router-id**, and **nv set vrf default router bgp neighbor swpX remote-as external**. EVPN (Lab 5) adds the **l2vpn-evpn** address family to these same neighbours.
- **1.2** — Rail-optimised leaves with a shared spine ASN give valley-free, two-hop paths between any two leaves; each additional spine adds another equal-cost path.
- **2.1** — Loopbacks are advertised as /32s and become VTEP addresses; verify with **nv show vrf default router bgp neighbor**, **vttysh show ip bgp summary** and **show ip route**.
- **2.2 / 5.3** — ECMP multipath is on by default; losing a session halves the paths without breaking reachability, which is why path counts matter for low-latency, high-bandwidth verification.
- **2.4** — Air lets you rehearse the full underlay and its failures on Cumulus VX; hashing detail and forwarding performance are not representative of Spectrum hardware.

Review questions

1. Why can every leaf use **remote-as external** on both uplinks, and what would break if you typed the wrong numeric ASN instead?
2. Why does hxb-spine01 have no route to hxb-spine02's loopback, and is that a problem?
3. In **show ip route 10.255.0.14** on hxb-leaf-r1, what tells you the underlay is using ECMP, and what tells you the sessions are unnumbered?
4. Why do you ping with **-I 10.255.0.11** rather than letting the switch pick a source address?

5. A leaf can still reach every other leaf, but `show ip route` shows one next hop for each. Name two likely causes and the first command you would run.

Answers

1. `remote-as external` accepts any ASN different from the local one, so the same line works for any eBGP peer. A wrong numeric ASN makes the leaf reject the spine's OPEN with a **Bad Peer AS** notification, and the session never establishes.
2. Both spines use AS 65100. A path to spine02's loopback through any leaf has AS path `65101 65100`, which contains spine01's own ASN, so BGP loop prevention drops it. It is not a problem: spines do not need to reach each other in a leaf-spine fabric, and this prevents valley paths.
3. ECMP: two `via` lines, both marked `*` (installed), one through swp31 and one through swp32. Unnumbered: the next hops are IPv6 link-local addresses (`fe80::...`) on swp ports, with no IPv4 next hop.
4. The fabric ports have no IPv4 addresses, and only the loopback /32 is advertised. Sourcing from the loopback makes sure the far leaf has a route back, and it tests the same address pair that VXLAN tunnels will use.
5. One uplink or BGP session is down: a port set to admin down (or a cable fault), or a misconfigured neighbour such as a wrong `remote-as`. Start with `nv show vrf default router bgp neighbor` (or `sudo vtysh -c "show ip bgp summary"`), then `nv show interface` for the affected port.

Lab 3 — Lossless RoCE QoS and Soft-RoCE Hosts

Lab at a glance

Item	Detail
Book chapters	Chapter 5 (Lossless RoCE: QoS, PFC and ECN) · Chapter 3 (RDMA and RoCE basics) · Chapter 19 (perftest flags)
Exam objectives	2.1 Configure Spectrum-X switches for RoCE · 2.2 Enable QoS, ECN and PFC · supports 5.5 (perftest)
Time	About 90 minutes
Air resources used	hxb-leaf-r1, hxb-gpu01, hxb-gpu03 (plus oob-mgmt-server as jump host). Optional roll-out task touches every switch
Prerequisite labs	Lab 0 (topology running), Lab 1 (NVUE essentials), Lab 2 (underlay)

Objectives

By the end of this lab you will be able to:

- Build a temporary test VLAN on a leaf so two servers can exchange RoCE traffic without any routing.
- Enable the NVIDIA RoCE QoS profile with `nv set qos roce` and read back every default it sets: trust, DSCP to switch-priority mapping, PFC, ECN and buffers.
- Turn an Ubuntu server into a RoCE endpoint with Soft-RoCE (`rdma_rxe`), and inspect the RDMA device, port and GID table.
- Mark RoCE traffic with traffic class byte 106 (DSCP 26 + ECT), both per test and as the RDMA-CM default.
- Prove the marking on the wire with `tcpdump`, and recognise the "host sends DSCP 0" fault.
- Explain plainly what Cumulus VX in Air can and cannot show you about lossless behaviour.

Background

Chapter 5 described the Spectrum QoS pipeline: the switch **trusts** a marking, maps it to a **switch priority**, maps that to a **traffic class** (egress queue), places the packet in a **buffer pool**, and applies **PFC** or **ECN** when thresholds are crossed. On Cumulus Linux one command, `nv set qos roce`, builds the whole NVIDIA-validated profile: DSCP 24–31 lands in switch priority 3 (lossless, PFC on), and DSCP 48–55 (CNPs) lands in switch priority 6 (strict priority).

The switch only does half the job. The **host** must mark RoCE packets so they fall into the lossless class. The usual Helix-B value is traffic class byte **106**: $\text{DSCP } 26 \times 4 = 104$, plus $\text{ECT}(0) = 2$. If the host sends DSCP 0, the switch classifies the traffic into lossy TC 0, and everything on the switch still "looks configured". That mismatch is the most common real-world RoCE QoS fault, and this lab makes you see it on the wire.

Air has no Spectrum ASIC and no ConnectX NICs. So you use **Soft-RoCE** (`rdma_rxe`), a Linux kernel driver that implements RoCEv2 in software over any Ethernet interface. It produces genuine RoCEv2 packets (UDP destination port 4791) with a real IP header, so the DSCP marking is exactly what you would check on hardware. On Cumulus VX the `swp` ports are ordinary Linux interfaces and bridging is done by the Linux kernel, so you can run `tcpdump` on the leaf port itself. On a real Spectrum switch the data plane is in the ASIC and `tcpdump` on a `swp` only sees traffic punted to the CPU.

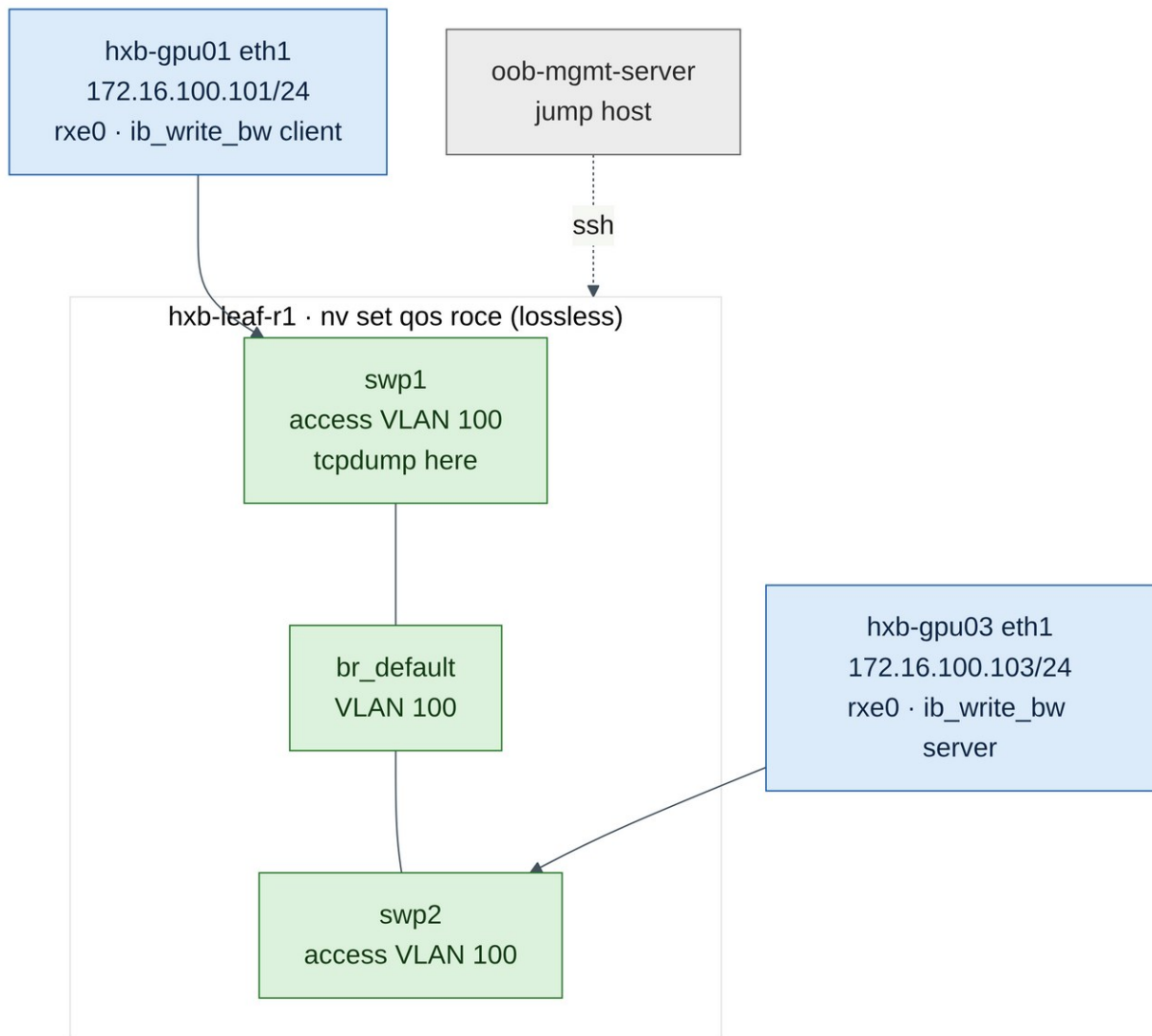


Figure 3.1: Lab 3 test topology

For this lab only, `hxb-gpu01 eth1` and `hxb-gpu03 eth1` sit in a plain VLAN 100 on the `hxb-leaf-r1` bridge, so Soft-RoCE traffic between them crosses one switch with no routing. The RoCE QoS profile is enabled on the leaf, and you capture on `swp1` to read the DSCP value. VLAN 100 is removed at the end because Lab 5 puts these ports into their EVPN tenants.

Warning: Cumulus VX has no Spectrum ASIC. It has no real buffers, no hardware queues, no PFC pause generation and no ECN marking under load. The RoCE QoS commands are accepted and `nv show qos roce` displays the intended configuration, but no packet is ever paused or ECN-marked by VX. In this lab you learn to **configure and verify intent** on the switch and to **verify marking** on the host and on the wire. Counters that depend on the ASIC (per-priority pause, ECN marks, buffer occupancy) are expected to stay at zero or be absent.

Step-by-step

Task 1 — Build temporary VLAN 100 on `hxb-leaf-r1`

1. From the `oob-mgmt-server`, log in to the leaf.

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-leaf-r1
cumulus@hxb-leaf-r1:~$
```

2. Put `swp1` and `swp2` into the default bridge as access ports in VLAN 100, then review and apply.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set bridge domain br_default vlan 100
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp1-2 bridge domain br_default access 100
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp1-2 description "LAB3-TEMP VLAN100"
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

3. Check the bridge membership.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show bridge domain br_default vlan
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp1-2 link
```

Expected output (illustrative): VLAN 100 is listed in `br_default`, and swp1 and swp2 show `oper-status up` with access VLAN 100.

- On each server, bring up eth1 and give it a Lab 3 address. These addresses are not persistent, which is what you want for a temporary VLAN.

```
ubuntu@oob-mgmt-server:~$ ssh ubuntu@hxb-gpu01
ubuntu@hxb-gpu01:~$ sudo ip link set eth1 up
ubuntu@hxb-gpu01:~$ sudo ip addr add 172.16.100.101/24 dev eth1
```

```
ubuntu@oob-mgmt-server:~$ ssh ubuntu@hxb-gpu03
ubuntu@hxb-gpu03:~$ sudo ip link set eth1 up
ubuntu@hxb-gpu03:~$ sudo ip addr add 172.16.100.103/24 dev eth1
ubuntu@hxb-gpu03:~$ ping -c 3 172.16.100.101
```

Expected output (illustrative):

```
64 bytes from 172.16.100.101: icmp_seq=1 ttl=64 time=0.912 ms
64 bytes from 172.16.100.101: icmp_seq=2 ttl=64 time=0.644 ms
64 bytes from 172.16.100.101: icmp_seq=3 ttl=64 time=0.701 ms
```

Checkpoint

- VLAN 100 exists on hxb-leaf-r1 and swp1-2 are access ports in it.
- hxb-gpu03 pings hxb-gpu01 on 172.16.100.0/24. `ttl=64` confirms there is no router in the path.

Task 2 — Enable the RoCE QoS profile

- Enable RoCE in its default mode and state the mode explicitly, so the intent is readable in the configuration.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set qos roce
cumulus@hxb-leaf-r1:mgmt:~$ nv set qos roce mode lossless
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

Expected output (illustrative): `nv config diff` shows a small block under `qos: roce:` with `enable: on` and `mode: lossless`. One NVUE line expands into the whole profile when NVUE renders the switch configuration.

Version note: On Cumulus VX, `nv config apply` normally accepts the RoCE profile (check on your release). If `apply` returns an error about the platform or `switchd`, copy the message into your lab notes, run `nv config detach` to drop the pending change, and continue with the read-only parts of this task using the book's reference output in Chapter 5. The host tasks (3 to 6) do not depend on it.

2. Read the full profile.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos roce
```

Expected output (illustrative, trimmed):

```

----- operational ----- applied -----
enable          on          on
mode            lossless   lossless
congestion-control
  congestion-mode ECN
  enabled-tc      0,3
  max-threshold   1.43 MB
  min-threshold   146.48 KB
  probability     100
pfc
  pfc-priority    3
  rx-enabled      enabled
  tx-enabled      enabled
trust
  trust-mode      pcp,dscp
RoCE PCP/DSCP->SP mapping configurations
  pcp  dscp          switch-prio
  3    24,25,26,27,28,29,30,31  3
  6    48,49,50,51,52,53,54,55  6
  ...
RoCE SP->TC mapping and ETS configurations
  switch-prio traffic-class scheduler-weight
  3           3             DWRR-50%
  6           6             strict-priority
  ...
RoCE pool config
  name          mode    size
lossy-default-ingress Dynamic 50%
roce-reserved-ingress Dynamic 50%
lossy-default-egress  Dynamic 50%
roce-reserved-egress  Dynamic inf

```

3. Record your own values in this table. It is the same table as Chapter 5, and the exam expects you to know it.

Setting	Default (book)	Your output
Mode	lossless	
Trust	pcp,dscp	
DSCP 26 maps to	switch priority 3, TC 3	
DSCP 48 maps to	switch priority 6, TC 6	
PFC priority	3 (RX and TX)	
ECN enabled on	TC 0 and TC 3	
ECN min / max / probability	146.48 KB / 1.43 MB / 100%	
RoCE TC 3 scheduling	DWRR 50%	
CNP TC 6 scheduling	Strict priority	

- Look at the same settings through the individual QoS objects. The RoCE profile populates the standard QoS profiles (**default-global**), so you can inspect each stage of the pipeline separately.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos mapping default-global
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos mapping default-global dscp 26
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos mapping default-global dscp 48
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos pfc default-global
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos congestion-control default-global
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos traffic-pool
```

Expected output (illustrative): **dscp 26** shows **switch-priority 3**, **dscp 48** shows **switch-priority 6**, PFC lists switch priority 3, and congestion control lists traffic classes 0 and 3 with ECN enabled. If one of these paths is not available on your release, **nv show qos** lists the objects that are (check on your release).

- For reference only, these are the standard QoS commands that the RoCE profile saves you from typing. **Do not apply them** on top of **nv set qos roce**: NVIDIA's profile is validated per ASIC, and mixing manual changes into it is how fabrics drift.

```
nv set qos mapping default-global trust l3
nv set qos mapping default-global dscp 26 switch-priority 3
nv set qos mapping default-global dscp 48 switch-priority 6
nv set qos pfc default-global switch-priority 3
nv set qos pfc default-global tx enable
nv set qos pfc default-global rx enable
nv set qos congestion-control default-global traffic-class 3 min-threshold <bytes>
nv set qos congestion-control default-global traffic-class 3 max-threshold <bytes>
nv set qos congestion-control default-global traffic-class 3 ecn enable
nv set qos traffic-pool <pool-name> memory-percent <percent>
```

- Check the per-port view on the two server-facing ports.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp1 qos roce status
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp1 qos roce counters
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface qos-roce-status-pool-map
```

Expected output: On hardware, **status** shows the profile active on the port with PFC on priority 3, and **counters** shows RoCE bytes, pause frames and ECN marks. On VX, status may show the configured intent while counters are zero or not supported. That is expected: the counters come from the ASIC.

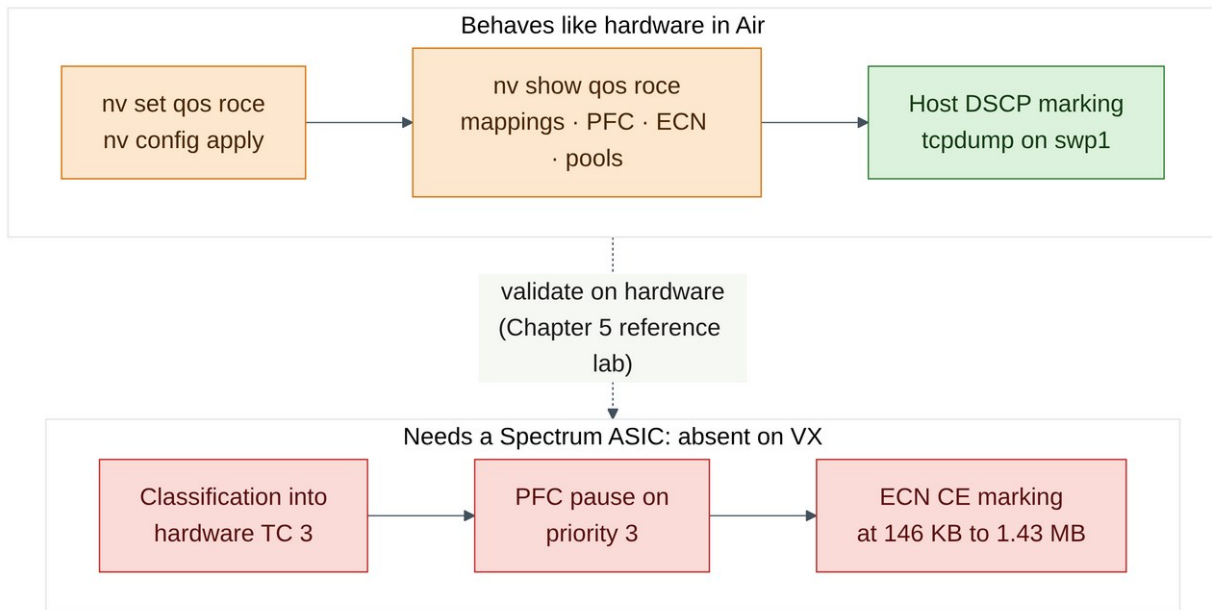


Figure 3.2: What VX shows and what it cannot

The configuration plane (NVUE, the rendered QoS files and `nv show`) behaves as on Spectrum hardware. Everything below it depends on the ASIC, so on VX it is absent: no hardware queues, no pause frames, no ECN marks and no buffer occupancy. Marking is still real because the hosts generate it.

Checkpoint

- `nv show qos roce` shows mode `lossless`, trust `pcp,dscp`, PFC priority 3 and ECN on TC 0 and 3.
- You can state which switch priority and TC DSCP 26 and DSCP 48 map to.
- You can say which of these settings VX can actually enforce (none of the data-plane ones).

Task 3 — Optional: roll the profile out to every switch

RoCE QoS must be consistent on every hop. Chapter 22 and Lab 12 automate this with Ansible. For now, a shell loop from the oob-mgmt-server is enough. It runs non-interactively because NVUE commands work over `ssh`.

```

ubuntu@oob-mgmt-server:~$ for sw in hxb-spine01 hxb-spine02 hxb-leaf-r2 hxb-leaf-r3 hxb-leaf-r4;
do
> echo "== $sw"; ssh cumulus@$sw "nv set qos roce && nv config apply -y && nv show qos roce |
grep -E '^mode|pfc-priority'"
> done
  
```

Expected output (illustrative): each switch prints `mode lossless` and `pfc-priority 3`.

Checkpoint

- Every switch in helix-b-air reports the same RoCE mode. A single switch left in its default QoS state is the classic "one hop is lossy" fault.

Task 4 — Install RDMA tools and create a Soft-RoCE device

Do steps 1–6 on **both** `hxb-gpu01` and `hxb-gpu03`. `hxb-gpu01` is shown.

1. Install the RDMA user space, the verbs utilities and `perftest`.

```

ubuntu@hxb-gpu01:~$ sudo apt-get update
ubuntu@hxb-gpu01:~$ sudo apt-get install -y rdma-core ibverbs-utils ibverbs-providers perftest
  
```

Field note: Air servers normally reach the Ubuntu mirrors through the OOB network. If `apt-get update` cannot resolve or reach the mirror, check the internet setting of your simulation in the Air UI before troubleshooting

anything else.

2. Load the Soft-RoCE kernel module.

```
ubuntu@hxb-gpu01:~$ sudo modprobe rdma_rxe
ubuntu@hxb-gpu01:~$ lsmod | grep rdma_rxe
```

Expected output (illustrative):

```
rdma_rxe                204800  0
ib_uverbs               200704  2 rdma_rxe,rdma_ucm
ib_core                 524288  6 rdma_cm,rdma_rxe,...
```

Version note: On some Ubuntu kernel flavours `rdma_rxe` ships in the extra modules package. If `modprobe` reports `Module rdma_rxe not found`, install it with `sudo apt-get install -y linux-modules-extra-$(uname -r)` and try again (check on your release).

3. Bind a Soft-RoCE device called `rxex0` to `eth1` and check the link.

```
ubuntu@hxb-gpu01:~$ sudo rdma link add rxex0 type rxe netdev eth1
ubuntu@hxb-gpu01:~$ rdma link show
```

Expected output (illustrative):

```
link rxex0/1 state ACTIVE physical_state LINK_UP netdev eth1
```

4. List RDMA devices and inspect the port.

```
ubuntu@hxb-gpu01:~$ ibv_devices
ubuntu@hxb-gpu01:~$ ibv_devinfo -d rxex0
```

Expected output (illustrative):

```
device                node GUID
-----
rxex0                 5054:00ff:fe12:3401

hca_id: rxex0
  transport:                InfiniBand (0)
  ...
  port:    1
    state:                PORT_ACTIVE (4)
    max_mtu:              4096 (5)
    active_mtu:           1024 (3)
    sm_lid:                0
    port_lid:              0
    port_lmc:              0x00
    link_layer:            Ethernet
```

Note three things. **transport: InfiniBand** is normal for RoCE: the verbs transport is InfiniBand's, carried over Ethernet. **link_layer: Ethernet** tells you it is RoCE. **active_mtu 1024** follows from `eth1`'s MTU of 1500 bytes, because a RoCE MTU must fit inside the Ethernet MTU with its headers (Lab 9 explores this).

- Read the GID table. Ubuntu's rdma-core does not ship the `show_gids` script that comes with DOCA-OFED/MLNX_OFED, so use `ibv_devinfo -v` or read sysfs directly.

```
ubuntu@hxb-gpu01:~$ ibv_devinfo -v -d rxe0 | grep GID
ubuntu@hxb-gpu01:~$ for i in 0 1 2 3; do
> g=$(cat /sys/class/infiniband/rxe0/ports/1/gids/$i 2>/dev/null)
> t=$(cat /sys/class/infiniband/rxe0/ports/1/gid_attrs/types/$i 2>/dev/null)
> echo "$i $g $t"
> done
```

Expected output (illustrative):

```

                GID[ 0]:      fe80::5054:ff:fe12:3401, RoCE v2
                GID[ 1]:      ::ffff:172.16.100.101, RoCE v2
0 fe80:0000:0000:0000:5054:00ff:fe12:3401 RoCE v2
1 0000:0000:0000:0000:0000:0000:ffff:ac10:6465 RoCE v2
2 0000:0000:0000:0000:0000:0000:0000:0000
3 0000:0000:0000:0000:0000:0000:0000:0000
```

The **RoCEv2 IPv4 GID** is the one that embeds your IPv4 address as `::ffff:172.16.100.101` (`ac10:6465` is 172.16.100.101 in hex). Its index, usually **1** on Soft-RoCE, is what `-x` means in `perftest`. On ConnectX adapters the same address is often index 3, because the NIC also lists RoCE v1 GIDs. Always read the table rather than assume.

- Repeat steps 1–5 on `hxb-gpu03` and record its RoCEv2 IPv4 GID index.

Checkpoint

- `rdma link show` reports `rxe0/1 state ACTIVE` bound to `eth1` on both servers.
- `ibv_devinfo` shows `PORT_ACTIVE` and `link_layer: Ethernet`.
- You know the GID index that holds `::ffff:172.16.100.10x` on each server.

Task 5 — Mark RoCE traffic with traffic class 106

There are three places a RoCE application's marking can come from. Know all three, because the exam and real incidents use all three.

Method	Applies to	How
<code>perftest --tclass=<value></code>	Tests that connect without RDMA-CM (you give <code>-x <GID index></code>). Sets the Traffic Class in the GRH, which becomes the IP ToS byte	<code>ib_write_bw -x 1 --tclass=106 ...</code>
<code>perftest -T <value> / --tos=<value></code>	Tests that connect with RDMA-CM (<code>-R</code>). <code>perftest</code> passes the ToS to RDMA-CM	<code>ib_write_bw -R -T 106 ...</code>
RDMA-CM default ToS (configs)	Every RDMA-CM connection on that device and port that doesn't set its own ToS	<code>default_roce_tos</code> under <code>/sys/kernel/config/rdma_cm/</code>

Version note: In current `perftest`, `--tclass` sets the GRH traffic class and `-T/--tos` is documented as "available only with -R". A command such as `ib_write_bw -R --tclass=106` may therefore **not** mark the packets on your version. Check `ib_write_bw --help | grep -iE 'tos|tclass'` on your release and use the table above. The `cma_roce_tos` helper script mentioned in Chapter 5 ships with DOCA-OFED/MLNX_OFED, not with Ubuntu's `rdma-core`, so here you use configs directly.

- Check the `perftest` options on your installed version.

```
ubuntu@hxb-gpu01:~$ ib_write_bw --help | grep -iE 'tos|tclass|rdma_cm|gid-index'
```

Expected output (illustrative):

```
-R, --rdma_cm Connect QPs with rdma_cm and run test on those QPs
-T, --tos=<tos value> Set <tos_value> to RDMA-CM QPs. available only with -R flag. values 0-256 (default off)
-x, --gid-index=<index> Test uses GID with GID index taken from command
--tclass=<value> Set the Traffic Class in GRH (if GRH is in use)
```

2. Set the RDMA-CM default ToS for rx0 to 106 on **both** servers. Creating the directory under **rdma_cm** makes the kernel populate it with the device's ports.

```
ubuntu@hxb-gpu01:~$ sudo modprobe rdma_cm
ubuntu@hxb-gpu01:~$ mount | grep -q configfs || sudo mount -t configfs none /sys/kernel/config
ubuntu@hxb-gpu01:~$ sudo mkdir -p /sys/kernel/config/rdma_cm/rxe0
ubuntu@hxb-gpu01:~$ echo 106 | sudo tee /sys/kernel/config/rdma_cm/rxe0/ports/1/default_roce_tos
ubuntu@hxb-gpu01:~$ cat /sys/kernel/config/rdma_cm/rxe0/ports/1/default_roce_tos
```

Expected output: 106. This setting is not persistent: it is lost when the server reboots or the rx0 device is deleted.

Checkpoint

- You can explain why **--tclass** goes with **-x** and **-T** goes with **-R**.
- **default_roce_tos** reads 106 on both servers.

Task 6 — Run perfest and prove the marking on the wire

1. On hxb-leaf-r1, start a capture on swp1 (the port facing hxb-gpu01). Leave it running.

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo tcpdump -i swp1 -nn -v -c 6 udp port 4791
```

2. On hxb-gpu03, start the **server** side of an RDMA WRITE bandwidth test using RDMA-CM.

```
ubuntu@hxb-gpu03:~$ ib_write_bw -d rxe0 -R -T 106 -F --report_gbits
```

3. On hxb-gpu01, run the **client** against the server's IP.

```
ubuntu@hxb-gpu01:~$ ib_write_bw -d rxe0 -R -T 106 -F --report_gbits 172.16.100.103
```

Expected output (client, illustrative):

```
RDMA_Write BW Test
Dual-port      : OFF      Device      : rxe0
Number of qps  : 1        Transport type : IB
Connection type : RC      Using SRQ      : OFF
TX depth       : 128
CQ Moderation  : 1
Mtu            : 1024[B]
Link type      : Ethernet
GID index      : 1
Max inline data : 0[B]
rdma_cm QPs    : ON
Data ex. method : rdma_cm   TOS      : 106
-----
local address: LID 0000 QPN 0x0011 PSN 0x2c61a2
GID: 00:00:00:00:00:00:00:00:00:255:255:172:16:100:101
remote address: LID 0000 QPN 0x0012 PSN 0x9a01be
```

```
GID: 00:00:00:00:00:00:00:00:00:00:255:255:172:16:100:103
```

```
-----
#bytes      #iterations    BW peak[Gb/sec]    BW average[Gb/sec]    MsgRate[Mpps]
65536       5000             1.62               1.48                   0.002823
-----
```

Ignore the bandwidth figure. Soft-RoCE runs on a virtual CPU, so a few Gb/s or less is normal (Lab 9 explains why). What matters here is **rdma_cm QPs : ON, GID index : 1** chosen automatically by RDMA-CM, and **TOS : 106**.

4. Read the capture on the leaf.

Expected output (illustrative):

```
tcpdump: listening on swp1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10:42:17.311508 IP (tos 0x6a,ECT(0), ttl 64, id 4412, offset 0, flags [DF], proto UDP (17),
length 1080)
    172.16.100.101.49153 > 172.16.100.103.4791: UDP, length 1052
...
6 packets captured
```

Decode the ToS byte: **0x6a** = 106 = binary **011010 10**. The top six bits **011010** are **DSCP 26**, and the bottom two bits **10** are **ECT(0)**. On a Spectrum switch with the RoCE profile, DSCP 26 falls in 24–31, so the packet goes to switch priority 3, TC 3, the lossless queue.

5. Now run the same test the other way: without RDMA-CM, with an explicit GID index and **--tclass**. Use the index you recorded in Task 4.

```
ubuntu@hxb-gpu03:~$ ib_write_bw -d rxe0 -x 1 --tclass=106 -F --report_gbits
ubuntu@hxb-gpu01:~$ ib_write_bw -d rxe0 -x 1 --tclass=106 -F --report_gbits 172.16.100.103
```

Capture again on swp1. You should see the same **tos 0x6a**. In this mode perftest exchanges QP details over a TCP socket (port 18515) first, and you can see that TCP session if you capture without the UDP filter.

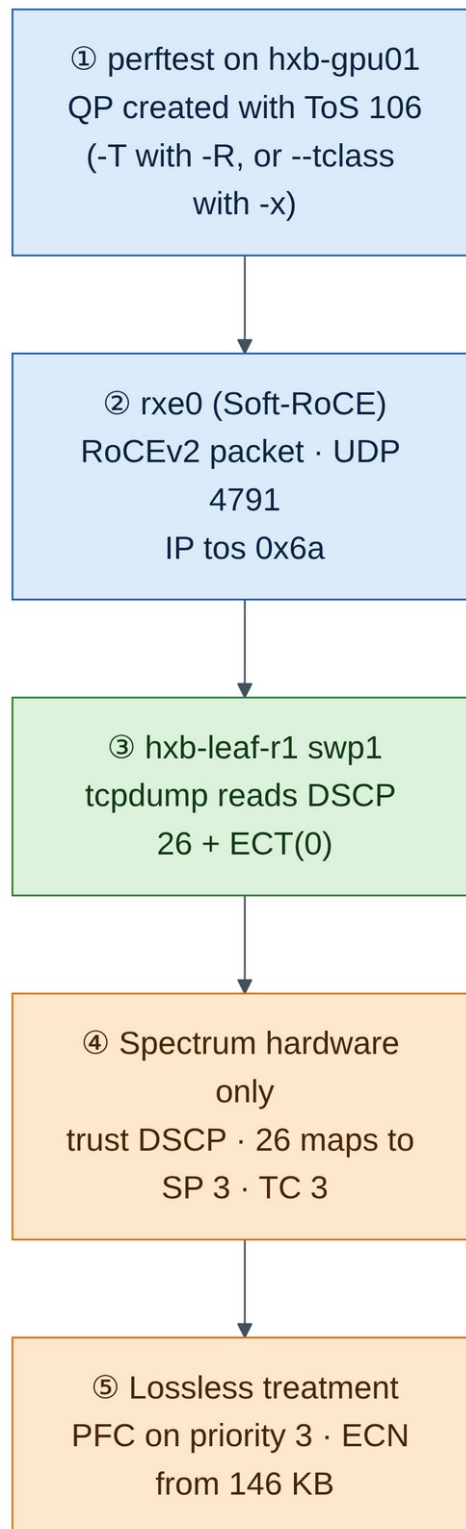


Figure 3.3: Following the marking from host to queue

The application (perftest) sets ToS 106 either with -T over RDMA-CM or with --tclass in the GRH. Soft-RoCE copies it into the IP header of every RoCEv2 packet (UDP 4791). The leaf reads DSCP 26 and, on Spectrum hardware, maps it to switch priority 3 and lossless TC 3.

Checkpoint

- **tcpdump** on hxb-leaf-r1 swp1 shows RoCEv2 packets to UDP 4791 with **tos 0x6a, ECT(0)**.
- You can convert a ToS byte to DSCP (divide by 4, ignore the remainder) and back (DSCP × 4 + ECN bits).
- You have run perftest in both connection modes, **-R -T 106** and **-x <index> --tclass=106**.

Verify

- [] VLAN 100 is up on hxb-leaf-r1 with swp1-2 as access ports, and the servers ping each other on 172.16.100.0/24.
- [] `nv show qos roce` shows `lossless`, trust `pcp,dscp`, PFC priority 3, ECN on TC 0 and 3, and DSCP 26 → SP 3, DSCP 48 → SP 6.
- [] `rdma link show` shows rxex0 ACTIVE on eth1 on hxb-gpu01 and hxb-gpu03.
- [] You know the RoCEv2 IPv4 GID index on each server.
- [] `default_roce_tos` is 106 on both servers.
- [] A capture on swp1 shows RoCEv2 (UDP 4791) with ToS 0x6a (DSCP 26, ECT(0)).
- [] You can list what VX cannot show: hardware TC classification, PFC pauses, ECN marks, buffer occupancy.

Break and fix

Fault 1 — The host sends DSCP 0

Inject. On both servers, set the RDMA-CM default back to 0, then run the test with `-R` and **no** `-T`, as an application that "forgot" its traffic class would.

```
ubuntu@hxb-gpu03:~$ echo 0 | sudo tee /sys/kernel/config/rdma_cm/rxe0/ports/1/default_roce_tos
ubuntu@hxb-gpu01:~$ echo 0 | sudo tee /sys/kernel/config/rdma_cm/rxe0/ports/1/default_roce_tos
ubuntu@hxb-gpu03:~$ ib_write_bw -d rxe0 -R -F --report_gbits
ubuntu@hxb-gpu01:~$ ib_write_bw -d rxe0 -R -F --report_gbits 172.16.100.103
```

Symptoms. The test completes normally. Nothing on the switch has changed, and `nv show qos roce` is still perfect. On real hardware the job would be fine when idle and would suffer drops and erratic throughput under incast, while the RoCE priority-3 counters on the leaf stay almost empty.

Diagnosis. Capture on the leaf port.

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo tcpdump -i swp1 -nn -v -c 3 udp port 4791
```

Expected output (illustrative):

```
10:51:02.114873 IP (tos 0x0, ttl 64, id 5120, offset 0, flags [DF], proto UDP (17), length 1080)
172.16.100.101.49154 > 172.16.100.103.4791: UDP, length 1052
```

`tos 0x0` means DSCP 0 and Not-ECT. DSCP 0 maps to switch priority 0 and **TC 0**, which is lossy: no PFC, and packets can be dropped. It is also not ECN-capable, so DCQCN gets no early feedback. The client's own output confirms it: there is no **TOS : 106** on the **Data ex. method** line.

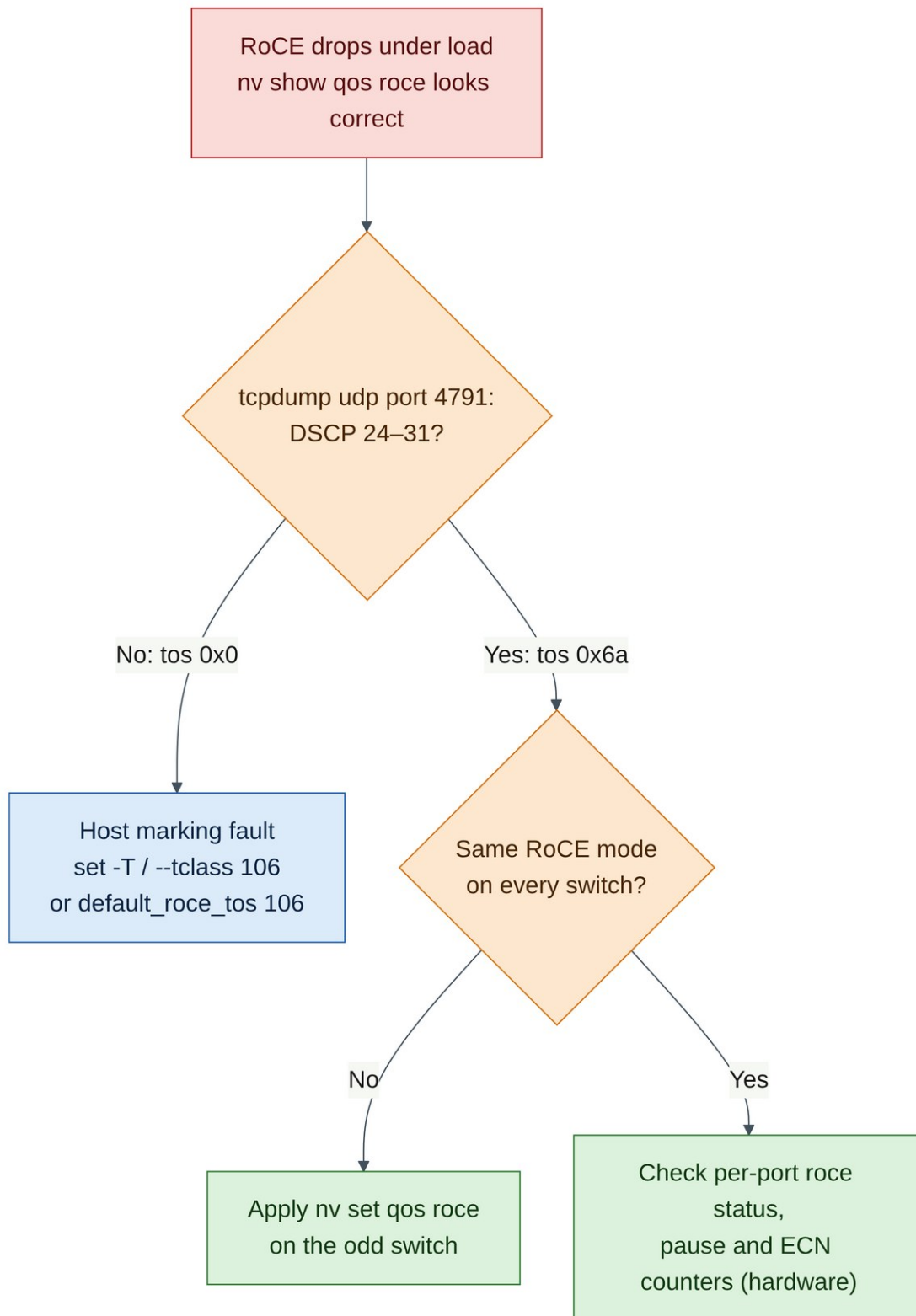


Figure 3.4: Diagnosing "RoCE configured but still dropping"

Start from the wire, not the switch configuration. If the captured ToS is not DSCP 24-31, fix the host marking. Only if the marking is right do you move on to switch trust, mapping and PFC state.

Fix. Restore the default ToS on both servers and re-run the capture to confirm **tos 0x6a**.

```

ubuntu@hxb-gpu01:~$ echo 106 | sudo tee /sys/kernel/config/rdma_cm/rxe0/ports/1/default_roce_tos
ubuntu@hxb-gpu03:~$ echo 106 | sudo tee /sys/kernel/config/rdma_cm/rxe0/ports/1/default_roce_tos
  
```

Because the RDMA-CM default now applies, even `ib_write_bw -R` without `-T` marks the traffic with ToS 106. In production the equivalent fixes are `NCCL_IB_TC=106` for NCCL, the device's RDMA-CM default ToS, and trust `dscp` on the NIC (Chapter 5).

Fault 2 — One switch drops to lossy mode

Inject. On `hxb-leaf-r1`, change the mode and apply.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set qos roce mode lossy
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

Symptoms. On hardware, RoCE still reaches TC 3 and gets ECN, but the leaf no longer sends or honours PFC on priority 3, so incast bursts that outrun congestion control are dropped instead of paused. On VX there is no visible traffic effect.

Diagnosis.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos roce | grep -A3 -E '^mode|^pfc'
cumulus@hxb-leaf-r1:mgmt:~$ nv config history | head
```

`mode lossy` and the missing PFC priority give it away. `nv config history` shows who applied the change and when.

Fix.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set qos roce mode lossless
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos roce | grep -E '^mode|^pfc-priority'
```

Clean-up / save state

Lab 5 puts `swp1` and `swp2` into the AURORA and BOREALIS tenants, and the servers' `eth1` addresses change to 172.16.10.101 and 172.17.10.103. Remove the temporary VLAN now, but **keep the RoCE QoS profile**: every later lab assumes it.

1. On `hxb-leaf-r1`, remove VLAN 100 and the temporary port settings, then save.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv unset interface swp1-2 bridge
cumulus@hxb-leaf-r1:mgmt:~$ nv unset interface swp1-2 description
cumulus@hxb-leaf-r1:mgmt:~$ nv unset bridge domain br_default vlan 100
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r1:mgmt:~$ nv config save
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos roce | grep -E '^mode'
```

Expected output: `mode lossless`. If `br_default` has no other members, NVUE may also remove the bridge. Lab 5 re-creates it.

2. On each server, remove the Lab 3 address. Leave `rxex0` in place: Lab 9 reuses it on the same `eth1` interface, and its GID table follows the new address automatically.

```
ubuntu@hxb-gpu01:~$ sudo ip addr del 172.16.100.101/24 dev eth1
ubuntu@hxb-gpu03:~$ sudo ip addr del 172.16.100.103/24 dev eth1
```

3. If you are stopping here, sleep the simulation from the Air UI to save compute-hour credits. Note that the `rxex0` device and the configs ToS are **not persistent**: after a server reboot, repeat Task 4 steps 2–3 and Task 5 step 2.

Exam tie-in

- **2.1:** `nv set qos roce` (then `nv config apply`) enables the RoCE profile, default mode `lossless`. `nv set qos roce mode lossy` removes PFC but keeps classification and ECN. Verify with `nv show qos roce` and `nv show interface <if> qos roce status/counters`.
- **2.2:** Know the defaults: trust PCP+DSCP, DSCP 24–31 → SP 3 → TC 3 (DWRR 50%, PFC, ECN), DSCP 48–55 → SP 6 → TC 6 (strict, CNPs), ECN 146.48 KB / 1.43 MB / 100% on TC 0 and 3.
- **2.1 and 2.2 (host side):** Traffic class 106 = DSCP 26 × 4 + ECT(0). A host sending DSCP 0 lands in lossy TC 0 even when every switch is correct.
- **5.5:** `perftest` on RoCE needs the right GID (`-x`, or `-R` to let RDMA-CM resolve it) and the right marking (`--tclass` with `-x`, `-T` with `-R`).

Review questions

1. A captured RoCEv2 packet shows `tos 0x6a`. What DSCP and ECN values does it carry, and which switch priority and traffic class does the default RoCE profile give it?
2. Which command changes an existing lossless RoCE configuration to lossy, and what is the single behavioural difference between the two modes?
3. You run `ib_write_bw -x 1 -T 106 172.16.100.103` and the capture shows `tos 0x0`. Why?
4. On `hxb-leaf-r1` in Air, `nv show qos roce` is correct but `nv show interface swp1 qos roce counters` shows no pause frames or ECN marks after a `perftest` run. Is the fabric broken?
5. Why does `ibv_devinfo` report `active_mtu 1024` for `rxex0` when `eth1` has an MTU of 1500?

Answers

1. `0x6a` = 106 = DSCP 26 (106 ÷ 4 = 26 remainder 2) with ECN bits `10`, ECT(0). DSCP 26 is in 24–31, so switch priority 3 and traffic class 3, the lossless RoCE queue with PFC and ECN.
2. `nv set qos roce mode lossy` followed by `nv config apply`. Lossy mode keeps trust, mappings, scheduling and ECN, but does not enable PFC on priority 3.
3. `-T/--tos` only applies to RDMA-CM connections (`-R`). Without `-R`, `perftest` connects over its TCP socket with the GID from `-x`, and the marking must be set with `--tclass=106`.
4. No. Cumulus VX has no Spectrum ASIC, so there are no hardware queues, pause frames or ECN marks to count. You verify intent with `nv show` and marking with `tcpdump`. Data-plane counters must be validated on hardware.
5. The RoCE MTU is one of the InfiniBand sizes (256, 512, 1024, 2048, 4096), and the whole RoCEv2 packet (Ethernet, IP, UDP, BTH headers plus payload) must fit in the Ethernet MTU. 2048 does not fit in 1500, so the largest valid value is 1024. With a 9000-byte MTU it becomes 4096 (Lab 9).

Lab 4 — Adaptive Routing, ECMP and Telemetry Configuration

Lab at a glance

Item	Detail
Book chapters	Chapter 6 (Adaptive routing, performance isolation and telemetry) · Chapter 10 (monitoring) · Chapter 2 (Clos fabrics and ECMP)
Exam objectives	2.2 Enable QoS, ECN, PFC, adaptive routing and telemetry · supports 2.5 (diagnose congestion)
Time	About 60 minutes
Air resources used	hxb-leaf-r1 and hxb-leaf-r3 (configuration), hxb-spine01 and hxb-spine02 (path observation), oob-mgmt-server (jump host, telemetry destination)
Prerequisite labs	Lab 2 (eBGP unnumbered underlay with loopbacks advertised). Lab 3 is recommended but not required

Objectives

By the end of this lab you will be able to:

- Enable adaptive routing (AR) globally and on each uplink with NVUE, and verify it, while being clear about what VX can and cannot do with it.
- Read and change the ECMP hash configuration (`nv show system forwarding`, hash fields, hash seed) and know where `cl-ecmpcalc` fits.
- Observe ECMP path selection in the Air underlay by varying the source port of a probe and watching it move between the two spines.
- Configure OTLP telemetry export and interface, buffer and histogram statistics with the commands from Chapter 6.
- State, for each feature, whether Air gives you a configuration-level result or a data-plane result.

Background

Hash-based ECMP pins each flow to one path. For the few, huge, long-lived RoCE flows of AI training, that means collisions: two elephant flows share one uplink while another sits idle. Spectrum-X **adaptive routing** solves this by choosing the egress port **per packet** from live port load, and relies on the SuperNIC to put packets back in order at the receiver. On Cumulus Linux it is enabled globally with `nv set router adaptive-routing state enabled` and then per eligible Layer-3 uplink. AR applies to RoCEv2 unicast in the default VRF; everything else still uses ECMP hashing, which is why the hash settings still matter.

Telemetry is the other half of objective 2.2. Counters averaged over seconds hide microbursts, so Spectrum switches add **histograms** (distributions of queue occupancy, latency and counters), **high-frequency telemetry** (HFT, Spectrum-4) and **OTLP export** to an OpenTelemetry collector over gRPC.

This lab is honest about Air. NVIDIA's documentation lists **Cumulus VX as not supported** for adaptive routing, and histograms, HFT and OTLP export need Spectrum-2 or later (HFT needs Spectrum-4). On VX you can type, diff and in many cases apply the configuration, and read it back with `nv show`. You cannot see packets sprayed or histograms filled. ECMP, on the other hand, is real in Air: VX forwards in the Linux kernel, which load-shares across the two spines, so you can watch flows move between paths.

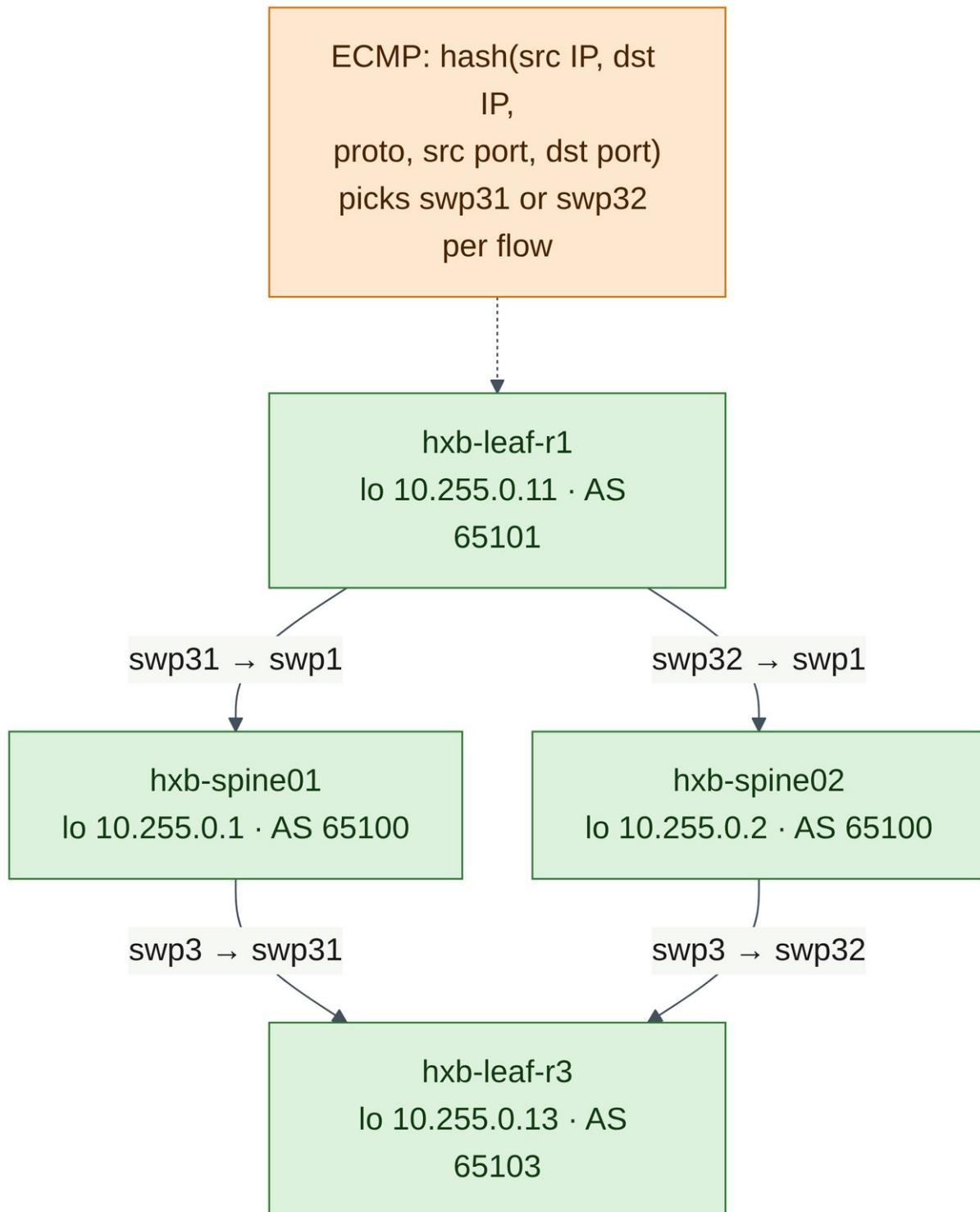


Figure 4.1: Two equal-cost paths from hxb-leaf-r1 to hxb-leaf-r3

Each leaf has two uplinks, swp31 to hxb-spine01 and swp32 to hxb-spine02, so the route to 10.255.0.13 has two next hops. ECMP picks one per flow by hashing header fields. Adaptive routing (hardware only) would pick per packet by load.

Step-by-step

Task 1 — Confirm the two equal-cost paths

1. Log in to hxb-leaf-r1 and check the route to hxb-leaf-r3's loopback.

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-leaf-r1
cumulus@hxb-leaf-r1:~$ nv show vrf default router rib ipv4 route 10.255.0.13/32
```

```
cumulus@hxb-leaf-r1:mgmt:~$ ip route show 10.255.0.13
```

Expected output (illustrative):

```
10.255.0.13 nhid 62 proto bgp metric 20
    nexthop via inet6 fe80::4638:39ff:fe00:1 dev swp31 weight 1
    nexthop via inet6 fe80::4638:39ff:fe00:3 dev swp32 weight 1
```

Two next hops (IPv6 link-local, because the underlay is BGP unnumbered from Lab 2) means ECMP is in play.

Checkpoint

- The route to 10.255.0.13/32 has two next hops, one via swp31 and one via swp32.

Task 2 — Enable adaptive routing (configuration level)

1. Enable AR globally and on both uplinks. The book's Chapter 6 example uses **swp49-52**; on helix-b-air the uplinks are **swp31-32**.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set router adaptive-routing state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp31-32 router adaptive-routing state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
```

Expected output (illustrative):

```
- set:
  router:
    adaptive-routing:
      state: enabled
  interface:
    swp31:
      router:
        adaptive-routing:
          state: enabled
    swp32:
      router:
        adaptive-routing:
          state: enabled
```

2. Try to apply.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

Version note: NVIDIA documents adaptive routing as supported on Spectrum-4 at 200G and 400G, and **not supported on Cumulus VX**. Depending on your release, VX either accepts the configuration (and **nv show** displays it with no data-plane effect) or rejects the apply with a platform error (check on your release). If apply fails, record the message, run **nv config detach** to discard the pending change, and complete step 3 using **nv config diff** as your evidence of correct intent.

3. Verify.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show router adaptive-routing
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp31 router adaptive-routing
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp32 router adaptive-routing
```

Expected output (illustrative):

	operational	applied
-----	-----	-----
state	enabled	enabled
link-utilization-threshold	disabled	disabled

- Look at the one AR tunable Chapter 6 covers: the link-utilisation threshold (default 70%). Do not change it in this lab; just read the syntax.

```
nv set interface swp31 router adaptive-routing link-utilization-threshold <1-100>
nv set router adaptive-routing link-utilization-threshold enabled
```

- Repeat steps 1–3 on hxb-leaf-r3 so that both ends of the path under study carry the same intent.

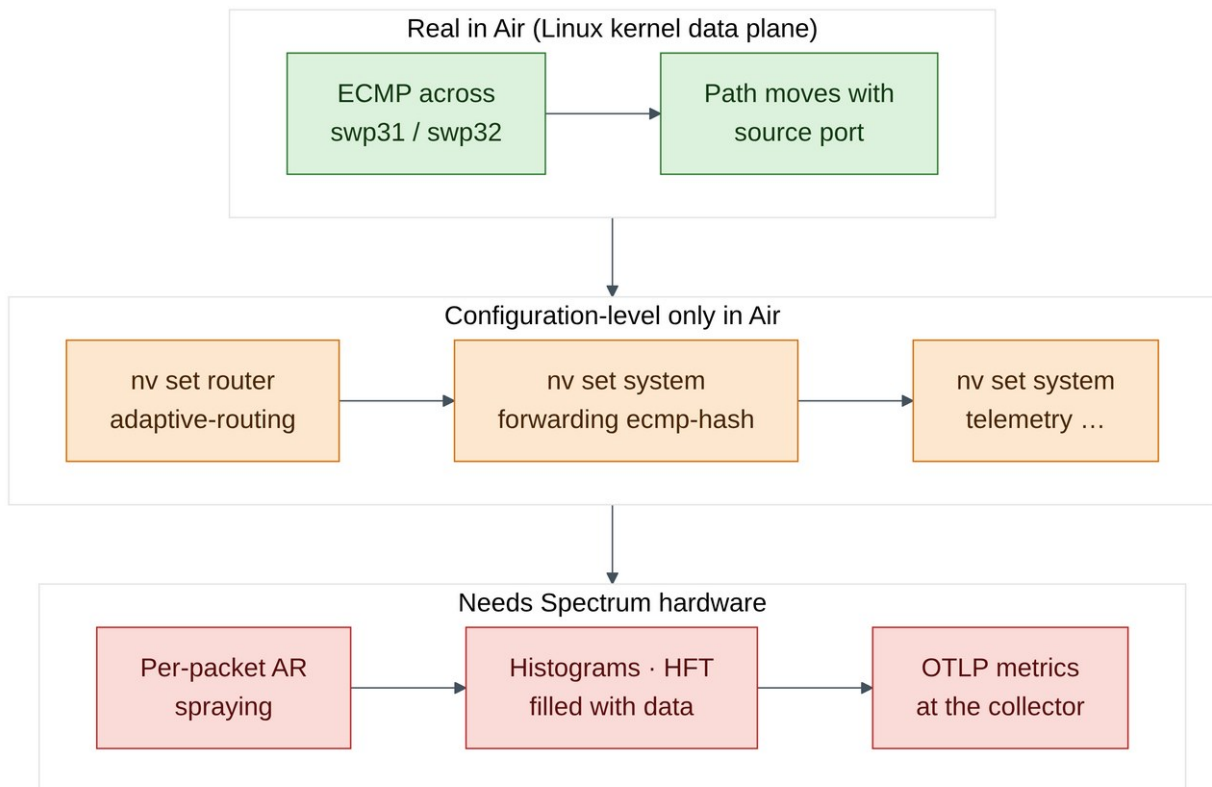


Figure 4.2: What this lab proves in Air, layer by layer

ECMP forwarding is real in Air because VX forwards in the Linux kernel. Adaptive routing, histograms and OTLP data depend on the Spectrum ASIC, so in Air you prove the configuration and leave data-plane proof to hardware.

Checkpoint

- `nv config diff` (or `nv show router adaptive-routing`) shows AR enabled globally and on swp31 and swp32, on both hxb-leaf-r1 and hxb-leaf-r3.
- You can explain why AR still needs per-interface enablement after the global command.

Task 3 — Inspect and tune ECMP hashing

- Read the forwarding and hash settings.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show system forwarding
cumulus@hxb-leaf-r1:mgmt:~$ nv show system forwarding ecmp-hash
```

Expected output (illustrative):

```
operational  applied
```

```

-----
destination-ip    enabled        on
destination-port  enabled        on
ip-protocol      enabled        on
source-ip        enabled        on
source-port      enabled        on
...

```

2. Set a unique hash seed on this leaf. Chapter 6 explains why: identical seeds on every tier make leaf and spine take correlated decisions (polarisation).

```

cumulus@hxb-leaf-r1:mgmt:~$ nv set system forwarding hash-seed 50
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r1:mgmt:~$ nv show system forwarding | grep -i seed

```

Warning: On Spectrum hardware these settings program the ASIC hash, through `switchd`. On VX there is no ASIC, so they do **not** change how the Linux kernel hashes the traffic you observe in Task 4. The kernel's own policy is controlled by `net.ipv4.fib_multipath_hash_policy`. Keep that distinction clear.

3. Try the path calculator.

```

cumulus@hxb-leaf-r1:mgmt:~$ sudo cl-ecmpcalc -i swp1 -s 10.255.0.11 -d 10.255.0.13 -p udp --sport
49152 --dport 4791

```

Expected output: On Spectrum hardware, `cl-ecmpcalc` prints the egress interface the ASIC hash would choose for that flow. On VX it is either absent or fails because there is no ASIC to query (check on your release). Record which one you see. The book's form of the command, with every field (interface, source, destination, protocol, ports) supplied, is what matters for the exam.

Checkpoint

- You can list the ECMP hash fields and name the commands that remove L4 ports from the hash (`nv set system forwarding ecmp-hash source-port off / destination-port off`) and set the seed.
- You know which tool predicts a flow's path on hardware.

Task 4 — Watch ECMP move flows between spines

On VX the kernel does the forwarding, so you can see real ECMP decisions. Your shell on a Cumulus switch runs in the management VRF (note `:mgmt:` in the prompt), so run underlay tests in the **default** VRF with `ip vrf exec default`.

1. Check that the kernel includes Layer-4 ports in its multipath hash.

```

cumulus@hxb-leaf-r1:mgmt:~$ sysctl net.ipv4.fib_multipath_hash_policy

```

Expected output: `net.ipv4.fib_multipath_hash_policy = 1` (L3 + L4). If it shows `0` (L3 only), every flow between the same two loopbacks takes the same path. Set it to 1 for this lab only: `sudo sysctl -w net.ipv4.fib_multipath_hash_policy=1` (it reverts on reboot).

2. Ask the kernel which next hop it would use for several flows that differ only by source port. The destination port is 4791, as for RoCEv2.

```

cumulus@hxb-leaf-r1:mgmt:~$ for sp in 49152 49153 49154 49155 49156 49157; do
> echo -n "sport $sp: "
> ip route get 10.255.0.13 from 10.255.0.11 ipproto udp sport $sp dport 4791 | grep -o 'dev
swp3[12]'

```

```
> done
```

Expected output (illustrative):

```
sport 49152: dev swp31
sport 49153: dev swp32
sport 49154: dev swp32
sport 49155: dev swp31
sport 49156: dev swp31
sport 49157: dev swp32
```

3. Confirm with real probes. **tracert** **-U** sends UDP to one fixed destination port (instead of incrementing it per probe), and **--sport** fixes the source port, so each run is one "flow".

```
cumulus@hxb-leaf-r1:mgmt:~$ for sp in 49152 49153 49154 49155; do
> sudo ip vrf exec default traceroute -n -q 1 -U -p 4791 --sport=$sp -s 10.255.0.11 10.255.0.13
| tail -2
> done
```

Expected output (illustrative):

```
1 10.255.0.1 0.901 ms
2 10.255.0.13 1.412 ms
1 10.255.0.2 0.874 ms
2 10.255.0.13 1.380 ms
1 10.255.0.2 0.911 ms
2 10.255.0.13 1.502 ms
1 10.255.0.1 0.820 ms
2 10.255.0.13 1.355 ms
```

Hop 1 is the spine's loopback (unnumbered interfaces borrow it), so **10.255.0.1** means hxb-spine01 and **10.255.0.2** means hxb-spine02. Different source ports, different spines: that is per-flow ECMP. Every probe of one run takes the same path, because the 5-tuple does not change. That is exactly why a single elephant RoCE flow can never use more than one uplink under ECMP.

Field note: If **tracert** is not installed or does not recognise **--sport**, use step 2 alone: **ip route get** with **sport/dport** shows the kernel's decision directly (check on your release).

4. Now remove the source port from the kernel hash and repeat step 2. With **fib_multipath_hash_policy=0**, all six flows pick the same uplink. Put the value back afterwards.

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo sysctl -w net.ipv4.fib_multipath_hash_policy=0
cumulus@hxb-leaf-r1:mgmt:~$ sudo sysctl -w net.ipv4.fib_multipath_hash_policy=1
```

This is the kernel equivalent of **nv set system forwarding ecmp-hash source-port off** on hardware: fewer hash fields, less entropy, more collisions.

Checkpoint

- Different source ports map to different uplinks (swp31/swp32) and different first-hop spines.
- Without L4 fields in the hash, all flows between the same two addresses take one path.

Task 5 — Configure telemetry: OTLP export, statistics and histograms

1. On the oob-mgmt-server, find its management address. It is the OTLP destination for this lab.

```
ubuntu@oob-mgmt-server:~$ ip -4 addr show eth0 | grep inet
```

Expected output (illustrative): `inet 192.168.200.1/24 . . .`. Use your own value in place of `192.168.200.1` below.

2. On hxb-leaf-r1, enable telemetry and OTLP export over the management VRF (the Chapter 6 command set).

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry export otlp state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry export otlp grpc destination 192.168.200.1
port 4317
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry export otlp grpc insecure enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry export vrf mgmt
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry interface-stats export state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry interface-stats sample-interval 1
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry buffer-stats export state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry histogram export state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry platform-stats export state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry router export state enabled
```

Warning: `insecure enabled` sends telemetry without TLS. It is for the lab only. In production, install the collector's CA certificate with `nv set system telemetry export otlp grpc certificate <ca-certificate>`.

3. Add histograms for the RoCE queue. An egress-buffer histogram on traffic class 3 answers "how often does the lossless queue go above X?". Add it on the server-facing and uplink ports, together with a counter histogram on the uplinks.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry histogram egress-buffer bin-min-boundary 960
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry histogram egress-buffer histogram-size 12288
cumulus@hxb-leaf-r1:mgmt:~$ nv set system telemetry histogram egress-buffer sample-interval 1024
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp1-2,swp31-32 telemetry histogram egress-buffer
traffic-class 3
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp31-32 telemetry histogram counter counter-type
tx-byte
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

Version note: OTLP export, buffer statistics and histograms need a Spectrum-2 or later ASIC (egress-buffer histograms Spectrum-1 and later), and HFT needs Spectrum-4. On VX, apply may succeed with no data produced, or be rejected for unsupported objects (check on your release). If it is rejected, `nv config detach`, then re-enter the commands without the objects named in the error and keep a note of which ones VX refused.

4. Read the configuration back.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show system telemetry
cumulus@hxb-leaf-r1:mgmt:~$ nv show system telemetry export
cumulus@hxb-leaf-r1:mgmt:~$ nv show system telemetry histogram
cumulus@hxb-leaf-r1:mgmt:~$ nv show system telemetry histogram interface
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp31 telemetry histogram egress-buffer traffic-
class 3
```

Expected output (illustrative): `export` shows OTLP **enabled**, destination 192.168.200.1 port 4317, VRF **mgmt**. **histogram interface** lists swp1, swp2, swp31 and swp32. On hardware, the per-interface histogram view shows bins with sample counts. On VX expect the configuration and empty or missing bins.

- Optional: see whether the leaf tries to reach the collector. There is no collector running, so any connection is refused, but the attempt shows the export path and VRF are right.

```
ubuntu@oob-mgmt-server:~$ sudo tcpdump -i eth0 -nn -c 5 tcp port 4317
```

Expected output: SYN packets from hxb-leaf-r1's management address to port 4317, answered by RST, if the VX build starts the exporter at all. No packets is also a valid VX result: record it.

- For reference, the HFT block from Chapter 6. Type it into `nv config diff` only if you want to see how NVUE renders it, then detach; HFT needs Spectrum-4.

```
nv set system telemetry hft sample-interval-usec 1000
nv set system telemetry hft counter tc-occupancy
nv set system telemetry hft egress-buffer traffic-class 3
nv set interface swp31-32 telemetry hft state enabled
nv set system telemetry hft duration 120
nv set system telemetry hft export state enabled
```

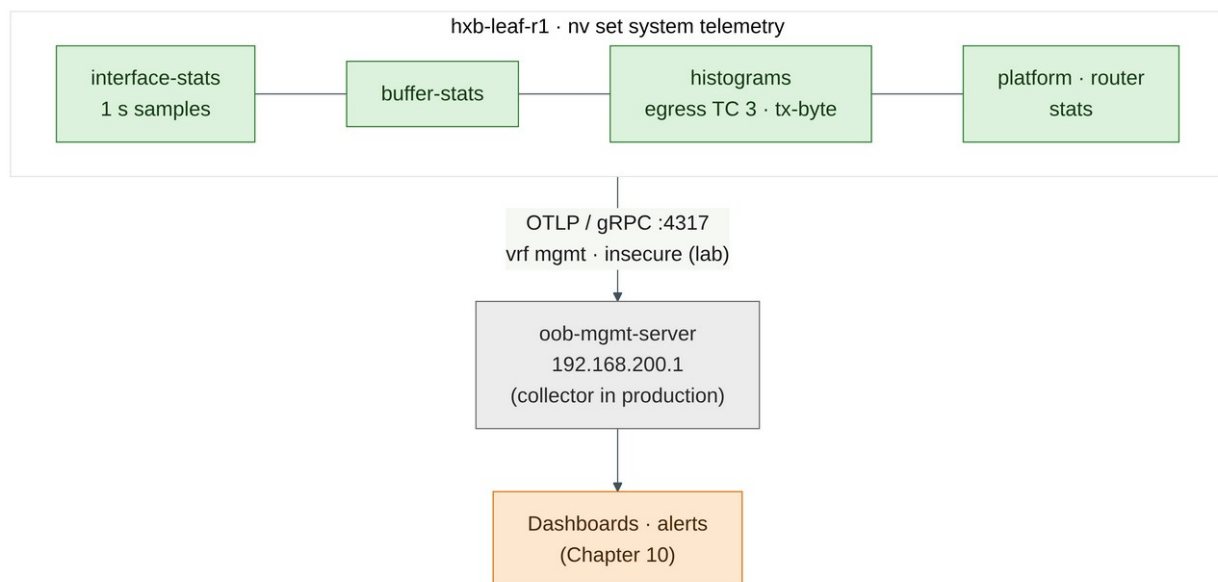


Figure 4.3: Telemetry configured on hxb-leaf-r1

Interface, buffer, platform and routing statistics plus TC 3 histograms feed the OTLP exporter, which streams over gRPC (port 4317) in the management VRF to the collector on the oob-mgmt-server. On hardware the collector would feed Prometheus or Grafana. On VX the pipeline is configured but carries little or no data.

Checkpoint

- `nv show system telemetry export` shows the OTLP destination, port and VRF you set, or you have a recorded note of what VX refused.
- You can name the four telemetry layers in Chapter 6 (counters, histograms, HFT, OTLP) and the ASIC generation each needs.

Verify

- [] The route to 10.255.0.13/32 has two next hops on hxb-leaf-r1.
- [] AR intent (global and swp31–32) is visible in `nv show router adaptive-routing` or in `nv config diff`, on hxb-leaf-r1 and hxb-leaf-r3.

- [] `nv show system forwarding` shows your hash seed and the hash fields.
- [] `ip route get ... sport <n>` or `traceroute --sport` shows different flows taking different spines.
- [] Telemetry export and TC 3 histograms are configured on hxb-leaf-r1 (or you recorded exactly which objects VX refused).
- [] You can say, for AR, ECMP, histograms and OTLP, which part Air proved and which part needs hardware.

Break and fix

Fault 1 — AR enabled on only one uplink

This is the Chapter 6 troubleshooting scenario in miniature.

Inject. On hxb-leaf-r1, disable AR on swp32 as if a typo had left it out of the range.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp32 router adaptive-routing state disabled
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

Symptoms. On hardware, RoCE traffic that the leaf sends out of swp32 is still placed by ECMP hash, so uplink utilisation is uneven and ECN marks appear on one uplink only, even though "AR is enabled".

Diagnosis. Check the global state, then every uplink. Use JSON when you script this across many leaves.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show router adaptive-routing
cumulus@hxb-leaf-r1:mgmt:~$ for i in swp31 swp32; do echo -n "$i: "; nv show interface $i router
adaptive-routing -o json | grep -o '"state": *"[a-z]*"'; done
```

Expected output (illustrative):

```
swp31: "state": "enabled"
swp32: "state": "disabled"
```

Fix.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp31-32 router adaptive-routing state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

To stop it happening again, generate leaf configuration from a template (Lab 11) and check AR on every uplink automatically.

Fault 2 — No entropy in the hash

Inject. On hxb-leaf-r1, set the kernel policy to L3 only (standing in for `ecmp-hash source-port off` and `destination-port off` on hardware).

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo sysctl -w net.ipv4.fib_multipath_hash_policy=0
```

Symptoms. Re-run Task 4 step 2. All flows between 10.255.0.11 and 10.255.0.13 take the same uplink, whatever the source port.

Diagnosis. `sysctl net.ipv4.fib_multipath_hash_policy` shows 0. On hardware, `nv show system forwarding ecmp-hash` would show the port fields disabled.

Fix. `sudo sysctl -w net.ipv4.fib_multipath_hash_policy=1`, then re-run the loop and confirm both uplinks appear. On hardware: `nv set system forwarding ecmp-hash source-port on` and `destination-port on`, then apply.

Clean-up / save state

1. Decide what to keep. The AR and telemetry configuration is harmless on VX and matches the Helix-B design, so keep it if it applied cleanly. If VX rejected any object, make sure nothing is left pending.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config detach      # only if the diff shows leftover pending
changes
```

2. If you prefer a clean baseline for Lab 5, remove the telemetry objects (keep AR and the hash seed).

```
cumulus@hxb-leaf-r1:mgmt:~$ nv unset system telemetry
cumulus@hxb-leaf-r1:mgmt:~$ nv unset interface swp1-2,swp31-32 telemetry
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

3. Confirm the kernel hash policy is back to **1**, then save on every switch you changed.

```
cumulus@hxb-leaf-r1:mgmt:~$ sysctl net.ipv4.fib_multipath_hash_policy
cumulus@hxb-leaf-r1:mgmt:~$ nv config save
cumulus@hxb-leaf-r3:mgmt:~$ nv config save
```

4. Sleep the simulation from the Air UI if you are stopping here.

Exam tie-in

- **2.2 (AR):** `nv set router adaptive-routing state enabled` (global) and `nv set interface <swp> router adaptive-routing state enabled` (per uplink), verified with `nv show router adaptive-routing` and `nv show interface <swp> router adaptive-routing`. AR applies to RoCEv2 unicast on L3 uplinks in the default VRF, and needs SuperNIC reordering.
- **2.2 (telemetry):** OTLP export (`nv set system telemetry export otlp ...`, gRPC destination and port, VRF), statistics (`interface-stats`, `buffer-stats`, `histogram`, `platform-stats`, `router`), histograms per interface and TC, HFT for microbursts on Spectrum-4.
- **2.2 and 2.5 (ECMP):** ECMP is per flow. Hash fields, `hash-seed` and `cl-ecmpcalc` still matter for non-RoCE traffic and for any RoCE traffic AR doesn't handle.
- **Exam trap:** AR is not "a better hash". It does not hash RoCE at all; it chooses per packet by load.

Review questions

1. After `nv set router adaptive-routing state enabled` and `nv config apply`, RoCE on uplink swp32 is still placed by hash. What was missed?
2. Why does changing `nv set system forwarding hash-seed` on VX not change the paths you observed with `ip route get`?
3. Four `traceroute` runs with different `--sport` values show first hops 10.255.0.1, 10.255.0.2, 10.255.0.2, 10.255.0.1. What does this show, and why does each individual run stay on one spine?
4. Which NVUE settings stream interface statistics to an OpenTelemetry collector at 10.10.0.50 over the management VRF?
5. A 1-second interface average shows 30% utilisation on a leaf uplink, yet ECN marks keep appearing. Which telemetry feature would show why, and on which ASIC generation?

Answers

1. Per-interface enablement: `nv set interface swp32 router adaptive-routing state enabled` (usually for all uplinks, e.g. `swp31-32`), then apply. The global command alone does not enable AR on any port.
2. On VX there is no ASIC. The NVUE ECMP settings program the Spectrum hash through `switchd`, while VX forwards in the Linux kernel, whose hash is governed by `net.ipv4.fib_multipath_hash_policy`.
3. It shows per-flow ECMP across the two spines: the source port is part of the hash, so different flows take different paths. Within one run the 5-tuple is constant (`-U -p 4791 --sport=<n>`), so every probe hashes to the same next hop.
4. `nv set system telemetry export otlp state enabled`, `nv set system telemetry export otlp grpc destination 10.10.0.50 port 4317`, `nv set system telemetry export vrf mgmt`, `nv set system telemetry interface-stats export state enabled` (plus a certificate, or `insecure enabled` in a lab), then `nv config apply`.
5. High-frequency telemetry (HFT), sampling `tc-occupancy` and `tx-byte` at sub-millisecond intervals, reveals the microbursts that the average hides. HFT needs Spectrum-4 or later and benefits from PTP time.

Lab 5 — BGP-EVPN Multi-Tenancy

Lab at a glance

Item	Detail
Book chapters	Chapter 7 (Multi-Tenancy with BGP-EVPN) · Chapter 4 (NVUE, underlay) · Chapter 10 (NetQ EVPN checks, concept only)
Exam objectives	2.3 Configure multi-tenancy BGP-EVPN to isolate tenant workloads · supports 2.6 (validation) and 5.3 (verification)
Time	About 120 minutes
Air resources used	hxb-spine01-02, hxb-leaf-r1-r4, hxb-gpu01-04, oob-mgmt-server (hxa-ibsim can stay idle)
Prerequisite labs	Lab 0 (setup), Lab 1 (NVUE), Lab 2 (eBGP unnumbered underlay with loopbacks advertised). Lab 3's temporary VLAN 100 must be removed (Task 1)

Objectives

By the end of this lab you will be able to:

1. Enable EVPN on a Cumulus Linux 5.x fabric: VTEP source address, global EVPN state and the **l2vpn-evpn** address family on leaves and spines.
2. Build two isolated tenants, **AURORA** (L3 VNI 104001) and **BOREALIS** (L3 VNI 104002), each with two L2 VNIs.
3. Configure a **distributed anycast gateway** (VRR **.1**) in every tenant subnet with the Cumulus Linux 5.15+ **ip v4** syntax.
4. Configure the four servers with rail-aligned addressing and per-NIC policy routing using netplan.
5. Verify EVPN with NVUE and vtysh (VNIs, type-2 and type-5 routes, MAC tables), prove bridged and routed (symmetric IRB) reachability, and prove that AURORA cannot reach BOREALIS.
6. Diagnose and fix two classic EVPN faults: a mismatched L2 VNI and a missing L3 VNI.

Background

In Lab 2 you built a routed underlay: eBGP unnumbered on **swp31–swp32**, with every leaf loopback (10.255.0.11–.14) reachable from every other leaf. That underlay knows nothing about tenants. In this lab each leaf becomes a **VTEP** (VXLAN tunnel endpoint) using its loopback as the tunnel source, and **BGP-EVPN** runs over the same eBGP sessions to tell every VTEP which MAC addresses, host IPs and prefixes live behind which other VTEP (book Section 7.2).

Each tenant gets a **VRF** mapped to an **L3 VNI**, and each tenant subnet gets a VLAN mapped to an **L2 VNI**. With **symmetric IRB**, the ingress leaf routes a packet into the tenant's L3 VNI, and the egress leaf routes it out of the L3 VNI into the destination VLAN. A leaf therefore needs only the VLANs it serves locally, plus the L3 VNI (book Section 7.3). Every leaf answers on the same gateway address and MAC (the **anycast gateway**, implemented as VRR), so a host's gateway is always its own leaf.

The helix-b-air topology is **rail-aligned**, as a real GPU cluster would be. Each "GPU server" has one NIC on an odd rail leaf (r1 or r3) and one on an even rail leaf (r2 or r4). VLAN 110 (AURORA) and VLAN 210 (BOREALIS) live on leaves r1 and r3, while VLAN 111 and VLAN 211 live on leaves r2 and r4. So traffic between two servers on the **same rail** is bridged across the fabric in an L2 VNI, and traffic **between rails** is routed through the tenant's L3 VNI. You will see both.

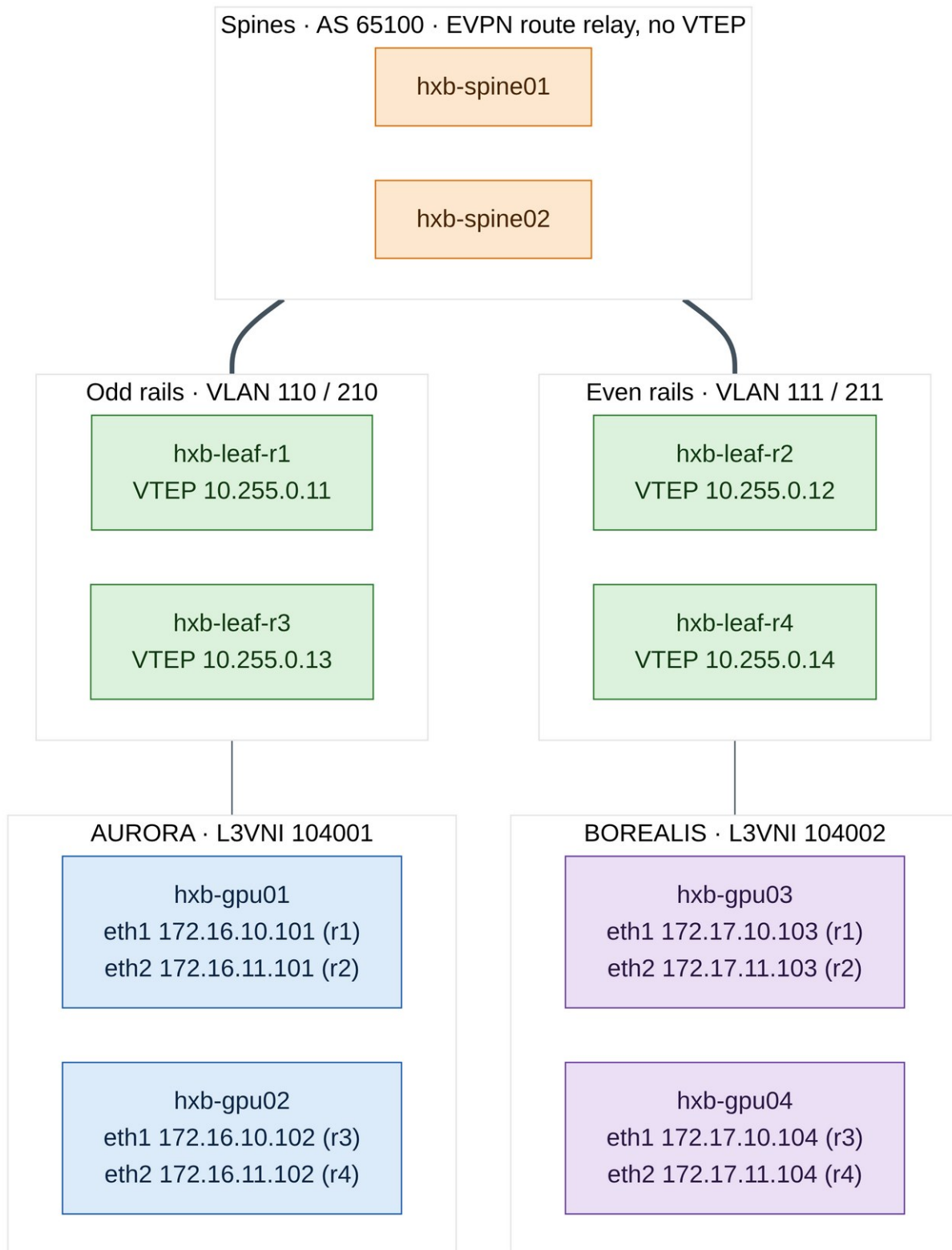


Figure 5.1: Lab 5 tenant plan on helix-b-air

Each server has one NIC on an odd rail (VLAN 110 or 210) and one on an even rail (VLAN 111 or 211). AURORA (green) and BOREALIS (purple) share every leaf and spine but live in separate VRFs with separate L3 VNIs. Every leaf is a VTEP on its loopback; the spines only relay EVPN routes.

The tenant plan used throughout this lab:

Tenant	VRF / L3 VNI	VLAN / L2 VNI	Subnet	Anycast gateway	Leaves	Leaf ports
AURORA	AURORA / 104001	110 / 10110	172.16.10.0/24	172.16.10.1	r1, r3	swp1

Tenant	VRF / L3 VNI	VLAN / L2 VNI	Subnet	Anycast gateway	Leaves	Leaf ports
AURORA	AURORA / 104001	111 / 10111	172.16.11.0/24	172.16.11.1	r2, r4	swp1
BOREALIS	BOREALIS / 104002	210 / 10210	172.17.10.0/24	172.17.10.1	r1, r3	swp2
BOREALIS	BOREALIS / 104002	211 / 10211	172.17.11.0/24	172.17.11.1	r2, r4	swp2

Each SVI also has a **unique** address per leaf, using the leaf's loopback last octet: **.11** on leaf-r1, **.12** on leaf-r2, **.13** on leaf-r3 and **.14** on leaf-r4 (for example 172.16.10.11 on leaf-r1 and 172.16.10.13 on leaf-r3). The VRR MAC is **00:00:5e:00:01:01** on every leaf, as in the book.

Version note: Your switches run Cumulus Linux 5.16.1. From Cumulus Linux 5.15, NVUE moves SVI addressing under an `ipv4` object and VRF membership directly under the interface: `nv set interface vlan110 vrf AURORA, ... ipv4 address ..., ... ipv4 vrr address ..., ... ipv4 vrr mac-address ...` and `... ipv4 vrr state enabled`. The book's Chapter 7 shows the earlier 5.x form (`... ip vrf AURORA, ... ip address ..., ... ip vrr ... state up`). The concepts are identical. If a command is rejected on your release, press **Tab** after `nv set interface vlan110` to see which form it expects, and check the Cumulus Linux user guide for your release.

Step-by-step

Task 1 — Pre-checks and Lab 3 clean-up

- From the oob-mgmt-server, log in to hxb-leaf-r1 and confirm the underlay is healthy. Every leaf should have two Established sessions, one per spine.

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-leaf-r1
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show bgp ipv4 unicast summary"
cumulus@hxb-leaf-r1:mgmt:~$ ip route show 10.255.0.13
```

Expected output (illustrative):

```
Neighbor      V      AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  State/PfxRcd
PfxSnt Desc
hxb-spine01(swp31) 4    65100    412     409      0     0     0 03:21:07          5
6 N/A
hxb-spine02(swp32) 4    65100    410     409      0     0     0 03:21:05          5
6 N/A

10.255.0.13 nhid 38 proto bgp metric 20
    nexthop via inet6 fe80::4ab0:2dff:fe11:2201 dev swp31 weight 1
    nexthop via inet6 fe80::4ab0:2dff:fe8c:9a02 dev swp32 weight 1
```

Checkpoint:

- Both uplink sessions are Established on every leaf (repeat on r2-r4).
 - Each remote leaf loopback is reachable over **two** next hops (ECMP through both spines). VXLAN tunnels ride on these loopback routes, so do not continue until they are present.
- Remove Lab 3's temporary VLAN 100 from hxb-leaf-r1. The host ports will be re-assigned to tenant VLANs in Task 4.

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show bridge domain br_default vlan
cumulus@hxb-leaf-r1:mgmt:~$ nv unset interface vlan100
cumulus@hxb-leaf-r1:mgmt:~$ nv unset bridge domain br_default vlan 100
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

Checkpoint:

- `nv show bridge domain br_default vlan` no longer lists VLAN 100. (If Lab 3 left the SVI unconfigured, the `nv unset interface vlan100` line simply has nothing to remove.)
- Lab 3's QoS configuration (`nv show qos roce`) is untouched. You keep it; RoCE QoS and EVPN work together (book Section 7.5).
- 3. On hxb-gpu01 and hxb-gpu03, remove Lab 3's temporary 172.16.100.x address. List the netplan files and delete (or move aside) the file that Lab 3 created for `eth1`; Task 5 writes a new one for both NICs.

```
ubuntu@hxb-gpu01:~$ ls /etc/netplan/
ubuntu@hxb-gpu01:~$ sudo grep -l "172.16.100" /etc/netplan/*.yaml
ubuntu@hxb-gpu01:~$ sudo mv /etc/netplan/<lab3-file>.yaml /root/
ubuntu@hxb-gpu01:~$ sudo ip addr flush dev eth1
```

Checkpoint:

- `ip -br addr show eth1` shows no IPv4 address on hxb-gpu01 and hxb-gpu03.
- The file that configures `eth0` (the OOB management NIC, usually `50-cloud-init.yaml`) is still in place. Do not touch it, or you will lose SSH access from the oob-mgmt-server.

Task 2 — Enable EVPN on the spines

The spines are not VTEPs. They only need the `l2vpn-evpn` address family on their leaf-facing sessions so that they relay EVPN routes between leaves (book Chapter 7, Q1). Configure **both** spines.

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-spine01
cumulus@hxb-spine01:mgmt:~$ nv set evpn state enabled
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp address-family l2vpn-evpn state enabled
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp neighbor swp1 address-family l2vpn-evpn state enabled
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp neighbor swp2 address-family l2vpn-evpn state enabled
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp neighbor swp3 address-family l2vpn-evpn state enabled
cumulus@hxb-spine01:mgmt:~$ nv set vrf default router bgp neighbor swp4 address-family l2vpn-evpn state enabled
cumulus@hxb-spine01:mgmt:~$ nv config diff
cumulus@hxb-spine01:mgmt:~$ nv config apply -y
```

Repeat exactly the same commands on hxb-spine02.

Version note: Some Cumulus Linux releases enable EVPN route relay on the spines with the address family alone, without `nv set evpn state enabled`. Setting it is harmless on a spine because the spine has no VNIs and no `nve vxlan source`, so it never becomes a VTEP. If NVUE rejects it on your release, leave it out.

Checkpoint:

- `nv config diff` showed only the `evpn` and `l2vpn-evpn` changes, with no change to the IPv4 underlay.
- `nv show vrf default router bgp neighbor swp1 address-family` lists `l2vpn-evpn` as enabled (the EVPN sessions only come up once the leaves are configured in Task 3).

Task 3 — Make every leaf a VTEP

On each leaf, enable EVPN, set the VTEP source to the leaf's loopback and activate the `l2vpn-evpn` address family on both uplinks. Start with hxb-leaf-r1.

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-leaf-r1
cumulus@hxb-leaf-r1:mgmt:~$ nv set evpn state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set nve vxlan source address 10.255.0.11
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf default router bgp address-family l2vpn-evpn state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf default router bgp neighbor swp31 address-family l2vpn-evpn state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf default router bgp neighbor swp32 address-family l2vpn-evpn state enabled
```

Do not apply yet: you will apply the complete leaf configuration at the end of Task 4. The only per-leaf value in this task is the VTEP source address:

Leaf	<code>nv set nve vxlan source address ...</code>
hxb-leaf-r1	10.255.0.11
hxb-leaf-r2	10.255.0.12
hxb-leaf-r3	10.255.0.13
hxb-leaf-r4	10.255.0.14

Checkpoint:

- `nv config diff` on each leaf shows the four EVPN/NVE lines and the two neighbour address-family lines.
- The VTEP source is the leaf's own /32 loopback from Lab 2, which every other leaf can already reach (Task 1).

Task 4 — Tenant VRFs, VNIs, anycast gateways and access ports

The complete tenant configuration for each leaf follows. Enter it on top of Task 3's pending changes, check the diff, then apply. Note which VLANs each leaf carries: a leaf only needs its **local** VLANs plus both L3 VNIs (symmetric IRB).

1. **hxb-leaf-r1** (VLANs 110 and 210, SVI host part .11):

```
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf AURORA evpn vni 104001
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf BOREALIS evpn vni 104002
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf AURORA router bgp address-family ipv4-unicast redistribute
connected state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf AURORA router bgp address-family ipv4-unicast route-export
to-evpn state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf BOREALIS router bgp address-family ipv4-unicast
redistribute connected state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set vrf BOREALIS router bgp address-family ipv4-unicast route-
export to-evpn state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set bridge domain br_default vlan 110 vni 10110
cumulus@hxb-leaf-r1:mgmt:~$ nv set bridge domain br_default vlan 210 vni 10210
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan110 vrf AURORA
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan110 ipv4 address 172.16.10.11/24
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan110 ipv4 vrr address 172.16.10.1/24
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan110 ipv4 vrr mac-address 00:00:5e:00:01:01
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan110 ipv4 vrr state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan210 vrf BOREALIS
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan210 ipv4 address 172.17.10.11/24
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan210 ipv4 vrr address 172.17.10.1/24
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan210 ipv4 vrr mac-address 00:00:5e:00:01:01
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface vlan210 ipv4 vrr state enabled
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp1 bridge domain br_default access 110
cumulus@hxb-leaf-r1:mgmt:~$ nv set interface swp2 bridge domain br_default access 210
```

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

What each group does:

Commands	Purpose	Book reference
<code>vrf ... evpn vni 104001/104002</code>	Creates the tenant VRF and maps it to its L3 VNI . NVUE allocates the hidden L3 VNI VLAN automatically	Section 7.4 Step 2
<code>redistribute connected + route-export to-evpn</code>	Advertises each tenant subnet as an EVPN type-5 route	Step 5
<code>bridge domain br_default vlan ... vni ...</code>	Creates the VLAN and maps it to its L2 VNI on the single VXLAN device	Step 3
<code>interface vlanX vrf .../ipv4 address/ipv4 vrr ...</code>	SVI in the tenant VRF with a unique address and the anycast gateway .1	Step 4
<code>interface swpX bridge domain br_default access ...</code>	Host-facing access port in the tenant VLAN	Step 6

Warning: The VRR MAC must be identical on every leaf for the same gateway. A different MAC on one leaf breaks the anycast gateway after a host moves or when ARP caches age (book Chapter 7, common mistakes). Copy the MAC exactly.

2. **hxb-leaf-r2** (VLANs 111 and 211, SVI host part **.12**). Run the six **vrf** lines exactly as on leaf-r1, then:

```
cumulus@hxb-leaf-r2:mgmt:~$ nv set bridge domain br_default vlan 111 vni 10111
cumulus@hxb-leaf-r2:mgmt:~$ nv set bridge domain br_default vlan 211 vni 10211
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan111 vrf AURORA
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan111 ipv4 address 172.16.11.12/24
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan111 ipv4 vrr address 172.16.11.1/24
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan111 ipv4 vrr mac-address 00:00:5e:00:01:01
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan111 ipv4 vrr state enabled
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan211 vrf BOREALIS
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan211 ipv4 address 172.17.11.12/24
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan211 ipv4 vrr address 172.17.11.1/24
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan211 ipv4 vrr mac-address 00:00:5e:00:01:01
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface vlan211 ipv4 vrr state enabled
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp1 bridge domain br_default access 111
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp2 bridge domain br_default access 211
cumulus@hxb-leaf-r2:mgmt:~$ nv config diff
cumulus@hxb-leaf-r2:mgmt:~$ nv config apply -y
```

3. **hxb-leaf-r3** is the same as leaf-r1 (VLANs 110 and 210) with SVI addresses **.13: 172.16.10.13/24** on vlan110 and **172.17.10.13/24** on vlan210. The VRR addresses and MAC do not change.

4. **hxb-leaf-r4** is the same as leaf-r2 (VLANs 111 and 211) with SVI addresses **.14: 172.16.11.14/24** on vlan111 and **172.17.11.14/24** on vlan211.

Summary of per-leaf values:

Leaf	VTEP	VLAN → VNI	Unique SVI addresses	swp1 access	swp2 access
hxb-leaf-r1	10.255.0.11	110 → 10110 · 210 → 10210	172.16.10.11 · 172.17.10.11	110	210
hxb-leaf-r2	10.255.0.12	111 → 10111 · 211 → 10211	172.16.11.12 · 172.17.11.12	111	211
hxb-leaf-r3	10.255.0.13	110 → 10110 · 210 → 10210	172.16.10.13 · 172.17.10.13	110	210
hxb-leaf-r4	10.255.0.14	111 → 10111 · 211 → 10211	172.16.11.14 · 172.17.11.14	111	211

Field note: Four leaves typed by hand is exactly where one wrong digit creeps in. Break/fix exercise 1 is that digit. In Lab 11 you generate this same configuration from one NVUE template, and in Lab 12 from Ansible variables, so every leaf is built from one source.

Checkpoint (on each leaf):

- `nv config apply` completed without errors.
- `nv show vrf AURORA evpn` shows VNI 104001 and `nv show vrf BOREALIS evpn` shows 104002.
- `nv show bridge domain br_default vlan` lists only the two local tenant VLANs, each with its VNI.
- `nv show interface vlan110` (or `vlan111`) shows the unique address, the VRR address `.1` and VRF AURORA.

Task 5 — Configure the servers with netplan

Each server has one NIC in each of its tenant's two subnets. Both subnets are therefore directly connected, and by default Linux reaches the other rail's subnet **directly through the other NIC** (bridged in its L2 VNI), never through the gateway. To exercise routing across rails through the L3 VNI, as a real rail-aligned GPU node does, each NIC gets its own **routing table** and a **source-based rule**:

- Traffic sourced from the eth1 address uses table 101, which reaches the rest of the tenant (`172.16.0.0/16` or `172.17.0.0/16`) through eth1's anycast gateway.
- Traffic sourced from the eth2 address uses table 102, through eth2's gateway.
- A higher-priority rule keeps each NIC's own subnet on the main table, so same-subnet traffic stays bridged.
- The main table also gets a tenant supernet route via eth1's gateway, so any future subnet of the same tenant is reachable. The OOB default route on `eth0` is untouched.

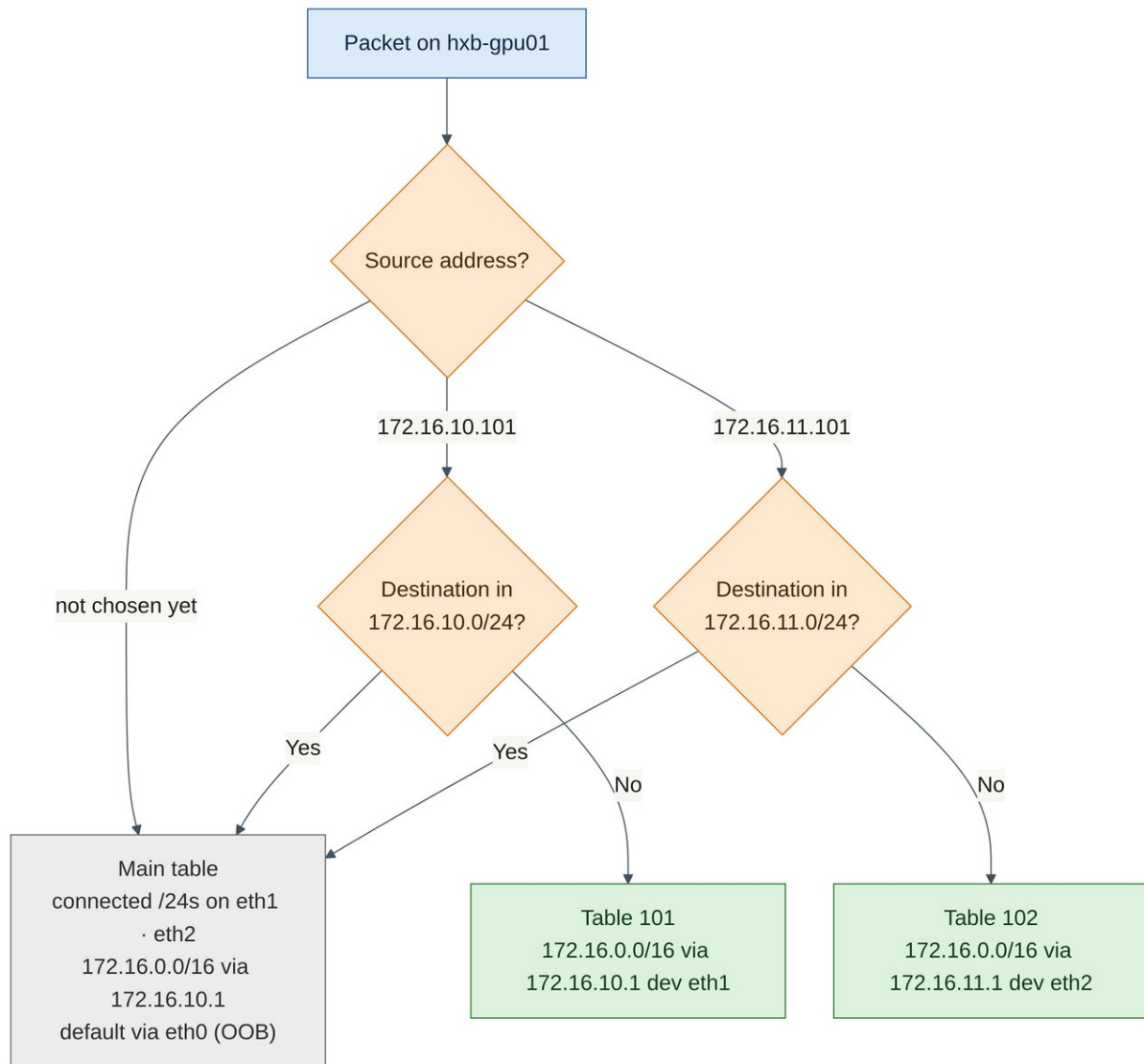


Figure 5.2: Per-NIC routing on hxb-gpu01

Source-based rules pick a routing table per NIC. Traffic from 172.16.10.101 leaves eth1 towards leaf-r1's anycast gateway, so a packet to the other rail (172.16.11.0/24) is routed through L3 VNI 104001. Traffic without a chosen source uses the main table and reaches 172.16.11.0/24 directly on eth2.

1. On hxb-gpu01, create `/etc/netplan/60-fabric.yaml`:

```
ubuntu@oob-mgmt-server:~$ ssh ubuntu@hxb-gpu01
ubuntu@hxb-gpu01:~$ sudo nano /etc/netplan/60-fabric.yaml
```

```
network:
  version: 2
  ethernet:
    eth1:
      mtu: 9000
      addresses: [172.16.10.101/24]
      routes:
        - to: 172.16.0.0/16          # rest of AURORA, main table
          via: 172.16.10.1
          metric: 200
        - to: 172.16.0.0/16          # rest of AURORA, eth1-sourced traffic
          via: 172.16.10.1
          on-link: true
```

```

    table: 101
  routing-policy:
    - from: 172.16.10.101
      to: 172.16.10.0/24
      table: 254                # own subnet stays on the main table (bridged)
      priority: 100
    - from: 172.16.10.101
      table: 101
      priority: 110
  eth2:
    mtu: 9000
    addresses: [172.16.11.101/24]
    routes:
      - to: 172.16.0.0/16
        via: 172.16.11.1
        on-link: true
        table: 102
    routing-policy:
      - from: 172.16.11.101
        to: 172.16.11.0/24
        table: 254
        priority: 100
      - from: 172.16.11.101
        table: 102
        priority: 110

```

2. Apply it:

```

ubuntu@hxb-gpu01:~$ sudo chmod 600 /etc/netplan/60-fabric.yaml
ubuntu@hxb-gpu01:~$ sudo netplan try
ubuntu@hxb-gpu01:~$ sudo netplan apply
ubuntu@hxb-gpu01:~$ ip -br addr show eth1; ip -br addr show eth2
ubuntu@hxb-gpu01:~$ ip rule show
ubuntu@hxb-gpu01:~$ ip route show table 101

```

Expected output (illustrative):

```

eth1          UP          172.16.10.101/24 fe80::4ab0:2dff:fe31:a101/64
eth2          UP          172.16.11.101/24 fe80::4ab0:2dff:fe31:a102/64
0:            from all lookup local
100:          from 172.16.10.101 to 172.16.10.0/24 lookup main
100:          from 172.16.11.101 to 172.16.11.0/24 lookup main
110:          from 172.16.10.101 lookup 101
110:          from 172.16.11.101 lookup 102
32766:        from all lookup main
32767:        from all lookup default
172.16.0.0/16 via 172.16.10.1 dev eth1 proto static onlink

```

netplan try rolls back automatically after 120 seconds unless you press Enter, which protects you from a typo. Press Enter when it asks.

- Repeat on the other three servers, changing only the values in this table (the structure is identical; BOREALIS servers use **172.17.0.0/16** as the tenant supernet):

Server	eth1 address / gateway	eth2 address / gateway	Tenant supernet
hxb-gpu01	172.16.10.101/24 · 172.16.10.1	172.16.11.101/24 · 172.16.11.1	172.16.0.0/16
hxb-gpu02	172.16.10.102/24 · 172.16.10.1	172.16.11.102/24 · 172.16.11.1	172.16.0.0/16

Server	eth1 address / gateway	eth2 address / gateway	Tenant supernet
hxb-gpu03	172.17.10.103/24 · 172.17.10.1	172.17.11.103/24 · 172.17.11.1	172.17.0.0/16
hxb-gpu04	172.17.10.104/24 · 172.17.10.1	172.17.11.104/24 · 172.17.11.1	172.17.0.0/16

Remember to change the **from:** and **to:** values in both **routing-policy** blocks, not just the addresses.

Version note: The MTU of 9000 leaves room for the roughly 50 bytes of VXLAN overhead inside the 9216-byte fabric MTU (book Section 7.5). If Lab 2 left the uplinks at a smaller MTU, set the hosts to 1500 for now and see Lab 7. Netplan keys shown (**routes**, **on-link**, **table**, **routing-policy**, **priority**) are documented for Ubuntu 24.04's netplan; **sudo netplan try** will reject a malformed file without changing anything.

- On every server, ping the anycast gateway from **each** NIC. This populates the leaves' ARP and MAC tables, so every host is advertised in EVPN before the tests in Task 6.

```
ubuntu@hxb-gpu01:~$ ping -c 2 -I 172.16.10.101 172.16.10.1
ubuntu@hxb-gpu01:~$ ping -c 2 -I 172.16.11.101 172.16.11.1
```

Checkpoint:

- Every server reaches both of its gateways with 0% loss.
- ip rule show** on each server shows the four rules at priorities 100 and 110.
- ip route** on each server still shows the default route on **eth0**.

Task 6 — Verify the EVPN control plane

- Check the EVPN sessions and VNIs on hxb-leaf-r1:

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show bgp l2vpn evpn summary"
cumulus@hxb-leaf-r1:mgmt:~$ nv show evpn vni
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show evpn vni"
```

Expected output (illustrative):

```
BGP router identifier 10.255.0.11, local AS number 65101 vrf-id 0
Neighbor      V  AS  MsgRcvd MsgSent  Up/Down State/PfxRcd PfxSnt
hxb-spine01(swp31) 4 65100    96     88 00:12:40      18    10
hxb-spine02(swp32) 4 65100    95     88 00:12:38      18    10

VNI      Type VxLAN IF      # MACs  # ARPs  # Remote VTEPs  Tenant VRF
10110    L2   vxlan48      3       4       1              AURORA
10210    L2   vxlan48      3       4       1              BOREALIS
104001   L3   vxlan48      4       4       n/a            AURORA
104002   L3   vxlan48      4       4       n/a            BOREALIS
```

What to notice: two L2 VNIs and two L3 VNIs, each tied to the correct VRF. Each L2 VNI has exactly **one** remote VTEP, hxb-leaf-r3, because only leaf-r1 and leaf-r3 carry VLANs 110 and 210. (The book's Chapter 7 output shows 7 remote VTEPs because its full Helix-B has 8 rail leaves carrying every VLAN.) **nv show evpn vni** shows the same VNIs in NVUE format.

- Look at the type-2 (MAC/IP) and type-5 (IP prefix) routes:

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show bgp l2vpn evpn route type macip"
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show bgp l2vpn evpn route type prefix"
```

Expected output (illustrative, abbreviated):

```

Route Distinguisher: 10.255.0.13:2
*> [2]:[0]:[48]:[4a:b0:2d:32:a1:01]:[32]:[172.16.10.102]
      10.255.0.13                                0 65100 65103 i
      RT:65103:10110 RT:65103:104001 ET:8 Rmac:4a:b0:2d:0c:13:00
Route Distinguisher: 10.255.0.12:4
*> [5]:[0]:[24]:[172.16.11.0]
      10.255.0.12                                0 65100 65102 ?
      RT:65102:104001 ET:8 Rmac:4a:b0:2d:0c:12:00
Route Distinguisher: 10.255.0.14:5
*> [5]:[0]:[24]:[172.17.11.0]
      10.255.0.14                                0 65100 65104 ?
      RT:65104:104002 ET:8 Rmac:4a:b0:2d:0c:14:00

```

What to notice:

- A **type-2** route for hxb-gpu02's eth1 carries **two** route targets: the L2 VNI's (...:10110) for bridging, and the L3 VNI's (...:104001) plus the router MAC for symmetric routing.
- **Type-5** routes carry the tenant subnets with only the L3 VNI route target. AURORA prefixes carry ...:104001, BOREALIS prefixes ...:104002. These route targets are the isolation mechanism (book Section 7.2).
- The next hop of every route is the **originating leaf's VTEP loopback**, not the spine. The spines relayed the routes without changing the next hop.

Version note: Recent FRR releases also accept the numeric form, for example `show bgp l2vpn evpn route type 2` and ... `type 5`. The book uses `macip` and `prefix`, which work on all Cumulus Linux 5.x releases.

3. Check the MAC tables per VNI and the tenant routing tables:

```

cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show evpn mac vni all"
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show ip route vrf AURORA"
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show ip route vrf BOREALIS"

```

Expected output (illustrative, abbreviated):

```

VNI 10110 #MACs (local and remote) 3
MAC                Type   Flags Intf/Remote ES/VTEP          VLAN  Seq #'s
4a:b0:2d:31:a1:01  local          swp1              110    0/0
4a:b0:2d:32:a1:01  remote        10.255.0.13      0/0
00:00:5e:00:01:01  local          vlan110           110    0/0

VRF AURORA:
C>* 172.16.10.0/24 is directly connected, vlan110, 00:14:02
B>* 172.16.11.0/24 [20/0] via 10.255.0.12, vlan4024_l3 onlink, weight 1, 00:13:51
    *               via 10.255.0.14, vlan4024_l3 onlink, weight 1, 00:13:51
B>* 172.16.11.101/32 [20/0] via 10.255.0.12, vlan4024_l3 onlink, weight 1, 00:09:12
B>* 172.16.11.102/32 [20/0] via 10.255.0.14, vlan4024_l3 onlink, weight 1, 00:08:57

```

Checkpoint:

- `show evpn mac vni all` lists the local host MAC on `swp1` (or `swp2`) and the remote host MAC behind the other odd-rail leaf's VTEP.
- VRF AURORA contains **only** 172.16.x prefixes and host routes, reached through the L3 VNI interface (the auto-created L3 VNI VLAN name, such as `vlan4024_l3`, varies). VRF BOREALIS contains only 172.17.x.
- Repeat the checks on hxb-leaf-r2: its L2 VNIs are 10111 and 10211, with hxb-leaf-r4 as the remote VTEP.

Task 7 — Prove bridged and routed reachability within a tenant

1. **Same rail, same subnet (bridged in L2 VNI 10110).** From hxb-gpu01 eth1 to hxb-gpu02 eth1:

```
ubuntu@hxb-gpu01:~$ ping -c 3 -I 172.16.10.101 172.16.10.102
```

Expected output (illustrative):

```
PING 172.16.10.102 (172.16.10.102) from 172.16.10.101 : 56(84) bytes of data.
64 bytes from 172.16.10.102: icmp_seq=1 ttl=64 time=1.84 ms
64 bytes from 172.16.10.102: icmp_seq=2 ttl=64 time=1.12 ms
64 bytes from 172.16.10.102: icmp_seq=3 ttl=64 time=1.09 ms
```

TTL 64 means no router touched the packet. It crossed leaf-r1 → spine → leaf-r3 inside VXLAN VNI 10110, but it was **bridged**.

2. **Across rails, different subnet (routed in L3 VNI 104001).** From hxb-gpu01 eth1 (VLAN 110 on leaf-r1) to hxb-gpu02 eth2 (VLAN 111 on leaf-r4):

```
ubuntu@hxb-gpu01:~$ ping -c 3 -I 172.16.10.101 172.16.11.102
ubuntu@hxb-gpu01:~$ ping -c 1 -t 1 -I 172.16.10.101 172.16.11.102
```

Expected output (illustrative):

```
64 bytes from 172.16.11.102: icmp_seq=1 ttl=62 time=2.41 ms
64 bytes from 172.16.11.102: icmp_seq=2 ttl=62 time=1.37 ms
64 bytes from 172.16.11.102: icmp_seq=3 ttl=62 time=1.33 ms

From 172.16.10.11 icmp_seq=1 Time to live exceeded
```

What to notice: the reply arrives with **TTL 62**. The reply was routed **twice**, once by leaf-r4 (ingress for the reply) and once by leaf-r1 (egress). That is symmetric IRB: both VTEPs route, and the L3 VNI carries the packet between them (book Figure 7.4). The TTL-1 probe is answered by leaf-r1's own SVI address (**.11**), which proves that the first routing hop is the local leaf, the anycast gateway. The source address shown may differ on your release.

3. Compare with the default path without a chosen source:

```
ubuntu@hxb-gpu01:~$ ping -c 2 172.16.11.102
```

Expected: replies with **TTL 64**. With no source chosen, the main table sends the packet out of eth2 directly into VLAN 111, so it is bridged through leaf-r2 and leaf-r4. This is why real rail-aligned hosts use per-NIC routing: the NIC choice decides which rail carries the traffic.

4. Repeat the equivalent tests for BOREALIS from hxb-gpu03:

```
ubuntu@hxb-gpu03:~$ ping -c 3 -I 172.17.10.103 172.17.10.104
ubuntu@hxb-gpu03:~$ ping -c 3 -I 172.17.10.103 172.17.11.104
```

Checkpoint:

- Same-subnet tests succeed with TTL 64 (bridged over the L2 VNI).
- Cross-rail tests succeed with TTL 62 (routed over the L3 VNI) for both tenants.
- Large frames also pass: `ping -c 2 -M do -s 8972 -I 172.16.10.101 172.16.10.102` succeeds, which confirms that the 9216-byte underlay carries 9000-byte tenant frames plus VXLAN overhead.

Task 8 — Prove tenant isolation

A well-behaved server in AURORA has no route to BOREALIS at all, so a simple ping would leave through **eth0** and prove nothing about the fabric. To test the fabric, temporarily point hxb-gpu01 at its gateway for the BOREALIS range, as a misconfigured or hostile host might.

1. Add a temporary route and try to reach BOREALIS hosts:

```
ubuntu@hxb-gpu01:~$ sudo ip route add 172.17.0.0/16 via 172.16.10.1 dev eth1 table 101
ubuntu@hxb-gpu01:~$ ping -c 3 -I 172.16.10.101 172.17.10.103
ubuntu@hxb-gpu01:~$ ping -c 3 -I 172.16.10.101 172.17.11.104
```

Expected output (illustrative):

```
From 172.16.10.11 icmp_seq=1 Destination Net Unreachable
From 172.16.10.11 icmp_seq=2 Destination Net Unreachable
--- 172.17.10.103 ping statistics ---
3 packets transmitted, 0 received, +2 errors, 100% packet loss
```

hxb-gpu03 is attached to **the same leaf and the same physical switch** (leaf-r1 swp2), yet it is unreachable. The packet entered VRF AURORA on leaf-r1, and VRF AURORA has no route to 172.17.0.0/16. Depending on the release you may see "Destination Net Unreachable" from the leaf, or simply 100% loss.

2. Prove it on the leaf:

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show ip route vrf AURORA 172.17.10.103"
cumulus@hxb-leaf-r1:mgmt:~$ ip route show vrf AURORA | grep 172.17
cumulus@hxb-leaf-r1:mgmt:~$ nv show vrf AURORA evpn
```

Expected: % **Network not in table** and no output from the **grep**. The BOREALIS type-5 and type-2 routes reached leaf-r1 (you saw them in Task 6), but they carry BOREALIS's route targets (**...:104002**), so VRF AURORA never imported them.

3. Remove the temporary route:

```
ubuntu@hxb-gpu01:~$ sudo ip route del 172.17.0.0/16 via 172.16.10.1 dev eth1 table 101
```

Checkpoint:

- AURORA → BOREALIS fails even between hosts on the same leaf.
- **show ip route vrf AURORA** contains only 172.16.x routes, and **show ip route vrf BOREALIS** only 172.17.x routes.
- The temporary route is removed (**ip route show table 101** shows only 172.16.0.0/16).

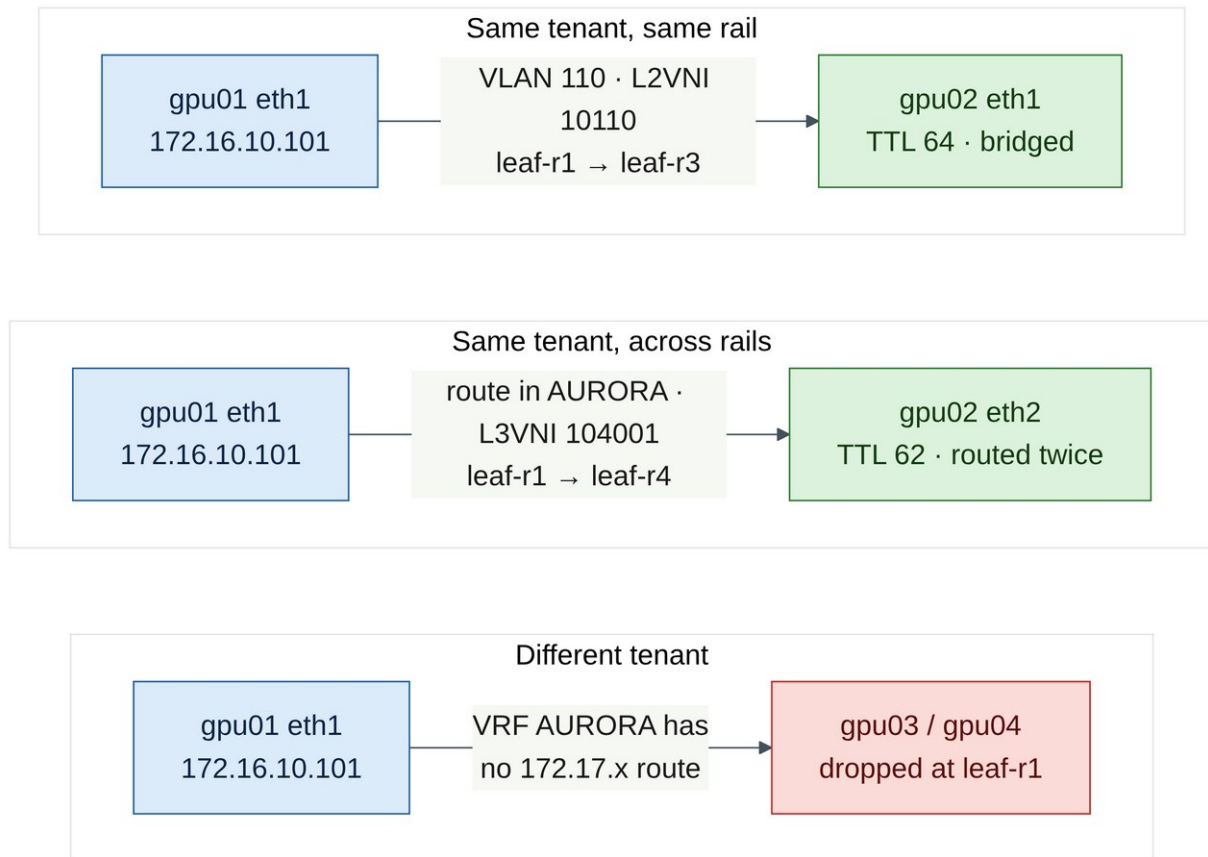


Figure 5.3: What each test in Tasks 7 and 8 proves

The same physical fabric carries three kinds of flow. Same-rail traffic is bridged in an L2 VNI (TTL 64). Cross-rail traffic in the same tenant is routed twice, through the L3 VNI (TTL 62). Traffic between tenants has no route in the source VRF and is dropped at the first leaf.

Verify

- [] Both spines have `l2vpn-evpn` enabled on swp1–swp4 and are not VTEPs (`nv show nve vxlan` shows no source address on the spines).
- [] Each leaf has `nv show nve vxlan` source = its loopback, and `sudo vtysh -c "show bgp l2vpn evpn summary"` shows two Established EVPN sessions.
- [] `nv show evpn vni` on each leaf lists its two local L2 VNIs and both L3 VNIs (104001 → AURORA, 104002 → BOREALIS).
- [] `show bgp l2vpn evpn route type macip` shows remote host MAC/IP routes with both L2 and L3 route targets; `type prefix` shows the four tenant subnets.
- [] `show evpn mac vni all` shows local and remote host MACs.
- [] Same-rail pings within a tenant succeed with TTL 64; cross-rail pings within a tenant (sourced with `-I`) succeed with TTL 62.
- [] AURORA cannot reach BOREALIS even with a forced route on the host, and each VRF contains only its own prefixes.

Break and fix

Run each fault on its own, fix it, and re-verify before starting the next.

Fault 1 — Mismatched L2 VNI on one leaf

Inject (on hxb-leaf-r3, a "typo" in the VNI for VLAN 110):

```
cumulus@hxb-leaf-r3:mgmt:~$ nv unset bridge domain br_default vlan 110 vni 10110
cumulus@hxb-leaf-r3:mgmt:~$ nv set bridge domain br_default vlan 110 vni 10119
cumulus@hxb-leaf-r3:mgmt:~$ nv config apply -y
```

Symptoms:

```
ubuntu@hxb-gpu01:~$ ping -c 3 -I 172.16.10.101 172.16.10.102      # same subnet, bridged
3 packets transmitted, 0 received, 100% packet loss
ubuntu@hxb-gpu02:~$ ping -c 2 -I 172.16.10.102 172.16.10.1      # its gateway
2 packets transmitted, 2 received, 0% packet loss
```

hxb-gpu02 still reaches its gateway: the access port, VLAN and SVI on leaf-r3 are fine. Only the stretch of VLAN 110 between leaf-r1 and leaf-r3 is broken. Routed traffic to 172.16.10.102 from the other rail may still work, because gpu02's type-2 route still carries the correct L3 VNI route target. A half-working tenant is a typical sign of a VNI mismatch.

Diagnosis:

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show evpn vni 10110"
cumulus@hxb-leaf-r3:mgmt:~$ sudo vtysh -c "show evpn vni"
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show evpn mac vni 10110"
cumulus@hxb-leaf-r3:mgmt:~$ nv show bridge domain br_default vlan
```

- On leaf-r1, VNI 10110 now has **0 remote VTEPs**: leaf-r3 no longer sends a type-3 route for 10110.
- On leaf-r3, **show evpn vni** lists **10119** instead of 10110, also with 0 remote VTEPs.
- gpu02's MAC is missing from **show evpn mac vni 10110** on leaf-r1.
- **nv show bridge domain br_default vlan** on leaf-r3 shows VLAN 110 → 10119. Compare with the tenant plan.

Fix:

```
cumulus@hxb-leaf-r3:mgmt:~$ nv unset bridge domain br_default vlan 110 vni 10119
cumulus@hxb-leaf-r3:mgmt:~$ nv set bridge domain br_default vlan 110 vni 10110
cumulus@hxb-leaf-r3:mgmt:~$ nv config apply -y
ubuntu@hxb-gpu01:~$ ping -c 3 -I 172.16.10.101 172.16.10.102
```

Checkpoint: VNI 10110 shows 1 remote VTEP on both leaf-r1 and leaf-r3, and the bridged ping succeeds with TTL 64.

Fault 2 — Missing L3 VNI mapping

This is the book's Chapter 7 troubleshooting scenario, reproduced on your fabric.

Inject (on hxb-leaf-r4, "built from an old template"):

```
cumulus@hxb-leaf-r4:mgmt:~$ nv unset vrf AURORA evpn vni 104001
cumulus@hxb-leaf-r4:mgmt:~$ nv config apply -y
```

Symptoms:

```
ubuntu@hxb-gpu01:~$ ping -c 3 -I 172.16.10.101 172.16.11.102    # cross-rail, routed
3 packets transmitted, 0 received, 100% packet loss
ubuntu@hxb-gpu01:~$ ping -c 3 -I 172.16.11.101 172.16.11.102    # same subnet on the even rail,
bridged
3 packets transmitted, 3 received, 0% packet loss
ubuntu@hxb-gpu03:~$ ping -c 3 -I 172.17.10.103 172.17.11.104    # BOREALIS cross-rail
```

```
3 packets transmitted, 3 received, 0% packet loss
```

Bridged AURORA traffic and all of BOREALIS still work. Only **routed** AURORA traffic to or from hosts behind leaf-r4 fails.

Diagnosis: follow book Figure 7.7.

```
cumulus@hxb-leaf-r4:mgmt:~$ sudo vtysh -c "show evpn vni"
cumulus@hxb-leaf-r4:mgmt:~$ nv show vrf AURORA evpn
cumulus@hxb-leaf-r4:mgmt:~$ sudo vtysh -c "show ip route vrf AURORA"
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show ip route vrf AURORA"
```

- **show evpn vni** on leaf-r4 lists L2 VNIs 10111 and 10211 and L3 VNI 104002, but **no L3 VNI 104001**.
- **nv show vrf AURORA evpn** on leaf-r4 has no VNI.
- VRF AURORA on leaf-r4 has only its connected subnet: no remote /32s or type-5 routes were imported.
- On leaf-r1, the host route to 172.16.11.102 behind leaf-r4 has disappeared (or the traffic is black-holed), because leaf-r4 no longer advertises AURORA's routed information with the L3 VNI.

Fix:

```
cumulus@hxb-leaf-r4:mgmt:~$ nv set vrf AURORA evpn vni 104001
cumulus@hxb-leaf-r4:mgmt:~$ nv config apply -y
ubuntu@hxb-gpu01:~$ ping -c 3 -I 172.16.10.101 172.16.11.102
```

Checkpoint: **show evpn vni** on leaf-r4 lists 104001 → AURORA again, and the cross-rail ping succeeds with TTL 62. In production you would also fix the template (Lab 11) and add an EVPN validation (**netq check evpn**, book Chapter 10) to the post-change checks.

Clean-up / save state

1. Confirm that all faults are fixed and that the Verify checklist passes.
2. NVUE auto-saves the applied configuration to **/etc/nvue.d/startup.yaml** by default. Save explicitly on each switch in case auto-save is off:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config save
```

3. Keep this configuration. Labs 7, 9, 10, 12 and 13 build on the AURORA and BOREALIS tenants and the server netplan files.
4. In the Air console, take a checkpoint of the simulation (Lab 6 covers checkpoints in detail), then put the simulation to sleep to stop spending compute-hour credits.

Exam tie-in

- **2.3 (multi-tenancy BGP-EVPN):** you configured every building block the objective names: tenant VRFs mapped to L3 VNIs (**nv set vrf <T> evpn vni**), VLAN-to-L2 VNI mappings, the VTEP source, the **l2vpn-evpn** address family, anycast gateways and type-5 export.
- **Isolation mechanism:** route targets decide which VRF imports a route. You proved that AURORA ignores BOREALIS routes even on the same leaf.
- **Route types:** type 2 (MAC/IP with both L2 and L3 route targets), type 3 (flood list, visible as the remote VTEP count) and type 5 (tenant prefixes).
- **Symmetric IRB:** a leaf needs only its local VLANs plus the L3 VNI. The TTL-62 test shows that both ingress and egress VTEPs route.

- **Troubleshooting (domain 5):** a missing L3 VNI breaks only routed traffic; a mismatched L2 VNI breaks only bridged traffic in one VLAN. Know which verification command reveals each.

Review questions

1. hxb-leaf-r1's `show evpn vni` shows one remote VTEP for VNI 10110, while the book's example shows seven. Which is wrong?
2. Why do the spines need the `l2vpn-evpn` address family but no `nve vxlan source` or tenant VRFs?
3. A cross-rail ping between two AURORA hosts returns TTL 62. What does that tell you about the path?
4. On one leaf, `show evpn vni` lists the L2 VNIs but not L3 VNI 104001. Which traffic fails and which still works?
5. hxb-gpu01 and hxb-gpu03 connect to the same physical leaf. What prevents them from communicating, even if a host adds a route through its gateway?

Answers

1. Neither. The remote VTEP count per L2 VNI equals the number of other leaves that carry that VLAN. In helix-b-air only leaf-r3 also carries VLAN 110, so the answer is one. The book's full Helix-B stretches each VLAN across eight rail leaves, so it shows seven.
2. In the eBGP design the spines relay EVPN routes between leaves, so the address family must be active on their sessions. They never encapsulate or decapsulate tenant traffic (they route only the outer VTEP-to-VTEP packets), so they need no VTEP address, VNIs or tenant VRFs.
3. The packet was routed by two routers: the ingress leaf (into the L3 VNI) and the egress leaf (out of the L3 VNI into the destination VLAN). That is symmetric IRB. A bridged path would return TTL 64.
4. Routed traffic for AURORA to or from hosts behind that leaf fails, because the leaf cannot send or receive AURORA traffic in the L3 VNI and does not import AURORA's routed type-2 and type-5 information. Bridged traffic in the local L2 VNIs, and all BOREALIS traffic, still works.
5. The tenant VRFs. The ports are in different VLANs whose SVIs are in different VRFs (AURORA and BOREALIS). VRF AURORA contains no BOREALIS routes because BOREALIS routes carry BOREALIS's route targets, which AURORA does not import. The packet is dropped in VRF AURORA on the first leaf.

Lab 6 — Air Power Skills: Checkpoints, Cloning, Services and the SDK

Lab at a glance

Book chapters	Chapter 9 (Lab simulation with NVIDIA Air) · Chapter 4 (NVUE startup configuration) · Chapters 21–22 (testing templates and Ansible in a twin)
Exam objectives	2.4 Use NVIDIA Air to simulate network environments · supports 6.1/6.2 (testing automation in a digital twin)
Time	75–90 minutes
Air resources used	The whole helix-b-air simulation (about 26 vCPU while running). Some tasks run with the simulation stopped. Your own computer for SSH and the Python SDK
Prerequisite labs	Lab 0 (topology imported, OOB network enabled). Labs 1–5 recommended, so there is configuration worth protecting

Objectives

By the end of this lab you will be able to:

1. Save switch configuration so it survives a restart, and stop a simulation with a checkpoint.
2. Work with checkpoints: start from one, mark a favourite, and revert a simulation to one.
3. Clone a simulation and export its topology as JSON.
4. Read the **History** tab and estimate how many compute hours (credits) your work uses.
5. Reach the simulation directly from your laptop with an SSH service on **oob-mgmt-server**.
6. Configure servers automatically with node instructions and cloud-init.
7. Use the DSX Air Python SDK (**nv-air-sdk**) to list, start and sleep simulations from a script.

Background

By Lab 5 you have a working EVPN fabric that took hours to build. This lab is about protecting that work and using Air efficiently. A free-trial account has **10,000 compute hours** for a year and a **60 vCPU / 60 GiB** concurrent limit. A simulation that runs all night uses credits whether you type anything or not. Stopping it with a checkpoint costs nothing while it's stopped and brings you back to exactly where you were.

Air saves work at two layers, and you need both. **Inside each switch**, NVUE holds the running (applied) configuration and a startup configuration in **/etc/nvue.d/startup.yaml** (Chapter 4). Only the startup configuration is loaded at boot. **Around the whole simulation**, Air stores **checkpoints**: snapshots of every node's disk state. When you stop a simulation with **Stop Simulation and Store Checkpoint**, Air shuts the nodes down, saves their disks and releases the compute resources. Air keeps up to **three checkpoints** per simulation. The oldest non-favourite checkpoint is deleted automatically when a new one is saved, so mark the important ones as favourites.

The last part of the lab turns Air into something you can drive from code. The **Services** tab publishes a port on a node (usually SSH on **oob-mgmt-server**) to the internet. **Node instructions** and **cloud-init** configure servers automatically. The **Python SDK** lets a CI pipeline (Chapter 22) create, start, test and sleep simulations without anyone opening a browser.

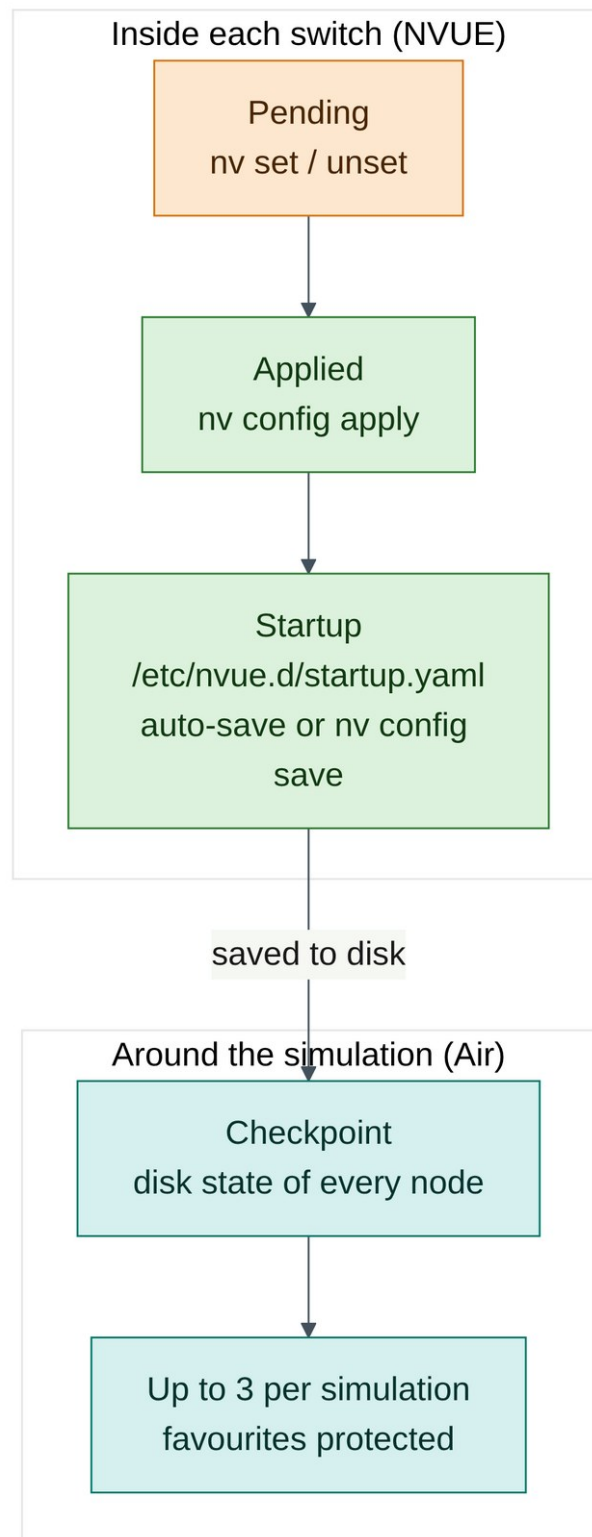


Figure 6.1: Two layers of saved state

NVUE startup configuration lives inside each switch's disk. An Air checkpoint saves every node's disk. Both are needed: a checkpoint of a switch whose change was never written to startup.yaml brings back the old configuration after the restart.

Version note: The DSX Air user interface changes often. The labels in this lab (**Stop Simulation and Store Checkpoint, Checkpoints, History, Services, Enable SSH, Settings, Node Instructions**) come from the DSX Air user guide at the time of writing. If a button has moved or been renamed, look for the same function nearby, and check docs.nvidia.com/dsx-air/simulation-management.

Step-by-step

Task 1 — Save configuration on every switch

A checkpoint saves disks, not running processes. Anything that exists only in a switch's applied configuration, and not in `startup.yaml`, might not come back after a stop and start. Make sure every switch's startup configuration matches its applied configuration.

1. Open the console of `oob-mgmt-server` (or SSH to it) and log in as `ubuntu`.
2. Connect to the first leaf and check whether anything is pending or unsaved:

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-leaf-r1
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff applied startup
```

Expected output (illustrative): both commands print nothing. The first shows pending changes that haven't been applied. The second shows differences between applied and startup.

3. If the first command shows output, you have pending changes. Review them, then apply or detach them (`nv config apply` or `nv config detach`). If the second shows output, save:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config save
```

4. Confirm that auto-save is on, so future applies are saved automatically:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show system config auto-save
```

Expected output (illustrative):

```
operational  applied
-----
state  enabled      enabled
```

5. Repeat on every switch. A loop from `oob-mgmt-server` is faster. It assumes the SSH key set-up from Lab 0; otherwise you are asked for the password on each switch:

```
ubuntu@oob-mgmt-server:~$ for sw in hxb-spine01 hxb-spine02 hxb-leaf-r1 hxb-leaf-r2 hxb-leaf-r3
hxb-leaf-r4; do
  echo "== $sw"; ssh cumulus@$sw "nv config save && nv config diff applied startup | wc -l"
done
```

Expected output (illustrative): each switch prints its name followed by `0` (no lines of difference).

6. On the Ubuntu servers, confirm the tenant addresses from Lab 5 are in a netplan file, not only added with `ip addr add`:

```
ubuntu@oob-mgmt-server:~$ ssh ubuntu@hxb-gpu01 "sudo netplan get ethernet"
```

Checkpoint:

- Every switch shows no difference between applied and startup.
- Auto-save is enabled (the default in Cumulus Linux 5.x).
- Server addresses are persistent (netplan), or you have noted which ones you will re-add after a restart.

Exam focus (2.4): A checkpoint is only as good as what is on disk. The NVUE answer to "what makes a change survive a reboot?" is that it is in the `startup` revision (`/etc/nvue.d/startup.yaml`). `nv config save` writes the applied configuration to startup. It does **not** apply pending changes.

Task 2 — Stop the simulation and store a checkpoint

1. In the Air UI, open the **helix-b-air** simulation.
2. In the action bar at the top of the page, click **Stop Simulation and Store Checkpoint**.
3. Watch the status move through **Shutting Down** and **Saving to Inactive**. This can take several minutes for 13 nodes.
4. Open the **Checkpoints** tab. You should see one checkpoint with the time you stopped the simulation.

Warning: The drop-down next to the stop button also offers **Stop Simulation Without Checkpoint**. That discards everything since the last checkpoint. Use it only when you really want to throw work away, for example after a destructive test.

Checkpoint:

- The simulation state is **Inactive**.
- The **Checkpoints** tab shows a new checkpoint of type **standard**.

Task 3 — Mark a favourite and start from a checkpoint

Air keeps at most three checkpoints per simulation. When a fourth is saved, the oldest checkpoint that isn't a favourite is deleted. Protect your "known good after Lab 5" state now.

1. On the **Checkpoints** tab, mark the checkpoint you just created as a favourite (use the favourite control on its row). Its **Type** column changes from **standard** to **Favorites**. If you can rename it, call it **lab05-evpn-good**.
2. Click the drop-down arrow next to **Start Simulation**. A list of checkpoints appears.
3. Select the checkpoint and start the simulation.
4. When the state is **Active**, log in to a leaf and check that the EVPN configuration from Lab 5 is present:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff applied startup
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show bgp summary"
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show evpn vni"
```

Expected output (illustrative): the BGP summary lists both unnumbered neighbours (**swp31**, **swp32**) as Established and **show evpn vni** lists L2 VNIs 10110, 10111, 10210, 10211 and L3 VNIs 104001, 104002.

Checkpoint:

- The favourite checkpoint is marked and named.
- The simulation came back with the Lab 5 configuration intact.

Field note: Checkpoint before every risky lab, not only at the end of the day. Before Lab 12 (Ansible) or Lab 13 (capstone faults), stop with a checkpoint, favourite it, and start again. It costs a few minutes and saves an evening of rebuilding.

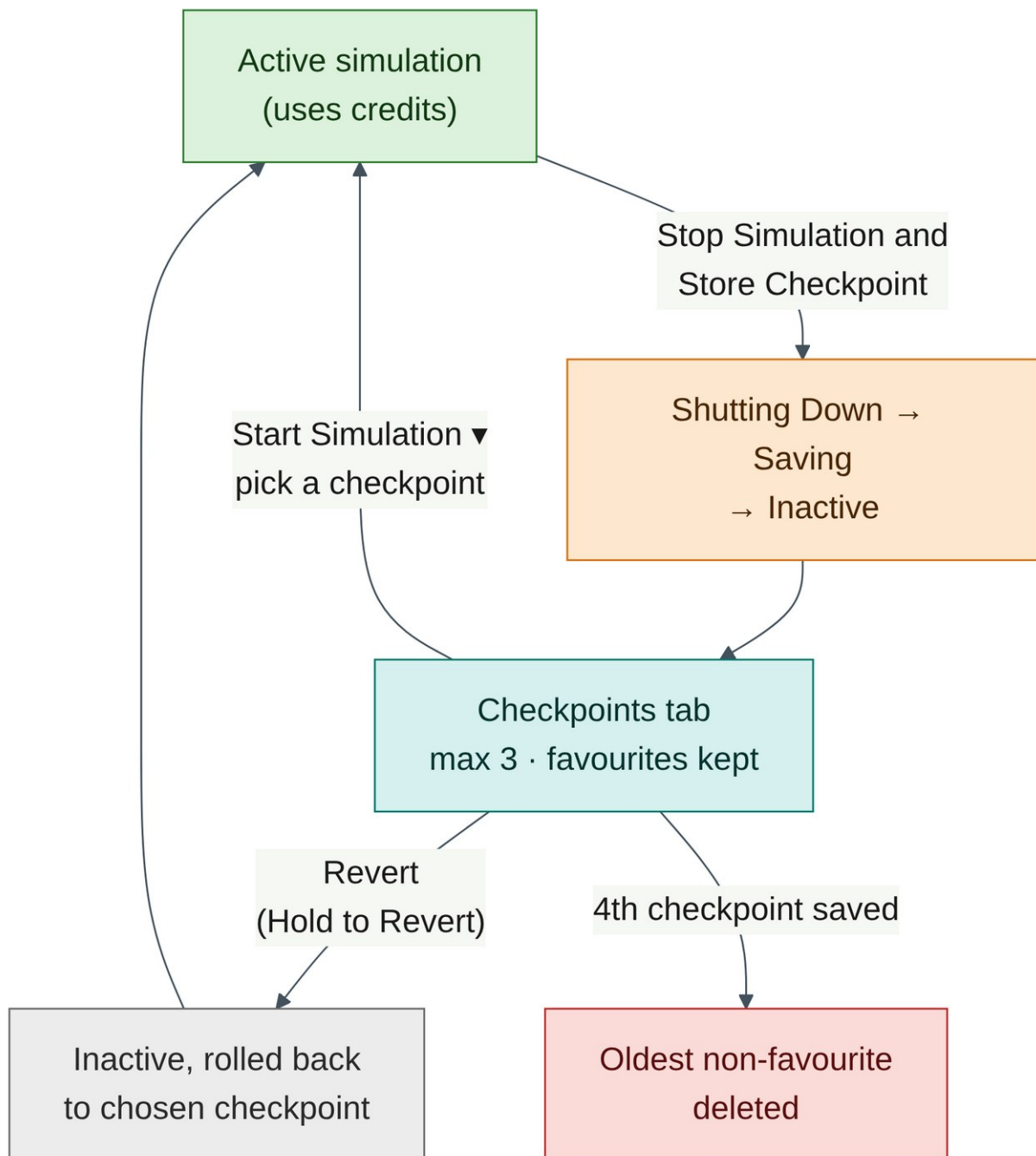


Figure 6.2: Checkpoint lifecycle in Air

Stopping with a checkpoint saves disks and frees resources. Starting from any stored checkpoint restores that state. Revert rolls an inactive simulation back to a checkpoint. When a fourth checkpoint is saved, the oldest non-favourite one is removed.

Task 4 — Break something, then revert

Revert rolls the simulation back to a checkpoint. It needs the simulation to be **Inactive** and to have at least one checkpoint.

1. With the simulation active, make a deliberate mess on `hxb-leaf-r1`:

```

cumulus@hxb-leaf-r1:mgmt:~$ nv unset vrf default router bgp neighbor swp31
cumulus@hxb-leaf-r1:mgmt:~$ nv unset vrf default router bgp neighbor swp32
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c "show bgp summary"
  
```

Expected output (illustrative): no neighbours listed. Leaf-r1 has dropped out of the fabric, and auto-save has written the broken configuration to startup.

2. Stop the simulation **without** a checkpoint (drop-down next to the stop button → **Stop Simulation Without Checkpoint**). You don't want to keep the broken state.
3. When the simulation is **Inactive**, click the **revert** icon in the action bar, choose the **lab05-evpn-good** checkpoint, and press and hold **Hold to Revert** until it completes.
4. Start the simulation and re-check **show bgp summary** on **hxb-leaf-r1**.

Checkpoint:

- Both BGP neighbours on **hxb-leaf-r1** are Established again.
- You understand the difference: *start from a checkpoint* picks a state for this run; *revert* rolls the simulation back to that checkpoint.

Warning: You can't revert an active simulation, and the revert option needs at least one checkpoint. If the revert icon is greyed out, check the state first.

Task 5 — Clone the simulation

A clone is a second, independent copy of the simulation. It's useful for a "what if" test that you might want to throw away, while the original stays safe.

1. From the simulation list (or the simulation's action menu), choose the clone option. At the time of writing, Air added "clone a simulation directly from the UI" in its August 2026 release. Look for **Clone** or **Duplicate** in the simulation's menu (check on your release).
2. Name the clone **helix-b-air-whatif**.
3. **Don't start both at once.** Check the quota arithmetic first:

Resource	helix-b-air (approx.)	Two copies	Free-trial limit
vCPU	26	52	60
Memory	about 46–48 GiB	about 94 GiB	60 GiB

Two copies fit the vCPU limit but not the memory limit. Stop the original with a checkpoint before starting the clone.

Checkpoint:

- The clone appears in your simulation list.
- You know that only one copy of this topology can run at a time on a free-trial account.

Field note: If your UI has no clone option, the same result comes from exporting the topology (Task 6) and importing it as a new simulation. An imported topology starts from fresh images, not from your checkpoints, so re-apply configuration from saved **startup.yaml** files or your Lab 11/12 automation.

Task 6 — Export the topology

1. Open the **Topology** tab of the original simulation.
2. Click the **Export** icon in the toolbar. Air downloads the topology as a JSON file.
3. Open the file in a text editor and find **hxb-leaf-r1**, its OS image, its vCPU and memory, and its links.
4. Commit the file to the Git repository you will use for Lab 11 and Lab 12, for example as **air/helix-b-air.json**.

Checkpoint:

- You have a JSON file that describes the nodes and links (not the configuration inside them).

- It is under version control next to your templates and playbooks.

Exam focus (2.4): Air simulations can be built with the UI builder, from a **DOT** or **JSON** topology file, or from a pre-built demo. An exported topology file is how a digital twin stays in step with the production design (Chapter 9).

Task 7 — Read the History tab and estimate credits

1. Open the **History** tab of the simulation. Each entry has a **Timestamp**, **Actor**, **Category** (INFO, WARNING or ERROR) and **Description**.
2. Find the events from Tasks 2–4: the stop, the checkpoint save, the revert and the start. Note who (**Actor**) did each one.
3. Estimate what one hour of this lab costs. DSX Air bills in **compute hours**. One compute hour is **1 vCPU for 1 hour**, or **8 GB of memory for 1 hour**, and the two are counted separately.

Item	Working
vCPU per hour	12 nodes × 2 vCPU + <code>oob-mgmt-switch</code> ≈ 26 vCPU → about 26 compute hours per hour
Memory per hour	about 46–48 GB ÷ 8 ≈ 6 compute hours per hour
Total	about 32 compute hours per running hour
Free-trial allowance	10,000 ÷ 32 ≈ 310 hours of running time

A simple rule of thumb for planning is **vCPU × hours**, and then add about a quarter for memory on this topology. An evening of three hours uses about 100 compute hours. Leaving it running for a week by mistake uses over 5,000.

4. Look for the rate or usage display in the UI. Air's August 2026 release notes say the UI "now shows the rate required to run a simulation" (check where it appears on your release). Compare it with your estimate.
5. Open the simulation's **Edit** dialog and look at **Sleep date**. Free-trial simulations get an automatic sleep date that can't be removed. When it is reached, Air saves the simulation's state and stops it.

Checkpoint:

- You can find a stop, checkpoint and revert event in **History**.
- You have a credit estimate for one hour and one evening of work.

Warning: The **Sleep date** is a safety net, not a plan. Stop the simulation yourself at the end of each session, with a checkpoint.

Task 8 — SSH from your laptop with a Service

The console in the browser works, but a proper SSH session from your laptop is faster and lets you copy and paste freely. Password authentication is disabled on `oob-mgmt-server` by default, so you must add a public key first.

1. On your laptop, create a key if you don't have one, and print the public half:

```
you@laptop:~$ ssh-keygen -t ed25519 -C "helix-air"
you@laptop:~$ cat ~/.ssh/id_ed25519.pub
```

2. In the Air UI, click your user name, choose **Settings**, and find the SSH keys section. Fill in **Name** (for example `laptop`) and **Public Key** (paste the whole `ssh-ed25519 ...` line), then select **Add**.
3. Keys you add are included automatically in the simulation's authorised keys on `oob-mgmt-server`. If the simulation was already running, stop and start it (or restart `oob-mgmt-server`) if the key doesn't work straight away (check on your release).

4. Open the simulation's **Services** tab and click **Enable SSH**. This is only available when the OOB network is enabled. (The same tab has **+ New Service** for other ports, with the fields **Service Name**, **Interface**, **Service Type** and **Service Port**.)
5. Air shows an external host name and port for the service, with a ready-made command. It looks like this (illustrative):

```
you@laptop:~$ ssh -p 24562 ubuntu@worker07.air.nvidia.com
```

6. Run the command shown in **your** Services tab. From **oob-mgmt-server**, go on to any node by name:

```
ubuntu@oob-mgmt-server:~$ ssh cumulus@hxb-spine01
```

Checkpoint:

- You can SSH from your laptop straight to **oob-mgmt-server** with your key.
- From there, you can reach every node by host name.

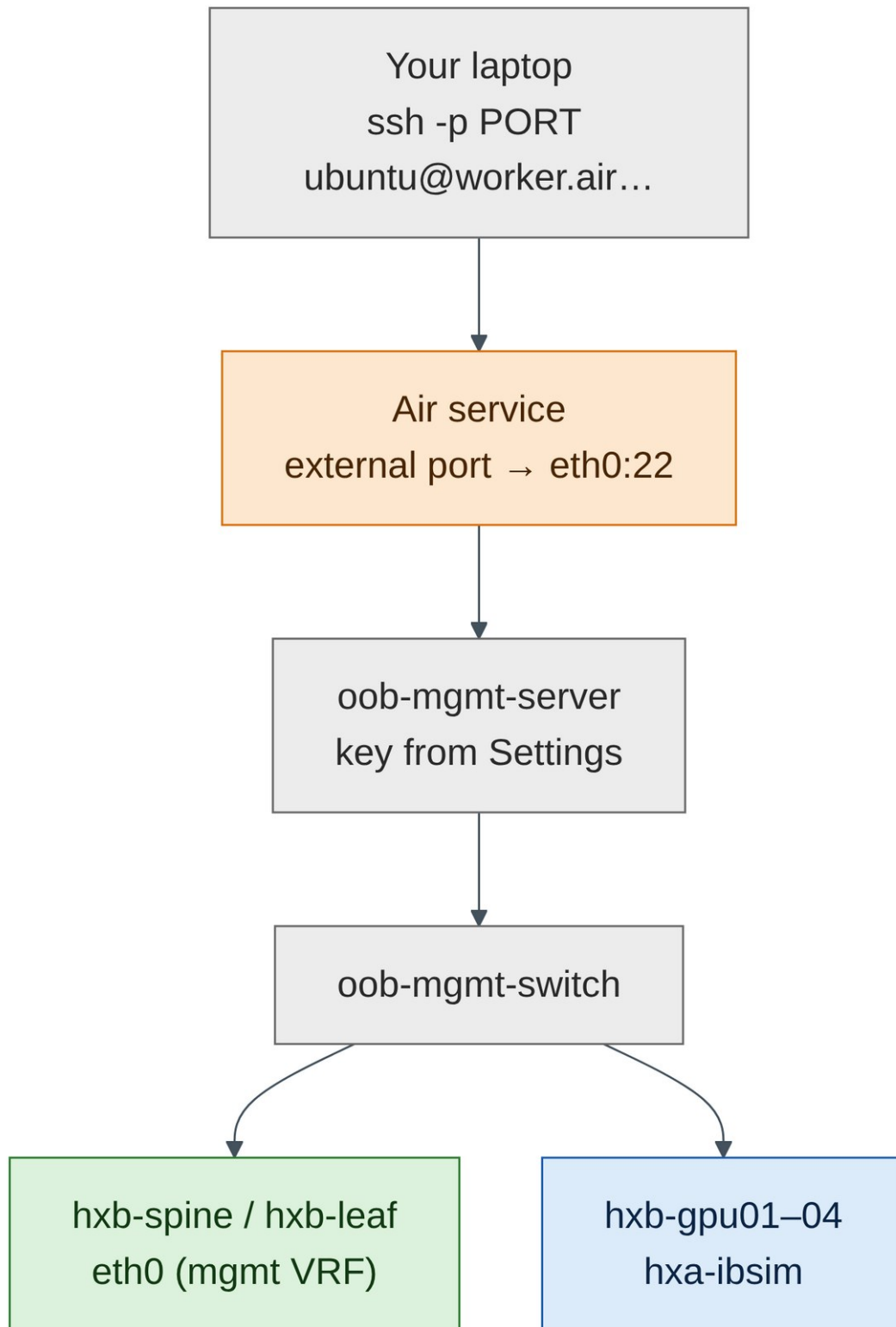


Figure 6.3: Reaching the simulation from your laptop

The SSH service publishes port 22 on oob-mgmt-server at an external host name and port. Your laptop's public key, added in Settings, is installed on oob-mgmt-server. Every other node is reached over the OOB network by host name.

Warning: A published SSH service is reachable from the internet. Keep password authentication disabled, use keys only, and disable the service when you no longer need it.

Task 9 — Configure servers automatically: node instructions and cloud-init

In Labs 3 and 5 you configured each server by hand. Air offers two ways to do this automatically.

Method	When it runs	How you add it	Best for
Node instructions	After the node boots, run by the Air agent on the node	UI: Nodes tab → menu on the node → Node Instructions (or right-click the node on the Topology canvas). Also the instructions API	Changes to an existing, running simulation
Cloud-init	On the node's first boot (NoCloud data source)	API or SDK: create UserConfig objects (user-data, meta-data) and assign them to nodes	Building servers from scratch, reproducibly

Air-provided **generic/ubuntu** images include cloud-init. Assign cloud-init configuration **before** a node's first boot. Assignments made after a node has booted take effect only when the node is rebuilt.

Part A — a node instruction for hxb-gpu01

1. In the **Nodes** tab, open the menu for **hxb-gpu01** and choose **Node Instructions**.
2. Create a shell-type instruction (choose the shell or script option the dialog offers; executor names vary by release) with this content. It makes the Lab 5 AURORA addresses persistent and installs the RDMA tools used in Labs 3 and 9:

```
#!/bin/bash
cat > /etc/netplan/60-helix.yaml <<'EOF'
network:
  version: 2
  ethernet:
    eth1:
      addresses: [172.16.10.101/24]
      routes: [{to: 172.16.0.0/16, via: 172.16.10.1}]
    eth2:
      addresses: [172.16.11.101/24]
EOF
chmod 600 /etc/netplan/60-helix.yaml
netplan apply
apt-get update && apt-get install -y rdma-core ibverbs-utils perftest
```

3. Leave **wait for network** at its default (enabled). With the OOB network enabled, the Air agent waits until it can reach **oob-mgmt-server** before it runs the instruction. These network-readiness settings can't be changed after the instruction is created.
4. Save the instruction and watch it run (the node's instruction status shows when it has completed).
5. Check the result:

```
ubuntu@oob-mgmt-server:~$ ssh ubuntu@hxb-gpu01 "ip -br addr show eth1; dpkg -l perftest | tail - 1"
```

Expected output (illustrative):

```
eth1          UP                172.16.10.101/24 fe80::.../64
ii perftest    24.x...      amd64      Infiniband verbs performance tests
```

Warning: The routes in the example assume the AURORA design from Lab 5 (a route to the other tenant subnets through the anycast gateway). Adjust them to match what you built. If eth1 and eth2 already have these addresses in another netplan file, remove the duplicate first.

Part B — cloud-init with the SDK (optional, for a new simulation)

Cloud-init is the cleaner choice when you build a new simulation, for example the clone from Task 5 before its first start, or a CI simulation in Lab 12. The user-data file is standard `#cloud-config`:

```
#cloud-config
package_update: true
packages: [rdma-core, ibverbs-utils, perftest, jq]
write_files:
  - path: /etc/netplan/60-helix.yaml
    permissions: '0600'
    content: |
      network:
        version: 2
        ethernets:
          eth1: {addresses: [172.16.10.102/24]}
          eth2: {addresses: [172.16.11.102/24]}
runcmd:
  - netplan apply
```

The SDK (Task 10) creates the configuration and assigns it to a node. This follows the DSX Air SDK cloud-init example:

```
from pathlib import Path
from air_sdk import AirApi
from air_sdk.endpoints import UserConfig

api = AirApi.with_ngc_config()
sim = next(s for s in api.simulations.list() if s.name == "helix-b-air-whatif")
node = next(n for n in sim.nodes.list() if n.name == "hxb-gpu02")

user_data = api.user_configs.create(
    name="hxb-gpu02-userdata",
    kind=UserConfig.KIND_CLOUD_INIT_USER_DATA,
    content=Path("hxb-gpu02-user-data.yaml"),
)
meta_data = api.user_configs.create(
    name="hxb-gpu02-metadata",
    kind=UserConfig.KIND_CLOUD_INIT_META_DATA,
    content="instance-id: hxb-gpu02\nlocal-hostname: hxb-gpu02\n",
)
sim.node_bulk_assign(nodes=[{"node": node, "user_data": user_data, "meta_data": meta_data}])
```

Checkpoint:

- **hxb-gpu01** has its tenant addresses and the RDMA tools installed by a node instruction.
- You can explain when to use node instructions (running simulation) and when to use cloud-init (first boot).

Version note: The `sim.nodes.list()` call and the node `name` attribute follow the SDK's naming pattern, but this exact combination was not tested for this guide (check on your release, at docs.nvidia.com/dsx-air/sdk). The rest of the example is taken from NVIDIA's cloud-init SDK example.

Task 10 — Drive Air from Python with the SDK

The DSX Air SDK is the `nv-air-sdk` package. It needs Python 3.10 or later and authenticates with an **NGC API key** that has the **NVIDIA Air** service selected.

1. **Create an API key.** Sign in to NGC, generate a Personal API key (see the NGC user guide, "Generating a Personal API Key"), and select **NVIDIA Air** in the services list. The key starts **nvapi-**. Copy it once; NGC doesn't show it again.

Version note: Older Air documentation and the older **air-sdk** package used an API token created in the Air UI and a user name and password style login. The current DSX Air SDK uses NGC keys: **AirApi.with_api_key(...)**, **AirApi.with_ngc_config()** (reads `~/.ngc/config`), or **AirApi.with_device_login(...)**. Confirm the method for your account at docs.nvidia.com/dsx-air/authentication and docs.nvidia.com/dsx-air/sdk.

2. **Install the SDK on your laptop**, in a virtual environment. Run it on your laptop, not on **oob-mgmt-server**: a script that sleeps the simulation would cut off the machine it runs on.

```
you@laptop:~$ python3 -m venv ~/air-venv
you@laptop:~$ source ~/air-venv/bin/activate
(air-venv) you@laptop:~$ pip install nv-air-sdk
(air-venv) you@laptop:~$ export AIR_API_KEY='nvapi-...your key...'
```

AIR_API_KEY is simply the variable name this lab uses. It keeps the key out of the script and out of Git.

3. **Write the script.** Save it as **helix_air.py**:

```
#!/usr/bin/env python3
"""List, start and sleep DSX Air simulations (Lab 6)."""
import os
import sys

from air_sdk import AirApi, SimState

api = AirApi.with_api_key(api_key=os.environ["AIR_API_KEY"])

def list_sims():
    for sim in api.simulations.list():
        print(f"{sim.name:<28} {str(sim.state):<12} {sim.id}")

def find(name):
    for sim in api.simulations.list():
        if sim.name == name:
            return sim
    sys.exit(f"No simulation called {name!r}")

def start(name):
    sim = find(name)
    sim.start()
    sim.wait_for_state(SimState.ACTIVE, error_states=SimState.INACTIVE)
    print(f"{name} is ACTIVE")

def sleep(name):
    sim = find(name)
    sim.shutdown() # stores a checkpoint unless create_checkpoint=False
    sim.wait_for_state(SimState.INACTIVE)
    print(f"{name} is INACTIVE, checkpoint stored")
    for cp in sim.checkpoints.list():
        print(f"  checkpoint {cp.name} (id={cp.id}) state={cp.state}")
```

```
if __name__ == "__main__":
    if len(sys.argv) < 2 or sys.argv[1] not in ("list", "start", "sleep"):
        sys.exit("usage: helix_air.py list | start <sim> | sleep <sim>")
    if sys.argv[1] == "list":
        list_sims()
    elif sys.argv[1] == "start":
        start(sys.argv[2])
    else:
        sleep(sys.argv[2])
```

4. Run it.

```
(air-venv) you@laptop:~$ python3 helix_air.py list
```

Expected output (illustrative):

helix-b-air	ACTIVE	3f6c1d2e-...
helix-b-air-whatif	INACTIVE	9a41b7c0-...

```
(air-venv) you@laptop:~$ python3 helix_air.py sleep helix-b-air
(air-venv) you@laptop:~$ python3 helix_air.py start helix-b-air
```

Expected output (illustrative): the sleep command waits several minutes, prints **helix-b-air is INACTIVE, checkpoint stored** and lists the checkpoints. The start command waits until the nodes are up and prints **helix-b-air is ACTIVE**.

5. Open the **History** tab. The API calls appear as events with your account as the **Actor**.

Checkpoint:

- **list** shows your simulations with their state.
- **sleep** creates a checkpoint, the same as **Stop Simulation and Store Checkpoint** in the UI.
- You know the other SDK calls that exist for CI: **sim.shutdown(create_checkpoint=False)**, **sim.rebuild(checkpoint=cp.id)**, **sim.export()**, **sim.get_history()**, and **cp.update(name=..., favorite=True)**.

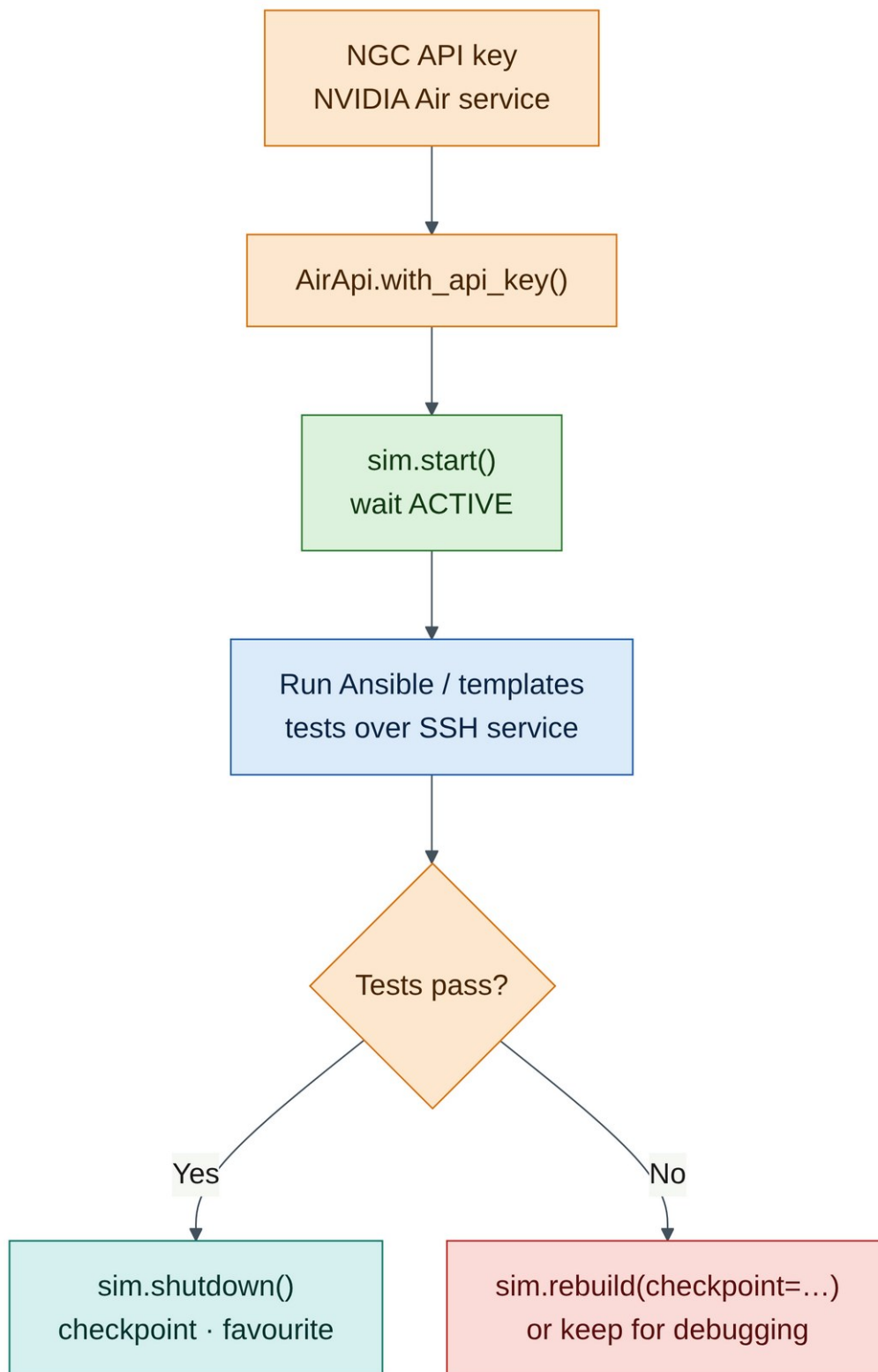


Figure 6.4: Air in a CI pipeline with the SDK

A pipeline authenticates with an NGC API key, starts the twin, runs automation and tests against it, then sleeps it with a checkpoint. The same calls you used by hand in this lab are the building blocks of Chapter 22's Air CI workflow.

Field note: Put a `sleep` step in the pipeline's clean-up stage so it runs even when tests fail. A failed CI run that leaves a 26 vCPU simulation running overnight uses around 400 compute hours.

Verify

- [] Every switch shows no difference in `nv config diff applied startup`.
- [] You stopped the simulation with **Stop Simulation and Store Checkpoint** and saw a new checkpoint.
- [] One checkpoint is a favourite, named `lab05-evpn-good`.
- [] You started from a checkpoint, and reverted an inactive simulation to one.
- [] A clone exists, and you know that only one copy can run at a time on a trial account.
- [] The topology JSON is exported and committed to Git.
- [] You can find stop, checkpoint and revert events in **History**, and you have a credit estimate.
- [] You can SSH from your laptop to `oob-mgmt-server` through the SSH service.
- [] A node instruction configured `hxb-gpu01`.
- [] `helix_air.py list, start` and `sleep` work.

Break and fix

Fault 1 — "My changes disappeared after the restart"

Symptom: You changed `hxb-leaf-r2` (for example `nv set interface swp1 link mtu 9000`), stopped the simulation with a checkpoint, and started it the next day. The MTU is back to 9216.

Diagnosis:

1. `nv config history` on the leaf shows no revision with the change, or `nv config diff applied startup` shows it was never saved.
2. `nv show system config auto-save` shows `disabled`. Someone turned auto-save off (the fault injection for this exercise is `nv set system config auto-save state disabled` followed by `nv config apply -y`, run before the change).
3. The checkpoint faithfully saved a disk whose `startup.yaml` didn't contain the change.

Fix: Re-apply the change, run `nv config save`, and turn auto-save back on with `nv set system config auto-save state enabled` and `nv config apply -y`. Then stop with a checkpoint again. The lesson: Air checkpoints save disks, and the switch decides what is on disk.

Fault 2 — "Permission denied (publickey)"

Symptom: The SSH command from the **Services** tab fails with `Permission denied (publickey)`.

Diagnosis:

1. Password authentication is disabled on `oob-mgmt-server`, so only a key works.
2. Check the key in **Settings**: a common mistake is pasting the private key, or a public key with a line break in the middle.
3. Run `ssh -v -p <port> ubuntu@<host>` and check which key file your SSH client offers. If you created a non-default key, add `-i ~/.ssh/<keyfile>`.
4. If the key was added while the simulation was running, it may not have reached `oob-mgmt-server` yet.

Fix: Paste the single-line public key (`ssh-ed25519 AAAA... comment`), select **Add**, restart the simulation or `oob-mgmt-server` if needed, and connect with the right `-i` key file.

Clean-up / save state

1. Make sure every switch is saved (Task 1 loop).
2. Stop the simulation with **Stop Simulation and Store Checkpoint**, or run `python3 helix_air.py sleep helix-b-air`.
3. Mark this checkpoint as a favourite if it's the state you want Lab 7 to start from. With three slots, keep: `lab05-evpn-good` (favourite), the latest working state, and one spare.

4. If you don't need the clone, delete it so it doesn't confuse your SDK scripts.
5. Disable the SSH service if you won't use it for a while.

Exam tie-in

- **2.4 Use NVIDIA Air:** know the lifecycle words (start, stop with or without checkpoint, sleep date, checkpoints, revert, rebuild) and what each keeps.
- **2.4:** simulations are built from the UI, from DOT/JSON topology files, or from demos, and can be exported as JSON. Air adds an OOB network with `oob-mgmt-server` and `oob-mgmt-switch`.
- **2.4:** Air is automated through the API/SDK (`nv-air-sdk`, from `air_sdk import AirApi`, NGC API key). That is how a digital twin fits into CI.
- **6.1/6.2:** templates and playbooks are tested against an Air twin started and stopped by a pipeline (Chapters 21–22).
- **Chapter 4 link:** a change survives a reboot only if it is in `startup.yaml`. `nv config save` saves; `nv config apply` applies.

Review questions

1. What does **Stop Simulation and Store Checkpoint** save, and what happens to the compute resources?
2. A simulation has three standard checkpoints and one favourite. You stop it with a checkpoint again. What happens?
3. Why must you run `nv config save` (or have auto-save enabled) before you stop a simulation with a checkpoint?
4. Your simulation uses 26 vCPU and about 48 GB of memory. Roughly how many compute hours does a 4-hour session use?
5. You add a cloud-init configuration to a server that has already booted. Why doesn't it take effect, and what should you use instead?

Answers

1. It shuts the nodes down, saves a checkpoint of every node's disk state, and releases the compute resources, so an inactive simulation doesn't use compute hours. The status moves through Shutting Down and Saving to Inactive.
2. A new checkpoint is saved and the oldest **non-favourite** checkpoint is deleted automatically. The favourite is kept. (Wording in the docs: up to 3 checkpoints per simulation; favourites are protected from automatic deletion.)
3. A checkpoint saves disks. At boot, a Cumulus switch loads `/etc/nvue.d/startup.yaml`. A change that is applied but not saved to startup is lost after the restart, even though the checkpoint itself worked.
4. About 26 vCPU-hours per hour plus about 6 for memory ($48 \div 8$), so about 32 per hour, or roughly **130 compute hours** for 4 hours. The quick $\text{vCPU} \times \text{hours}$ estimate gives 104.
5. Cloud-init runs on first boot; assignments made after a node has booted take effect only when the node is rebuilt. On a running simulation, use a **node instruction**, which the Air agent runs after boot.

Lab 7 — Monitoring and Troubleshooting a Spectrum-X-Style Fabric

Lab at a glance

Book chapters	10 (NetQ, WJH, telemetry), 18 (cl-resource-query and WJH troubleshooting)
Exam objectives	2.5, 2.6, 5.1, 5.2
Time	60 minutes
Air resources	Full helix-b-air simulation
Prerequisites	Labs 1, 2 and 5 (EVPN tenants working)

Objectives

- Build a repeatable health check for the Helix-B fabric from NVUE, FRR and Linux.
- Read interface, BGP and EVPN state quickly and know which command answers which question.
- Try `cl-resource-query` and `nv show platform asic resource` on Cumulus VX and interpret what you get.
- Understand what What Just Happened (WJH) and NetQ add on real hardware, and why they aren't in this lab.
- Diagnose three injected faults with a layered method.

Background

On a Spectrum-X fabric you'd have three observability layers: **on-box** state (NVUE, FRR, Linux counters), **ASIC-level** drop and resource data (WJH, `cl-resource-query`), and **fabric-wide** history and validation (NetQ). Cumulus VX gives you the first layer in full. It has no Spectrum ASIC, so WJH isn't available and ASIC resource data is limited or absent. NetQ needs its own server, which is large for the free trial (Chapter 10), so this lab builds a scripted "mini-NetQ" health check instead and uses the book's WJH examples to practise reading drop reasons.

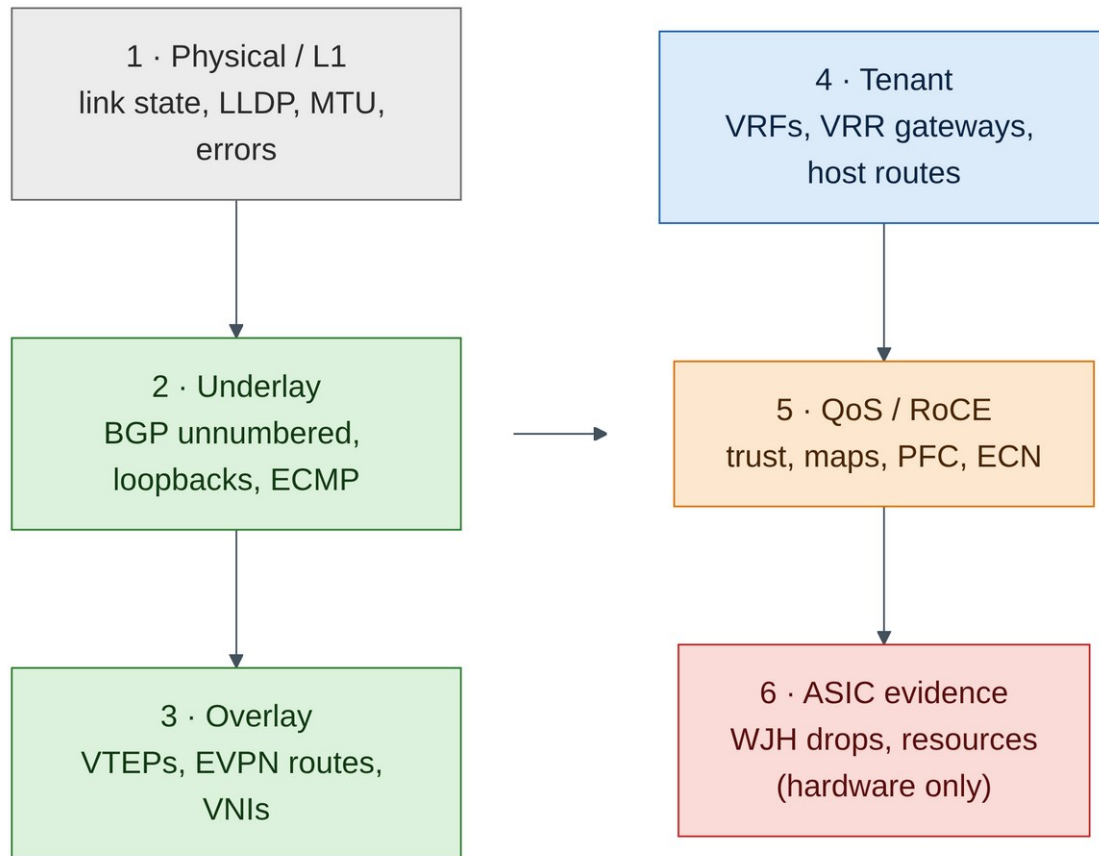


Figure 7.1: Troubleshooting layer by layer

Work bottom-up. A failure at one layer explains every failure above it, so don't debug EVPN before the underlay is clean.

Step-by-step

Task 1 — The five-minute health check on one switch

Run these on `hxb-leaf-r1`. Each answers one question.

Question	Command
Are the ports up, with the right MTU?	<code>nv show interface</code>
Who is cabled where?	<code>nv show interface swp31 lldp</code> or <code>sudo lldpctl</code>
Are the BGP sessions up and exchanging routes?	<code>nv show vrf default router bgp neighbor</code>
Are both spines installed as next hops?	<code>sudo vtysh -c 'show ip route 10.255.0.14/32'</code>
Is EVPN learning MACs and routes?	<code>nv show evpn vni</code> · <code>sudo vtysh -c 'show bgp l2vpn evpn summary'</code>
Are tenant gateways present?	<code>nv show interface vlan110</code>

```

cumulus@hxb-leaf-r1:mgmt:~$ nv show interface | grep -E 'swp|vlan|lo '
lo          up      65536  loopback  IP Address: 10.255.0.11/32
swp1        up      9216   swp
swp2        up      9216   swp
swp31       up      9216   swp

```

```
swp32    up    9216    swp
vlan110  up    9216    svi    IP Address: 172.16.10.11/24
vlan210  up    9216    svi    IP Address: 172.17.10.11/24
```

(Illustrative; your columns and addresses depend on the release and on Lab 5.)

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c 'show bgp summary' | grep -E 'Neighbor|swp'
Neighbor      V      AS    MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  State/PfxRcd
PfxSnt Desc
swp31         4      65100      812      805        0    0    0 06:41:12          5
7 N/A
swp32         4      65100      810      806        0    0    0 06:41:10          5
7 N/A
```

(Illustrative.) Both sessions **Established** (a number in *State/PfxRcd*, not a word like *Active* or *Idle*).

Checkpoint

- [] You can answer every question in the table for hxb-leaf-r1 in under five minutes.

Task 2 — Counters: NVUE and Linux

- Interface counters through NVUE:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface swp31 counters
```

- The same data from Linux. On Cumulus VX the switch ports are ordinary Linux interfaces, so standard tools work:

```
cumulus@hxb-leaf-r1:mgmt:~$ ip -s link show swp31
cumulus@hxb-leaf-r1:mgmt:~$ sudo ethtool -S swp31 | head -20
```

- Generate some traffic (a 60-second ping flood inside AURORA from hxb-gpu01 to hxb-gpu02) and watch the counters move on the uplinks. Which spine carries it tells you where ECMP hashed the flow (Lab 4).

```
ubuntu@hxb-gpu01:~$ sudo ping -f -c 20000 -s 1400 172.16.10.102
cumulus@hxb-leaf-r1:mgmt:~$ watch -n 2 "nv show interface swp31-32 counters | grep -iE 'packet|octet'"
```

Field note: On Spectrum hardware, `nv show interface <swp> counters` also shows per-queue and PFC counters, and `nv show interface <swp> qos roce counters` shows RoCE-specific counters (Chapter 5). On VX those sections are empty or missing because there is no ASIC.

Task 3 — ASIC resources: try cl-resource-query

Chapter 18 uses `cl-resource-query` to catch forwarding-table exhaustion. Try it on VX:

```
cumulus@hxb-leaf-r1:mgmt:~$ sudo cl-resource-query
cumulus@hxb-leaf-r1:mgmt:~$ nv show platform asic resource          # 5.11–5.14 syntax
cumulus@hxb-leaf-r1:mgmt:~$ nv show platform asic                # find the ASIC id on
5.15+
```

What you'll see depends on the VX release: the command may be missing, may print zeros or maximums of zero, or may print a partial table. Any of these is expected: the numbers come from the Spectrum ASIC driver. Compare with the hardware output in Chapter 18 and answer:

- Which rows would grow when you add a tenant VRF with many /32 host routes?
- Which row would grow if you added more ECMP next hops?

Exam focus: Know the resource categories (host/neighbour entries, IPv4/IPv6 routes, ECMP next hops, MAC entries, ACL regions) and the symptom of exhaustion: routes present in FRR but not programmed in hardware, so traffic falls back or drops.

Task 4 — WJH and NetQ: read what the hardware would tell you

WJH classifies every dropped packet in the Spectrum ASIC (Chapters 10 and 18). It isn't available on VX. Practise instead with the book's example output. For each drop reason, say which layer of Figure 7.1 it belongs to and what you would check next.

WJH drop (from Chapter 18)	Layer	Next check
L2 · Ingress VLAN filtering	Tenant	Is the VLAN allowed on the bridge port?
Router · Blackhole route	Tenant / underlay	Which route points to blackhole, and why?
Router · TTL value is too small	Underlay	Routing loop? traceroute
Buffer · Tail drop / WRED	QoS	Congestion on which egress queue; RoCE marked correctly?
ACL · Ingress port ACL	Policy	Which rule matched?

For NetQ, write down (from Chapter 10) the three commands you would run on netq-01 for this lab's topology: **netq check bgp**, **netq check evpn**, and **netq show interfaces** or **netq check mtu**. Task 5 builds a small substitute.

Task 5 — Build a mini fabric check from the oob-mgmt-server

This script gives you a NetQ-style one-screen health view for the whole fabric using SSH.

```
ubuntu@oob-mgmt-server:~$ mkdir -p ~/checks && nano ~/checks/fabric-check.sh
```

```
#!/bin/bash
# Mini fabric check for helix-b-air: BGP, EVPN and link state on every switch
SWITCHES="hxb-spine01 hxb-spine02 hxb-leaf-r1 hxb-leaf-r2 hxb-leaf-r3 hxb-leaf-r4"
for s in $SWITCHES; do
    echo "==== $s"
    ssh -o BatchMode=yes cumulus@$s '
        echo -n "BGP established: "; sudo vtysh -c "show bgp summary json" | python3 -c "import
sys,json;d=json.load(sys.stdin);p=d.get(\"ipv4Unicast\",{}).get(\"peers\",{});print(sum(1 for v
in p.values() if v.get(\"state\")==\"Established\"),\"/\",len(p))"
        echo -n "Down swp ports: "; ip -br link | awk "/^swp/ && \$2!=\"UP\" {printf \"%1\" \"%}";
    echo
        echo -n "EVPN VNIs:          "; sudo vtysh -c "show evpn vni" 2>/dev/null | grep -cE "^ *[0-9]+
"
    '
done
```

```
ubuntu@oob-mgmt-server:~$ chmod +x ~/checks/fabric-check.sh && ~/checks/fabric-check.sh
==== hxb-spine01
BGP established: 4 / 4
Down swp ports:
EVPN VNIs:      0
==== hxb-leaf-r1
BGP established: 2 / 2
```

```
Down swp ports:
EVPN VNIs:      4
```

(*Illustrative.*) Run it now and save the output as your **baseline**: `~/checks/fabric-check.sh > ~/checks/baseline.txt`. In the break/fix tasks, compare with `diff`.

Warning: The script relies on the passwordless SSH set up in Lab 0 and on `sudo` without a password for the `cumulus` user (the Cumulus default). If `sudo` prompts, run the commands manually instead.

Break and fix

Ask a colleague to inject a fault without telling you which, or inject it yourself and come back later. Diagnose with Figure 7.1, starting at layer 1.

Fault A — MTU mismatch on an uplink

- *Inject:* on hxb-spine01, `nv set interface swp3 mtu 1500 && nv config apply -y`.
- *Symptoms:* BGP to hxb-leaf-r3 stays up (small packets). Small pings between gpu01 and gpu02 work, but large ones fail when the flow hashes through spine01: `ping -M do -s 8000 172.16.10.102`.
- *Diagnosis:* `nv show interface swp3` on spine01 shows MTU 1500 against 9216 on leaf-r3 swp31. VXLAN adds 50 bytes to every host frame, so host MTU 9000 frames can't cross a 1500-byte link.
- *Fix:* `nv set interface swp3 mtu 9216 && nv config apply -y`. Re-run the large ping.

Fault B — A BGP neighbour down

- *Inject:* on hxb-leaf-r2, `nv set interface swp32 link state down && nv config apply -y`.
- *Symptoms:* the fabric check shows leaf-r2 at 1 / 2 established and swp32 in the down list. Traffic still flows (ECMP falls back to spine01) but capacity is halved.
- *Diagnosis:* `nv show interface swp32` shows admin down; LLDP neighbour missing.
- *Fix:* `nv set interface swp32 link state up && nv config apply -y`.

Fault C — VLAN missing from the bridge on one leaf

- *Inject:* on hxb-leaf-r3, `nv unset bridge domain br_default vlan 110 && nv config apply -y`.
- *Symptoms:* gpu01 can no longer reach gpu02 in 172.16.10.0/24; BOREALIS is unaffected. `nv show evpn vni` on leaf-r3 no longer lists 10110.
- *Diagnosis:* layers 1–3 are clean (script output matches the baseline except VNI count on leaf-r3). At layer 4, the VLAN-to-VNI mapping and the access port's VLAN are gone. On hardware, WJH would show *Ingress VLAN filtering* drops on leaf-r3 swp1.
- *Fix:* restore `nv set bridge domain br_default vlan 110 vni 10110`, the access port VLAN and the SVI settings from Lab 5, then `nv config apply -y`. Or roll back with `nv config history` and `nv config apply <rev>` (Lab 1).

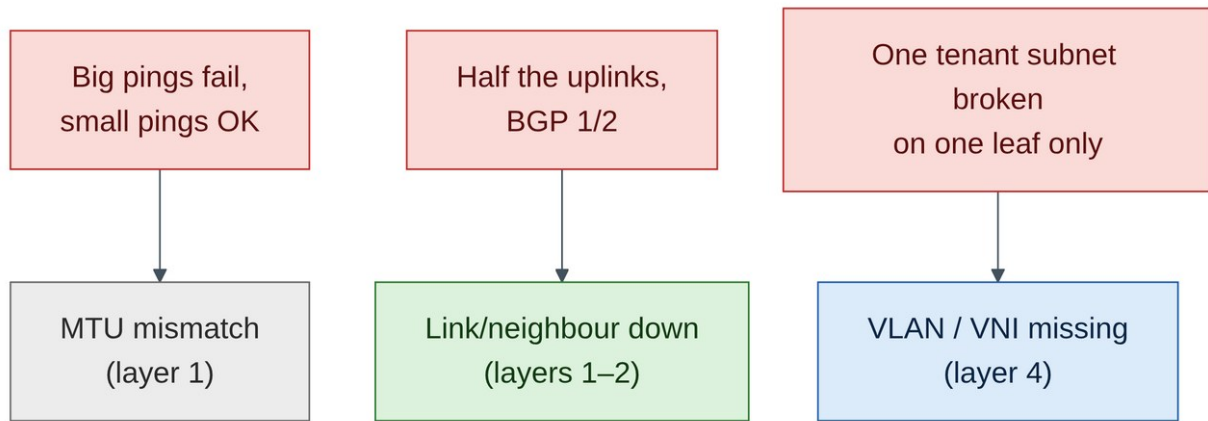


Figure 7.2: Symptom to cause for the three faults

The first diverging layer points to the fault. Keep a baseline so "different from normal" is obvious.

Verify

- [] Your fabric check runs cleanly and matches the baseline.
- [] You found and fixed Faults A, B and C, each starting at layer 1.
- [] You can name what WJH, cl-resource-query and NetQ add on real hardware.

Clean-up / save state

Make sure all three faults are reverted, run the fabric check once more, `nv config save` on every switch, and store a checkpoint (`lab07-monitoring`).

Exam tie-in

- 5.1: `cl-resource-query` categories and the symptom of table exhaustion.
- 5.2: WJH drop categories (L1, L2, router, tunnel, ACL, buffer) and how they map to fixes.
- 2.5/2.6: NetQ checks (`netq check bgp|evpn|mtu`) and telemetry are how you'd run this lab at scale.
- The layered method (physical → underlay → overlay → tenant → QoS) is what most scenario questions test.

Review questions

1. BGP is up on every link but large pings fail between tenants' hosts across leaves. What do you check first?
2. Why doesn't `cl-resource-query` give useful numbers on Cumulus VX?
3. Which WJH drop reason would you expect for Fault C on real hardware?
4. What does a baseline add to a health check?
5. Name two NetQ checks that would have caught Faults A and C.

Answers

1. MTU on every hop of the path, including VXLAN overhead (host 9000 + 50 bytes must fit the underlay MTU).
2. The data comes from the Spectrum ASIC's forwarding tables. VX forwards in the Linux kernel, so there's no ASIC table to report.
3. L2 *Ingress VLAN filtering* on the leaf port where gpu02's frames arrive, because VLAN 110 is no longer allowed on the bridge.
4. It turns "is this normal?" into a diff. Faults show up as differences from a known-good state.

5. `netq check mtu` (Fault A) and `netq check evpn` (Fault C; it flags VNI inconsistencies across VTEPs).

Lab 8 — An InfiniBand Fabric with ibsim: SM, HA, Partitions and Diagnostics

Lab at a glance

Book chapters	11 (IB architecture and SM), 12 (provisioning and HA), 13 (PKeys), 14 (routing), 15 and 20 (monitoring and health), 17 (diagnostic toolkit)
Exam objectives	3.1, 3.2, 3.3, 3.4, 5.4, 5.5
Time	90 minutes
Air resources	hxa-ibsim only (2 vCPU, 2 GiB); the rest of the simulation can stay running or be stopped
Prerequisites	Lab 0

Objectives

- Build a small Helix-A InfiniBand fabric in the **ibsim** simulator.
- Run **OpenSM**, watch it discover the fabric, assign LIDs and program forwarding tables.
- Use the Chapter 17 toolkit: **sminfo**, **ibnodes**, **ibswitches**, **ibhosts**, **ibnetdiscover**, **iblinkinfo**, **smpquery**, **ibroute**, **ibtracert**, **perfquery**, **ibqueryerrors**, **saquery**.
- Run a **master and a standby SM**, then fail over and fail back (Chapter 12).
- Create **partitions** (PKeys) for the RESEARCH, PRODUCTION and SHARED tenants (Chapter 13) and prove membership.
- Inject **symbol errors** and a **failed link**, and find them the way you would on a real fabric (Chapters 15, 17 and 20).

Background

DSX Air cannot simulate InfiniBand switches, so this lab uses **ibsim**, the open-source InfiniBand fabric simulator from the linux-rdma project. ibsim models switches and HCAs from a text topology file in the same format that **ibnetdiscover** prints. The real OpenSM and the real **infiniband-diags** tools run against it through a small library, **libumad2sim.so**, that redirects their management datagrams (MADs) from the kernel to the simulator. The helper **ibsim-run <command>** loads that library for you.

What you practise is therefore genuine: the same SM state machine, LID assignment, PKey tables, forwarding tables and diagnostic tools that run on a Quantum fabric. The simulator has limits: links report SDR speed (2.5 Gb/s per lane) rather than NDR, sysfs-based tools such as **ibstat** and **ibv_devinfo** don't see simulated HCAs, and NVIDIA-specific tools (ibdiagnet, UFM, SHARP, the **ar_updn** routing engine) aren't part of it.

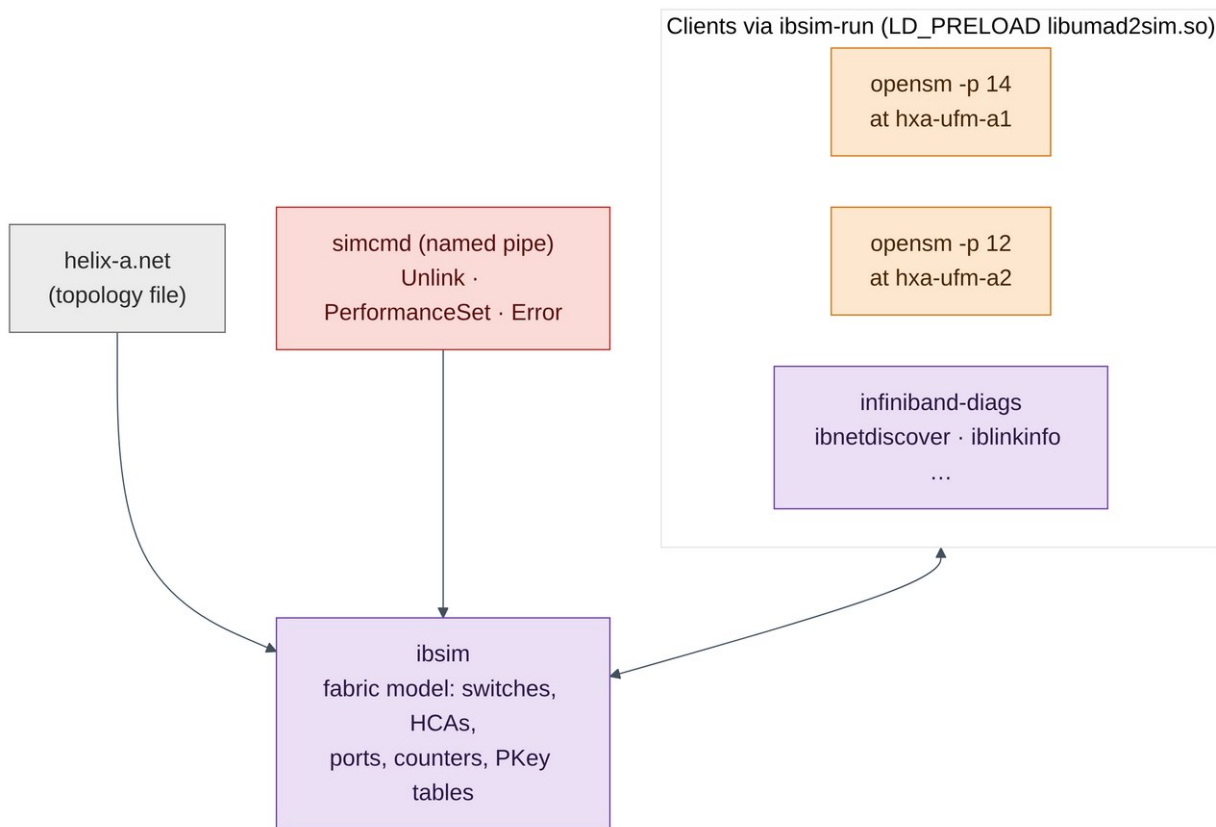


Figure 8.1: How ibsim works

ibsim holds the fabric model. OpenSM and the diagnostic tools run unchanged, with `libumad2sim.so` redirecting their MADs to the simulator instead of a real HCA. Commands typed into the simulator console (fed through a named pipe) change the fabric, for example by failing a link.

The simulated Helix-A fabric

The fabric is a two-SU slice of Helix-A: two spines, four rail leaves, four GPU nodes with two HCAs each (one per rail), and the two UFM servers of the HA pair. The first node in the file, `hxa-ufm-a1`, is where `ibsim` attaches clients by default, so that is where the master SM runs, just as UFM hosts the SM on Helix-A.

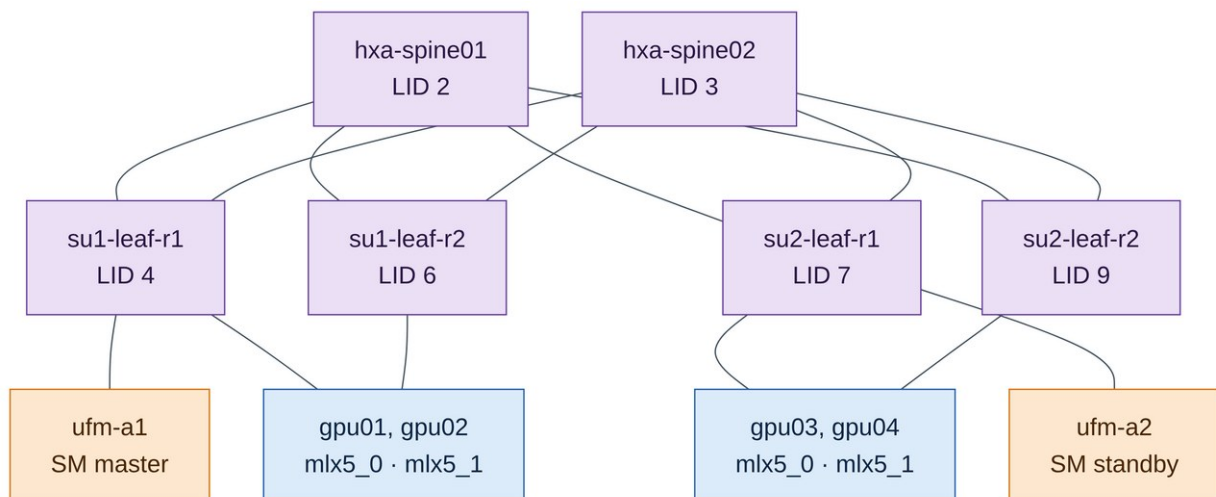


Figure 8.2: The Helix-A mini fabric in ibsim

Leaf ports 1–2 go to GPU HCAs, port 30 to a UFM server, and ports 33 and 34 are uplinks to spine01 and spine02. Rail 1 HCAs (`mlx5_0`) connect to the r1 leaves, rail 2 HCAs (`mlx5_1`) to the r2 leaves.

Step-by-step

Task 1 — Install the simulator and tools on hxa-ibsim

1. From the oob-mgmt-server, log in and become root. Running everything as root keeps the simulator and its clients under the same user, which ibsim needs.

```
ubuntu@oob-mgmt-server:~$ ssh ubuntu@hxa-ibsim
ubuntu@hxa-ibsim:~$ sudo -i
root@hxa-ibsim:~#
```

2. Install ibsim, OpenSM and the diagnostic tools. They are in Ubuntu 24.04's **universe** repository and download through the oob-mgmt-server's NAT gateway.

```
root@hxa-ibsim:~# apt-get update
root@hxa-ibsim:~# apt-get install -y ibsim-utils opensm infiniband-diags
root@hxa-ibsim:~# ibsim-run -h | head -3
ibsim-run: run commands with an ibsim-smulated stack
```

Checkpoint

- `[ ] which ibsim opensm ibnetdiscover` prints three paths.

Task 2 — Create the fabric file and start the simulator

1. Create a working directory and the topology file. Download **helix-a.net** from the companion site, or type it in from **Appendix B**.

```
root@hxa-ibsim:~# mkdir -p ~/ibsim && cd ~/ibsim
root@hxa-ibsim:~/ibsim# nano helix-a.net          # paste Appendix B, save
root@hxa-ibsim:~/ibsim# grep -c '^Switch' helix-a.net ; grep -c '^Hca' helix-a.net
6
10
```

2. Each node record is a header line (**Switch 36 "name"** or **Hca 1 "name"**) followed by one line per connected port: `[local-port] "remote-node"[remote-port]`. Look at one leaf:

```
Switch 36 "hxa-su1-leaf-r1"
[1]    "hxa-gpu01 mlx5_0"[1]
[2]    "hxa-gpu02 mlx5_0"[1]
[30]   "hxa-ufm-a1 mlx5_0"[1]
[33]   "hxa-spine01"[1]
[34]   "hxa-spine02"[1]
```

3. Start the simulator in the background. Its console reads commands from a **named pipe**, **simcmd**, so you can change the fabric later without a second terminal.

```
root@hxa-ibsim:~/ibsim# mkfifo simcmd
root@hxa-ibsim:~/ibsim# nohup sh -c 'tail -f ~/ibsim/simcmd | ibsim -s ~/ibsim/helix-a.net' >
~/ibsim/sim.log 2>&1 &
root@hxa-ibsim:~/ibsim# echo 'Help' > simcmd ; sleep 1 ; grep -A6 'Commands:' sim.log
sim> Commands:
    !<filename> - run commands from the file
    Start network
    Dump ["nodeid"] : dump node information in network
    Route <from-lid> <to-lid>
    Link "nodeid"[port] "remoteid"[port]
    ReLink "nodeid" : restore previously unconnected link(s) of the node
```

Field note: `sim.log` also contains many `ibwarn: parse_port_connection_data: cannot parse remote lid` lines. They are harmless: the parser expects the optional LID fields that a real `ibnetdiscover` dump contains.

Task 3 — Start the Subnet Manager and discover the fabric

1. Start OpenSM on `hxa-ufm-a1` with priority 14 (Chapter 12: the preferred master gets the highest priority, 0–15).

```
root@hxa-ibsim:~/ibsim# nohup ibsim-run opensm -p 14 -f ~/ibsim/opensm-a1.log > osm-a1.out 2>&1 &
root@hxa-ibsim:~/ibsim# sleep 8 ; grep -E 'Priority|state' osm-a1.out
Priority = 14
Entering DISCOVERING state
Entering MASTER state
```

2. Confirm which SM is master:

```
root@hxa-ibsim:~/ibsim# ibsim-run sminfo 2>/dev/null
sminfo: sm lid 1 sm guid 0x100001, activity count 506 priority 14 state 3 SMINFO_MASTER
```

3. Inventory the fabric (Chapter 17, section 17.3):

```
root@hxa-ibsim:~/ibsim# alias ib='ibsim-run'          # saves typing; tools print ibwarn noise on
stderr
root@hxa-ibsim:~/ibsim# ib ibswitches 2>/dev/null
Switch : 0x0000000000200005 ports 36 "hxa-su2-leaf-r2" base port 0 lid 9 lmc 0
Switch : 0x0000000000200004 ports 36 "hxa-su2-leaf-r1" base port 0 lid 7 lmc 0
Switch : 0x0000000000200003 ports 36 "hxa-su1-leaf-r2" base port 0 lid 6 lmc 0
Switch : 0x0000000000200001 ports 36 "hxa-spine02" base port 0 lid 3 lmc 0
Switch : 0x0000000000200000 ports 36 "hxa-spine01" base port 0 lid 2 lmc 0
Switch : 0x0000000000200002 ports 36 "hxa-su1-leaf-r1" base port 0 lid 4 lmc 0
root@hxa-ibsim:~/ibsim# ib ibhosts 2>/dev/null | wc -l
10
```

4. Build the LID and port-GUID map you'll need for partitions. `ibnetdiscover -p` prints one line per port:

```
root@hxa-ibsim:~/ibsim# ib ibnetdiscover -p 2>/dev/null | grep '^CA' | grep gpu | sort -k2 -n
CA    5  1 0x0000000000100003 4x SDR - SW    4  1 0x0000000000200002 ( 'hxa-gpu01 mlx5_0' -
'hxa-su1-leaf-r1' )
CA    8  1 0x0000000000100005 4x SDR - SW    6  1 0x0000000000200003 ( 'hxa-gpu01 mlx5_1' -
'hxa-su1-leaf-r2' )
CA   10  1 0x0000000000100007 4x SDR - SW    4  2 0x0000000000200002 ( 'hxa-gpu02 mlx5_0' -
'hxa-su1-leaf-r1' )
CA   11  1 0x0000000000100009 4x SDR - SW    6  2 0x0000000000200003 ( 'hxa-gpu02 mlx5_1' -
'hxa-su1-leaf-r2' )
CA   12  1 0x000000000010000b 4x SDR - SW    7  1 0x0000000000200004 ( 'hxa-gpu03 mlx5_0' -
'hxa-su2-leaf-r1' )
CA   13  1 0x000000000010000d 4x SDR - SW    9  1 0x0000000000200005 ( 'hxa-gpu03 mlx5_1' -
'hxa-su2-leaf-r2' )
CA   14  1 0x000000000010000f 4x SDR - SW    7  2 0x0000000000200004 ( 'hxa-gpu04 mlx5_0' -
'hxa-su2-leaf-r1' )
CA   15  1 0x0000000000100011 4x SDR - SW    9  2 0x0000000000200005 ( 'hxa-gpu04 mlx5_1' -
'hxa-su2-leaf-r2' )
```

Read each line as: node type, **LID**, port number, **port GUID**, width and speed, then the peer's type, LID, port and GUID, then the two node descriptions. Record the table below; the rest of the lab uses it.

HCA	LID	Port GUID
hxa-gpu01 mlx5_0	5	0x0000000000100003
hxa-gpu02 mlx5_0	10	0x0000000000100007
hxa-gpu03 mlx5_0	12	0x000000000010000b
hxa-gpu04 mlx5_0	14	0x000000000010000f

Exam trap: Node GUID and port GUID are different numbers. **ibhosts** shows the node GUID (...002 for gpu01 mlx5_0); partitions and SA records use the **port GUID** (...003). Mixing them up is the most common reason a partition "doesn't work".

Checkpoint

- [] **sminfo** shows a MASTER at LID 1 with priority 14.
- [] 6 switches and 10 CAs discovered; your LID table matches.

Task 4 — Read ports, links and forwarding tables

1. Check one HCA port the way Chapter 11 describes the port state machine:

```
root@hxa-ibsim:~/ibsim# ib smquery portinfo 5 1 2>/dev/null | grep -E 'SMLid|LinkState|
PhysLinkState|LinkWidthActive|LinkSpeedActive'
SMLid:.....1
LinkWidthActive:.....4X
LinkState:.....Active
PhysLinkState:.....LinkUp
LinkSpeedActive:.....2.5 Gbps
```

Active + LinkUp + 4X is healthy. (A real NDR port shows 106.25 Gbps per lane; the simulator models SDR.)

2. See every link from the switches' point of view (Chapter 17, **iblinkinfo**):

```
root@hxa-ibsim:~/ibsim# ib iblinkinfo --switches-only -l 2>/dev/null | grep 'hxa-su1-leaf-r1' |
head -4
0x0000000000200002 "          hxa-su1-leaf-r1"      4    1[ ] == ( 4X          2.5 Gbps
Active/ LinkUp)=== 0x0000000000100002      5    1[ ] "hxa-gpu01 mlx5_0" (Could be 12X Could be
10.0 Gbps)
0x0000000000200002 "          hxa-su1-leaf-r1"      4    2[ ] == ( 4X          2.5 Gbps
Active/ LinkUp)=== 0x0000000000100006     10    1[ ] "hxa-gpu02 mlx5_0" (Could be 12X Could be
10.0 Gbps)
```

3. Dump a leaf's linear forwarding table (LFT). The SM spreads destinations across both uplinks (33 and 34):

```
root@hxa-ibsim:~/ibsim# ib ibroute 4 2>/dev/null | head -12
Unicast lids [0x0-0x10] of switch Lid 4 guid 0x0000000000200002 (hxa-su1-leaf-r1):
  Lid  Out  Destination
      Port   Info
0x0001 030 : (Channel Adapter portguid 0x0000000000100001: 'hxa-ufm-a1 mlx5_0')
0x0002 033 : (Switch portguid 0x0000000000200000: 'hxa-spine01')
0x0003 034 : (Switch portguid 0x0000000000200001: 'hxa-spine02')
0x0004 000 : (Switch portguid 0x0000000000200002: 'hxa-su1-leaf-r1')
0x0005 001 : (Channel Adapter portguid 0x0000000000100003: 'hxa-gpu01 mlx5_0')
0x0006 034 : (Switch portguid 0x0000000000200003: 'hxa-su1-leaf-r2')
0x0007 034 : (Switch portguid 0x0000000000200004: 'hxa-su2-leaf-r1')
```

```
0x0008 034 : (Channel Adapter portguid 0x0000000000100005: 'hxa-gpu01 mlx5_1')
```

- Trace the path from gpu01 rail 1 (LID 5) to gpu03 rail 1 (LID 12), which sits in the other SU:

```
root@hxa-ibsim:~/ibsim# ib ibtracert 5 12 2>/dev/null
From ca {0x0000000000100002} portnum 1 lid 5-5 "hxa-gpu01 mlx5_0"
[1] -> switch port {0x0000000000200002}[1] lid 4-4 "hxa-sul-leaf-r1"
[33] -> switch port {0x0000000000200000}[1] lid 2-2 "hxa-spine01"
[3] -> switch port {0x0000000000200004}[33] lid 7-7 "hxa-su2-leaf-r1"
[1] -> ca port {0x000000000010000b}[1] lid 12-12 "hxa-gpu03 mlx5_0"
To ca {0x000000000010000a} portnum 1 lid 12-12 "hxa-gpu03 mlx5_0"
```

Leaf → spine → leaf: exactly the rail-optimised path of Chapter 2. Traffic between two HCAs on the same rail leaf never leaves that leaf.

Check your understanding: Run `ib ibtracert 5 8` (gpu01 rail 1 to gpu01 rail 2). Why does it cross a spine even though both HCAs are in the same server? (Answer: rail 1 and rail 2 are separate leaves; in a rail-optimised fabric, cross-rail traffic goes leaf → spine → leaf. In a real node NCCL keeps it on NVLink instead.)

- Read the SM's routing choice from its log. OpenSM's default engine is **minhop**; NVIDIA's OpenSM in UFM adds **ar_updn** for adaptive routing (Chapter 14), which this open-source build doesn't include.

```
root@hxa-ibsim:~/ibsim# grep -iE 'routing engine|ucast' ~/ibsim/opensm-a1.log | head -3
```

Checkpoint

- [] You can explain each field of an `iblinkinfo` line and an `ibroute` entry.
- [] You traced a cross-SU path leaf → spine → leaf.

Task 5 — Master and standby SM; failover and failback

- Start a second OpenSM on `hxa-ufm-a2` with a lower priority (12). The `SIM_HOST` variable tells `umad2sim` which simulated node to attach to.

```
root@hxa-ibsim:~/ibsim# SIM_HOST="hxa-ufm-a2 mlx5_0" nohup ibsim-run opensm -p 12 -f
~/ibsim/opensm-a2.log > osm-a2.out 2>&1 &
root@hxa-ibsim:~/ibsim# sleep 10 ; grep 'state' osm-a2.out
Entering DISCOVERING state
Entering STANDBY state
```

- Fail the master by stopping it (the equivalent of losing `ufm-a1`):

```
root@hxa-ibsim:~/ibsim# pgrep -af 'opensm -p 14'
root@hxa-ibsim:~/ibsim# kill <pid-from-above>
```

- Watch the standby. It polls the master and takes over after several missed polls; in the simulator this takes about a minute.

```
root@hxa-ibsim:~/ibsim# sleep 90 ; grep 'state' osm-a2.out | tail -1
Entering MASTER state
root@hxa-ibsim:~/ibsim# ib sminfo 2>/dev/null
sminfo: sm lid 16 sm guid 0x100013, activity count 788 priority 12 state 3 SMINFO_MASTER
```

(The activity count will differ.) The master is now LID 16, `hxa-ufm-a2`. Existing LIDs and forwarding tables were kept: a failover does not renumber the fabric.

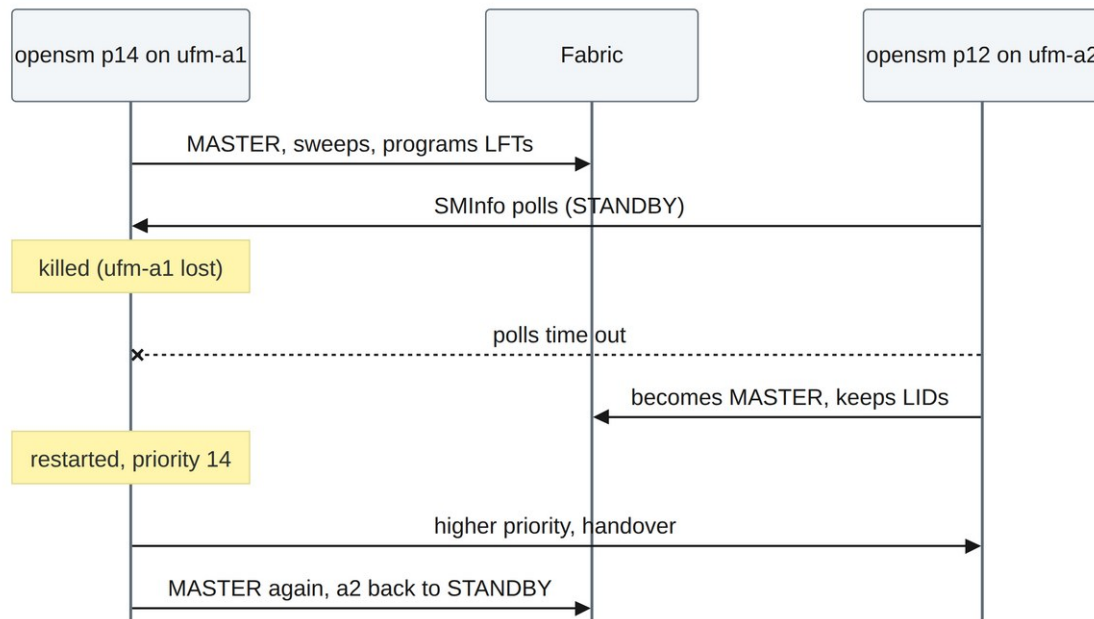


Figure 8.3: SM failover and failback in the lab

The standby polls the master. When polls fail, it takes over as master with the existing LIDs. When the higher-priority SM returns, it negotiates and takes mastership back (handover).

4. Fail back: restart the priority-14 SM on **hxa-ufm-a1**. Because it has the higher priority, it negotiates mastership back.

```

root@hxa-ibsim:~/ibsim# nohup ibsim-run opensm -p 14 -f ~/ibsim/opensm-a1.log > osm-a1.out 2>&1 &
root@hxa-ibsim:~/ibsim# sleep 45 ; ib sminfo 2>/dev/null
sminfo: sm lid 1 sm guid 0x100001, activity count 483 priority 14 state 3 SMINFO_MASTER
  
```

Exam focus: Highest priority wins; on equal priority the lowest port GUID wins. In UFM HA (Chapter 12) the SM follows the UFM master through Pacemaker, so you don't normally run two independent SMs by hand. This task shows the SM-level behaviour underneath.

Checkpoint

- [] You saw STANDBY → MASTER on ufm-a2 after the master failed.
- [] The priority-14 SM took mastership back.

Task 6 — Partitions (PKeys) for tenant isolation

1. Write **partitions.conf** using the **port GUIDs** from Task 3. RESEARCH gets gpu01 and gpu03 (rail 1), PRODUCTION gets gpu02 and gpu04 (rail 1), and SHARED is a limited partition that every port can use to talk to a full member only (the UFM server).

```

root@hxa-ibsim:~/ibsim# cat > partitions.conf <<'EOF'
Default=0x7fff, ipoib : ALL=full ;
RESEARCH=0x8001, ipoib : 0x0000000000100003=full, 0x000000000010000b=full ;
PRODUCTION=0x8002, ipoib : 0x0000000000100007=full, 0x000000000010000f=full ;
SHARED=0x0100 : ALL=limited, 0x0000000000100001=full ;
EOF
  
```

2. Restart the master SM with the partition file (**-P**). In production UFM manages this file for you (Chapter 13).

```

root@hxa-ibsim:~/ibsim# pkill -f 'opensm -p 14' ; sleep 2
root@hxa-ibsim:~/ibsim# nohup ibsim-run opensm -p 14 -P ~/ibsim/partitions.conf -f
  
```

```
~/ibsim/opensm-a1.log > osm-a1.out 2>&1 &
root@hxa-ibsim:~/ibsim# sleep 45 ; ib sminfo 2>/dev/null | grep -o 'priority 14.*'
priority 14 state 3 SMINFO_MASTER
```

3. Read the PKey tables the SM programmed. gpu03 rail 1 (LID 12) is in RESEARCH; gpu01 rail 2 (LID 8) is not:

```
root@hxa-ibsim:~/ibsim# ib smpquery pkeys 12 2>/dev/null | head -1
0: 0xffff 0x0100 0x8001 0x0000 0x0000 0x0000 0x0000 0x0000
root@hxa-ibsim:~/ibsim# ib smpquery pkeys 8 2>/dev/null | head -1
0: 0xffff 0x0100 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
```

Decode them with Chapter 13: **0xffff** is the default partition with the membership bit set (full member). **0x0100** is SHARED with the membership bit **clear**, so these ports are **limited** members and can talk only to full members of 0x0100. **0x8001** is RESEARCH, full member.

4. Change membership and reload without restarting. Add gpu02 rail 1 to RESEARCH, then send OpenSM a HUP so it re-reads the file:

```
root@hxa-ibsim:~/ibsim# sed -i 's/0x00000000010000b=full ;/0x00000000010000b=full,
0x000000000100007=full ;/' partitions.conf
root@hxa-ibsim:~/ibsim# pkill -HUP -f 'opensm -p 14' ; sleep 20
root@hxa-ibsim:~/ibsim# ib smpquery pkeys 10 2>/dev/null | head -1
0: 0xffff 0x0100 0x8002 0x8001 0x0000 0x0000 0x0000 0x0000
```

gpu02 rail 1 is now in both RESEARCH and PRODUCTION.

Warning: OpenSM discards partition lines it can't parse. Always check the log after a change: **grep -i partition ~/ibsim/opensm-a1.log | tail**.

Checkpoint

- [] LID 12 shows 0x8001; LID 8 does not.
- [] After the HUP, LID 10 shows 0x8001.

Task 7 — Counters, errors and a failed link

1. Read counters on a leaf uplink (Chapter 15). All zero on a healthy port:

```
root@hxa-ibsim:~/ibsim# ib perfquery 4 33 2>/dev/null | head -8
# Port counters: Lid 4 port 33 (CapMask: 0x1300)
PortSelect:.....33
CounterSelect:.....0x0000
SymbolErrorCounter:.....0
LinkErrorRecoveryCounter:.....0
LinkDownedCounter:.....0
PortRcvErrors:.....0
```

2. Inject a marginal cable: set symbol errors and link-downed events on **hxa-su1-leaf-r1** port 33 through the simulator console.

```
root@hxa-ibsim:~/ibsim# echo 'PerformanceSet "hxa-su1-leaf-r1"[33]
PortCounters.SymbolErrorCounter=250' > simcmd
root@hxa-ibsim:~/ibsim# echo 'PerformanceSet "hxa-su1-leaf-r1"[33]
PortCounters.LinkDownedCounter=3' > simcmd
```

3. Find it the way an operator would, with a fabric-wide error sweep (Chapter 17):

```

root@hxa-ibsim:~/ibsim# ib ibqueryerrors 2>/dev/null
Errors for 0x200002 "hxa-su1-leaf-r1"
  GUID 0x200002 port ALL: [SymbolErrorCounter == 250] [LinkDownedCounter == 3]
  GUID 0x200002 port 33: [SymbolErrorCounter == 250] [LinkDownedCounter == 3]

## Summary: 16 nodes checked, 1 bad nodes found
##          226 ports checked, 1 ports have errors beyond threshold

```

Diagnosis per Chapters 15 and 20: symbol errors plus link-down events on one uplink point to a marginal cable or optic between **hxa-su1-leaf-r1** port 33 and **hxa-spine01** port 1. On a real fabric you'd confirm with ibdiagnet's cable and BER checks and UFM's health report, isolate the port and replace the cable. Clear the counters afterwards (**perfquery -R**) so a recurrence is visible.

```

root@hxa-ibsim:~/ibsim# ib perfquery -R 4 33 2>/dev/null >/dev/null ; ib ibqueryerrors
2>/dev/null | tail -2

```

4. Now fail a link completely: unplug the spine02 uplink of **hxa-su2-leaf-r2**.

```

root@hxa-ibsim:~/ibsim# echo 'Unlink "hxa-su2-leaf-r2"[34]' > simcmd ; sleep 20
root@hxa-ibsim:~/ibsim# ib iblinkinfo --switches-only -l 2>/dev/null | grep 'hxa-su2-leaf-r2' |
grep -E ' 3[34]\['
0x00000000000200005 "          hxa-su2-leaf-r2"      9  33[ ]==( 4X          2.5 Gbps
Active/ LinkUp)==> 0x00000000000200000      2   4[ ] "hxa-spine01" (Could be 12X Could be 10.0
Gbps)
0x00000000000200005 "          hxa-su2-leaf-r2"      9  34[ ]==(          Down/
Polling)==>      [ ] "" ( )

```

Port 34 is **Down/Polling**: no physical link partner. The SM's next sweep reroutes: check **ib ibroute 9** and you'll see every destination now leaves through port 33 only. The fabric still works at half the uplink capacity on that leaf, which is exactly the "silent" degradation that Chapter 15 warns about.

5. Restore the link:

```

root@hxa-ibsim:~/ibsim# echo 'Relink "hxa-su2-leaf-r2"[34]' > simcmd ; sleep 20
root@hxa-ibsim:~/ibsim# ib iblinkinfo --switches-only -l 2>/dev/null | grep 'hxa-su2-leaf-r2' |
grep ' 34\['

```

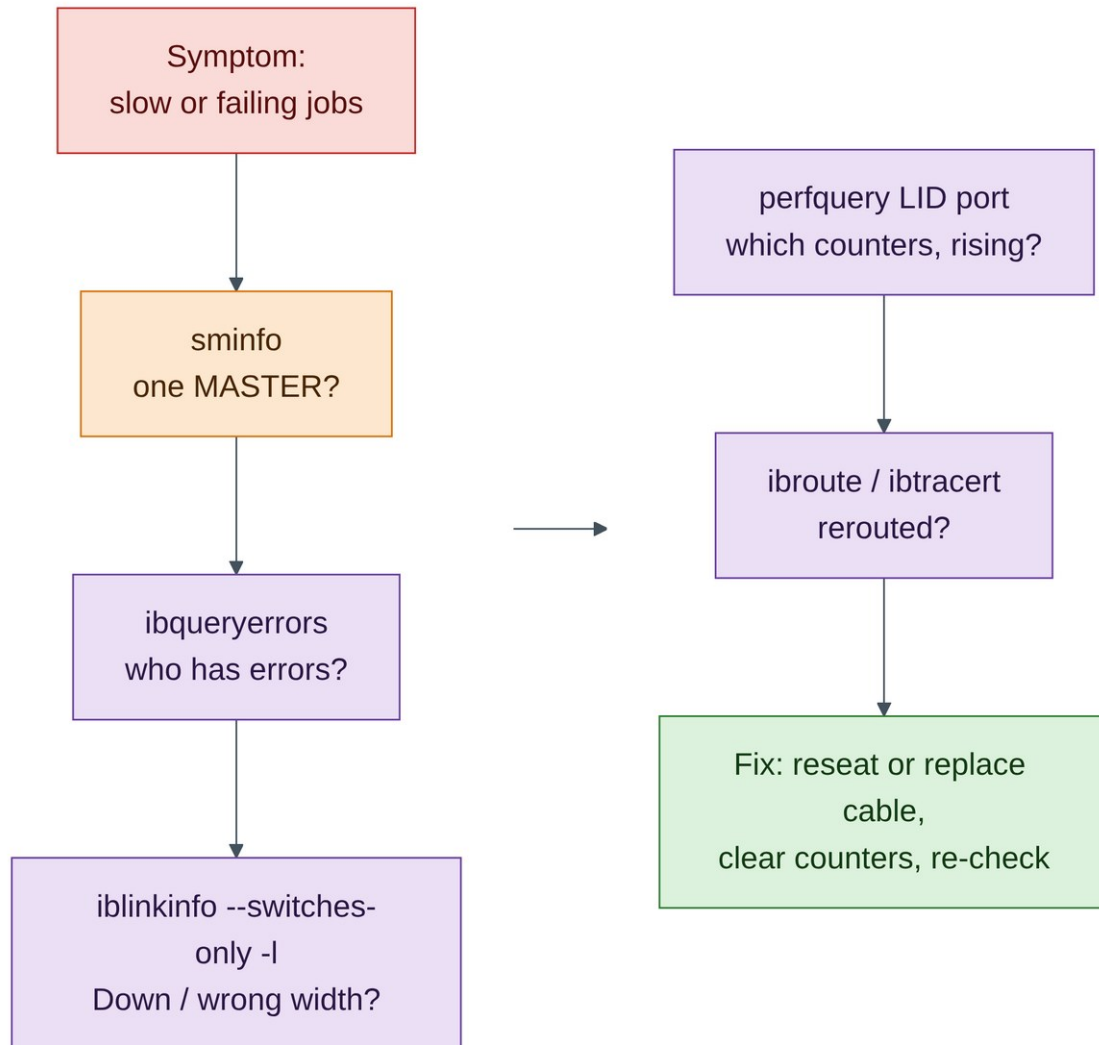


Figure 8.4: Finding a fabric problem with the toolkit

Start wide (errors and down links across the fabric), then narrow to the port, its counters and its routes. The same order works on a real fabric, where *ibdiagnet* and *UFM* add cable, BER and health data.

Checkpoint

- [] **ibqueryerrors** found the symbol errors on leaf LID 4 port 33.
- [] **iblinkinfo** showed port 34 Down/Polling, and it recovered after ReLink.

Verify

- [] Master SM on hxa-ufm-a1 (priority 14); standby on hxa-ufm-a2.
- [] Failover and failback both observed.
- [] PKeys 0x8001, 0x8002 and limited 0x0100 visible in PKey tables.
- [] Error injection and link failure found with **ibqueryerrors** and **iblinkinfo**.

Break and fix

Fault 1: "RESEARCH members can't reach each other."

- *Inject:* in `partitions.conf`, replace `gpu03`'s port GUID `...10000b` with its node GUID `...10000a`, then `pkill -HUP -f 'opensm -p 14'`.
- *Symptom:* `ib smpquery pkeys 12` no longer shows `0x8001`.
- *Diagnosis:* the partition references a GUID that isn't a port GUID. Compare with `ib ibnetdiscover -p`.
- *Fix:* use the port GUID and reload.

Fault 2: "Two masters fight after maintenance."

- *Inject:* start the `ufm-a2` SM with `-p 14` as well.
- *Symptom:* both logs show handover messages; mastership may move unexpectedly.
- *Diagnosis:* equal priorities, so the tie-break falls to the lowest GUID.
- *Fix:* set a clear priority order (14 on the preferred master, lower on the standby), as in Chapter 12.

Clean-up / save state

The simulator runs in memory. To stop it: `pkill -f opensm ; pkill -f 'ibsim -s' ; pkill -f 'tail -f'`. To restart later, repeat Tasks 2 (step 3), 3 (step 1), 5 (step 1) and 6 (step 2). Your files in `~/ibsim` survive a simulation checkpoint.

Exam tie-in

- 3.1: SM discovery, LID assignment, master/standby election by priority and GUID, failover without renumbering.
- 3.2: PKeys, full versus limited membership, port GUIDs in `partitions.conf`, reload with SIGHUP (UFM does this for you).
- 3.3: forwarding tables and routing engines (minhop here; `ar_updn` for AR on NVIDIA OpenSM).
- 3.4, 5.4: counters and error sweeps; interpreting `SymbolErrorCounter` and `LinkDownedCounter`.
- 5.5: `sminfo`, `ibnodes`, `ibswitches`, `ibhosts`, `ibnetdiscover`, `iblinkinfo`, `perfquery`, `ibqueryerrors`, `ibtracert`, `ibroute`.

Review questions

1. What does **Down/Polling** on a switch port mean?
2. Why does `partitions.conf` need port GUIDs rather than node GUIDs?
3. After the master SM failed, did the LIDs change? Why does that matter?
4. A PKey table shows `0x0100` rather than `0x8100`. What kind of member is the port?
5. Which two tools would you run first to find a marginal cable in a large fabric?

Answers

1. The port has no physical link: the port is down and the PHY is polling for a partner. Typical causes are an unplugged or failed cable or a powered-off peer.
2. Partition membership is enforced per port: the SM writes PKeys into each port's PKey table, identified by the port GUID. A node GUID identifies the whole device, not the port.
3. No. The standby took over with the existing LIDs and forwarding tables. Renumbering would break every established connection and address handle, so a failover must be invisible to running jobs.
4. A limited member: the top (membership) bit is clear. It can communicate only with full members of that partition, not with other limited members.
5. `ibqueryerrors` (fabric-wide error sweep) and `iblinkinfo` (link state, width and speed per port). Then `perfquery` on the suspect port to see which counters are rising. On a real fabric, `ibdiagnet` and UFM's health report add cable and BER data.

Lab 9 — Verifying End-to-End Performance with perftest over Soft-RoCE

Lab at a glance

Book chapters	19 (perftest and NCCL tests), 3 (RDMA fundamentals), 5 (RoCE marking)
Exam objectives	5.3 Verify low-latency network performance · 5.5 <code>ib_write_bw</code> / <code>ib_write_lat</code>
Time	60 minutes
Air resources	hxb-gpu01, hxb-gpu02 and the Ethernet fabric
Prerequisites	Lab 3 (Soft-RoCE tools installed on gpu01), Lab 5 (AURORA working across leaves)

Objectives

- Apply Chapter 19's **bottom-up test ladder** to a real (software) RDMA path across the EVPN fabric.
- Run the perftest suite: `ib_write_bw`, `ib_write_lat`, `ib_read_lat`, `ib_send_bw`, with RDMA-CM and with explicit GID indexes.
- Read latency output (`t_min`, `t_typical`, `t_avg`, 99% and 99.9% percentiles) and bandwidth sweeps.
- See how MTU changes the RoCE path MTU and the results.
- Diagnose the two classic setup mistakes: wrong GID index and MTU mismatch.

Background

On real hardware, Chapter 19's ladder goes: link health → single-pair RDMA with perftest → GPUDirect RDMA → multi-node NCCL. In Air you can climb the first two rungs with **Soft-RoCE** (`rdma_rxe`), the Linux kernel's software implementation of RoCEv2. It speaks exactly the same verbs and wire protocol as a ConnectX NIC, so every perftest option, the GID table and RDMA-CM behave as on hardware. What it can't match is speed: the NIC's work is done by a virtual CPU, so expect well under 10 Gb/s and tens of microseconds of latency. **In this lab the method matters, not the numbers.** Never compare these results with the hardware figures in Chapter 19.

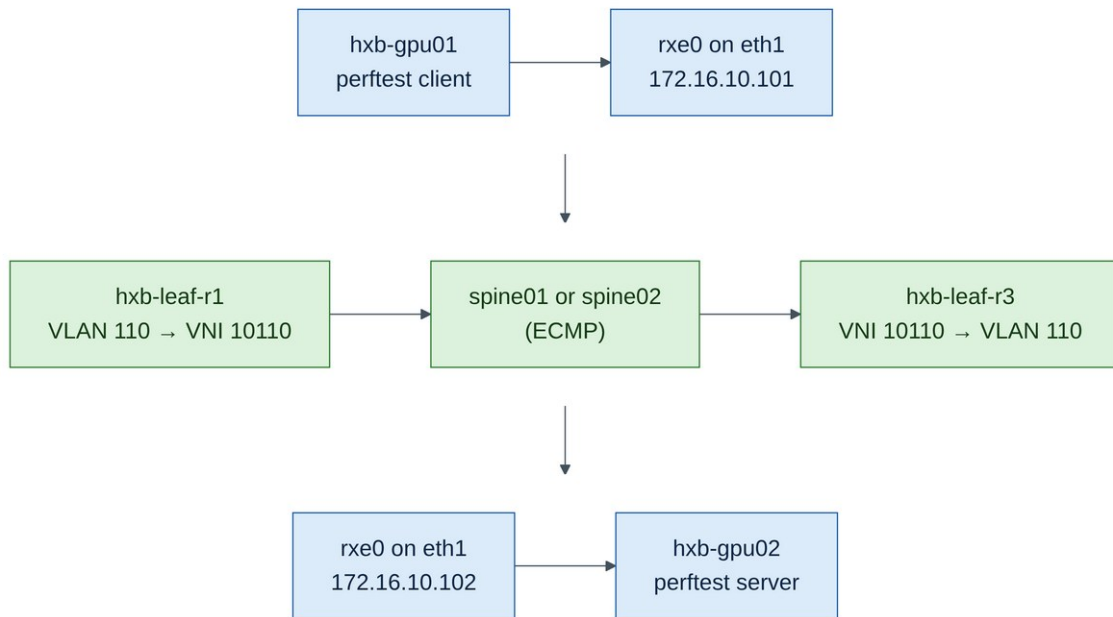


Figure 9.1: The RDMA path in this lab

gpu01's rxe0 sends RoCEv2 (UDP 4791) on eth1 into leaf-r1. The frame is bridged into VNI 10110, VXLAN-encapsulated across a spine and decapsulated on leaf-r3, which delivers it to gpu02 eth1.

Step-by-step

Task 1 — Rung 1: prove the path before testing RDMA

Chapter 19's first rule: never run perftest on a path you haven't checked.

1. Addresses and MTU on both hosts (Lab 5 set MTU 9000):

```
ubuntu@hxb-gpu01:~$ ip -br addr show eth1 ; ip link show eth1 | grep -o 'mtu [0-9]*'
eth1          UP          172.16.10.101/24
mtu 9000
```

2. Reachability with the largest frame that fits, with fragmentation forbidden:

```
ubuntu@hxb-gpu01:~$ ping -c 3 -M do -s 8972 172.16.10.102
3 packets transmitted, 3 received, 0% packet loss
```

8972 = 9000 - 20 (IP) - 8 (ICMP). If this fails while a small ping works, stop and fix MTU first (Lab 7, Fault A).

Checkpoint

- [] Jumbo ping gpu01 → gpu02 succeeds.

Task 2 — Soft-RoCE on both hosts

gpu01 has the tools from Lab 3; gpu02 doesn't yet. On **hxb-gpu02**:

```
ubuntu@hxb-gpu02:~$ sudo apt-get update && sudo apt-get install -y rdma-core ibverbs-utils
ibverbs-providers perftest
ubuntu@hxb-gpu02:~$ sudo modprobe rdma_rxe
ubuntu@hxb-gpu02:~$ sudo rdma link add rxe0 type rxe netdev eth1
```

On **hxb-gpu01**, check that rxex0 still exists (it doesn't survive a reboot or checkpoint restore) and recreate it if not:

```
ubuntu@hxb-gpu01:~$ rdma link show || true
ubuntu@hxb-gpu01:~$ sudo modprobe rdma_rxe ; sudo rdma link add rxex0 type rxe netdev eth1
2>/dev/null ; rdma link show
link rxex0/1 state ACTIVE physical_state LINK_UP netdev eth1
```

Check the port attributes on both hosts. Note **active_mtu**: the RoCE path MTU is the largest InfiniBand MTU (256–4096) that fits in the Ethernet MTU.

```
ubuntu@hxb-gpu01:~$ ibv_devinfo -d rxex0 | grep -E 'state|max_mtu|active_mtu|link_layer'
state:                PORT_ACTIVE (4)
max_mtu:              4096 (5)
active_mtu:           4096 (5)
link_layer:           Ethernet
```

(*Illustrative.*) With Ethernet MTU 9000 the active MTU is 4096; in Lab 3 at MTU 1500 it was 1024.

Find the RoCEv2 IPv4 GID index on each host (Lab 3, Task 4, step 5). With a fresh rxe device on eth1 it is usually 1:

```
ubuntu@hxb-gpu02:~$ ibv_devinfo -v -d rxex0 | grep GID
GID[ 0]:              fe80::5054:ff:fe12:3402, RoCE v2
GID[ 1]:              ::ffff:172.16.10.102, RoCE v2
```

Checkpoint

- [] **rdma link show** ACTIVE on both; active_mtu 4096; GID index recorded for both.

Task 3 — Rung 2: bandwidth with RDMA-CM

1. Server on gpu02, client on gpu01. **-R** uses RDMA-CM to exchange connection data and pick the right GID automatically; **-T 106** sets traffic class 106 (DSCP 26 + ECT(0)) as in Lab 3.

```
ubuntu@hxb-gpu02:~$ ib_write_bw -d rxex0 -R -T 106 -F --report_gbits
ubuntu@hxb-gpu01:~$ ib_write_bw -d rxex0 -R -T 106 -F --report_gbits 172.16.10.102
```

Expected output (client, illustrative):

#bytes	#iterations	BW peak[Gb/sec]	BW average[Gb/sec]	MsgRate[Mpps]
65536	5000	2.91	2.63	0.005016

2. Sweep all message sizes (**-a**) to see where bandwidth levels off:

```
ubuntu@hxb-gpu02:~$ ib_write_bw -d rxex0 -R -a -F --report_gbits
ubuntu@hxb-gpu01:~$ ib_write_bw -d rxex0 -R -a -F --report_gbits 172.16.10.102 | tee ~/bw-sweep.txt
```

Plot or sketch bandwidth against message size. The shape is the lesson: small messages are limited by per-message overhead (message rate), large messages by the path (here, the CPU doing the NIC's work).

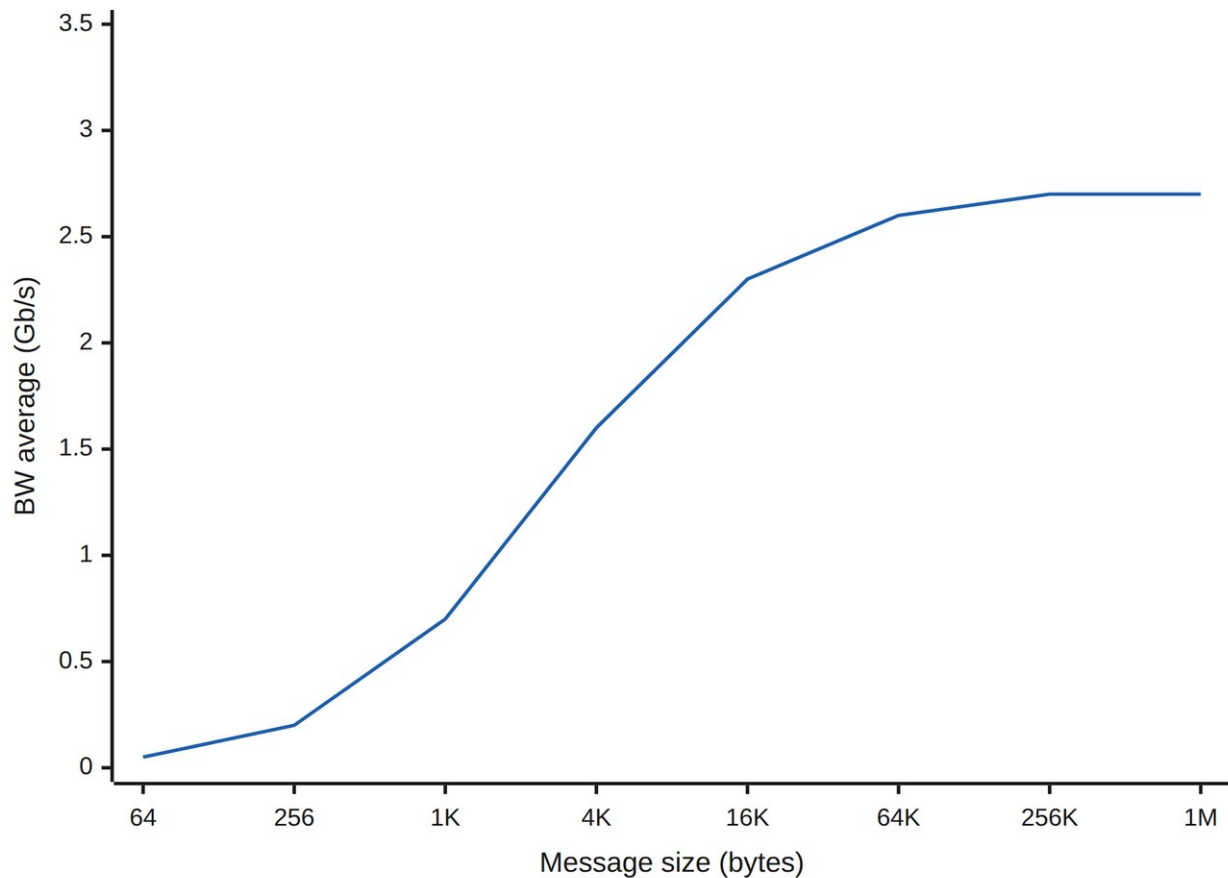


Figure 9.2: Bandwidth against message size (illustrative Soft-RoCE results)

Small messages are overhead-bound; bandwidth climbs and flattens once each message is large enough to amortise per-message cost. Hardware has the same shape at far higher numbers.

3. Try a send/receive test for comparison. Two-sided operations need the receiver to post buffers, so results differ slightly from RDMA WRITE:

```
ubuntu@hxb-gpu02:~$ ib_send_bw -d rxe0 -R -F --report_gbits
ubuntu@hxb-gpu01:~$ ib_send_bw -d rxe0 -R -F --report_gbits 172.16.10.102
```

Task 4 — Latency and tail latency

1. Write latency. It's a ping-pong test and reports **half the round trip**:

```
ubuntu@hxb-gpu02:~$ ib_write_lat -d rxe0 -R -F
ubuntu@hxb-gpu01:~$ ib_write_lat -d rxe0 -R -F 172.16.10.102
```

Expected output (illustrative):

#bytes	#iterations	t_min[usec]	t_max[usec]	t_typical[usec]	t_avg[usec]
2	1000	38.12	412.55	51.40	55.83
118.61		390.07			14.20

Read it the Chapter 19 way:

- **t_typical** (the median) is the number to quote. **t_min** is a best case you'll rarely see.
- The **99% and 99.9% percentiles** are the tail. Collective operations wait for the slowest message, so the tail matters more than the average in AI clusters.

- A large gap between typical and 99.9% means jitter. Here it comes from virtual CPUs being scheduled; on hardware it points to congestion, PFC pauses or a noisy neighbour.
2. Read latency reports the full request-response time, so expect roughly double:

```
ubuntu@hxb-gpu02:~$ ib_read_lat -d rxe0 -R -F
ubuntu@hxb-gpu01:~$ ib_read_lat -d rxe0 -R -F 172.16.10.102
```

3. Run the write latency test three times and record `t_typical` and 99.9% each time. Variance between runs is itself a finding.

Run	t_typical (µs)	99.9% (µs)
1		
2		
3		

Task 5 — Explicit GID index, and why it matters

Without RDMA-CM, perf test exchanges connection data over a TCP socket and uses the GID index you give with `-x`. Use the index you recorded in Task 2.

```
ubuntu@hxb-gpu02:~$ ib_write_bw -d rxe0 -x 1 --tclass=106 -F --report_gbits
ubuntu@hxb-gpu01:~$ ib_write_bw -d rxe0 -x 1 --tclass=106 -F --report_gbits 172.16.10.102
```

It should give the same result as Task 3. On ConnectX NICs the IPv4 RoCEv2 GID is often index 3, and NCCL uses `NCCL_IB_GID_INDEX` (Chapter 19) or picks it automatically in recent releases.

Task 6 — MTU experiment

1. Lower the Ethernet MTU on **both** hosts to 1500 and re-read `active_mtu`:

```
ubuntu@hxb-gpu01:~$ sudo ip link set eth1 mtu 1500 ; ibv_devinfo -d rxe0 | grep active_mtu
active_mtu:          1024 (3)
```

2. Re-run the 64 KB `ib_write_bw` test and compare with Task 3. More, smaller packets per message means more per-packet work; on Soft-RoCE that usually lowers bandwidth.
3. Restore MTU 9000 on both hosts: `sudo netplan apply` (Lab 5's netplan file sets 9000).

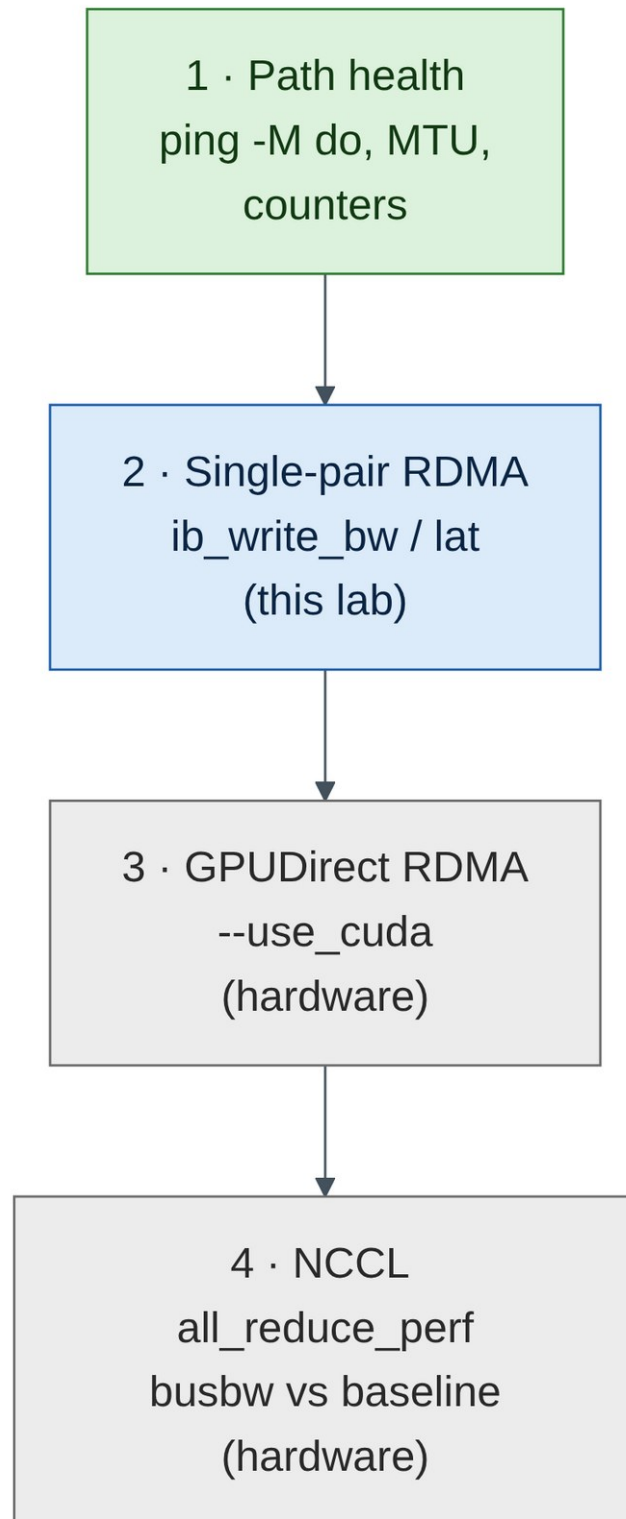


Figure 9.3: Chapter 19's test ladder, and how far Air takes you

Rungs 1–2 run in this lab. Rungs 3–4 need GPUs and ConnectX NICs; the method is the same.

Verify

- [] Jumbo ping works `gpu01 → gpu02`.
- [] `ib_write_bw` runs with `-R` and with `-x <index>`; a bandwidth sweep is saved in `~/bw-sweep.txt`.
- [] Latency recorded three times with `t_typical` and 99.9%.

- [] MTU experiment done and MTU restored to 9000.

Break and fix

Fault 1 — wrong GID index

- *Inject*: run the client with `-x 0` (the link-local IPv6 GID) while the server uses `-x 1`.
- *Symptom*: the test hangs or fails with errors such as `Unable to Connect the HCA's through the link` or a completion error with a retry-exceeded status.
- *Diagnosis*: `ibv_devinfo -v -d rxe0 | grep GID` on both hosts; the two sides must use GIDs they can route to each other (both the IPv4 RoCEv2 GID here).
- *Fix*: use `-x 1` on both sides, or `-R` so RDMA-CM chooses.

Fault 2 — MTU mismatch

- *Inject*: set MTU 1500 on gpu02 eth1 only.
- *Symptom*: small messages work; large `ib_write_bw` stalls or reports retries; jumbo ping from gpu01 fails.
- *Diagnosis*: Task 1's jumbo ping, and `active_mtu` differs between hosts.
- *Fix*: `sudo netplan apply` on gpu02 to restore 9000.

Clean-up / save state

Soft-RoCE devices aren't persistent. That's fine: Labs 10 and 13 recreate them if needed. Save switch configuration and store a checkpoint.

Exam tie-in

- 5.3: the bottom-up ladder; always prove path and MTU before blaming RDMA.
- 5.5: `ib_write_bw` / `ib_write_lat` server and client roles, `-d`, `-x`, `-R`, `-a`, `-F`, `--report_gbits`, `-T/--tclass`.
- Reading latency: typical versus tail percentiles; write latency is half round trip.
- GID index selection for RoCEv2 and why RDMA-CM avoids the problem.

Review questions

1. Why do you run `ping -M do -s 8972` before `perftest`?
2. What is the difference between `-R` and `-x 1`?
3. Which latency figure should you quote, and which should you watch for AI workloads?
4. Why is `active_mtu` 4096 with an Ethernet MTU of 9000, not 9000?
5. Why can't you compare this lab's results with Chapter 19's numbers?

Answers

1. To prove the path carries full-size frames without fragmentation. Otherwise an MTU problem looks like an RDMA problem.
2. `-R` uses RDMA-CM, which resolves addresses and chooses the GID automatically. `-x 1` exchanges connection data over a TCP socket and uses the GID at index 1 that you chose.
3. Quote `t_typical` (median); watch the 99% and 99.9% percentiles, because collectives wait for the slowest message.
4. RoCE uses InfiniBand path MTUs (256, 512, 1024, 2048, 4096). 4096 is the largest, and it fits inside a 9000-byte Ethernet frame with the RoCEv2 headers.
5. Soft-RoCE runs the NIC's work on a virtual CPU. Bandwidth and latency reflect CPU scheduling, not a ConnectX adapter or the fabric.

Lab 10 — Kubernetes and the NVIDIA Network Operator

Lab at a glance

Book chapters	16 (NVIDIA Network Operator: deploy and verify)
Exam objectives	4.1 Deploy the Network Operator · 4.2 Verify Network Operator functionality
Time	90 minutes
Air resources	hxb-gpu01 (control plane + worker), hxb-gpu02 (worker), EVPN fabric from Lab 5
Prerequisites	Lab 5 (AURORA VLAN 110 working between gpu01 and gpu02)

Objectives

- Build a two-node Kubernetes cluster with kubeadm on hxb-gpu01 and hxb-gpu02.
- Install the NVIDIA Network Operator with Helm in namespace `nvidia-network-operator`.
- Apply a **NicClusterPolicy** that deploys the secondary-network stack (Multus, CNI plugins, NV-IPAM).
- Create an NV-IPAM **IPPool** and a **MacvlanNetwork** on eth1 (AURORA 172.16.10.0/24).
- Attach pods to the high-speed network with the networks annotation and ping pod-to-pod across the EVPN fabric.
- Verify the operator the way objective 4.2 expects.

Background

On Helix, training pods need a second interface on the RDMA fabric in addition to the default cluster network. The Network Operator (Chapter 16) deploys everything needed from one custom resource, the NicClusterPolicy: the DOCA driver container, RDMA device plugins, Multus, CNI plugins and NV-IPAM. Network CRs such as MacvlanNetwork, HostDeviceNetwork and IPoIBNetwork then generate NetworkAttachmentDefinitions that pods request by annotation.

Air's servers have virtio NICs, not ConnectX, so this lab deploys only the parts that don't need NVIDIA hardware: **Multus, CNI plugins and NV-IPAM**. The driver container and RDMA device plugins are explained but left out. Everything else (the operator, the CRDs, the reconcile loop, the NADs and the pod attachments) is real.

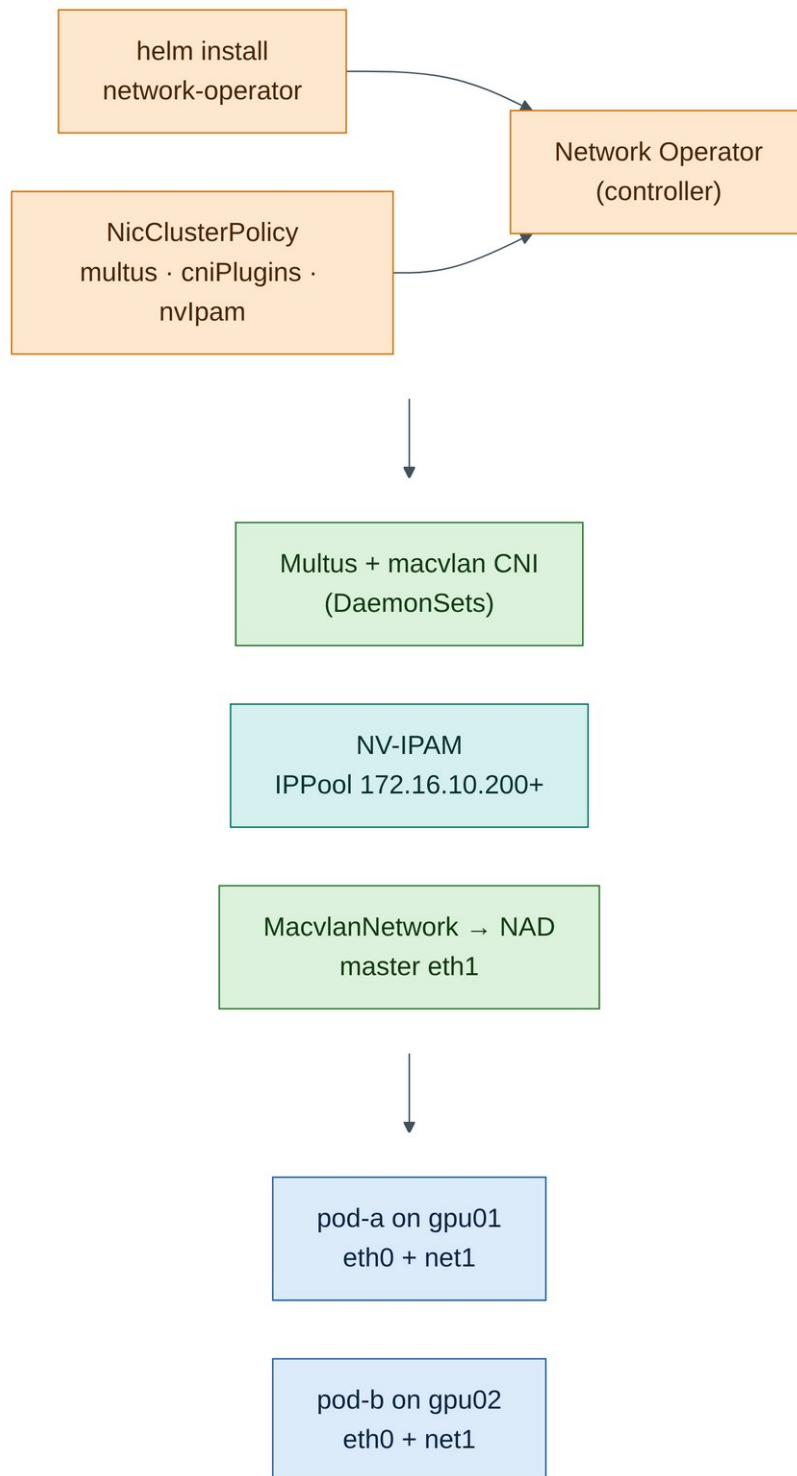


Figure 10.1: What this lab deploys

The operator reconciles the `NicClusterPolicy` into `Multus`, CNI plugins and `NV-IPAM` on both nodes. The `MacvlanNetwork` becomes a `NetworkAttachmentDefinition`; pods that name it in their annotation get `net1` on `eth1`, with an address from the IPPool.

Version note: Kubernetes, the Network Operator and container images move quickly. This lab was written against the Kubernetes 1.34 package repository and Network Operator 26.7 (image tag `network-operator-v26.7.0`, as in NVIDIA's quick-start guides at the time of writing). Use the versions that `helm search repo` and the Network Operator release notes show for your date.

Step-by-step

Task 1 — Prepare both nodes (gpu01 and gpu02)

Run every step in this task on **both** servers. The cluster uses the OOB network (eth0) for Kubernetes traffic, and eth1 stays free for the secondary network.

1. Record each node's eth0 address; you'll need gpu01's.

```
ubuntu@hxb-gpu01:~$ ip -4 -br addr show eth0
eth0                UP                192.168.200.21/24
```

(Illustrative: the OOB network uses 192.168.200.0/24.)

2. Kernel settings for Kubernetes networking, and swap off:

```
ubuntu@hxb-gpu01:~$ sudo swapoff -a ; sudo sed -i 's/^/#/' /etc/fstab
ubuntu@hxb-gpu01:~$ printf 'overlay\nbr_netfilter\n' | sudo tee /etc/modules-load.d/k8s.conf
ubuntu@hxb-gpu01:~$ sudo modprobe overlay ; sudo modprobe br_netfilter
ubuntu@hxb-gpu01:~$ printf 'net.bridge.bridge-nf-call-iptables=1\nnet.bridge.bridge-nf-call-
ip6tables=1\nnet.ipv4.ip_forward=1\n' | sudo tee /etc/sysctl.d/k8s.conf
ubuntu@hxb-gpu01:~$ sudo sysctl --system >/dev/null
```

3. Install containerd and use the systemd cgroup driver:

```
ubuntu@hxb-gpu01:~$ sudo apt-get update && sudo apt-get install -y containerd apt-transport-https
ca-certificates curl gpg
ubuntu@hxb-gpu01:~$ sudo mkdir -p /etc/containerd
ubuntu@hxb-gpu01:~$ containerd config default | sudo tee /etc/containerd/config.toml >/dev/null
ubuntu@hxb-gpu01:~$ sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
/etc/containerd/config.toml
ubuntu@hxb-gpu01:~$ sudo systemctl restart containerd
```

4. Install kubeadm, kubelet and kubectrl from the Kubernetes package repository:

```
ubuntu@hxb-gpu01:~$ KVER=v1.34
ubuntu@hxb-gpu01:~$ curl -fsSL https://pkgs.k8s.io/core:/stable:/$KVER/deb/Release.key | sudo gpg
--dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
ubuntu@hxb-gpu01:~$ echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/$KVER/deb/ /" | sudo tee
/etc/apt/sources.list.d/kubernetes.list
ubuntu@hxb-gpu01:~$ sudo apt-get update && sudo apt-get install -y kubelet kubeadm kubectrl
ubuntu@hxb-gpu01:~$ sudo apt-mark hold kubelet kubeadm kubectrl
```

Checkpoint

- [] **kubeadm version** works on both nodes; containerd is active.

Task 2 — Create the cluster

1. On **gpu01**, initialise the control plane on the OOB address, with the pod CIDR that Flannel expects:

```
ubuntu@hxb-gpu01:~$ sudo kubeadm init --apiserver-advertise-address=192.168.200.21 --pod-network-
cidr=10.244.0.0/16
...
Your Kubernetes control-plane has initialized successfully!
...
kubeadm join 192.168.200.21:6443 --token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:1f2e...
```

(*Illustrative: copy your own join command.*)

2. Set up kubectl for the **ubuntu** user and let pods run on the control-plane node (a two-node lab needs both):

```
ubuntu@hxb-gpu01:~$ mkdir -p ~/.kube && sudo cp /etc/kubernetes/admin.conf ~/.kube/config && sudo
chown $(id -u):$(id -g) ~/.kube/config
ubuntu@hxb-gpu01:~$ kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

3. Install the Flannel pod network (the primary CNI, **eth0** in pods):

```
ubuntu@hxb-gpu01:~$ kubectl apply -f
https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
```

4. On **gpu02**, run the join command from step 1 with **sudo**.

5. Back on gpu01:

```
ubuntu@hxb-gpu01:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES          AGE   VERSION   INTERNAL-IP
hxb-gpu01     Ready     control-plane   6m    v1.34.x   192.168.200.21
hxb-gpu02     Ready     <none>          1m    v1.34.x   192.168.200.22
```

(*Illustrative.*)

Checkpoint

- [] Both nodes **Ready**; **kubectl get pods -A** shows Flannel and CoreDNS running.

Task 3 — Install Helm and the Network Operator

```
ubuntu@hxb-gpu01:~$ curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-
3 | bash
ubuntu@hxb-gpu01:~$ helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
ubuntu@hxb-gpu01:~$ helm repo update
ubuntu@hxb-gpu01:~$ helm search repo nvidia/network-operator --versions | head -4
NAME                                CHART VERSION   APP VERSION     DESCRIPTION
nvidia/network-operator             26.7.0          v26.7.0         Nvidia network operator
...
```

(*Illustrative.*) Install the version shown at the top:

```
ubuntu@hxb-gpu01:~$ helm install network-operator nvidia/network-operator \
-n nvidia-network-operator --create-namespace --version 26.7.0 --wait
ubuntu@hxb-gpu01:~$ kubectl -n nvidia-network-operator get pods
NAME                                READY   STATUS    RESTARTS
network-operator-6d8c7b9f9-4kq2x    1/1     Running   0
network-operator-node-feature-discovery-master-... 1/1     Running   0
network-operator-node-feature-discovery-worker-... 1/1     Running   0
```

(*Illustrative.*) At this point only the controller (and Node Feature Discovery, if the chart enables it) runs. Nothing is deployed to the nodes until you create a NicClusterPolicy.

```
ubuntu@hxb-gpu01:~$ kubectl api-resources | grep -iE 'mellanox|nv-ipam'
```

You should see the CRDs from Chapter 16: **nicclusterpolicies**, **macvlannetworks**, **hostdevicenetworks**, **ipoibnetworks**, and NV-IPAM's **ippools** and **cidrpools**.

Task 4 — Apply a NicClusterPolicy for the secondary network

```
ubuntu@hxb-gpu01:~$ mkdir -p ~/netop && cat > ~/netop/nicclusterpolicy.yaml <<'EOF'
apiVersion: mellanox.com/v1alpha1
kind: NicClusterPolicy
metadata:
  name: nic-cluster-policy
spec:
  # No ofedDriver / rdmaSharedDevicePlugin: Air servers have no NVIDIA NICs.
  secondaryNetwork:
    multus:
      image: multus-cni
      repository: nvcr.io/nvidia/mellanox
      version: network-operator-v26.7.0
    cniPlugins:
      image: plugins
      repository: nvcr.io/nvidia/mellanox
      version: network-operator-v26.7.0
  nvIpam:
    image: nvidia-k8s-ipam
    repository: nvcr.io/nvidia/mellanox
    version: network-operator-v26.7.0
    enableWebhook: false
EOF
ubuntu@hxb-gpu01:~$ kubectl apply -f ~/netop/nicclusterpolicy.yaml
ubuntu@hxb-gpu01:~$ kubectl -n nvidia-network-operator get pods -w
```

Wait until the Multus, CNI-plugins and NV-IPAM pods are **Running** on both nodes (Ctrl+C to stop watching), then check the policy status:

```
ubuntu@hxb-gpu01:~$ kubectl get nicclusterpolicy nic-cluster-policy -o jsonpath='{.status.state}'
{"\n"}
ready
```

Exam focus: "NicClusterPolicy **ready**" is the first thing objective 4.2 asks you to verify. If it stays **notReady**, **kubectl describe nicclusterpolicy nic-cluster-policy** lists which component is failing, and the operator logs say why.

Checkpoint

- [] NicClusterPolicy state is **ready**; component pods Running on both nodes.
- [] **ls /etc/cni/net.d/** on each node shows a Multus configuration file.

Task 5 — IPPool and MacvlanNetwork on eth1

1. Create an NV-IPAM pool inside AURORA's 172.16.10.0/24. Keep it clear of host addresses (.101, .102) and the gateway (.1): hand out .200 onwards.

```
ubuntu@hxb-gpu01:~$ cat > ~/netop/ippool.yaml <<'EOF'
apiVersion: nv-ipam.nvidia.com/v1alpha1
kind: IPPool
metadata:
  name: aurora-pool
  namespace: nvidia-network-operator
spec:
  subnet: 172.16.10.0/24
  perNodeBlockSize: 8
  gateway: 172.16.10.1
```

```

exclusions:
  - startIP: 172.16.10.1
    endIP: 172.16.10.199
EOF
ubuntu@hxb-gpu01:~$ kubectl apply -f ~/netop/ippool.yaml
ubuntu@hxb-gpu01:~$ kubectl -n nvidia-network-operator get ippool aurora-pool -o yaml | grep -A8
'status:'

```

The status shows the per-node blocks NV-IPAM allocated, one block of 8 addresses per node.

Version note: If your NV-IPAM release rejects **exclusions**, remove that block and use a smaller dedicated range by choosing a narrower **subnet** (for example **172.16.10.192/27**) with the same gateway.

2. Create the MacvlanNetwork. The operator turns it into a NetworkAttachmentDefinition in the **default** namespace.

```

ubuntu@hxb-gpu01:~$ cat > ~/netop/macvlan.yaml <<'EOF'
apiVersion: mellanox.com/v1alpha1
kind: MacvlanNetwork
metadata:
  name: aurora-net
spec:
  networkNamespace: "default"
  master: "eth1"
  mode: "bridge"
  mtu: 9000
  ipam: |
    {
      "type": "nv-ipam",
      "poolName": "aurora-pool"
    }
EOF
ubuntu@hxb-gpu01:~$ kubectl apply -f ~/netop/macvlan.yaml
ubuntu@hxb-gpu01:~$ kubectl get macvlannetwork aurora-net -o jsonpath='{.status.state}{"\n"}'
ready
ubuntu@hxb-gpu01:~$ kubectl get network-attachment-definitions
NAME          AGE
aurora-net    10s

```

Task 6 — Pods on the high-speed network

1. Two test pods, pinned one per node:

```

ubuntu@hxb-gpu01:~$ cat > ~/netop/pods.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: pod-a
  annotations:
    k8s.v1.cni.cncf.io/networks: aurora-net
spec:
  nodeName: hxb-gpu01
  containers:
  - name: net
    image: nicolaka/netshoot
    command: ["sleep", "infinity"]
---
```

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-b
  annotations:
    k8s.v1.cni.cncf.io/networks: aurora-net
spec:
  nodeName: hxb-gpu02
  containers:
  - name: net
    image: nicolaka/netshoot
    command: ["sleep", "infinity"]
EOF
ubuntu@hxb-gpu01:~$ kubectl apply -f ~/netop/pods.yaml
ubuntu@hxb-gpu01:~$ kubectl get pods -o wide

```

2. Check that each pod has two interfaces:

```

ubuntu@hxb-gpu01:~$ kubectl exec pod-a -- ip -br addr
lo                UNKNOWN      127.0.0.1/8
eth0@if12         UP           10.244.0.12/24
net1@if3          UP           172.16.10.200/24
ubuntu@hxb-gpu01:~$ kubectl exec pod-b -- ip -br addr | grep net1
net1@if3          UP           172.16.10.208/24

```

(*Illustrative addresses.*) **eth0** is the Flannel cluster network; **net1** is the macvlan interface on the node's eth1, in AURORA.

3. Ping pod-to-pod across the EVPN fabric, and from a pod to a host and to the gateway:

```

ubuntu@hxb-gpu01:~$ kubectl exec pod-a -- ping -c 3 -I net1 172.16.10.208
ubuntu@hxb-gpu01:~$ kubectl exec pod-a -- ping -c 3 -I net1 172.16.10.102
ubuntu@hxb-gpu01:~$ kubectl exec pod-a -- ping -c 3 -I net1 172.16.10.1

```

pod-a's frames leave gpu01 eth1 into leaf-r1, cross the fabric in VNI 10110 and reach pod-b on gpu02 behind leaf-r3. Confirm on leaf-r1 that EVPN learned the pods' MAC addresses:

```

cumulus@hxb-leaf-r1:mgmt:~$ sudo vtysh -c 'show evpn mac vni 10110'

```

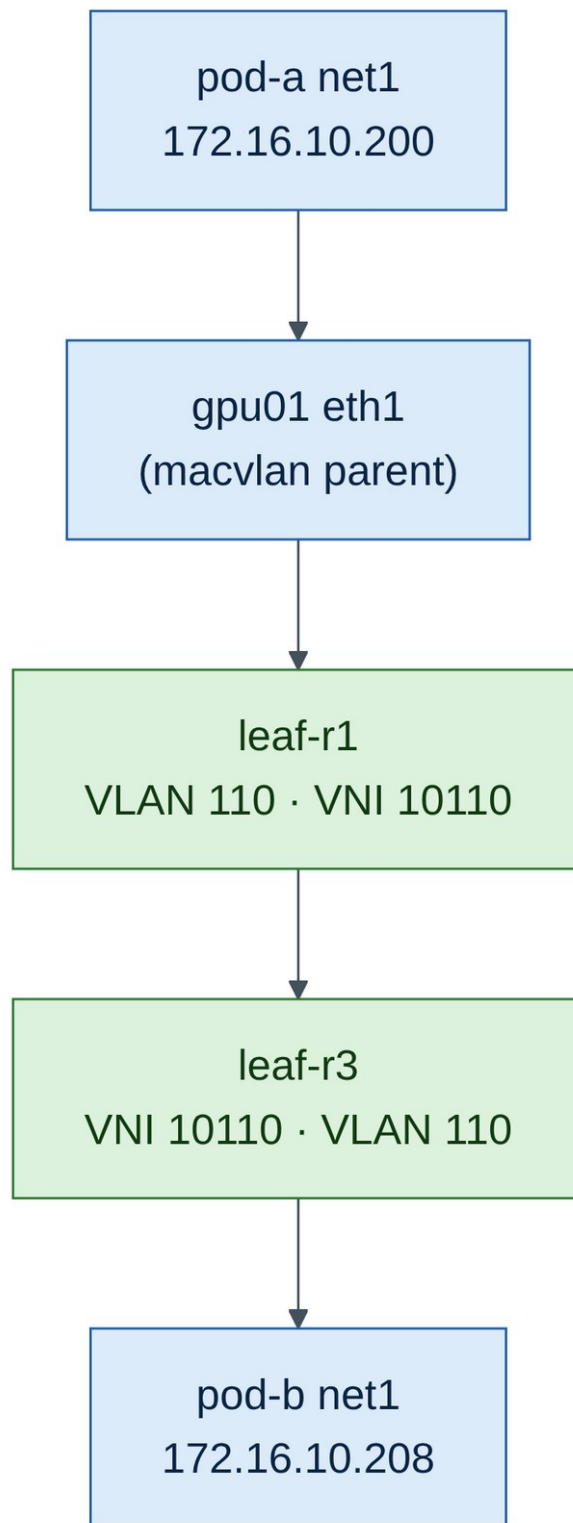


Figure 10.2: Pod-to-pod across the fabric

Each pod's net1 is a macvlan child of the node's eth1, so to the fabric a pod looks like another host in VLAN 110. EVPN advertises its MAC and IP like any server.

Checkpoint

- [] Both pods Running with net1 in 172.16.10.0/24 from the IPPool.
- [] pod-a ↔ pod-b ping works over net1; EVPN on leaf-r1 shows the pods' MACs.

Task 7 — What RDMA hardware would add

On Helix-B's real nodes you'd add these to the same NicClusterPolicy (Chapter 16), and each needs NVIDIA NICs:

Component	What it adds	Why it isn't in this lab
<code>ofedDriver</code> (DOCA driver container)	Installs the DOCA-OFED drivers on each node	Needs ConnectX/BlueField hardware
<code>rdmaSharedDevicePlugin</code>	Advertises <code>rdma/rdma_shared_device_a</code> in node Allocatable; pods request it with <code>IPC_LOCK</code>	Selects RDMA devices of physical NICs
SR-IOV device plugin / network operator	One VF per pod per rail (nvidia.com/rail0 ...) — the Spectrum-X quick start pattern	Needs SR-IOV capable NICs
NIC Configuration Operator	Applies the Spectrum-X NIC profile (<code>spectrumX0ptimized</code>)	Firmware settings on SuperNICs
<code>ib-kubernetes</code>	Adds pod GUIDs to UFM PKeys on InfiniBand	Needs UFM and an IB fabric

With those in place, the verification ladder from Chapter 16 continues: Allocatable shows the RDMA resource, the pod sees `/dev/infiniband`, `ibv_devinfo` works in the pod, and `ib_write_bw` runs pod-to-pod.

Verify

- [] Operator pods Running; NicClusterPolicy `ready`.
- [] IPPool allocated per-node blocks; MacvlanNetwork `ready`; NAD `aurora-net` exists.
- [] Pods got net1 addresses from the pool and reach each other across leaves.

Break and fix

Fault 1 — wrong master interface

- *Inject*: change `master: "eth1"` to `master: "ens1f0"` in `macvlan.yaml`, apply, and recreate pod-b (`kubectl delete pod pod-b && kubectl apply -f ~/netop/pods.yaml`).
- *Symptom*: pod-b stuck in `ContainerCreating`.
- *Diagnosis*: `kubectl describe pod pod-b` shows a Multus/CNI error such as `failed to lookup master "ens1f0": Link not found`.
- *Fix*: set `master: "eth1"`, re-apply, recreate the pod.

Fault 2 — pool exhausted

- *Inject*: set `perNodeBlockSize: 1` in the IPPool (on a new pool name, say `tiny-pool`, referenced by a new MacvlanNetwork) and start two pods on the same node.
- *Symptom*: the second pod stays in `ContainerCreating` with an IPAM allocation error.
- *Diagnosis*: `kubectl get ippool tiny-pool -o yaml` shows a one-address block per node.
- *Fix*: size `perNodeBlockSize` for the pods per node you expect.

Clean-up / save state

```
ubuntu@hxb-gpu01:~$ kubectl delete -f ~/netop/pods.yaml
```

Leave the cluster in place if you plan to revisit it; it survives checkpoints. If memory is tight for later labs, `sudo systemctl stop kubelet containerd` on both nodes frees it.

Exam tie-in

- 4.1: Helm install into `nvidia-network-operator`; NicClusterPolicy decides what gets deployed; network CRs (Macvlan, HostDevice, IPoIB, SR-IOV) create NADs.
- 4.2: verify operator pods, NicClusterPolicy `ready`, node resources, NADs, pod interfaces, and finally an RDMA test between pods.
- Know which components need NVIDIA hardware (driver container, RDMA device plugin, SR-IOV, NIC configuration) and which don't (Multus, CNI plugins, NV-IPAM).

Review questions

1. What does the Network Operator deploy before you create a NicClusterPolicy?
2. How does a pod ask for the secondary network?
3. Which resource would a real RoCE pod request in addition, and which capability does it need?
4. `kubectl get nicclusterpolicy` shows `notReady`. What are your next two commands?
5. Why does pod-b's net1 appear to the fabric as another host in VLAN 110?

Answers

1. Only the operator controller (plus Node Feature Discovery if enabled in the chart). Components are deployed when a NicClusterPolicy exists.
2. With the annotation `k8s.v1.cni.cncf.io/networks: <network-name>`, which Multus resolves to the NetworkAttachmentDefinition created by the network CR.
3. An RDMA resource such as `rdma/rdma_shared_device_a` (or an SR-IOV VF resource), and the `IPC_LOCK` capability so it can pin memory.
4. `kubectl describe nicclusterpolicy nic-cluster-policy` (which component is failing) and `kubectl -n nvidia-network-operator logs deploy/network-operator` (why).
5. net1 is a macvlan child of gpu02's eth1 with its own MAC address, bridged onto eth1, which is an access port in VLAN 110. EVPN learns and advertises its MAC and IP like any server.

Lab 11 — NVUE Templates and Configuration Management

Lab at a glance

Book chapters	21 (NVUE templates and configuration management), 4 (NVUE revisions)
Exam objectives	6.1 Configure Spectrum-X using NVUE templates
Time	60 minutes
Air resources	oob-mgmt-server and the six switches
Prerequisites	Labs 1, 2 and 5

Objectives

- Keep per-switch **variables** in YAML and one **Jinja2 template** in git on the oob-mgmt-server.
- Render NVUE command files for all six switches and use them as the **golden configuration** for the underlay.
- Load them with `nv config patch`, gate with `nv config diff`, and apply with `--confirm`.
- See exactly why `nv config replace` with a partial file is dangerous, and recover safely.
- Push a change through the **NVUE REST API** (revision → PATCH → apply) from the oob-mgmt-server.
- Detect **drift** between a switch and its golden file.

Background

In Labs 1, 2 and 5 you typed configuration switch by switch. That's how drift starts (Chapter 21): six switches, six chances to miss a line. The fix is to describe each switch with a handful of variables, render its configuration from one template, keep both in git, and load the result with NVUE's file commands or REST API.

NVUE's file commands accept either YAML or a **plain text file of `nv set` / `nv unset` lines**. This lab renders the text form: it's easy to read and review, and it uses exactly the commands you already typed in Labs 1 and 2. Chapter 21 shows the YAML form, which you'd use for complete configurations.

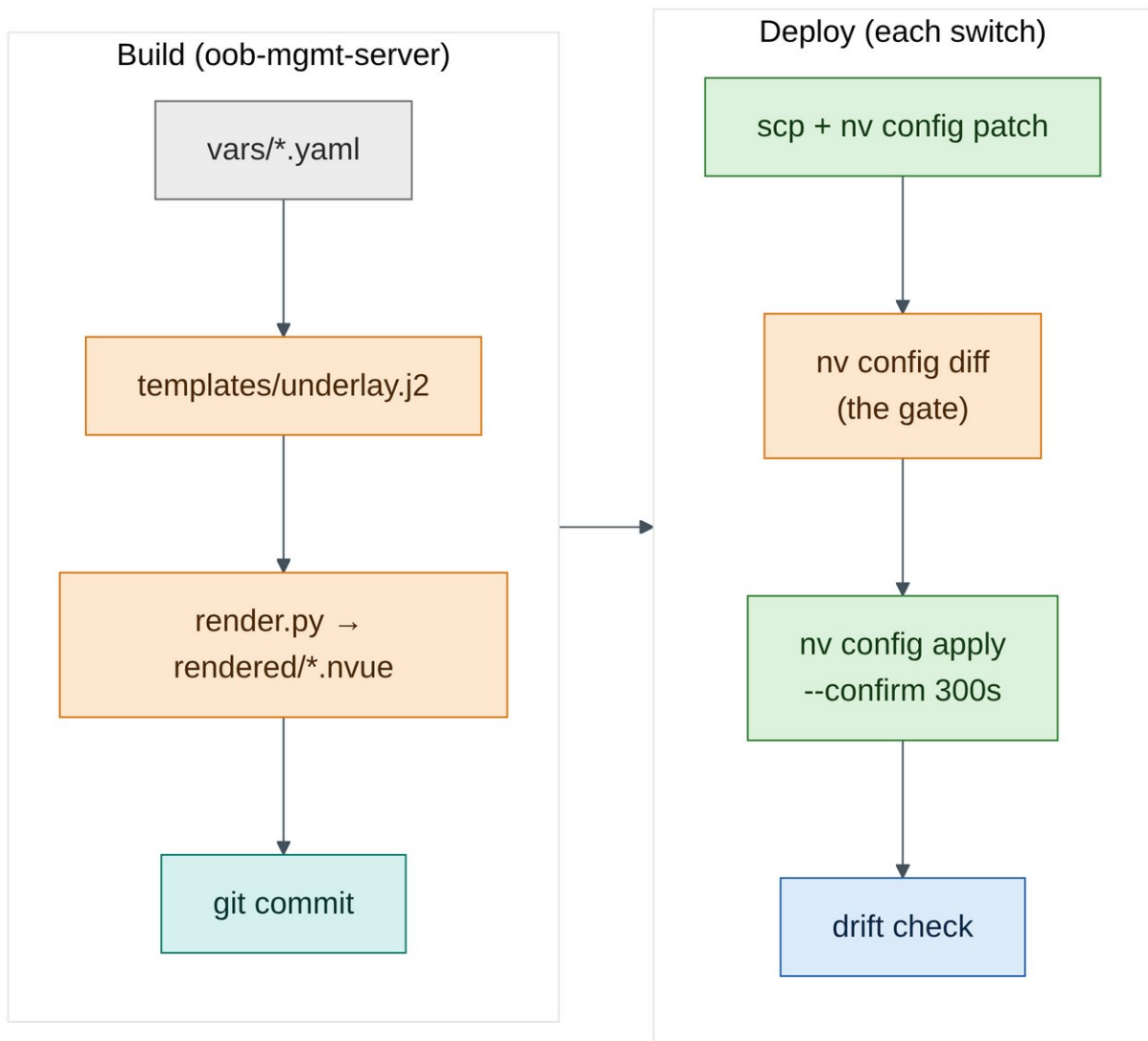


Figure 11.1: The lab's configuration pipeline

Variables and one template render a command file per switch. The file goes into git, is copied to the switch, loaded with patch, checked with diff, then applied with a confirm timer. A drift check compares the switch with the golden file afterwards.

Step-by-step

Task 1 — Set up the project on the oob-mgmt-server

```

ubuntu@oob-mgmt-server:~$ sudo apt-get update && sudo apt-get install -y python3-jinja2 python3-yaml git
ubuntu@oob-mgmt-server:~$ mkdir -p ~/helix-nvue/{vars,templates,rendered} && cd ~/helix-nvue
ubuntu@oob-mgmt-server:~/helix-nvue$ git init -q && git config user.name "Helix NetOps" && git config user.email netops@example.com
  
```

Task 2 — Variables

One file per switch. The values are the lab plan from Lab 0 and Lab 2.

```

ubuntu@oob-mgmt-server:~/helix-nvue$ cat > vars/hxb-leaf-r1.yaml <<'EOF'
hostname: hxb-leaf-r1
role: leaf
asn: 65101
loopback: 10.255.0.11
  
```

```
uplinks:
  swp31: to-hxb-spine01-swp1
  swp32: to-hxb-spine02-swp1
EOF
```

Create the other five with a small loop rather than by hand, which is the point of the exercise:

```
ubuntu@oob-mgmt-server:~/helix-nvue$ for n in 2 3 4; do
>   cat > vars/hxb-leaf-r$n.yaml <<EOF
>   hostname: hxb-leaf-r$n
>   role: leaf
>   asn: 6510$n
>   loopback: 10.255.0.1$n
>   uplinks:
>     swp31: to-hxb-spine01-swp$n
>     swp32: to-hxb-spine02-swp$n
> EOF
> done
ubuntu@oob-mgmt-server:~/helix-nvue$ for s in 1 2; do
>   cat > vars/hxb-spine0$s.yaml <<EOF
>   hostname: hxb-spine0$s
>   role: spine
>   asn: 65100
>   loopback: 10.255.0.$s
>   uplinks:
>     swp1: to-hxb-leaf-r1-swp3$s
>     swp2: to-hxb-leaf-r2-swp3$s
>     swp3: to-hxb-leaf-r3-swp3$s
>     swp4: to-hxb-leaf-r4-swp3$s
> EOF
> done
ubuntu@oob-mgmt-server:~/helix-nvue$ ls vars
hxb-leaf-r1.yaml hxb-leaf-r2.yaml hxb-leaf-r3.yaml hxb-leaf-r4.yaml hxb-spine01.yaml hxb-
spine02.yaml
```

Check your understanding: In the spine loop, why does the description end in **swp3\$s**? (Answer: *spine01 connects to each leaf's swp31 and spine02 to swp32, so the suffix is 31 or 32.*)

Task 3 — The template and the renderer

1. One template for every switch. It produces the same **nv set** lines as Labs 1 and 2, plus two NTP servers that are new:

```
ubuntu@oob-mgmt-server:~/helix-nvue$ cat > templates/underlay.j2 <<'EOF'
# {{ hostname }} - rendered from templates/underlay.j2 - do not edit by hand
nv set system hostname {{ hostname }}
nv set interface lo ip address {{ loopback }}/32
{% for port, desc in uplinks.items() %}
nv set interface {{ port }} description {{ desc }}
nv set interface {{ port }} link mtu 9216
nv set interface {{ port }} link state up
nv set vrf default router bgp neighbor {{ port }} remote-as external
{% endfor %}
nv set router bgp autonomous-system {{ asn }}
nv set router bgp router-id {{ loopback }}
nv set vrf default router bgp address-family ipv4-unicast network {{ loopback }}/32
nv set service ntp mgmt server 0.pool.ntp.org
```

```
nv set service ntp mgmt server 1.pool.ntp.org
EOF
```

2. The renderer fails loudly on a missing variable (**StrictUndefined**), so a typo in a vars file can't produce a half-empty configuration:

```
ubuntu@oob-mgmt-server:~/helix-nvue$ cat > render.py <<'EOF'
#!/usr/bin/env python3
import pathlib, yaml, jinja2
env = jinja2.Environment(loader=jinja2.FileSystemLoader("templates"),
                        undefined=jinja2.StrictUndefined, trim_blocks=True, lstrip_blocks=True)
for f in sorted(pathlib.Path("vars").glob("*.yaml")):
    data = yaml.safe_load(f.read_text())
    out = pathlib.Path("rendered") / f"{data['hostname']}.nvue"
    out.write_text(env.get_template("underlay.j2").render(**data))
    print("rendered", out)
EOF
ubuntu@oob-mgmt-server:~/helix-nvue$ python3 render.py
rendered rendered/hxb-leaf-r1.nvue
...
rendered rendered/hxb-spine02.nvue
ubuntu@oob-mgmt-server:~/helix-nvue$ cat rendered/hxb-leaf-r1.nvue
```

Expected output:

```
# hxb-leaf-r1 - rendered from templates/underlay.j2 - do not edit by hand
nv set system hostname hxb-leaf-r1
nv set interface lo ip address 10.255.0.11/32
nv set interface swp31 description to-hxb-spine01-swp1
nv set interface swp31 link mtu 9216
nv set interface swp31 link state up
nv set vrf default router bgp neighbor swp31 remote-as external
nv set interface swp32 description to-hxb-spine02-swp1
nv set interface swp32 link mtu 9216
nv set interface swp32 link state up
nv set vrf default router bgp neighbor swp32 remote-as external
nv set router bgp autonomous-system 65101
nv set router bgp router-id 10.255.0.11
nv set vrf default router bgp address-family ipv4-unicast network 10.255.0.11/32
nv set service ntp mgmt server 0.pool.ntp.org
nv set service ntp mgmt server 1.pool.ntp.org
```

3. Commit it. The rendered files are the golden configuration.

```
ubuntu@oob-mgmt-server:~/helix-nvue$ git add -A && git commit -qm "Underlay golden config for
helix-b-air" && git log --oneline
```

Version note: If your Cumulus release rejects the NTP lines, check the syntax with `nv set service ntp <Tab>` and correct the template, not the switch. That's the discipline this lab teaches.

Task 4 — Patch, diff, apply with confirm

1. Copy the file to hxb-leaf-r1 and load it with **patch** (merge into the pending revision):

```
ubuntu@oob-mgmt-server:~/helix-nvue$ scp rendered/hxb-leaf-r1.nvue cumulus@hxb-leaf-r1:/tmp/
ubuntu@oob-mgmt-server:~/helix-nvue$ ssh cumulus@hxb-leaf-r1
```

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config patch /tmp/hxb-leaf-r1.nvue
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
```

Expected output (illustrative):

```
- set:
  service:
    ntp:
      mgmt:
        server:
          0.pool.ntp.org: {}
          1.pool.ntp.org: {}
```

The diff is the gate. Only the NTP servers are new; everything else in the file already matches what you typed in Labs 1 and 2, so NVUE shows no change for it. If the diff shows anything you didn't expect, stop and investigate before applying.

2. Apply with a confirm timer, check, and keep:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply --confirm 300s -y
cumulus@hxb-leaf-r1:mgmt:~$ nv show service ntp mgmt server
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply --confirm=yes
```

3. Roll it out to the remaining switches with a loop. **nv config apply -y** without a timer is acceptable here because each change was just reviewed on leaf-r1 and is identical in form.

```
ubuntu@oob-mgmt-server:~/helix-nvue$ for h in hxb-leaf-r2 hxb-leaf-r3 hxb-leaf-r4 hxb-spine01
hxb-spine02; do
> scp -q rendered/$h.nvue cumulus@$h:/tmp/ &&
> ssh cumulus@$h "nv config patch /tmp/$h.nvue && nv config diff && nv config apply -y && nv
config save"
> done
```

Checkpoint

- [] **nv show service ntp mgmt server** lists both servers on all six switches.
- [] BGP is still up everywhere (**~/checks/fabric-check.sh** from Lab 7).

Task 5 — Why a partial replace is dangerous

1. On hxb-leaf-r1, load the same underlay-only file with **replace** instead of patch, and look at the diff. **Don't apply it.**

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config replace /tmp/hxb-leaf-r1.nvue
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff | head -40
```

The diff now has a large **unset:** section: the EVPN configuration, VRFs AURORA and BOREALIS, the bridge and VLANs, the server ports and the RoCE settings. Replace makes the pending configuration **exactly** the file, and the file only describes the underlay.

2. Throw the pending change away:

```
cumulus@hxb-leaf-r1:mgmt:~$ nv config detach
cumulus@hxb-leaf-r1:mgmt:~$ nv config diff
```

An empty diff means the pending revision matches applied again.

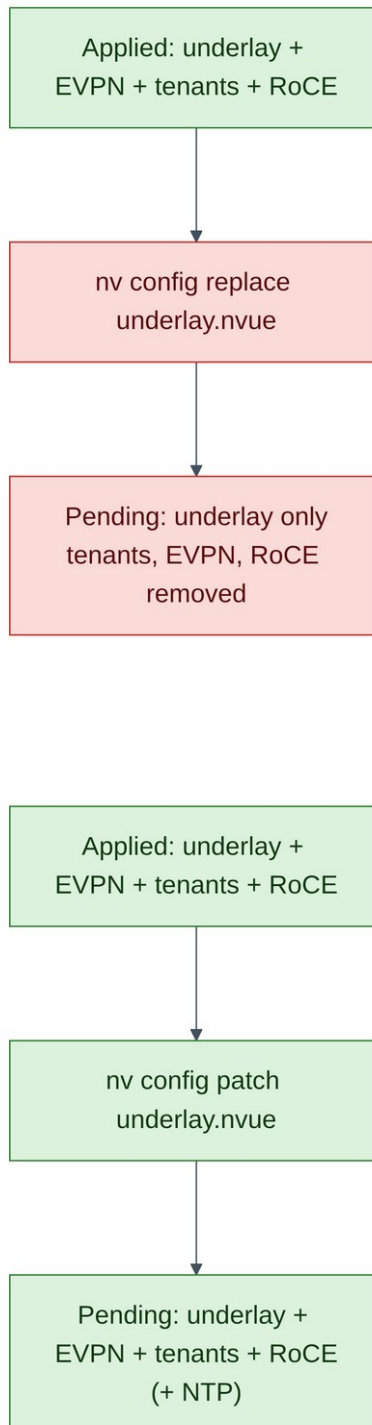


Figure 11.2: Patch versus replace with an underlay-only file

Patch merges the file into the current configuration, so the tenants survive. Replace makes the configuration equal to the file, so everything the file doesn't mention is removed.

Exam trap: Replace is for **complete** files, where the file describes the whole switch. With a partial file it is a mass deletion. Both patch and replace only change the pending revision; nothing happens until apply.

Task 6 — A change through the REST API

The API listens on localhost by default (Lab 1, Task 8). Let the oob-mgmt-server reach it on each switch's management address.

1. On hxb-leaf-r1, find eth0's address and add it as a listening address:

```
cumulus@hxb-leaf-r1:mgmt:~$ ip -4 -br addr show eth0
eth0                UP                192.168.200.11/24
cumulus@hxb-leaf-r1:mgmt:~$ nv set system api listening-address 192.168.200.11
cumulus@hxb-leaf-r1:mgmt:~$ nv config apply -y
```

(Illustrative address.)

2. From the oob-mgmt-server, run the revision workflow: create a revision, PATCH a description into it, apply it, and check its state.

```
ubuntu@oob-mgmt-server:~$ SW=https://hxb-leaf-r1:8765/nvue_v1 ; AUTH='-s -k -u
cumulus:CumulusLinux!'
ubuntu@oob-mgmt-server:~$ curl $AUTH -X POST $SW/revision
{"changeset/cumulus/2026-09-28_12.03.11_K3Q9": {"state": "pending", "transition": {"issue": {}},
"progress": ""}}
```

(Illustrative.) Copy the revision ID (older releases return a **changeset/...** name, recent ones a number). In the next commands, replace **REV** with it, URL-encoding any **/** as **%2F**.

```
ubuntu@oob-mgmt-server:~$ REV='changeset%2Fcumulus%2F2026-09-28_12.03.11_K3Q9'
ubuntu@oob-mgmt-server:~$ curl $AUTH -X PATCH "$SW/interface/swp1?rev=$REV" -H 'Content-Type:
application/json' \
-d '{"description": "hxb-gpu01-eth1"}'
ubuntu@oob-mgmt-server:~$ curl $AUTH -X PATCH "$SW/revision/$REV" -H 'Content-Type:
application/json' \
-d '{"state": "apply", "auto-prompt": {"ays": "ays_yes"}}'
ubuntu@oob-mgmt-server:~$ curl $AUTH "$SW/revision/$REV"
{"state": "applied", ...}
ubuntu@oob-mgmt-server:~$ curl $AUTH "$SW/interface/swp1?rev=applied" | python3 -m json.tool |
grep description
"description": "hxb-gpu01-eth1",
```

Warning: The single quotes around **AUTH** stop Bash from treating the **!** in the password as a history expansion. For anything beyond a lab, keep credentials out of your shell history: use **curl --netrc-file** or an NVUE API token. Chapter 21 recommends a token and a dedicated automation account in production.

3. The change came from outside git. That's drift from your golden file; Task 7 detects it.

Task 7 — Drift detection

A switch is compliant when its applied configuration contains everything the golden file says. Let NVUE do the comparison: patch the golden file into a pending revision, look at the diff, then detach.

```
ubuntu@oob-mgmt-server:~/helix-nvue$ cat > drift-check.sh <<'EOF'
#!/bin/bash
# Report switches whose applied config differs from the golden underlay file
for f in rendered/*.nvue; do
    h=$(basename $f .nvue)
    scp -q $f cumulus@$h:/tmp/golden.nvue
    d=$(ssh cumulus@$h "nv config patch /tmp/golden.nvue >/dev/null && nv config diff; nv config
detach >/dev/null")
```

```

    if [ -z "$d" ]; then echo "$h: compliant"; else echo "$h: DRIFT"; echo "$d" | sed 's/^/ /';
fi
done
EOF
ubuntu@oob-mgmt-server:~/helix-nvue$ chmod +x drift-check.sh && ./drift-check.sh
hxb-leaf-r1: compliant
...

```

The swp1 description you set through the API isn't flagged: the golden file doesn't mention swp1, and patch-based checks only find differences in what the file describes. Now simulate real drift: on hxb-leaf-r3 change swp31's MTU by hand (`nv set interface swp31 link mtu 9000 && nv config apply -y`) and run the check again.

```

hxb-leaf-r3: DRIFT
- set:
  interface:
    swp31:
      link:
        mtu: 9216

```

(*Illustrative.*) The golden value is 9216. Fix it the configuration-as-code way: re-run the deployment loop from Task 4 for hxb-leaf-r3, or decide the change was intended, update the vars/template, re-render, commit and deploy.

Field note: A **complete** golden file (Chapter 21's base + role + host templates) lets you use replace for both deployment and drift checks, which also finds additions like the swp1 description. That's the mature end state; this lab uses the safer partial pattern.

Verify

- [] Six rendered files committed in git.
- [] NTP applied to all switches via patch; BGP unaffected.
- [] You saw the replace diff and detached it without applying.
- [] A description changed through the REST API revision workflow.
- [] `drift-check.sh` found and you fixed the MTU drift on leaf-r3.

Break and fix

Fault — "Someone ran replace with the underlay file and applied it."

- *Inject (in a safe way):* on hxb-leaf-r4, `nv config replace /tmp/hxb-leaf-r4.nvue && nv config apply --confirm 120s -y`.
- *Symptom:* within seconds, gpu02 and gpu04 lose tenant connectivity; `nv show evpn vni` on leaf-r4 is empty.
- *Diagnosis:* `nv config history | head` shows the latest apply; `nv config diff <previous-rev> applied` shows the deletions.
- *Fix:* do nothing: after 120 seconds without confirmation, NVUE rolls back automatically. Check with `nv config history`. Without `--confirm`, you'd recover with `nv config apply <previous-revision>` (Lab 1).

Clean-up / save state

`nv config save` on every switch; commit anything new in `~/helix-nvue`; store a checkpoint (`lab11-templates`).

Exam tie-in

- 6.1: templates plus variables render NVUE configuration; load with `nv config patch` (merge) or `nv config replace` (whole configuration); gate with `nv config diff`; apply with `--confirm`.
- REST API workflow: POST revision → PATCH `?rev=` → PATCH `revision/<id>` with `state: apply`.
- Golden configuration in git and drift detection close the loop.

Review questions

1. What exactly does `nv config replace file` do to the pending revision?
2. Why use `StrictUndefined` in the renderer?
3. After `nv config patch`, the diff is empty. What does that mean?
4. What are the three REST calls in the revision workflow?
5. Why didn't the patch-based drift check flag the swp1 description?

Answers

1. It makes the pending configuration exactly the file's contents, removing anything the file doesn't include. Nothing changes on the switch until apply.
2. So a missing or misspelt variable stops rendering with an error instead of silently producing an incomplete configuration.
3. The switch already has everything the file describes: it's compliant for that scope.
4. `POST /nvue_v1/revision`, `PATCH /nvue_v1/<path>?rev=<id>` with the change, and `PATCH /nvue_v1/revision/<id>` with `{"state": "apply"}` (plus `auto-prompt` to answer yes).
5. A patch only compares what the file describes. The golden file doesn't mention swp1, so any setting there is invisible to the check. Complete files with replace-based checks catch additions too.

Lab 12 — Ansible Automation: VLANs and RoCE with nvidia.nvue

Lab at a glance

Book chapters	22 (Ansible automation for Spectrum-X), 21 (NVUE REST API)
Exam objectives	6.2 Automate network configuration using Ansible playbooks (VLAN creation, RoCE configuration)
Time	60 minutes
Air resources	oob-mgmt-server (control node) and the six switches
Prerequisites	Labs 1, 2, 5 and 11 (Task 6: REST API listening address)

Objectives

- Install Ansible and the **nvidia.nvue** collection on the oob-mgmt-server.
- Build an inventory with **group_vars** and **host_vars** that connects over **httpapi** to NVUE on port 8765.
- Run a playbook that creates two VLAN-to-VNI mappings on every leaf, read them back and assert the result.
- Run a playbook that enables lossless RoCE on every switch, one switch at a time.
- Use check mode, **--limit**, **serial** and a second run to understand idempotency.
- Diagnose authentication and connectivity failures.

Background

Chapter 22 describes three ways to drive NVUE from Ansible: the **nvidia.nvue** collection (the recommended path), **ansible.builtin.uri** against the REST API, and SSH with **nv** commands. The collection's modules don't use SSH at all. They talk to the NVUE REST API through Ansible's **httpapi** connection plugin, on TCP 8765, and each task becomes a revision that NVUE applies.

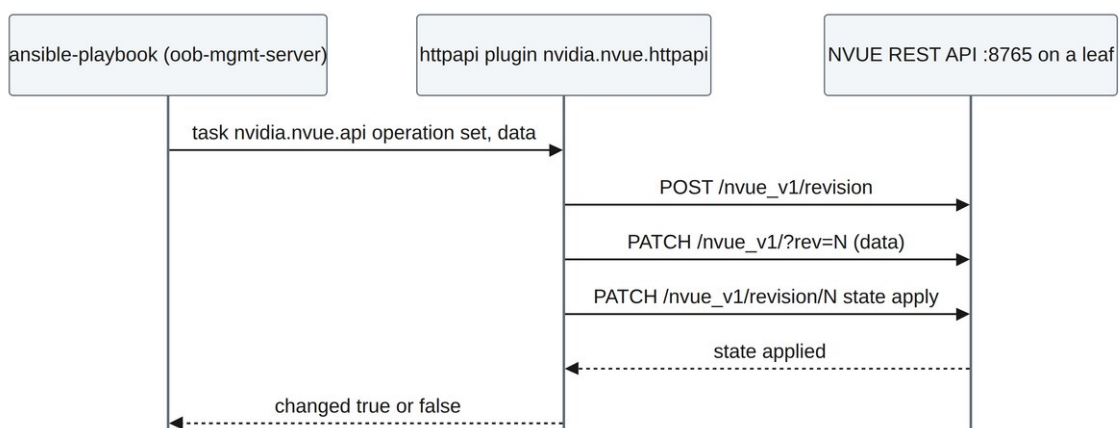


Figure 12.1: How a task reaches the switch

The playbook runs on the oob-mgmt-server. The httpapi plugin sends the task's data to NVUE on port 8765 over the OOB network; NVUE creates a revision, applies it and reports the result.

Step-by-step

Task 1 — Open the API on every switch

Lab 11 opened the API on hxb-leaf-r1 only. Do the rest from the oob-mgmt-server:

```
ubuntu@oob-mgmt-server:~$ for h in hxb-spine01 hxb-spine02 hxb-leaf-r1 hxb-leaf-r2 hxb-leaf-r3
hxb-leaf-r4; do
> ip=$(getent hosts $h | awk '{print $1}')
> ssh cumulus@$h "nv set system api listening-address $ip && nv config apply -y && nv config
save" >/dev/null && echo "$h $ip ok"
> done
ubuntu@oob-mgmt-server:~$ curl -s -k -u 'cumulus:CumulusLinux!'
https://hxb-spine02:8765/nvue_v1/system/hostname?rev=applied
```

Checkpoint

- [] The curl call returns spine02's hostname as JSON.

Task 2 — Install Ansible and the collection

```
ubuntu@oob-mgmt-server:~$ sudo apt-get update && sudo apt-get install -y pipx
ubuntu@oob-mgmt-server:~$ pipx install --include-deps ansible && pipx ensurepath && exec bash
ubuntu@oob-mgmt-server:~$ ansible --version | head -1
ubuntu@oob-mgmt-server:~$ ansible-galaxy collection install nvidia.nvue ansible.netcommon
ubuntu@oob-mgmt-server:~$ ansible-galaxy collection list | grep -E 'nvidia.nvue|
ansible.netcommon'
nvidia.nvue          1.2.9
ansible.netcommon    7.x.x
```

(*Illustrative versions.*) Read the module documentation for **your** version before writing tasks:

```
ubuntu@oob-mgmt-server:~$ ansible-doc nvidia.nvue.api | sed -n '1,60p'
```

Task 3 — Project, inventory and variables

```
ubuntu@oob-mgmt-server:~$ mkdir -p ~/helix-ansible/{group_vars,host_vars,playbooks} && cd
~/helix-ansible
ubuntu@oob-mgmt-server:~/helix-ansible$ cat > inventory.yml <<'EOF'
all:
  children:
    helixb:
      children:
        spines:
          hosts:
            hxb-spine01:
            hxb-spine02:
        leaves:
          hosts:
            hxb-leaf-r1:
            hxb-leaf-r2:
            hxb-leaf-r3:
            hxb-leaf-r4:
EOF
ubuntu@oob-mgmt-server:~/helix-ansible$ cat > group_vars/helixb.yml <<'EOF'
ansible_connection: ansible.netcommon.httpapi
ansible_network_os: nvidia.nvue.httpapi
ansible_httpapi_port: 8765
```

```

ansible_httpapi_use_ssl: true
ansible_httpapi_validate_certs: false      # lab only: self-signed certificates
ansible_user: cumulus
ansible_password: "{{ vault_nvue_password }}"
EOF
ubuntu@oob-mgmt-server:~/helix-ansible$ cat > group_vars/leaves.yml <<'EOF'
tenant_vlans:                             # staging VLANs for new AURORA subnets
- { vlan: 120, vni: 10120 }
- { vlan: 121, vni: 10121 }
EOF
ubuntu@oob-mgmt-server:~/helix-ansible$ cat > ansible.cfg <<'EOF'
[defaults]
inventory = inventory.yml
host_key_checking = False
EOF

```

Keep the password out of plain text with Ansible Vault:

```

ubuntu@oob-mgmt-server:~/helix-ansible$ ansible-vault create group_vars/vault.yml
# in the editor, type:  vault_nvue_password: "CumulusLinux!"
ubuntu@oob-mgmt-server:~/helix-ansible$ mv group_vars/vault.yml group_vars/all.yml

```

Field note: `group_vars/all.yml` applies to every host, so the vaulted password is available to the `helixb` variables. Run playbooks with `--ask-vault-pass` (or a vault password file in production pipelines).

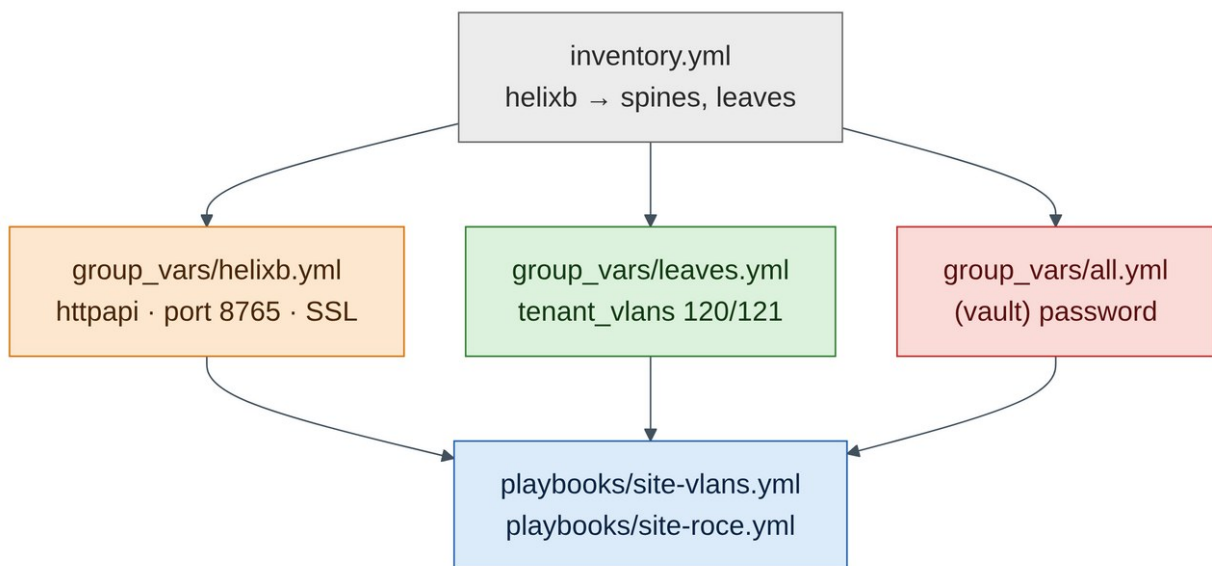


Figure 12.2: Project layout

Inventory groups decide which variables each switch gets. helixb holds the connection settings; leaves adds the tenant VLAN list; all holds the vaulted secret.

Task 4 — A read-only playbook first

Always prove connectivity with a read before you write.

```

ubuntu@oob-mgmt-server:~/helix-ansible$ cat > playbooks/facts.yml <<'EOF'
- name: Read hostname and BGP ASN through the NVUE API
  hosts: helixb
  gather_facts: false
  tasks:

```

```

- name: Get system hostname
  nvidia.nvue.api:
    operation: get
    path: /system/hostname
    register: hn
- name: Show it
  ansible.builtin.debug:
    msg: "{{ inventory_hostname }}" reports {{ hn.message }}"
EOF
ubuntu@oob-mgmt-server:~/helix-ansible$ ansible-playbook playbooks/facts.yml --ask-vault-pass
...
PLAY RECAP *****
hxb-leaf-r1 : ok=2  changed=0  unreachable=0  failed=0
...

```

Checkpoint

- [] All six hosts **ok=2** **failed=0**.

Task 5 — Playbook 1: VLANs and VNIs on every leaf

```

ubuntu@oob-mgmt-server:~/helix-ansible$ cat > playbooks/site-vlans.yml <<'EOF'
- name: Helix-B staging VLANs and VNIs
  hosts: leaves
  gather_facts: false
  serial: 2                                # two leaves at a time
  max_fail_percentage: 0                    # stop the rollout on the first failure
  tasks:
    - name: Build the NVUE bridge data from the VLAN plan
      ansible.builtin.set_fact:
        vlan_data: >-
          {{ vlan_data | default({}) | combine({ (item.vlan | string):
            { 'vni': { (item.vni | string): {} } } }) }}
        loop: "{{ tenant_vlans }}"

    - name: Merge VLANs and VNIs into br_default and apply
      nvidia.nvue.api:
        operation: set
        force: true
        wait: 15
        data:
          bridge:
            domain:
              br_default:
                vlan: "{{ vlan_data }}"

    - name: Read back the bridge VLANs
      nvidia.nvue.api:
        operation: get
        path: /bridge/domain/br_default/vlan
        register: vlans

    - name: Assert every VLAN has the right VNI
      ansible.builtin.assert:
        that:
          - (vlans.message[item.vlan | string].vni | list | first) == (item.vni | string)
        fail_msg: "VLAN {{ item.vlan }}" on "{{ inventory_hostname }}" is not mapped to VNI
          {{ item.vni }}"
        loop: "{{ tenant_vlans }}"

```

EOF

1. Dry run first, on one leaf:

```
ubuntu@oob-mgmt-server:~/helix-ansible$ ansible-playbook playbooks/site-vlans.yml --ask-vault-pass --check --limit hxb-leaf-r1
```

Whether the `nvidia.nvue` modules honour check mode depends on the collection version (Chapter 22). If the run reports a change but the switch doesn't show one, the module doesn't support check mode; that's why you test in Air before production.

2. Real run, all leaves, two at a time:

```
ubuntu@oob-mgmt-server:~/helix-ansible$ ansible-playbook playbooks/site-vlans.yml --ask-vault-pass
```

3. Verify on a switch:

```
cumulus@hxb-leaf-r2:mgmt:~$ nv show bridge domain br_default vlan
cumulus@hxb-leaf-r2:mgmt:~$ nv show evpn vni
```

VLANs 120 and 121 now appear with VNIs 10120 and 10121, alongside the Lab 5 tenant VLANs, which the merge left untouched.

4. **Idempotency:** run the playbook again. A well-behaved run reports `changed=0` for the merge task, because the desired state already exists. If your collection version reports `changed` on every run, note it: Chapter 22 explains why that matters for change control and how to guard against it (read first, compare, only write on a difference).

Checkpoint

- [] VLANs 120/121 with VNIs 10120/10121 on all four leaves; asserts passed.
- [] You know whether your module version is idempotent and check-mode aware.

Task 6 — Playbook 2: lossless RoCE on every switch

```
ubuntu@oob-mgmt-server:~/helix-ansible$ echo 'roce_mode: lossless' >> group_vars/helixb.yml
ubuntu@oob-mgmt-server:~/helix-ansible$ cat > playbooks/site-roce.yml <<'EOF'
- name: Helix-B RoCE QoS
  hosts: helixb                                # leaves AND spines
  gather_facts: false
  serial: 1                                    # one switch at a time for QoS changes
  max_fail_percentage: 0
  tasks:
    - name: Enable RoCE with the chosen mode
      nvidia.nvue.api:
        operation: set
        force: true
        wait: 20
        data:
          qos:
            roce:
              enable: "on"
              mode: "{{ roce_mode }}"

    - name: Read back the RoCE configuration
      nvidia.nvue.api:
        operation: get
```

```

    path: /qos/roce
    register: roce_show

- name: Assert lossless RoCE is active
  ansible.builtin.assert:
    that:
      - "'lossless' in (roce_show.message | string)"
      fail_msg: "RoCE is not {{ roce_mode }} on {{ inventory_hostname }}"
EOF
ubuntu@oob-mgmt-server:~/helix-ansible$ ansible-playbook playbooks/site-roce.yml --ask-vault-pass

```

Version note: The JSON keys under `/qos/roce` (`enable`, `mode`) follow Chapter 22. If NVUE rejects them on your release, read the current structure first with a `get` on `/qos/roce` (or `nv config show -o json` on a switch where you enabled RoCE by hand in Lab 3) and match it.

Confirm on a spine and a leaf with the CLI: `nv show qos roce`. On Cumulus VX this verifies the configured intent; the lossless data plane itself needs Spectrum hardware (Lab 3).

Task 7 — Commit the project

```

ubuntu@oob-mgmt-server:~/helix-ansible$ git init -q && git add -A && git commit -qm "Helix-B VLAN
and RoCE playbooks"

```

The vault file is encrypted, so committing it is safe; the vault password is not in the repository.

Verify

- [] Read-only playbook succeeds on all six switches.
- [] VLANs 120/121 and VNIs 10120/10121 present on all leaves; second run behaviour recorded.
- [] RoCE lossless configured on all six switches, one at a time, with assertions passing.

Break and fix

Fault 1 — 401 Unauthorized

- *Inject:* change the vaulted password to a wrong value (`ansible-vault edit group_vars/all.yml`).
- *Symptom:* every host fails with an HTTP 401 error from the `httpapi` plugin.
- *Diagnosis:* `curl -k -u 'cumulus:<password>' https://hxb-leaf-r1:8765/nvue_v1/system` returns 401 with the wrong password and 200 with the right one.
- *Fix:* correct the vaulted password.

Fault 2 — connection refused on one switch

- *Inject:* on `hxb-leaf-r4`, `nv unset system api listening-address && nv config apply -y`.
- *Symptom:* only `hxb-leaf-r4` fails, with "connection refused" to port 8765. SSH to the switch still works.
- *Diagnosis:* `nv show system api` on `leaf-r4` shows `localhost` only. The collection uses the API, not SSH (Chapter 22 exam trap).
- *Fix:* re-add the listening address (Task 1) and re-run with `--limit hxb-leaf-r4`.

Clean-up / save state

The NVUE API applies each change, and auto-save writes it to startup on current releases. To be sure, run `nv config save` on all switches (Lab 11's loop), then store a checkpoint (`lab12-ansible`).

Exam tie-in

- 6.2: the `nvidia.nvue` collection over `ansible.netcommon.httpapi` with `ansible_network_os: nvidia.nvue.httpapi` on port 8765.
- VLAN creation is a merge into `bridge domain br_default vlan <id> vni <vni>`; RoCE is `qos roce` with `mode lossless`.
- `serial`, `max_fail_percentage`, `--limit`, check mode and idempotency are how you roll changes out safely.
- Read-back and `assert` turn a playbook from "sent a change" into "proved the result".

Review questions

1. Which connection plugin and `ansible_network_os` do the `nvidia.nvue` modules use?
2. SSH to a switch works but the playbook fails to connect. What do you check?
3. Why does the VLAN playbook merge rather than replace the bridge VLAN list?
4. What do `serial: 1` and `max_fail_percentage: 0` achieve in the RoCE playbook?
5. How do you tell whether a module is idempotent?

Answers

1. `ansible.netcommon.httpapi` with `ansible_network_os: nvidia.nvue.httpapi`, over HTTPS to port 8765.
2. The NVUE REST API: enabled, listening on the management address (not only localhost), port 8765 reachable, and the credentials valid.
3. A merge adds VLANs 120/121 and leaves the Lab 5 tenant VLANs alone. Replacing (or `overridden`) would delete any VLAN not in the playbook's list.
4. Changes go to one switch at a time, and the play stops at the first failure, so a bad change can't spread across the fabric.
5. Run it twice with the same inputs. An idempotent task reports `changed=0` on the second run because the desired state already exists.

Lab 13 — Capstone: Troubleshooting Challenge

Lab at a glance

Book chapters	Chapter 4 (NVUE) · 5 (RoCE QoS) · 7 (BGP-EVPN) · 11–13 (InfiniBand, SM HA, PKeys) · 17 (IB diagnostic tools) · 22 (Ansible) · 23 (exam strategy)
Exam objectives	Domain 5 Troubleshooting (5.4, 5.5) plus the objectives behind each fault: 2.1, 2.2, 2.3, 3.1, 3.2, 3.4, 6.1, 6.2
Time	3–4 hours for all ten scenarios (about 15–20 minutes each), or run them in two sessions
Air resources used	The whole helix-b-air simulation (about 26 vCPU while running). A favourite checkpoint taken before you start
Prerequisite labs	Labs 0–5, Lab 6 (checkpoints), Lab 8 (ibsim set up in <code>~/ibsim</code> on <code>hxa-ibsim</code>), Lab 12 (Ansible project). Labs 7, 9 and 11 help

Objectives

By the end of this lab you will be able to:

1. Work a fault from a vague ticket to a verified fix using a layer-by-layer method, not guesswork.
2. Diagnose Ethernet faults on the Helix-B twin: a BGP unnumbered neighbour that won't establish, an MTU mismatch, a VNI mismatch, a missing anycast gateway, a disabled lossless RoCE profile, and a wrong VLAN pushed by Ansible.
3. Diagnose InfiniBand faults in ibsim: a missing standby Subnet Manager and SM failover, a missing partition member, rising symbol errors on an uplink, and a failed link.
4. Prove every fix with the same commands that showed the fault.
5. Write troubleshooting notes that a colleague (or an examiner) can follow.

Background

This lab turns everything you built in Labs 0–12 into a practice ground. An instructor, a study partner or a script breaks one thing at a time. You get only a short **ticket**, written the way a user would report it. Your job is to find the fault, fix it, and prove the fix. The exam's troubleshooting domain (20%) tests exactly this skill in multiple-choice form: given symptoms and command output, pick the cause and the next step.

There are **ten scenarios**: six on the Helix-B Ethernet twin and four on the Helix-A InfiniBand simulator. Each scenario lists how the fault is injected (instructor only), what the student sees, three hints that give away progressively more, and the exam objective it practises. Full solutions are in a separate section so you don't read them by accident.

Use the same method every time. Start from the symptom and the scope (one host? one rail? one tenant? whole fabric?). Then work **up the layers**: physical link, then L2 (VLAN, MTU), then the underlay control plane (BGP), then the overlay (EVPN, VNIs, VRFs, gateways), then QoS and policy, and last the automation that put the configuration there. On InfiniBand the order is link state, then SM and LIDs, then partitions, then counters and routes. At every step, compare the broken device with a healthy peer: in a rail-optimised fabric there is always a twin that should look the same.

Warning: Cumulus VX has no Spectrum ASIC, and ibsim has no real links. Some faults don't produce the traffic symptoms they would in production (for example, lossy RoCE on VX doesn't drop anything, and ibsim symbol errors only rise when someone sets them). In those scenarios you diagnose from configuration and counters, which is also what most exam questions show you.

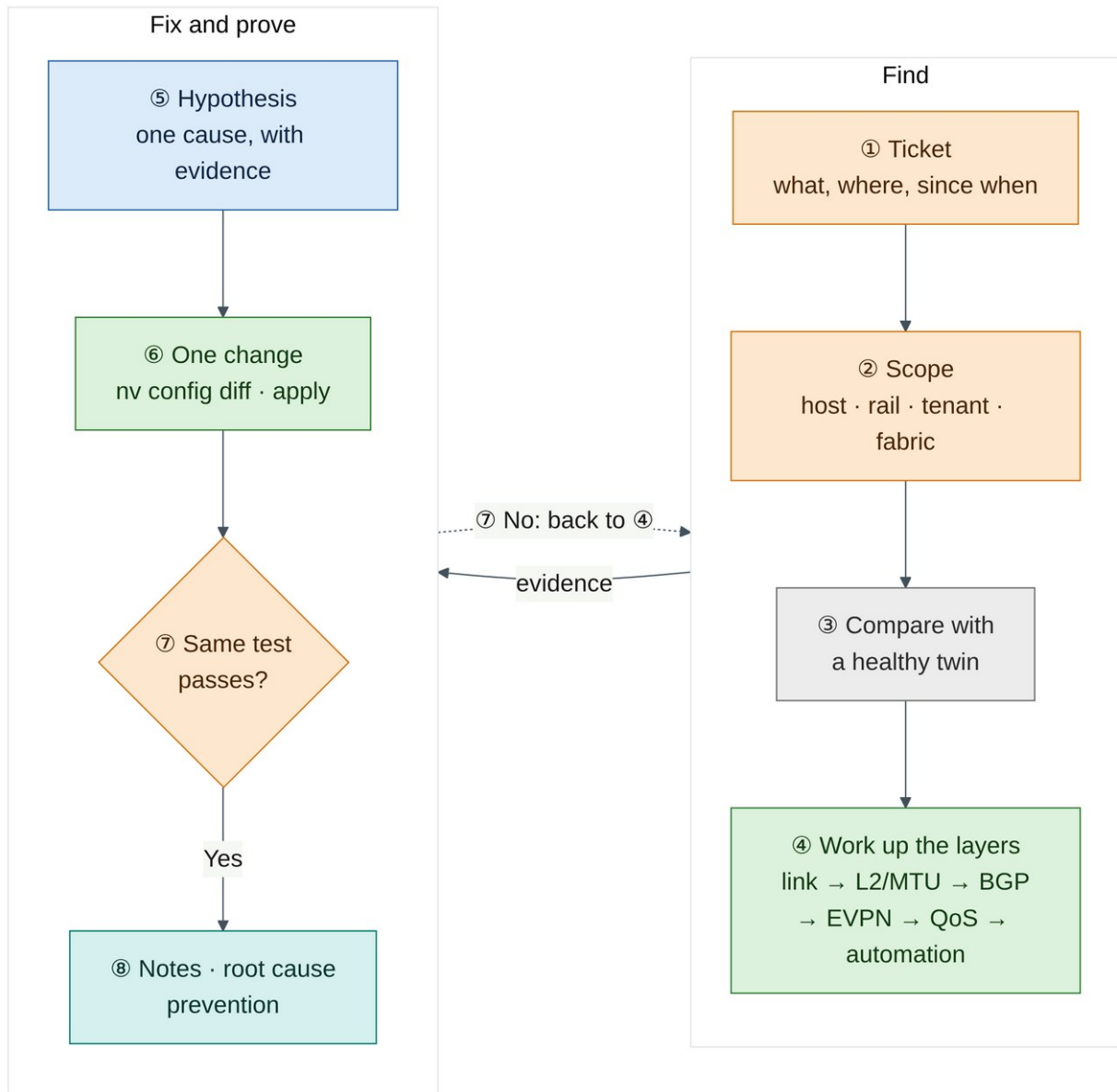


Figure 13.1: The capstone troubleshooting method

Every scenario follows the same loop. Scope the problem, compare with a healthy twin, work up the layers, change one thing, and verify with the command that first showed the fault. Write notes as you go, not afterwards.

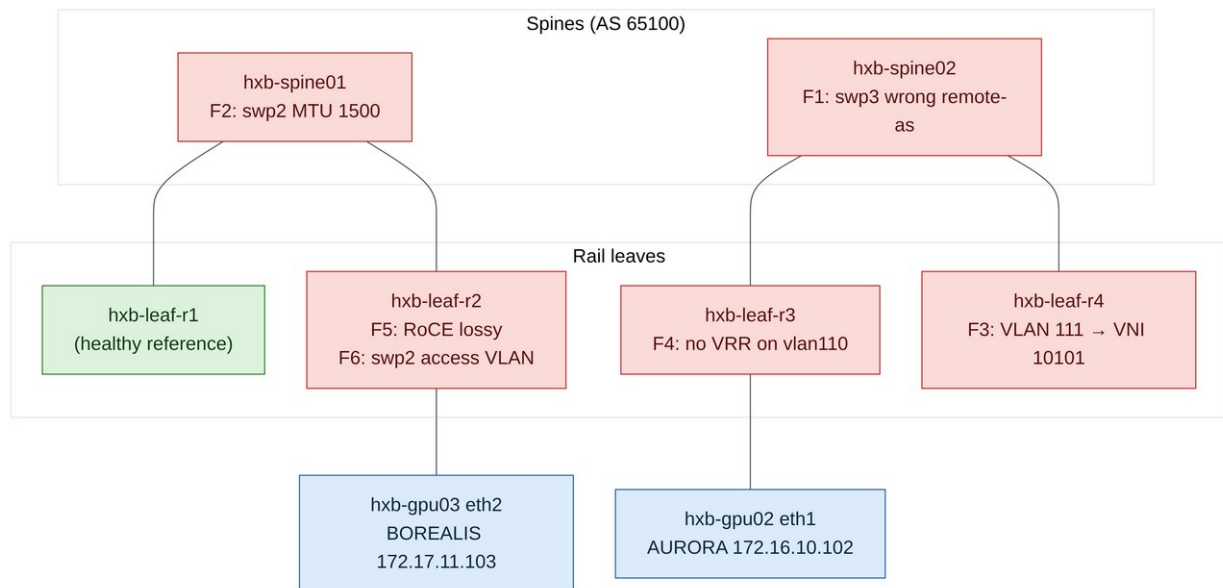


Figure 13.2: Where the Ethernet faults are injected

F1 breaks the spine02-leaf-r3 BGP session, F2 lowers the MTU on the spine01-leaf-r2 link, F3 changes VLAN 111's VNI on leaf-r4, F4 removes the VLAN 110 gateway on leaf-r3, F5 switches leaf-r2 RoCE to lossy, and F6 moves gpu03's access port on leaf-r2 into the wrong VLAN.

The figure shows only the links involved in the faults. Every leaf connects to both spines (swp31 to hxb-spine01, swp32 to hxb-spine02).

Step-by-step

Task 1 — Capture a baseline and take a checkpoint

You can only recognise "broken" if you have recorded "working". Capture a baseline from the healthy fabric before any fault is injected.

1. On **oob-mgmt-server**, create a working folder and capture the Ethernet baseline:

```

ubuntu@oob-mgmt-server:~$ mkdir -p ~/capstone/baseline && cd ~/capstone/baseline
ubuntu@oob-mgmt-server:~/capstone/baseline$ for sw in hxb-spine01 hxb-spine02 hxb-leaf-r1 hxb-leaf-r2 hxb-leaf-r3 hxb-leaf-r4; do
  ssh cumulus@$sw "sudo vtysh -c 'show bgp summary'; sudo vtysh -c 'show evpn vni'; nv show qos roce; nv config show" > $sw.txt 2>&1
done
ubuntu@oob-mgmt-server:~/capstone/baseline$ ls
  
```

Expected output (illustrative):

```

hxb-leaf-r1.txt hxb-leaf-r2.txt hxb-leaf-r3.txt hxb-leaf-r4.txt hxb-spine01.txt hxb-spine02.txt
  
```

2. Record the host reachability matrix. From **hxb-gpu01**, every AURORA address should answer:

```

ubuntu@hxb-gpu01:~$ for ip in 172.16.10.1 172.16.10.102 172.16.11.1 172.16.11.102; do ping -c 2 -W 1 $ip >/dev/null && echo "$ip ok" || echo "$ip FAIL"; done
  
```

Expected output (illustrative): four lines ending in **ok**. Repeat from **hxb-gpu03** for the BOREALIS addresses (172.17.10.1, 172.17.10.104, 172.17.11.1, 172.17.11.104). Check that a cross-tenant ping (for example **ping 172.17.10.103** from **hxb-gpu01**) fails, as Lab 5 intended.

3. Capture the InfiniBand baseline on **hxa-ibsim**. Start ibsim and OpenSM the way Lab 8 does, then:

```
ubuntu@hxa-ibsim:~$ cd ~/ibsim
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run sminfo > base-sminfo.txt
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run saquery -s > base-sm-ports.txt
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run ibnetdiscover > base-topo.txt
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run iblinkinfo --switches-only -l > base-links.txt
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run ibqueryerrors > base-errors.txt
ubuntu@hxa-ibsim:~/ibsim$ cp partitions.conf base-partitions.conf
```

saquery -s lists the ports that are SM-capable. With Lab 8's HA set-up it should show both SM hosts, so you know the master's and the standby's LIDs.

4. Save everything and take a checkpoint. On every switch, **nv config diff applied startup** should be empty (Lab 6, Task 1). Then in Air use **Stop Simulation and Store Checkpoint**, mark the checkpoint as a favourite, and name it **pre-capstone**. Start the simulation from it.

Warning: An Air checkpoint saves disks, not running processes. After starting from a checkpoint, restart ibsim and the OpenSM instances as in Lab 8 before you do any InfiniBand scenario.

Checkpoint:

- Baseline files exist in **~/capstone/baseline** and **~/ibsim/base-***.
- A favourite checkpoint called **pre-capstone** exists.

Task 2 — Install the fault injector (instructor or study partner)

The student should not read this task. If you work alone, ask a friend to install the script, or install it without reading the **case** block and use **random** mode (Task 3).

1. On **oob-mgmt-server**, create **~/capstone/inject.sh**:

```
#!/bin/bash
# Lab 13 fault injector. Usage: inject.sh F1..F10 | F7b | random
set -euo pipefail
SVI_AF=ipv4 # use "ip" if Lab 5 used the pre-5.15 SVI syntax
ANSIBLE_DIR=~/.helix-ansible # your Lab 12 Ansible project
SIM=hxa-ibsim
VICTIM_GUID=0x0000000000000000 # port GUID of one PRODUCTION member (F8), from partitions.conf

nv_on() { local sw=$1; shift; ssh -o BatchMode=yes cumulus@"$sw" "$* && nv config apply -y"
>/dev/null; }
sim_cmd() { ssh -o BatchMode=yes ubuntu@"$SIM" "echo '$1' > ~/ibsim/simcmd"; }

id=${1:?usage: inject.sh F1..F10|F7b|random}
if [ "$id" = random ]; then
    id=F$(( (RANDOM % 10) + 1 ))
    echo "$id $(date)" | sudo tee -a /root/capstone-answers >/dev/null
    echo "A fault has been injected. Good luck."
fi

case $id in
    F1) nv_on hxb-spine02 "nv set vrf default router bgp neighbor swp3 remote-as 65200" ;;
    F2) nv_on hxb-spine01 "nv set interface swp2 link mtu 1500" ;;
    F3) nv_on hxb-leaf-r4 "nv unset bridge domain br_default vlan 111 vni 10111 && nv set bridge
domain br_default vlan 111 vni 10101" ;;
    F4) nv_on hxb-leaf-r3 "nv unset interface vlan110 $SVI_AF vrr" ;;
    F5) nv_on hxb-leaf-r2 "nv set qos roce mode lossy" ;;
    F6) cd "$ANSIBLE_DIR"
        cat > playbooks/access-ports.yml <<'EOF'
```

```

- name: Tidy leaf access ports
  hosts: hxb-leaf-r2
  gather_facts: false
  tasks:
    - name: Set access VLAN on swp2
      nvidia.nvue.api:
        operation: set
        force: true
        wait: 15
        data:
          interface:
            swp2:
              bridge:
                domain:
                  br_default:
                    access: 111
EOF
git add playbooks/access-ports.yml && git commit -qm "Tidy access ports"
ansible-playbook playbooks/access-ports.yml >/dev/null ;;
F7) ssh ubuntu@$SIM "pkill -f '[o]pensm.*-p 12'" ;; # standby SM disappears
F7b) ssh ubuntu@$SIM "pkill -f '[o]pensm.*-p 14'" ;; # later: master fails too
F8) ssh ubuntu@$SIM "cd ~/ibsim && sed -i.bak 's/${VICTIM_GUID}[^,;]*, *//; s/, *$
{VICTIM_GUID}[^,;]*//' partitions.conf && pkill -HUP -f '[o]pensm.*-p 14'" ;;
F9) ( for n in 400 1500 4200 9800; do
      sim_cmd "PerformanceSet \"hxa-su1-leaf-r1\"[33] PortCounters.SymbolErrorCounter=$n";
sleep 120
done ) >/dev/null 2>&1 & ;;
F10) sim_cmd 'Unlink "hxa-su2-leaf-r2"[34]' ;;
*) echo "unknown fault $id"; exit 1 ;;
esac

```

2. Make it executable and check it parses:

```

ubuntu@oob-mgmt-server:~$ chmod +x ~/capstone/inject.sh
ubuntu@oob-mgmt-server:~$ bash -n ~/capstone/inject.sh && echo syntax ok

```

Notes on the script (check on your set-up):

- **nv_on** assumes the passwordless SSH from **oob-mgmt-server** to the switches that Lab 0 and Lab 12 set up. Without it, **BatchMode=yes** makes the script fail instead of hanging on a password prompt.
- **SVI_AF** must match the SVI syntax you used in Lab 5. From Cumulus Linux 5.15, NVUE uses **interface vlan110 ipv4 vrr**. Earlier 5.x releases use **ip vrr**.
- F6 uses the Lab 12 connection settings (**nvidia.nvue.api** over the NVUE REST API on port 8765). If your Lab 12 inventory uses a different project path or host names, adjust **ANSIBLE_DIR** and **hosts**.
- F7, F7b and F8 assume that Lab 8 started the master OpenSM with priority 14 (**-p 14**) and the standby with priority 12 (**-p 12**), and that the partition file is **~/ibsim/partitions.conf**. The **[o]pensm** pattern stops **pkill** from matching its own shell. OpenSM re-reads the partition file on its next heavy sweep, which **SIGHUP** triggers (check on your OpenSM version; if the change doesn't appear, restart the master OpenSM the Lab 8 way).
- The ibsim console reads commands from the named pipe **~/ibsim/simcmd** that Lab 8 creates. If nothing is reading the pipe, **echo ... > simcmd** blocks. In the Lab 8 topology, leaf ports 33 and above are spine uplinks; confirm the port numbers in **~/ibsim/base-links.txt**.

Task 3 — Run the challenge

With an instructor: the instructor runs one scenario at a time (**~/capstone/inject.sh F3**, for example), hands the student the ticket text from the **Break and fix** section, and starts a timer.

On your own: run `~/capstone/inject.sh random`. The script picks a fault, writes the answer to `/root/capstone-answers` (readable only with `sudo`), and tells you nothing else. Read the ticket list in **Break and fix**, decide which ticket matches what you see, and start.

Rules:

1. Target 15–20 minutes per scenario. After 10 minutes stuck, take Hint 1; after 15, Hint 2.
2. Change **one thing at a time**. On switches, review `nv config diff` before every `nv config apply`.
3. Prove the fix with the same test that showed the fault.
4. Restore before the next scenario. Either apply the fix from the solution, or start again from the `pre-capstone` checkpoint (Lab 6). A checkpoint restore is the only guaranteed clean slate, but it takes several minutes and you must restart ibsim afterwards.

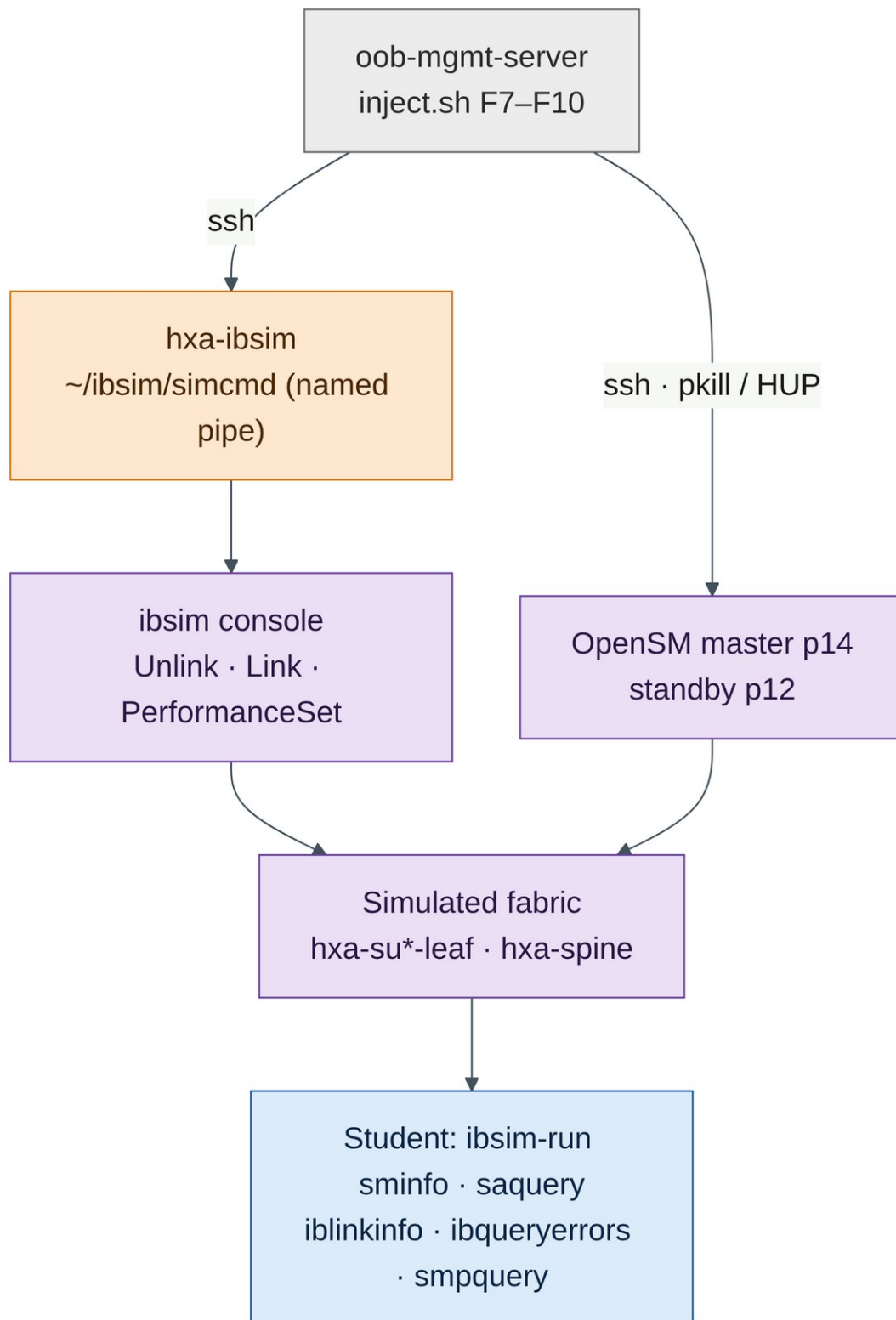


Figure 13.3: How the InfiniBand faults reach ibsim

The injector on oob-mgmt-server writes simulator commands into the named pipe that Lab 8 feeds to the ibsim console, or signals the OpenSM processes. Students see the effects only through the diagnostic tools run with ibsim-run.

Task 4 — Keep troubleshooting notes

Good notes are part of the score. Copy this template into `~/capstone/notes-F<n>.md` at the start of each scenario and fill it in **as you go**:

```
# Ticket F<n> – <one-line summary in your own words>
```

Start: 14:05 Finish: 14:22 Hints used: 0

Symptom and scope

- What fails: gpu02 eth1 (172.16.10.102) cannot reach its gateway 172.16.10.1
- What still works: gpu02 → gpu01 172.16.10.101 (same VLAN, other leaf)
- Scope: one leaf (hxb-leaf-r3), one VLAN (110); leaf-r1 VLAN 110 fine

Evidence (command → key line)

- gpu02: ip neigh show 172.16.10.1 → FAILED
- leaf-r3: nv show interface vlan110 → no vrr address
- leaf-r1: nv show interface vlan110 → vrr 172.16.10.1/24, 00:00:5e:00:01:01

Root cause

VRR (anycast gateway) removed from vlan110 on hxb-leaf-r3 (nv config history: rev 41).

Fix (exact commands)

nv set interface vlan110 ipv4 vrr address 172.16.10.1/24 ... ; nv config diff ; nv config apply

Verification (same test as the symptom)

- gpu02: ping 172.16.10.1 → 0% loss; ping 172.16.11.101 → 0% loss

Prevention

Gateway settings come from the Lab 11 template; add a post-change ping check to the pipeline.

What makes notes good:

Good notes	Weak notes
State scope: what works as well as what fails	"Network is broken"
Quote the one line of output that proves each point	Paste 200 lines of output with no comment
Name the device, interface, VLAN/VNI or port and LID	"Something on the leaf"
Separate the root cause from the symptom	"Fixed by rebooting"
Show the exact fix and the exact verification	"Changed config, works now"
End with prevention: template, check or alert	No lesson recorded

Task 5 — Score yourself

Score each scenario out of 10 with this rubric. An instructor scores from the student's notes and a short verbal walk-through.

Criterion	0	1	2	3
Scope and isolation (max 2)	No scope stated	Right area (Ethernet or IB, one tenant or rail)	Exact device and interface, VLAN or port	—
Root cause (max 3)	Wrong or none	Right layer, wrong detail	Right cause, weak evidence	Right cause, proved with the key output line
Fix (max 2)	None, or it broke something else	Worked but untidy (several changes, no diff review)	Minimal, reviewed with nv config diff or equivalent	—
Verification (max 2)	None	A different test from the symptom	Same test as the symptom, plus a check that nothing else broke	—
Notes (max 1)	Missing or unreadable	Complete template, including prevention	—	—

Adjustment	Points
Each hint used	-1 (Hint 3 counts as two)
Solved in under 10 minutes	+1
Needed the solution	Score the notes and verification only

Total over ten scenarios	Rating
85-110	Exam-ready for Domain 5: fast, evidence-based, tidy
65-84	Solid. Revisit the chapters for the scenarios where you lost points
Below 65	Repeat the lowest-scoring scenarios after re-reading their chapters and labs

Break and fix

Each scenario is written in four parts: **Ticket** (all the student gets), **Injection** (instructor only), **Symptoms** (what the student should find), and **Hints**. Full solutions follow in the next section.

F1 — "Rail 3 is running on one leg"

Ticket: "Monitoring says `hxb-leaf-r3` has only one path to the spines. Jobs still run. Can you check before tonight's training run?"

Injection: `inject.sh F1`. On `hxb-spine02`, the unnumbered neighbour on `swp3` (the link to `hxb-leaf-r3 swp32`) is changed from `remote-as external` to `remote-as 65200`.

Symptoms:

- On `hxb-leaf-r3`, `sudo vtysh -c "show bgp summary"` shows `swp31` Established but `swp32` in `Active`, `Connect` or `Idle`, with 0 prefixes received.
- `sudo vtysh -c "show ip route 10.255.0.11"` on `leaf-r3` shows one next hop (via `swp31`) instead of two.
- The physical link is up: `nv show interface swp32` shows `up`, and LLDP shows `hxb-spine02 swp3`.
- On `hxb-spine02`, `show bgp neighbor swp3` shows an OPEN error / bad peer AS as the last reset reason (wording varies by FRR version).

Hints:

1. The link is up. Which layer is left, and which end would you look at?
2. Compare `nv show vrf default router bgp neighbor swp3` on `spine02` with `swp1` on the same spine.
3. With BGP unnumbered, both ends normally say `remote-as external`. What happens if one end expects a specific AS?

Exam objective: 2.3 (the BGP underlay that EVPN runs on) · 6.1 (finding the change with `nv config history`) · 1.2 (rail-optimised design: why ECMP to both spines matters).

F2 — "Small pings work, big transfers hang"

Ticket: "From `hxb-gpu01` to `hxb-gpu02` on AURORA VLAN 111, ping works but copying a large file with `scp` stalls. Other pairs seem fine."

Injection: `inject.sh F2`. On `hxb-spine01`, `swp2` (the link to `hxb-leaf-r2 swp31`) gets `link mtu 1500`. The leaf side stays at 9216.

Symptoms:

- `ping 172.16.11.102` from `hxb-gpu01` (over eth2, leaf-r2 → spine → leaf-r4) works with default-size packets.
- A full-size, don't-fragment ping fails or partly fails: `ping -M do -s 1472 -c 5 172.16.11.102`. VXLAN adds 50 bytes, so a 1500-byte host packet becomes a 1550-byte underlay packet, which doesn't fit a 1500-byte link. Whether it fails depends on which spine the flow hashes to.
- **BGP stays Established** on every link. Unlike OSPF, BGP doesn't check MTU, so the control plane looks healthy.
- An underlay test from `hxb-leaf-r2` to `hxb-spine01`'s loopback with a jumbo packet fails, while the same test to `hxb-spine02` works.

Hints:

1. Small works, large fails: think about packet size before you think about routing.
2. Test the underlay between leaf-r2 and each spine with `ping -M do -s 8972` to the spine loopbacks.
3. Compare the MTU on **both ends** of each leaf-r2 uplink.

Exam objective: 2.1 (link settings on Spectrum-X switches, MTU 9216) · 2.3 (VXLAN overhead in an EVPN fabric) · 5.3 (verifying the data path, not just the control plane).

F3 — "gpu02 can't see gpu01 on VLAN 111"

Ticket: "After last night's maintenance on rail 4, `hxb-gpu02` eth2 (172.16.11.102) can't reach `hxb-gpu01` eth2 (172.16.11.101). It can reach its gateway."

Injection: `inject.sh F3`. On `hxb-leaf-r4`, VLAN 111 is re-mapped from VNI 10111 to VNI 10101.

Symptoms:

- From `hxb-gpu02`: `ping 172.16.11.1` works (the gateway is local VRR on leaf-r4). `ping 172.16.11.101` fails.
- Traffic between different subnets may still work (for example `gpu02` eth2 to 172.16.10.101), because symmetric routing uses the L3 VNI 104001, not the L2 VNI. That partial success is a strong clue.
- On leaf-r4, `sudo vtysh -c "show evpn vni"` lists VNI **10101** with no remote VTEPs, and no 10111.
- On leaf-r2, `sudo vtysh -c "show evpn vni 10111"` lists remote VTEPs 10.255.0.11 and .13 as usual, but not 10.255.0.14.

Hints:

1. Same subnet fails, gateway works. Which part of EVPN carries same-subnet traffic between leaves?
2. Run `show evpn vni` on leaf-r4 and on leaf-r2 and compare the VNI lists.
3. Look at `nv show bridge domain br_default vlan 111` on leaf-r4.

Exam objective: 2.3 (L2 VNI mapping, route type 2/3, symmetric IRB).

F4 — "One server can't leave its subnet"

Ticket: "`hxb-gpu02` eth1 (172.16.10.102) can't reach anything outside 172.16.10.0/24. It can still ping `hxb-gpu01` on 172.16.10.101."

Injection: `inject.sh F4`. On `hxb-leaf-r3`, the VRR (anycast gateway) configuration is removed from `vlan110`.

Symptoms:

- From `hxb-gpu02`: `ping 172.16.10.101` works (L2 across VNI 10110). `ping 172.16.10.1` and `ping 172.16.11.101` fail.
- `ip neigh show 172.16.10.1` on `hxb-gpu02` shows **FAILED** or **INCOMPLETE**: nothing answers ARP for the gateway on leaf-r3.
- On leaf-r3, `nv show interface vlan110` has no VRR address. On leaf-r1, the same command shows `172.16.10.1/24` and MAC `00:00:5e:00:01:01`.
- BGP and EVPN sessions are all healthy.

Hints:

1. The host can bridge but not route. Who is the host's router?
2. Check the host's ARP entry for 172.16.10.1.
3. Compare `nv show interface vlan110` on leaf-r3 and leaf-r1.

Exam objective: 2.3 (distributed anycast gateway with VRR on every leaf).

F5 — "RoCE audit failed on rail 2"

Ticket: "The weekly RoCE audit flagged `hxb-leaf-r2`. `perftest` still runs. Is it a real problem?"

Injection: `inject.sh F5`. On `hxb-leaf-r2`, `nv set qos roce mode lossy` is applied. (Variant for a harder round: change the switch's QoS trust so DSCP is no longer trusted, for example `nv set qos mapping default-global trust l2`; check on your release whether VX accepts it and how it interacts with the RoCE profile.)

Symptoms:

- Soft-RoCE `perftest` still works. VX has no Spectrum ASIC, so there is no PFC, pause or buffer behaviour to observe, and nothing drops.
- `nv show qos roce` on leaf-r2 shows `mode lossy` and no PFC on switch priority 3. Leaf-r1 shows `mode lossless` with PFC on priority 3, ECN enabled and trust `pcp,dscp`.
- `nv config history` on leaf-r2 shows a recent revision that changed it.

Hints:

1. VX can't show you drops. What can it show you?
2. Diff `nv show qos roce` between leaf-r2 and leaf-r1.
3. Which single NVUE command sets the lossless defaults, and what does `mode lossy` remove?

Exam objective: 2.1 (configure RoCE with `nv set qos roce`) · 2.2 (PFC and ECN) · 5.3 (verify the low-latency, lossless configuration end to end).

F6 — "BOREALIS host sees AURORA broadcast traffic"

Ticket: "`hxb-gpu03` eth2 (BOREALIS, 172.17.11.103) lost its gateway this morning. `tcpdump` on it shows ARP requests for 172.16.11.x addresses, which belong to another tenant."

Injection: `inject.sh F6`. A new playbook, committed to the Lab 12 Git repository as "Tidy access ports", sets `hxb-leaf-r2 swp2` to access VLAN 111 (AURORA) instead of 211 (BOREALIS).

Symptoms:

- From `hxb-gpu03`: `ping 172.17.11.1` and `ping 172.17.11.104` fail.
- `sudo tcpdump -ni eth2 arp` on `hxb-gpu03` shows ARP for 172.16.11.x. The host is in the wrong broadcast domain, which is a **tenant isolation breach**, not only an outage.
- On leaf-r2, `nv show interface swp2 bridge domain br_default` shows access VLAN 111. On leaf-r1, `swp2` shows 210, and on leaf-r4, `swp2` shows 211.
- `nv config history` on leaf-r2 shows a revision applied through the API by the automation user, not an interactive CLI user.
- `git log --oneline -3` in the Ansible project shows the "Tidy access ports" commit.

Hints:

1. Which VLAN is gpu03's port in, and which should it be in?
2. Who changed it? `nv config history` tells you whether it came from a person or from the API.
3. Fix the source of truth, not just the switch. Otherwise the next playbook run puts the fault back.

Exam objective: 6.2 (Ansible playbooks for VLANs) · 6.1 (NVUE configuration management and history) · 2.3 (tenant isolation).

F7 — "Confirm Subnet Manager HA before the change freeze"

Ticket: "Change freeze starts tomorrow. Please confirm the Helix-A SM pair is healthy: one master, one standby." (If the student reports HA as healthy, the instructor later runs `inject.sh F7b` and sends a second ticket: "New nodes on Helix-A are stuck in Initializing and `sminfo` times out.")

Injection: `inject.sh F7` stops the **standby** OpenSM (priority 12). `inject.sh F7b` later stops the **master** (priority 14).

Symptoms (stage 1):

- `ibsim-run sminfo` still shows a master (priority 14, state `SMINFO_MASTER`). Everything looks fine at first glance.
- `ibsim-run sminfo <standby LID>` (the standby LID from `base-sm-ports.txt`) doesn't return a standby SM (it times out or shows it not active), and `ps -ef | grep [o]pensm` shows only one OpenSM.

Symptoms (stage 2, after F7b):

- `ibsim-run sminfo` times out. There is no master SM.
- Existing routes keep working, because the switches keep their programmed forwarding tables, but ports that come up now stay in **Initializing**.

Hints:

1. "One master" is only half the question. How do you query a specific SM rather than the master?
2. Compare `ps -ef | grep [o]pensm` with how Lab 8 started the HA pair.
3. SM election uses the highest priority, then the lowest GUID. Which priority must the restarted standby have so that it doesn't take over?

Exam objective: 3.1 (SM high availability) · 5.4 (SM and UFM health) · 5.5 (`sminfo`, `saquery`).

F8 — "One PRODUCTION node can't join the job"

Ticket: "A node on Helix-A can't talk to the other PRODUCTION nodes over the PRODUCTION partition. Everything else about it looks normal."

Injection: `inject.sh F8` removes one member (`VICTIM_GUID`) from the PRODUCTION line of `~/ibsim/partitions.conf` and makes the master SM re-read the file.

Symptoms:

- The node's port is **Active** in `ibsim-run iblinkinfo`, and it has a LID.
- `ibsim-run smpquery pkeys <LID>` for that node lists the default partition but **not 0x8002** (PRODUCTION, full member). A healthy PRODUCTION node lists `0x8002`.
- `diff ~/ibsim/base-partitions.conf ~/ibsim/partitions.conf` shows the missing GUID (and there is a `partitions.conf.bak`).

Hints:

1. The port is up and has a LID, so the SM has seen it. What else does the SM program into a port?
2. Read the node's PKey table with `smpquery pkeys` and compare it with a healthy PRODUCTION node.
3. Where does OpenSM get partition membership from?

Exam objective: 3.2 (configure PKeys: `partitions.conf`, full and limited membership, `0x8002` = full member of `0x0002`).

F9 — "Errors climbing on an SU1 uplink"

Ticket: "UFM-style health alerts every few minutes about `hxa-su1-leaf-r1`. No outage yet."

Injection: `inject.sh F9` sets `SymbolErrorCounter` on `hxa-su1-leaf-r1` port 33 (a spine uplink) to 400, 1500, 4200 and 9800 at two-minute intervals, using the `ibsim` console command `PerformanceSet "hxa-su1-leaf-r1"[33] PortCounters.SymbolErrorCounter=N`.

Symptoms:

- `ibsim-run ibqueryerrors` lists `hxa-su1-leaf-r1` port 33 with a non-zero `SymbolErrorCounter`. Running it again a few minutes later shows a higher value.
- The link is still `LinkUp / Active` at full width in `ibsim-run iblinkinfo --switches-only -l`.
- `base-errors.txt` from Task 1 shows no errors on that port.

Hints:

1. The link is up. Which tool lists only ports with non-zero error counters?
2. One reading tells you little. What does a **rising** symbol error count mean?
3. Which end of the link, and which cable, would you replace, and how do you confirm the fix in the simulator?

Exam objective: 5.5 (`ibqueryerrors`, `perfquery`, `iblinkinfo`) · 3.4 and 5.4 (monitoring link health and acting on alerts).

F10 — "SU2 lost bandwidth to the spines"

Ticket: "Jobs spanning both SUs slowed down about 3% after lunch. Nothing is down, as far as we can see."

Injection: `inject.sh F10` sends `Unlink "hxa-su2-leaf-r2"[34]` to the `ibsim` console through the named pipe (`echo 'Unlink "hxa-su2-leaf-r2"[34]' > ~/ibsim/simcmd`).

Symptoms:

- `ibsim-run iblinkinfo --switches-only -l` shows `hxa-su2-leaf-r2` port 34 as `Down` (physical state `Polling` or `Disabled`, depending on the `ibsim` build), with no peer.
- `diff ~/ibsim/base-links.txt <(ibsim-run iblinkinfo --switches-only -l)` highlights that one line.
- `ibsim-run ibqueryerrors` may show nothing useful: in the simulator, unlinking doesn't necessarily raise error counters. That is why the baseline diff matters.
- The fabric keeps working. After its next sweep, the SM routes around the missing uplink, so only bandwidth drops.

Hints:

1. "Slower, nothing down" often means a missing link in an ECMP-like fabric. Which tool shows every link?
2. Compare the current `iblinkinfo --switches-only -l` with your baseline.
3. Which spine and port was leaf-r2 port 34 connected to? Your baseline knows.

Exam objective: 5.5 (`iblinkinfo`, `ibnetdiscover`) · 3.1 (fabric provisioning and link validation).

Solutions**F1 solution — wrong remote-as on spine02**

1. Scope: only leaf-r3's `swp32` session is down. Its other uplink is fine, and other leaves' sessions to spine02 are fine, so the problem is this one link or its two ends.
2. The physical layer is fine (`nv show interface swp32` is up, LLDP shows the right neighbour).
3. On `hxb-spine02`, compare neighbours:

```
cumulus@hxb-spine02:mgmt:~$ nv show vrf default router bgp neighbor swp3
cumulus@hxb-spine02:mgmt:~$ nv show vrf default router bgp neighbor swp1
cumulus@hxb-spine02:mgmt:~$ nv config history
```

Expected output (illustrative): `swp3` shows `remote-as 65200` and `swp1` shows `remote-as external`. The history shows the revision that changed it.

4. Fix and verify:

```
cumulus@hxb-spine02:mgmt:~$ nv set vrf default router bgp neighbor swp3 remote-as external
cumulus@hxb-spine02:mgmt:~$ nv config diff
cumulus@hxb-spine02:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r3:mgmt:~$ sudo vtysh -c "show bgp summary"
cumulus@hxb-leaf-r3:mgmt:~$ sudo vtysh -c "show ip route 10.255.0.11"
```

Checkpoint: swp32 is Established with prefixes received, and the route to 10.255.0.11 has two next hops again.

Root cause: the peer AS on spine02 swp3 didn't match leaf-r3's real AS (65103). The leaf's OPEN carried AS 65103, the spine expected 65200, and the session was rejected. **remote-as external** accepts any AS different from the local one, which is why unnumbered fabrics use it.

F2 solution — MTU mismatch on spine01 swp2

1. Prove it's size-related from the host:

```
ubuntu@hxb-gpu01:~$ ping -c 3 172.16.11.102
ubuntu@hxb-gpu01:~$ ping -M do -s 1472 -c 5 172.16.11.102
```

2. Test the underlay from **hxb-leaf-r2** to each spine loopback. Your SSH session is in the management VRF (the **:mgmt:** in the prompt), so run the ping in the default VRF:

```
cumulus@hxb-leaf-r2:mgmt:~$ sudo ip vrf exec default ping -M do -s 8972 -c 3 10.255.0.1
cumulus@hxb-leaf-r2:mgmt:~$ sudo ip vrf exec default ping -M do -s 8972 -c 3 10.255.0.2
```

Expected output (illustrative): the ping to 10.255.0.2 (spine02) gets replies. The ping to 10.255.0.1 (spine01) shows 100% packet loss or **message too long**.

3. Compare both ends of the leaf-r2 → spine01 link:

```
cumulus@hxb-leaf-r2:mgmt:~$ nv show interface swp31 link | grep -i mtu
cumulus@hxb-spine01:mgmt:~$ nv show interface swp2 link | grep -i mtu
```

Expected output (illustrative): leaf-r2 swp31 shows 9216, spine01 swp2 shows 1500.

4. Fix and verify:

```
cumulus@hxb-spine01:mgmt:~$ nv set interface swp2 link mtu 9216
cumulus@hxb-spine01:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r2:mgmt:~$ sudo ip vrf exec default ping -M do -s 8972 -c 3 10.255.0.1
ubuntu@hxb-gpu01:~$ ping -M do -s 1472 -c 5 172.16.11.102
```

Checkpoint: both pings succeed with no loss.

Root cause: one side of a routed link had MTU 1500. BGP stayed up because its packets are small and BGP doesn't negotiate MTU. VXLAN-encapsulated full-size frames were dropped on that link only, so failures depended on the ECMP hash.

Version note: How Cumulus VX handles an oversized received frame (drop, or accept) depends on the virtual NIC. If the jumbo ping from leaf-r2 to spine01 still works in your simulation, test in the other direction (from spine01 to 10.255.0.12), where spine01's 1500-byte egress MTU applies.

F3 solution — VNI mismatch on leaf-r4

```
cumulus@hxb-leaf-r4:mgmt:~$ sudo vtysh -c "show evpn vni"
cumulus@hxb-leaf-r4:mgmt:~$ nv show bridge domain br_default vlan 111
cumulus@hxb-leaf-r2:mgmt:~$ sudo vtysh -c "show evpn vni 10111"
```

Expected output (illustrative): leaf-r4 lists VNI 10101 (L2, 0 remote VTEPs) and no 10111. Leaf-r2's VNI 10111 lists remote VTEPs 10.255.0.11 and 10.255.0.13 only.

Fix and verify:

```
cumulus@hxb-leaf-r4:mgmt:~$ nv unset bridge domain br_default vlan 111 vni 10101
cumulus@hxb-leaf-r4:mgmt:~$ nv set bridge domain br_default vlan 111 vni 10111
cumulus@hxb-leaf-r4:mgmt:~$ nv config diff
cumulus@hxb-leaf-r4:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r4:mgmt:~$ sudo vtysh -c "show evpn vni 10111"
ubuntu@hxb-gpu02:~$ ping -c 3 172.16.11.101
```

Checkpoint: leaf-r4 shows VNI 10111 with remote VTEPs, and the same-subnet ping works.

Root cause: VLAN 111 on leaf-r4 was mapped to the wrong VNI, so its type-3 (inclusive multicast) and type-2 routes were in a VNI no other leaf used. Same-subnet traffic had no remote VTEPs to go to. Routed traffic could still work through the L3 VNI. This is exactly the typo from Chapter 22's opening story, and the reason for the Ansible `assert` in `site-vlans.yml`.

F4 solution — missing VRR on leaf-r3

```
ubuntu@hxb-gpu02:~$ ip neigh show 172.16.10.1
cumulus@hxb-leaf-r3:mgmt:~$ nv show interface vlan110
cumulus@hxb-leaf-r1:mgmt:~$ nv show interface vlan110
cumulus@hxb-leaf-r3:mgmt:~$ nv config history
```

Fix (use the same syntax family you used in Lab 5; the 5.15+ form is shown):

```
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface vlan110 ipv4 vrr address 172.16.10.1/24
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface vlan110 ipv4 vrr mac-address 00:00:5e:00:01:01
cumulus@hxb-leaf-r3:mgmt:~$ nv set interface vlan110 ipv4 vrr state enabled
cumulus@hxb-leaf-r3:mgmt:~$ nv config diff
cumulus@hxb-leaf-r3:mgmt:~$ nv config apply -y
```

On releases before 5.15, the equivalent is `nv set interface vlan110 ip vrr address 172.16.10.1/24, ... ip vrr mac-address 00:00:5e:00:01:01` and `... ip vrr state up`.

Verify:

```
ubuntu@hxb-gpu02:~$ ping -c 3 172.16.10.1
ubuntu@hxb-gpu02:~$ ping -c 3 172.16.11.101
```

Checkpoint: both pings work, and `ip neigh` shows 172.16.10.1 as **REACHABLE** with MAC 00:00:5e:00:01:01.

Root cause: the anycast gateway exists on every leaf, so each host's gateway is always its local leaf. Removing it from one leaf breaks routing only for hosts attached to that leaf, while bridging across the fabric still works.

F5 solution — RoCE lossy on leaf-r2

```
cumulus@hxb-leaf-r2:mgmt:~$ nv show qos roce
cumulus@hxb-leaf-r1:mgmt:~$ nv show qos roce
cumulus@hxb-leaf-r2:mgmt:~$ nv config history
```

Expected output (illustrative): leaf-r2 shows `mode lossy` and no PFC on switch priority 3. Leaf-r1 shows `mode lossless`, PFC on priority 3, ECN enabled and trust `pcp, dscp`.

Fix and verify:

```
cumulus@hxb-leaf-r2:mgmt:~$ nv set qos roce mode lossless
cumulus@hxb-leaf-r2:mgmt:~$ nv config apply -y
cumulus@hxb-leaf-r2:mgmt:~$ nv show qos roce
```

For the trust variant, remove the trust override (`nv unset qos mapping default-global trust`) and confirm that `nv show qos roce` reports trust `pcp,dscp` again (check on your release).

Checkpoint: `nv show qos roce` on leaf-r2 matches leaf-r1 line for line.

Root cause: one leaf's RoCE profile had been changed to lossy (PFC removed from priority 3). On real Spectrum hardware, RoCE traffic through that leaf would lose its lossless guarantee and drop under congestion, which causes retransmissions and slow collectives. VX can't show that, so the audit (comparing configuration against the standard) is the right detection method. Chapter 5: hosts mark DSCP 26 (traffic class 106), switches trust DSCP and map it to switch priority 3 with PFC and ECN.

F6 solution — Ansible pushed the wrong access VLAN

1. Find what changed and who changed it:

```
cumulus@hxb-leaf-r2:mgmt:~$ nv show interface swp2 bridge domain br_default
cumulus@hxb-leaf-r2:mgmt:~$ nv config history
cumulus@hxb-leaf-r2:mgmt:~$ nv config diff <previous-rev> <latest-rev>
ubuntu@oob-mgmt-server:~/helix-ansible$ git log --oneline -3
ubuntu@oob-mgmt-server:~/helix-ansible$ git show HEAD --stat
```

Expected output (illustrative): access VLAN 111 on swp2. The history shows an API revision by the automation user. `git log` shows `Tidy access ports`, which added `playbooks/access-ports.yml`.

2. Fix the source of truth first, then the switch. Revert the commit and correct the port through Ansible (or NVUE, if your Lab 12 project doesn't manage access ports):

```
ubuntu@oob-mgmt-server:~/helix-ansible$ git revert --no-edit HEAD
cumulus@hxb-leaf-r2:mgmt:~$ nv set interface swp2 bridge domain br_default access 211
cumulus@hxb-leaf-r2:mgmt:~$ nv config apply -y
```

3. Verify the host and check that the automation now agrees with the fabric:

```
ubuntu@hxb-gpu03:~$ ping -c 3 172.17.11.1
ubuntu@hxb-gpu03:~$ ping -c 3 172.17.11.104
ubuntu@oob-mgmt-server:~/helix-ansible$ ansible-playbook playbooks/site-vlans.yml --check --diff
```

Checkpoint: gpu03 reaches its gateway and its BOREALIS peer. No AURORA ARP is seen on eth2. The check run reports no changes.

Root cause: an unreviewed playbook applied a wrong access VLAN to one port. It broke gpu03 and put a BOREALIS host into the AURORA broadcast domain. Prevention: peer review (pull requests) for playbooks, `--check --diff` in CI against the Air twin (Lab 6 SDK script), and an `assert` step that checks access ports against the tenant plan.

F7 solution — standby SM missing, then master failure

Stage 1 (the proactive check):

```
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run sminfo
ubuntu@hxa-ibsim:~/ibsim$ cat base-sm-ports.txt
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run sminfo <standby LID>
ubuntu@hxa-ibsim:~/ibsim$ ps -ef | grep [o]pensm
```

Expected output (illustrative):

```
sminfo: sm lid 1 sm guid 0x...a1, activity count 5120 priority 14 state 3 SMINFO_MASTER
```

The query to the standby's LID fails or doesn't show a standby state, and **ps** shows one OpenSM (priority 14).

Fix: start the standby exactly as Lab 8 does, with **priority 12** on its own port GUID. Lab 8's command has this shape (use yours):

```
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run opensm -g <standby port GUID> -p 12 -P
~/ibsim/partitions.conf -B
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run sminfo <standby LID>
```

Expected output (illustrative): ... priority 12 state 2 SMINFO_STANDBY.

Stage 2 (after F7b, if HA was missed): **ibsim-run sminfo** times out. Start the standby first, which becomes master because it is the only SM. Then prove the failover path works:

```
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run sminfo
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run opensm -g <master port GUID> -p 14 -P ~/ibsim/partitions.conf
-B
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run sminfo
```

Checkpoint: one **SMINFO_MASTER** (priority 14) and one **SMINFO_STANDBY** (priority 12). Optional: stop the master and confirm the standby takes over within a few sweeps, then restore it.

Root cause: HA had silently degraded to a single SM. Nobody noticed until the master failed. With no master, existing routes kept forwarding but new or flapping ports couldn't become Active. Prevention: alert on SM state (UFM health, Chapter 15), and check the standby explicitly, not only the master.

Field note: Watch priorities when you restart SMs. If the standby is restarted with a **higher** priority than the master (15, say), it takes over mastership. That is legal but not what the design says, and it makes the next failover go the wrong way. OpenSM's election rule is the highest priority, then the lowest GUID. OpenSM's **-p** accepts 0-15.

F8 solution — missing partition member

```
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run iblinkinfo | grep <node name>
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run smpquery pkeys <LID of affected node>
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run smpquery pkeys <LID of a healthy PRODUCTION node>
ubuntu@hxa-ibsim:~/ibsim$ diff base-partitions.conf partitions.conf
```

Expected output (illustrative): the healthy node lists **0xffff** (or **0x7fff**) and **0x8002**. The affected node lists only the default partition. The diff shows the node's GUID missing from the **PRODUCTION=0x0002** line.

Fix: add the GUID back (restore the line from the baseline copy, or edit it), then make OpenSM re-read the file and check the PKey table:

```
ubuntu@hxa-ibsim:~/ibsim$ cp base-partitions.conf partitions.conf
ubuntu@hxa-ibsim:~/ibsim$ pkill -HUP -f '[o]pensm.*-p 14'
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run smpquery pkeys <LID of affected node>
```

Checkpoint: **0x8002** appears in the node's PKey table. If it doesn't after a minute, restart the master OpenSM the Lab 8 way.

Root cause: the node's port GUID had been removed from the PRODUCTION partition, so the SM didn't program 0x8002 into its PKey table. Packets carrying PKey 0x8002 to or from it are dropped by PKey enforcement, while the port looks perfectly healthy. Remember the encoding: 0x8002 is partition 0x0002 with the full-membership bit set.

F9 solution — rising symbol errors on an uplink

```
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run ibqueryerrors
```

Expected output (illustrative):

```
Errors for "hxa-su1-leaf-r1"
  GUID 0x... port 33: [SymbolErrorCounter == 1500]
```

Run it again two minutes later and the count has risen (4200). Confirm that the link is still up and find its far end:

```
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run iblinkinfo --switches-only -l | grep -E '"hxa-su1-leaf-r1".*\b33\b'
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run perfquery <leaf LID> 33
```

Diagnosis: a steadily rising **SymbolErrorCounter** on a link that is still up means a marginal physical link: a dirty or damaged cable or optic, or a bad port. Symbol errors are corrected or retried, so there's no outage yet, but throughput and latency suffer, and the link may go down or retrain at a lower width. In production you would check the physical layer with **mLxlink** (BER, eye), look at both ends, and replace the cable or optic in a maintenance window. UFM would flag it through its health checks.

Fix in the simulator (the equivalent of "replace the cable and clear the counters"):

```
ubuntu@hxa-ibsim:~/ibsim$ echo 'PerformanceSet "hxa-su1-leaf-r1"[33]
PortCounters.SymbolErrorCounter=0' > ~/ibsim/simcmd
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run ibqueryerrors
```

The injection loop runs on **oob-mgmt-server**, so the instructor should also stop it there (**kill %1** in the shell that ran it, or wait until it finishes after eight minutes). Clearing counters with **perfquery -R <LID> 33** also works on real fabrics (check how your ibsim build handles it).

Checkpoint: **ibqueryerrors** shows no errors on port 33, and a second run a few minutes later is still clean.

F10 solution — failed spine uplink on SU2 leaf-r2

```
ubuntu@hxa-ibsim:~/ibsim$ ibsim-run iblinkinfo --switches-only -l > now-links.txt
ubuntu@hxa-ibsim:~/ibsim$ diff base-links.txt now-links.txt
ubuntu@hxa-ibsim:~/ibsim$ grep -E '"hxa-su2-leaf-r2".*\b34\b' base-links.txt
```

Expected output (illustrative): the diff shows **hxa-su2-leaf-r2** port 34 changed from **4X ... LinkUp** with a spine peer to **Down** with no peer. The baseline line names the peer, for example **"hxa-spine02"** port 2.

Fix: reconnect the link through the ibsim console using the peer from your baseline:

```
ubuntu@hxa-ibsim:~/ibsim$ echo 'Link "hxa-su2-leaf-r2"[34] "hxa-spine02"[2]' > ~/ibsim/simcmd
ubuntu@hxa-ibsim:~/ibsim$ sleep 15; ibsim-run iblinkinfo --switches-only -l | grep -E '"hxa-su2-leaf-r2".*\b34\b'
```

Some ibsim builds also have **ReLink "hxa-su2-leaf-r2"**, which restores the node's disconnected links. Type **Help** at the ibsim console to see the commands your build supports.

Checkpoint: port 34 is `LinkUp / Active` at 4X again, and `diff base-links.txt <(ibsim-run iblinkinfo --switches-only -l)` shows nothing. The SM brings the port to Active on its next sweep (default sweep interval about 10 seconds).

Root cause: one spine uplink failed. The SM routed around it, so nothing broke, but SU2's bandwidth to the spines dropped by one uplink's share. Only a comparison with a known-good baseline (or UFM link monitoring) shows it quickly. In production, also check `LinkDownedCounter` on both ends and the physical layer before you declare it fixed.

Verify

- [] You captured Ethernet and InfiniBand baselines and a `pre-capstone` favourite checkpoint.
- [] For each scenario you solved, the notes follow the template, with scope, evidence, root cause, fix, verification and prevention.
- [] Every fix was proved with the same test that showed the fault.
- [] After the last scenario, all switches match the baseline: `nv config diff applied startup` is empty and `show bgp summary`, `show evpn vni` and `nv show qos roce` match the baseline files.
- [] On `hxa-ibsim`, `sminfo` shows a master (priority 14), the standby answers (priority 12), `ibqueryerrors` is clean and `iblinkinfo` matches `base-links.txt`.
- [] Your rubric total is recorded, with the chapters to revisit for any scenario that scored below 7.

Clean-up / save state

1. If you fixed each fault by hand, run a final comparison with the baseline (the Task 1 loop into a new folder, then `diff -r`).
2. If anything is uncertain, stop the simulation **without** a checkpoint and start it from `pre-capstone` (Lab 6). Restart ibsim and OpenSM afterwards.
3. Remove the capstone playbook if it's still in the Ansible project, and check `git status` is clean.
4. Stop the simulation with **Stop Simulation and Store Checkpoint**. Keep `pre-capstone` as a favourite: you can run the challenge again before the exam.

Exam tie-in

- **Domain 5 (20%)** questions look like these tickets: a symptom, a few lines of output, and "what is the most likely cause?" or "what should you check next?". Practise reading the one line that matters.
- **5.5:** know which InfiniBand tool answers which question: `sminfo` (who is master), `saquery/smpquery` (SM-capable ports, PKey tables), `iblinkinfo` (every link's state, width and speed), `ibqueryerrors` and `perfquery` (error counters), `ibnetdiscover` (topology for baseline diffs).
- **2.1, 2.2, 2.3:** control plane up does not mean data plane healthy (F2). Same subnet broken but routing working points to the L2 VNI (F3). Bridging working but routing broken points to the gateway (F4). RoCE settings must be identical on every switch (F5).
- **3.1, 3.2:** SM HA means checking the standby too (F7). A port can be Active and still be missing from a partition (F8).
- **6.1, 6.2:** `nv config history` shows whether a change came from a person or from automation. Fix the source of truth, then the device (F6).

Review questions

1. BGP is Established on every link, but large packets between two hosts on different leaves fail and small ones succeed. What do you check first, and why doesn't BGP show the problem?
2. A host can ping another host in the same subnet on a different leaf, but not its default gateway. Which configuration is most likely missing, and where?

3. `sminfo` shows a healthy master SM. Why is that not enough to confirm SM high availability, and how do you check the standby?
4. `ibqueryerrors` shows `SymbolErrorCounter == 4200` on a leaf uplink that is still Active, and the value was 1500 five minutes ago. What does this indicate and what is the usual fix?
5. A leaf's VLAN is wrong, and `nv config history` shows the change came through the API from the automation account. Why should you fix the Git repository before (or together with) the switch?

Answers

1. Check MTU on both ends of every link in the path (and test with `ping -M do -s 8972` in the underlay). BGP doesn't negotiate or test MTU and its packets are small, so sessions stay up while full-size VXLAN frames (inner frame plus 50 bytes) are dropped on the smaller link.
2. The anycast gateway (VRR) on that SVI on the host's local leaf. In EVPN with a distributed gateway, every leaf answers for the gateway address. Bridging uses the L2 VNI, so it still works without the SVI's VRR address.
3. `sminfo` without arguments shows only the current master. HA needs a standby that is running and in `SMINFO_STANDBY` state. Find the SM-capable ports with `saquery -s`, then query the standby directly with `sminfo <standby LID>`, and check that its priority is lower than the master's.
4. A marginal physical link: a dirty or damaged cable or optic, or a bad port. It hasn't failed yet but is getting worse. Check the physical layer (`mlxlink` on real hardware), reseal or replace the cable or optic in a maintenance window, clear the counters, and confirm they stay at zero.
5. The repository is the source of truth. If you fix only the switch, the next playbook run pushes the wrong VLAN again. Revert or correct the commit, run the playbook in check and diff mode, and add review and an `assert` so it can't happen again.

Appendices

Appendix A — helix-b-air.json

Save this as **helix-b-air.json** and import it in Lab 0, Task 3. It is also on the companion site (<https://cloudfoxy-ltd.github.io/ncp-ain-guide/>).

```
{
  "format": "JSON",
  "title": "helix-b-air",
  "ztp": null,
  "content": {
    "nodes": {
      "hxb-spine01": {"cpu": 2, "memory": 4096, "storage": 20, "os": "cumulus-vx-5.16.1"},
      "hxb-spine02": {"cpu": 2, "memory": 4096, "storage": 20, "os": "cumulus-vx-5.16.1"},
      "hxb-leaf-r1": {"cpu": 2, "memory": 4096, "storage": 20, "os": "cumulus-vx-5.16.1"},
      "hxb-leaf-r2": {"cpu": 2, "memory": 4096, "storage": 20, "os": "cumulus-vx-5.16.1"},
      "hxb-leaf-r3": {"cpu": 2, "memory": 4096, "storage": 20, "os": "cumulus-vx-5.16.1"},
      "hxb-leaf-r4": {"cpu": 2, "memory": 4096, "storage": 20, "os": "cumulus-vx-5.16.1"},
      "hxb-gpu01": {"cpu": 2, "memory": 4096, "storage": 20, "os": "generic/ubuntu2404"},
      "hxb-gpu02": {"cpu": 2, "memory": 4096, "storage": 20, "os": "generic/ubuntu2404"},
      "hxb-gpu03": {"cpu": 2, "memory": 4096, "storage": 20, "os": "generic/ubuntu2404"},
      "hxb-gpu04": {"cpu": 2, "memory": 4096, "storage": 20, "os": "generic/ubuntu2404"},
      "hxa-ibsim": {"cpu": 2, "memory": 2048, "storage": 10, "os": "generic/ubuntu2404"}
    },
    "links": [
      [{"node": "hxb-leaf-r1", "interface": "swp31"}, {"node": "hxb-spine01", "interface": "swp1"}],
      [{"node": "hxb-leaf-r1", "interface": "swp32"}, {"node": "hxb-spine02", "interface": "swp1"}],
      [{"node": "hxb-leaf-r2", "interface": "swp31"}, {"node": "hxb-spine01", "interface": "swp2"}],
      [{"node": "hxb-leaf-r2", "interface": "swp32"}, {"node": "hxb-spine02", "interface": "swp2"}],
      [{"node": "hxb-leaf-r3", "interface": "swp31"}, {"node": "hxb-spine01", "interface": "swp3"}],
      [{"node": "hxb-leaf-r3", "interface": "swp32"}, {"node": "hxb-spine02", "interface": "swp3"}],
      [{"node": "hxb-leaf-r4", "interface": "swp31"}, {"node": "hxb-spine01", "interface": "swp4"}],
      [{"node": "hxb-leaf-r4", "interface": "swp32"}, {"node": "hxb-spine02", "interface": "swp4"}],
      [{"node": "hxb-leaf-r1", "interface": "swp1"}, {"node": "hxb-gpu01", "interface": "eth1"}],
      [{"node": "hxb-leaf-r2", "interface": "swp1"}, {"node": "hxb-gpu01", "interface": "eth2"}],
      [{"node": "hxb-leaf-r1", "interface": "swp2"}, {"node": "hxb-gpu03", "interface": "eth1"}],
      [{"node": "hxb-leaf-r2", "interface": "swp2"}, {"node": "hxb-gpu03", "interface": "eth2"}],
      [{"node": "hxb-leaf-r3", "interface": "swp1"}, {"node": "hxb-gpu02", "interface": "eth1"}],
      [{"node": "hxb-leaf-r4", "interface": "swp1"}, {"node": "hxb-gpu02", "interface": "eth2"}],
      [{"node": "hxb-leaf-r3", "interface": "swp2"}, {"node": "hxb-gpu04", "interface": "eth1"}],
      [{"node": "hxb-leaf-r4", "interface": "swp2"}, {"node": "hxb-gpu04", "interface": "eth2"}]
    ],
    "oob": {"nodes": {"oob-mgmt-server": {"cpu": 2, "memory": 4096, "storage": 20}}}
  }
}
```

Resource budget

Nodes	Count	vCPU each	Memory each	Total vCPU	Total memory
Cumulus VX switches (20 GB disk each, Air's minimum)	6	2	4 GiB	12	24 GiB
Ubuntu GPU servers	4	2	4 GiB	8	16 GiB
hxa-ibsim	1	2	2 GiB	2	2 GiB
oob-mgmt-server	1	2	4 GiB	2	4 GiB
oob-mgmt-switch	1	set by Air	set by Air	~1-2	~1-2 GiB
Total				about 26	about 48 GiB

This fits the free trial's 60 vCPU / 60 GiB concurrent limits. You can't run two copies at once; sleep one before starting another (Lab 6).

Appendix B — helix-a.net (InfiniBand simulator fabric)

Used in Lab 8. Save as `~/ibsim/helix-a.net` on hxa-ibsim. The first node listed hosts the master SM.

```
# Helix-A mini InfiniBand fabric for ibsim (NCP-AIN Lab Guide)
# The first node listed hosts the Subnet Manager (the UFM server).

Hca      1 "hxa-ufm-a1 mlx5_0"
[1]      "hxa-su1-leaf-r1"[30]

Switch 36 "hxa-spine01"
[1]      "hxa-su1-leaf-r1"[33]
[2]      "hxa-su1-leaf-r2"[33]
[3]      "hxa-su2-leaf-r1"[33]
[4]      "hxa-su2-leaf-r2"[33]

Switch 36 "hxa-spine02"
[1]      "hxa-su1-leaf-r1"[34]
[2]      "hxa-su1-leaf-r2"[34]
[3]      "hxa-su2-leaf-r1"[34]
[4]      "hxa-su2-leaf-r2"[34]

Switch 36 "hxa-su1-leaf-r1"
[1]      "hxa-gpu01 mlx5_0"[1]
[2]      "hxa-gpu02 mlx5_0"[1]
[30]     "hxa-ufm-a1 mlx5_0"[1]
[33]     "hxa-spine01"[1]
[34]     "hxa-spine02"[1]

Switch 36 "hxa-su1-leaf-r2"
[1]      "hxa-gpu01 mlx5_1"[1]
[2]      "hxa-gpu02 mlx5_1"[1]
[33]     "hxa-spine01"[2]
[34]     "hxa-spine02"[2]

Switch 36 "hxa-su2-leaf-r1"
[30]     "hxa-ufm-a2 mlx5_0"[1]
[1]      "hxa-gpu03 mlx5_0"[1]
[2]      "hxa-gpu04 mlx5_0"[1]
[33]     "hxa-spine01"[3]
[34]     "hxa-spine02"[3]

Switch 36 "hxa-su2-leaf-r2"
[1]      "hxa-gpu03 mlx5_1"[1]
[2]      "hxa-gpu04 mlx5_1"[1]
[33]     "hxa-spine01"[4]
[34]     "hxa-spine02"[4]

Hca      1 "hxa-gpu01 mlx5_0"
[1]      "hxa-su1-leaf-r1"[1]

Hca      1 "hxa-gpu01 mlx5_1"
[1]      "hxa-su1-leaf-r2"[1]

Hca      1 "hxa-gpu02 mlx5_0"
[1]      "hxa-su1-leaf-r1"[2]
```

```
Hca      1 "hxa-gpu02 mlx5_1"
[1]      "hxa-su1-leaf-r2"[2]

Hca      1 "hxa-gpu03 mlx5_0"
[1]      "hxa-su2-leaf-r1"[1]

Hca      1 "hxa-gpu03 mlx5_1"
[1]      "hxa-su2-leaf-r2"[1]

Hca      1 "hxa-gpu04 mlx5_0"
[1]      "hxa-su2-leaf-r1"[2]

Hca      1 "hxa-gpu04 mlx5_1"
[1]      "hxa-su2-leaf-r2"[2]

Hca      1 "hxa-ufm-a2 mlx5_0"
[1]      "hxa-su2-leaf-r1"[30]
```

Appendix C — Quick Reference

Credentials used in this guide

Node	User	Password
Cumulus VX switches	cumulus	<code>cumulus</code> at first login, changed to <code>CumulusLinux!</code> (Lab 0)
Ubuntu servers, oob-mgmt-server	ubuntu	<code>nvidia</code> (check the node panel or console banner if different)

Everyday commands

Task	Command
Reach any node	<code>ssh cumulus@hxb-leaf-r1 / ssh ubuntu@hxb-gpu01</code> from oob-mgmt-server
Stage, review, apply, save	<code>nv set ... → nv config diff → nv config apply -y → nv config save</code>
Trial apply with auto-rollback	<code>nv config apply --confirm 300s -y</code> then <code>nv config apply --confirm-yes</code>
Roll back	<code>nv config history → nv config apply <revision></code>
BGP and routes	<code>nv show vrf default router bgp neighbor · sudo vtysh -c 'show bgp summary'</code>
EVPN	<code>nv show evpn vni · sudo vtysh -c 'show bgp l2vpn evpn route'</code>
RoCE QoS	<code>nv set qos roce mode lossless · nv show qos roce</code>
Soft-RoCE	<code>sudo modprobe rdma_rxe · sudo rdma link add rxe0 type rxe netdev eth1 · ibv_devinfo -d rxe0</code>
perftest	<code>server ib_write_bw -d rxe0 -R -F --report_gbits · client ... <server-ip></code>
ibsim tools	<code>ibsim-run sminfo · ibsim-run iblinkinfo --switches-only -l · ibsim-run ibqueryerrors</code>
ibsim console	<code>echo 'Unlink "node"[port]' > ~/ibsim/simcmd · ReLink · PerformanceSet</code>
Network Operator	<code>kubectl get nicclusterpolicy · kubectl get network-attachment-definitions</code>
Ansible	<code>ansible-playbook playbooks/site-vlans.yml --ask-vault-pass [--check] [--limit host]</code>

Appendix D — Troubleshooting the Lab Environment

Problem	Likely cause	What to do
Air rejects the JSON	Image name not available	Pick the nearest Cumulus VX 5.x / Ubuntu 24.04 image per node in the canvas (Lab 0 version note)
"Quota exceeded" when starting	Another simulation is running	Sleep the other simulation first; check concurrent vCPU/memory
Node unreachable by name	Still booting or no OOB lease	Wait; open its console; reboot the node from the canvas
<code>apt-get</code> fails on a server	No internet via the OOB NAT	Check the simulation's internet setting; test <code>ping 8.8.8.8</code> from the oob-mgmt-server
<code>modprobe rdma_rxe</code> fails	Module in the extras package	<code>sudo apt-get install -y linux-modules-extra-\$(uname -r)</code>
rx0 missing after a restart	Soft-RoCE isn't persistent	Re-run <code>modprobe</code> and <code>rdma link add</code>
NVUE command rejected	Syntax differs by release	Press Tab to explore the path; check the Cumulus Linux guide for your version
Ansible "connection refused"	API listening only on localhost	<code>nv set system api listening-address <eth0-ip></code> (Lab 12, Task 1)
ibsim tools hang	Simulator not running or stopped	<code>pgrep -af ibsim</code> ; restart it (Lab 8, Task 2)
Credits running low	Simulation left running	Always sleep with a checkpoint; set a sleep date

Appendix E — Where Each Exam Objective Is Practised

Objective	Labs
1.1–1.3 AI data centre design	0, 5, 8, 9
2.1 Spectrum-X / Cumulus configuration	1, 2, 3
2.2 RoCE, adaptive routing, congestion control	3, 4
2.3 BGP-EVPN multi-tenancy	5
2.4 NVIDIA Air	0, 6
2.5–2.6 Monitoring and telemetry	4, 7
2.7–2.8 DOCA, BlueField, SuperNIC	10 (concepts), book Chapter 8
3.1–3.4 InfiniBand SM, PKeys, routing, monitoring	8
4.1–4.2 Network Operator	10
5.1–5.2 cl-resource-query, WJH	7
5.3 Low-latency verification	9
5.4–5.5 UFM health, IB tools	8, 13
6.1 NVUE templates	11
6.2 Ansible	12
All	13